

set check range on

set check range off

Set range checking on or off, overriding the default setting for the current working language. A warning is issued if the setting does not match the language default. If a range error occurs and range checking is on, then a message is printed and evaluation of the expression is aborted.

set check range warn

Output messages when the GDB range checker detects a range error, but attempt to evaluate the expression anyway. Evaluating the expression may still be impossible for other reasons, such as accessing memory that the process does not own (a typical example from many Unix systems).

show check range

Show the current setting of the range checker, and whether or not it is being set automatically by GDB.

15.4 Supported Languages

GDB supports C, C++, D, Go, Objective-C, Fortran, OpenCL C, Pascal, Rust, assembly, Modula-2, and Ada. Some GDB features may be used in expressions regardless of the language you use: the GDB `@` and `::` operators, and the `{type}addr` construct (see Section 10.1 [Expressions], page 147) can be used with the constructs of any supported language.

The following sections detail to what degree each source language is supported by GDB. These sections are not meant to be language tutorials or references, but serve only as a reference guide to what the GDB expression parser accepts, and what input and output formats should look like for different languages. There are many good books written on each of these languages; please look to these for a language reference or tutorial.

15.4.1 C and C++

Since C and C++ are so closely related, many features of GDB apply to both languages. Whenever this is the case, we discuss those languages together.

The C++ debugging facilities are jointly implemented by the C++ compiler and GDB. Therefore, to debug your C++ code effectively, you must compile your C++ programs with a supported C++ compiler, such as GNU `g++`, or the HP ANSI C++ compiler (`aCC`).

15.4.1.1 C and C++ Operators

Operators must be defined on values of specific types. For instance, `+` is defined on numbers, but not on structures. Operators are often defined on groups of types.

For the purposes of C and C++, the following definitions hold:

- *Integral types* include `int` with any of its storage-class specifiers; `char`; `enum`; and, for C++, `bool`.
- *Floating-point types* include `float`, `double`, and `long double` (if supported by the target platform).
- *Pointer types* include all types defined as `(type *)`.
- *Scalar types* include all of the above.

The following operators are supported. They are listed here in order of increasing precedence:

,	The comma or sequencing operator. Expressions in a comma-separated list are evaluated from left to right, with the result of the entire expression being the last expression evaluated.
=	Assignment. The value of an assignment expression is the value assigned. Defined on scalar types.
op=	Used in an expression of the form <code>a op= b</code> , and translated to <code>a = a op b</code> . <code>op=</code> and <code>=</code> have the same precedence. The operator <code>op</code> is any one of the operators <code> </code> , <code>^</code> , <code>&</code> , <code><<</code> , <code>>></code> , <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> .
?:	The ternary operator. <code>a ? b : c</code> can be thought of as: if <code>a</code> then <code>b</code> else <code>c</code> . The argument <code>a</code> should be of an integral type.
	Logical OR. Defined on integral types.
&&	Logical AND. Defined on integral types.
	Bitwise OR. Defined on integral types.
^	Bitwise exclusive-OR. Defined on integral types.
&	Bitwise AND. Defined on integral types.
==, !=	Equality and inequality. Defined on scalar types. The value of these expressions is 0 for false and non-zero for true.
<, >, <=, >=	Less than, greater than, less than or equal, greater than or equal. Defined on scalar types. The value of these expressions is 0 for false and non-zero for true.
<<, >>	left shift, and right shift. Defined on integral types.
@	The GDB “artificial array” operator (see Section 10.1 [Expressions], page 147).
+, -	Addition and subtraction. Defined on integral types, floating-point types and pointer types.
*, /, %	Multiplication, division, and modulus. Multiplication and division are defined on integral and floating-point types. Modulus is defined on integral types.
++, --	Increment and decrement. When appearing before a variable, the operation is performed before the variable is used in an expression; when appearing after it, the variable’s value is used before the operation takes place.
*	Pointer dereferencing. Defined on pointer types. Same precedence as <code>++</code> .
&	Address operator. Defined on variables. Same precedence as <code>++</code> . For debugging C++, GDB implements a use of ‘&’ beyond what is allowed in the C++ language itself: you can use ‘&(&ref)’ to examine the address where a C++ reference variable (declared with ‘&ref’) is stored.
-	Negative. Defined on integral and floating-point types. Same precedence as <code>++</code> .
!	Logical negation. Defined on integral types. Same precedence as <code>++</code> .

<code>~</code>	Bitwise complement operator. Defined on integral types. Same precedence as <code>++</code> .
<code>., -></code>	Structure member, and pointer-to-structure member. For convenience, GDB regards the two as equivalent, choosing whether to dereference a pointer based on the stored type information. Defined on <code>struct</code> and <code>union</code> data.
<code>.*, ->*</code>	Dereferences of pointers to members.
<code>[]</code>	Array indexing. <code>a[i]</code> is defined as <code>*(a+i)</code> . Same precedence as <code>-></code> .
<code>()</code>	Function parameter list. Same precedence as <code>-></code> .
<code>::</code>	C++ scope resolution operator. Defined on <code>struct</code> , <code>union</code> , and <code>class</code> types.
<code>::</code>	Doubled colons also represent the GDB scope operator (see Section 10.1 [Expressions], page 147). Same precedence as <code>::</code> , above.

If an operator is redefined in the user code, GDB usually attempts to invoke the redefined version instead of using the operator's predefined meaning.

15.4.1.2 C and C++ Constants

GDB allows you to express the constants of C and C++ in the following ways:

- Integer constants are a sequence of digits. Octal constants are specified by a leading '0' (i.e. zero), and hexadecimal constants by a leading '0x' or '0X'. Constants may also end with a letter 'l', specifying that the constant should be treated as a `long` value.
- Floating point constants are a sequence of digits, followed by a decimal point, followed by a sequence of digits, and optionally followed by an exponent. An exponent is of the form: 'e[+|-]*nnn*', where *nnn* is another sequence of digits. The '+' is optional for positive exponents. A floating-point constant may also end with a letter 'f' or 'F', specifying that the constant should be treated as being of the `float` (as opposed to the default `double`) type; or with a letter 'l' or 'L', which specifies a `long double` constant.
- Enumerated constants consist of enumerated identifiers, or their integral equivalents.
- Character constants are a single character surrounded by single quotes ('), or a number—the ordinal value of the corresponding character (usually its ASCII value). Within quotes, the single character may be represented by a letter or by *escape sequences*, which are of the form '*nnn*', where *nnn* is the octal representation of the character's ordinal value; or of the form '*x*', where '*x*' is a predefined special character—for example, '\n' for newline.

Wide character constants can be written by prefixing a character constant with 'L', as in C. For example, 'L'*x*' is the wide form of '*x*'. The target wide character set is used when computing the value of this constant (see Section 10.21 [Character Sets], page 190).

- String constants are a sequence of character constants surrounded by double quotes ("). Any valid character constant (as described above) may appear. Double quotes within the string must be preceded by a backslash, so for instance "*a\"b'c*" is a string of five characters.

Wide string constants can be written by prefixing a string constant with 'L', as in C. The target wide character set is used when computing the value of this constant (see Section 10.21 [Character Sets], page 190).

- Pointer constants are an integral value. You can also write pointers to constants using the C operator ‘&’.
- Array constants are comma-separated lists surrounded by braces ‘{’ and ‘}’; for example, ‘{1,2,3}’ is a three-element array of integers, ‘{{1,2}, {3,4}, {5,6}}’ is a three-by-two array, and ‘{&"hi", &"there", &"fred"}’ is a three-element array of pointers.

15.4.1.3 C++ Expressions

GDB expression handling can interpret most C++ expressions.

Warning: GDB can only debug C++ code if you use the proper compiler and the proper debug format. Currently, GDB works best when debugging C++ code that is compiled with the most recent version of GCC possible. The DWARF debugging format is preferred; GCC defaults to this on most popular platforms. Other compilers and/or debug formats are likely to work badly or not at all when using GDB to debug C++ code. See Section 4.1 [Compilation], page 33.

1. Member function calls are allowed; you can use expressions like

```
count = aml->GetOriginal(x, y)
```
2. While a member function is active (in the selected stack frame), your expressions have the same namespace available as the member function; that is, GDB allows implicit references to the class instance pointer `this` following the same rules as C++. `using` declarations in the current scope are also respected by GDB.
3. You can call overloaded functions; GDB resolves the function call to the right definition, with some restrictions. GDB does not perform overload resolution involving user-defined type conversions, calls to constructors, or instantiations of templates that do not exist in the program. It also cannot handle ellipsis argument lists or default arguments.

It does perform integral conversions and promotions, floating-point promotions, arithmetic conversions, pointer conversions, conversions of class objects to base classes, and standard conversions such as those of functions or arrays to pointers; it requires an exact match on the number of function arguments.

Overload resolution is always performed, unless you have specified `set overload-resolution off`. See Section 15.4.1.7 [GDB Features for C++], page 237.

You must specify `set overload-resolution off` in order to use an explicit function signature to call an overloaded function, as in

```
p 'foo(char,int)('x', 13)
```

The GDB command-completion facility can simplify this; see Section 3.3 [Command Completion], page 24.

4. GDB understands variables declared as C++ lvalue or rvalue references; you can use them in expressions just as you do in C++ source—they are automatically dereferenced.
 In the parameter list shown when GDB displays a frame, the values of reference variables are not displayed (unlike other variables); this avoids clutter, since references are often used for large structures. The *address* of a reference variable is always shown, unless you have specified ‘`set print address off`’.
5. GDB supports the C++ name resolution operator `::`—your expressions can use it just as expressions in your program do. Since one scope may be defined in another, you can use

:: repeatedly if necessary, for example in an expression like ‘*scope1::scope2::name*’. GDB also allows resolving name scope by reference to source files, in both C and C++ debugging (see Section 10.3 [Program Variables], page 149).

6. GDB performs argument-dependent lookup, following the C++ specification.

15.4.1.4 C and C++ Defaults

If you allow GDB to set range checking automatically, it defaults to **off** whenever the working language changes to C or C++. This happens regardless of whether you or GDB selects the working language.

If you allow GDB to set the language automatically, it recognizes source files whose names end with `.c`, `.C`, or `.cc`, etc, and when GDB enters code compiled from one of these files, it sets the working language to C or C++. See Section 15.1.3 [Having GDB Infer the Source Language], page 230, for further details.

15.4.1.5 C and C++ Type and Range Checks

By default, when GDB parses C or C++ expressions, strict type checking is used. However, if you turn type checking off, GDB will allow certain non-standard conversions, such as promoting integer constants to pointers.

Range checking, if turned on, is done on mathematical operations. Array indices are not checked, since they are often used to index a pointer that is not itself an array.

15.4.1.6 GDB and C

The `set print union` and `show print union` commands apply to the `union` type. When set to ‘on’, any `union` that is inside a `struct` or `class` is also printed. Otherwise, it appears as ‘{...}’.

The `@` operator aids in the debugging of dynamic arrays, formed with pointers and a memory allocation function. See Section 10.1 [Expressions], page 147.

15.4.1.7 GDB Features for C++

Some GDB commands are particularly useful with C++, and some are designed specifically for use with C++. Here is a summary:

`breakpoint menus`

When you want a breakpoint in a function whose name is overloaded, GDB has the capability to display a menu of possible breakpoint locations to help you specify which function definition you want. See Section 10.2 [Ambiguous Expressions], page 148.

`rbreak regex`

Setting breakpoints using regular expressions is helpful for setting breakpoints on overloaded functions that are not members of any special classes. See Section 5.1.1 [Setting Breakpoints], page 60.

`catch throw`

`catch rethrow`

`catch catch`

Debug C++ exception handling using these commands. See Section 5.1.3 [Setting Catchpoints], page 71.

ptype *typename*

Print inheritance relationships as well as other information for type *typename*. See Chapter 16 [Examining the Symbol Table], page 265.

info vtbl *expression*.

The **info vtbl** command can be used to display the virtual method tables of the object computed by *expression*. This shows one entry per virtual table; there may be multiple virtual tables when multiple inheritance is in use.

demangle *name*

Demangle *name*. See Chapter 16 [Symbols], page 265, for a more complete description of the **demangle** command.

set print demangle**show print demangle****set print asm-demangle****show print asm-demangle**

Control whether C++ symbols display in their source form, both when displaying code as C++ source and when displaying disassemblies. See Section 10.9 [Print Settings], page 159.

set print object**show print object**

Choose whether to print derived (actual) or declared types of objects. See Section 10.9 [Print Settings], page 159.

set print vtbl**show print vtbl**

Control the format for printing virtual function tables. See Section 10.9 [Print Settings], page 159. (The **vtbl** commands do not work on programs compiled with the HP ANSI C++ compiler (**aCC**).)

set overload-resolution on

Enable overload resolution for C++ expression evaluation. The default is on. For overloaded functions, GDB evaluates the arguments and searches for a function whose signature matches the argument types, using the standard C++ conversion rules (see Section 15.4.1.3 [C++ Expressions], page 236, for details). If it cannot find a match, it emits a message.

set overload-resolution off

Disable overload resolution for C++ expression evaluation. For overloaded functions that are not class member functions, GDB chooses the first function of the specified name that it finds in the symbol table, whether or not its arguments are of the correct type. For overloaded functions that are class member functions, GDB searches for a function whose signature *exactly* matches the argument types.

show overload-resolution

Show the current setting of overload resolution.

Overloaded symbol names

You can specify a particular definition of an overloaded symbol, using the same notation that is used to declare such symbols in C++: type *symbol(types)*

rather than just *symbol*. You can also use the GDB command-line word completion facilities to list the available choices, or to finish the type list for you. See Section 3.3 [Command Completion], page 24, for details on how to do this.

Breakpoints in template functions

Similar to how overloaded symbols are handled, GDB will ignore template parameter lists when it encounters a symbol which includes a C++ template. This permits setting breakpoints on families of template functions or functions whose parameters include template types.

The `-qualified` flag may be used to override this behavior, causing GDB to search for a specific function or type.

The GDB command-line word completion facility also understands template parameters and may be used to list available choices or finish template parameter lists for you. See Section 3.3 [Command Completion], page 24, for details on how to do this.

Breakpoints in functions with ABI tags

The GNU C++ compiler introduced the notion of ABI “tags”, which correspond to changes in the ABI of a type, function, or variable that would not otherwise be reflected in a mangled name. See <https://developers.redhat.com/blog/2015/02/05/gcc5-and-the-c11-abi/> for more detail.

The ABI tags are visible in C++ demangled names. For example, a function that returns a `std::string`:

```
std::string function(int);
```

when compiled for the C++11 ABI is marked with the `cxx11` ABI tag, and GDB displays the symbol like this:

```
function[abi:cxx11](int)
```

You can set a breakpoint on such functions simply as if they had no tag. For example:

```
(gdb) b function(int)
Breakpoint 2 at 0x40060d: file main.cc, line 10.
(gdb) info breakpoints
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x0040060d in function[abi:cxx11](int)
                                     at main.cc:10
```

On the rare occasion you need to disambiguate between different ABI tags, you can do so by simply including the ABI tag in the function name, like:

```
(gdb) b ambiguous[abi:other_tag](int)
```

15.4.1.8 Decimal Floating Point format

GDB can examine, set and perform computations with numbers in decimal floating point format, which in the C language correspond to the `_Decimal32`, `_Decimal64` and `_Decimal128` types as specified by the extension to support decimal floating-point arithmetic.

There are two encodings in use, depending on the architecture: BID (Binary Integer Decimal) for x86 and x86-64, and DPD (Densely Packed Decimal) for PowerPC and S/390. GDB will use the appropriate encoding for the configured target.

Because of a limitation in `libdecnumber`, the library used by GDB to manipulate decimal floating point numbers, it is not possible to convert (using a cast, for example) integers wider than 32-bit to decimal float.

In addition, in order to imitate GDB's behavior with binary floating point computations, error checking in decimal float operations ignores underflow, overflow and divide by zero exceptions.

In the PowerPC architecture, GDB provides a set of pseudo-registers to inspect `_Decimal128` values stored in floating point registers. See Section 21.4.6 [PowerPC], page 364, for more details.

15.4.2 D

GDB can be used to debug programs written in D and compiled with GDC, LDC or DMD compilers. Currently GDB supports only one D specific feature — dynamic arrays.

15.4.3 Go

GDB can be used to debug programs written in Go and compiled with `gccgo` or `6g` compilers.

Here is a summary of the Go-specific features and restrictions:

The current Go package

The name of the current package does not need to be specified when specifying global variables and functions.

For example, given the program:

```
package main
var myglob = "Shall we?"
func main () {
    // ...
}
```

When stopped inside `main` either of these work:

```
(gdb) p myglob
(gdb) p main.myglob
```

Builtin Go types

The `string` type is recognized by GDB and is printed as a string.

Builtin Go functions

The GDB expression parser recognizes the `unsafe.Sizeof` function and handles it internally.

Restrictions on Go expressions

All Go operators are supported except `&^`. The Go `_` “blank identifier” is not supported. Automatic dereferencing of pointers is not supported.

15.4.4 Objective-C

This section provides information about some commands and command options that are useful for debugging Objective-C code. See also Chapter 16 [Symbols], page 265, and Chapter 16 [Symbols], page 265, for a few more commands specific to Objective-C support.

15.4.4.1 Method Names in Commands

The following commands have been extended to accept Objective-C method names as line specifications:

- `clear`
- `break`
- `info line`
- `jump`
- `list`

A fully qualified Objective-C method name is specified as

```
-[Class methodName]
```

where the minus sign is used to indicate an instance method and a plus sign (not shown) is used to indicate a class method. The class name *Class* and method name *methodName* are enclosed in brackets, similar to the way messages are specified in Objective-C source code. For example, to set a breakpoint at the `create` instance method of class `Fruit` in the program currently being debugged, enter:

```
break -[Fruit create]
```

To list ten program lines around the `initialize` class method, enter:

```
list +[NSText initialize]
```

In the current version of GDB, the plus or minus sign is required. In future versions of GDB, the plus or minus sign will be optional, but you can use it to narrow the search. It is also possible to specify just a method name:

```
break create
```

You must specify the complete method name, including any colons. If your program's source files contain more than one `create` method, you'll be presented with a numbered list of classes that implement that method. Indicate your choice by number, or type '0' to exit if none apply.

As another example, to clear a breakpoint established at the `makeKeyAndOrderFront:` method of the `NSWindow` class, enter:

```
clear -[NSWindow makeKeyAndOrderFront:]
```

15.4.4.2 The Print Command With Objective-C

The print command has also been extended to accept methods. For example:

```
print -[object hash]
```

will tell GDB to send the `hash` message to *object* and print the result. Also, an additional command has been added, `print-object` or `po` for short, which is meant to print the description of an object. However, this command may only work with certain Objective-C libraries that have a particular hook function, `_NSPrintForDebugger`, defined.

15.4.5 OpenCL C

This section provides information about GDB's OpenCL C support.

15.4.5.1 OpenCL C Datatypes

GDB supports the builtin scalar and vector datatypes specified by OpenCL 1.1. In addition the half- and double-precision floating point data types of the `cl_khr_fp16` and `cl_khr_fp64` OpenCL extensions are also known to GDB.

15.4.5.2 OpenCL C Expressions

GDB supports accesses to vector components including the access as `lvalue` where possible. Since OpenCL C is based on C99 most C expressions supported by GDB can be used as well.

15.4.5.3 OpenCL C Operators

GDB supports the operators specified by OpenCL 1.1 for scalar and vector data types.

15.4.6 Fortran

GDB can be used to debug programs written in Fortran. Note, that not all Fortran language features are available yet.

Some Fortran compilers (GNU Fortran 77 and Fortran 95 compilers among them) append an underscore to the names of variables and functions. When you debug programs compiled by those compilers, you will need to refer to variables and functions with a trailing underscore.

Fortran symbols are usually case-insensitive, so GDB by default uses case-insensitive matching for Fortran symbols. You can change that with the `'set case-insensitive'` command, see Chapter 16 [Symbols], page 265, for the details.

15.4.6.1 Fortran Types

In Fortran the primitive data-types have an associated `KIND` type parameter, written as `'type*kindparam'`, `'type*kindparam'`, or in the GDB-only dialect `'type_kindparam'`. A concrete example would be `'Real*4'`, `'Real(kind=4)'`, and `'Real_4'`. The kind of a type can be retrieved by using the intrinsic function `KIND`, see Section 15.4.6.3 [Fortran Intrinsics], page 243.

Generally, the actual implementation of the `KIND` type parameter is compiler specific. In GDB the kind parameter is implemented in accordance with its use in the GNU `gfortran` compiler. Here, the kind parameter for a given *type* specifies its size in memory — a Fortran `Integer*4` or `Integer(kind=4)` would be an integer type occupying 4 bytes of memory. An exception to this rule is the `Complex` type for which the kind of the type does not specify its entire size, but the size of each of the two `Real`'s it is composed of. A `Complex*4` would thus consist of two `Real*4`s and occupy 8 bytes of memory.

For every type there is also a default kind associated with it, e.g. `Integer` in GDB will internally be an `Integer*4` (see the table below for default types). The default types are the same as in GNU compilers but note, that the GNU default types can actually be changed by compiler flags such as `-fdefault-integer-8` and `-fdefault-real-8`.

Not every kind parameter is valid for every type and in GDB the following type kinds are available.

`Integer` `Integer*1`, `Integer*2`, `Integer*4`, `Integer*8`, and `Integer = Integer*4`.

Logical `Logical*1, Logical*2, Logical*4, Logical*8, and Logical = Logical*4.`
Real `Real*4, Real*8, Real*16, and Real = Real*4.`
Complex `Complex*4, Complex*8, Complex*16, and Complex = Complex*4.`

15.4.6.2 Fortran Operators and Expressions

Operators must be defined on values of specific types. For instance, `+` is defined on numbers, but not on characters or other non- arithmetic types. Operators are often defined on groups of types.

****** The exponentiation operator. It raises the first operand to the power of the second one.

: The range operator. Normally used in the form of `array(low:high)` to represent a section of array.

% The access component operator. Normally used to access elements in derived types. Also suitable for unions. As unions aren't part of regular Fortran, this can only happen when accessing a register that uses a gdbarch-defined union type.

:: The scope operator. Normally used to access variables in modules or to set breakpoints on subroutines nested in modules or in other subroutines (internal subroutines).

15.4.6.3 Fortran Ininsics

Fortran provides a large set of intrinsic procedures. GDB implements an incomplete subset of those procedures and their overloads. Some of these procedures take an optional `KIND` parameter, see Section 15.4.6.1 [Fortran Types], page 242.

ABS(a) Computes the absolute value of its argument *a*. Currently not supported for **Complex** arguments.

ALLOCATE(array)
 Returns whether *array* is allocated or not.

ASSOCIATED(pointer [, target])
 Returns the association status of the pointer *pointer* or, if *target* is present, whether *pointer* is associated with the target *target*.

CEILING(a [, kind])
 Computes the least integer greater than or equal to *a*. The optional parameter *kind* specifies the kind of the return type **Integer(kind)**.

CMPLX(x [, y [, kind]])
 Returns a complex number where *x* is converted to the real component. If *y* is present it is converted to the imaginary component. If *y* is not present then the imaginary component is set to 0.0 except if *x* itself is of **Complex** type. The optional parameter *kind* specifies the kind of the return type **Complex(kind)**.

FLOOR(a [, kind])
 Computes the greatest integer less than or equal to *a*. The optional parameter *kind* specifies the kind of the return type **Integer(kind)**.

- KIND(a)** Returns the kind value of the argument *a*, see Section 15.4.6.1 [Fortran Types], page 242.
- LBOUND(array [, dim [, kind]])**
Returns the lower bounds of an *array*, or a single lower bound along the *dim* dimension if present. The optional parameter *kind* specifies the kind of the return type *Integer(kind)*.
- LOC(x)** Returns the address of *x* as an *Integer*.
- MOD(a, p)** Computes the remainder of the division of *a* by *p*.
- MODULO(a, p)**
Computes the *a* modulo *p*.
- RANK(a)** Returns the rank of a scalar or array (scalars have rank 0).
- SHAPE(a)** Returns the shape of a scalar or array (scalars have shape '()').
- SIZE(array[, dim [, kind]])**
Returns the extent of *array* along a specified dimension *dim*, or the total number of elements in *array* if *dim* is absent. The optional parameter *kind* specifies the kind of the return type *Integer(kind)*.
- UBOUND(array [, dim [, kind]])**
Returns the upper bounds of an *array*, or a single upper bound along the *dim* dimension if present. The optional parameter *kind* specifies the kind of the return type *Integer(kind)*.

15.4.6.4 Special Fortran Commands

GDB has some commands to support Fortran-specific features, such as displaying common blocks.

info common [*common-name*]

This command prints the values contained in the Fortran **COMMON** block whose name is *common-name*. With no argument, the names of all **COMMON** blocks visible at the current program location are printed.

set fortran repack-array-slices [on|off]

show fortran repack-array-slices

When taking a slice from an array, a Fortran compiler can choose to either produce an array descriptor that describes the slice in place, or it may repack the slice, copying the elements of the slice into a new region of memory.

When this setting is on, then GDB will also repack array slices in some situations. When this setting is off, then GDB will create array descriptors for slices that reference the original data in place.

GDB will never repack an array slice if the data for the slice is contiguous within the original array.

GDB will always repack string slices if the data for the slice is non-contiguous within the original string as GDB does not support printing non-contiguous strings.

The default for this setting is **off**.

15.4.7 Pascal

Debugging Pascal programs which use sets, subranges, file variables, or nested functions does not currently work. GDB does not support entering expressions, printing values, or similar features using Pascal syntax.

The Pascal-specific command `set print pascal_static-members` controls whether static members of Pascal objects are displayed. See Section 10.9 [Print Settings], page 159.

15.4.8 Rust

GDB supports the Rust Programming Language (<https://www.rust-lang.org/>). Type- and value-printing, and expression parsing, are reasonably complete. However, there are a few peculiarities and holes to be aware of.

- Linespecs (see Section 9.2 [Location Specifications], page 128) are never relative to the current crate. Instead, they act as if there were a global namespace of crates, somewhat similar to the way `extern crate` behaves.

That is, if GDB is stopped at a breakpoint in a function in crate ‘A’, module ‘B’, then `break B::f` will attempt to set a breakpoint in a function named ‘f’ in a crate named ‘B’.

As a consequence of this approach, linespecs also cannot refer to items using ‘`self::`’ or ‘`super::`’.

- Because GDB implements Rust name-lookup semantics in expressions, it will sometimes prepend the current crate to a name. For example, if GDB is stopped at a breakpoint in the crate ‘K’, then `print ::x::y` will try to find the symbol ‘K::x::y’.

However, since it is useful to be able to refer to other crates when debugging, GDB provides the `extern` extension to circumvent this. To use the extension, just put `extern` before a path expression to refer to the otherwise unavailable “global” scope.

In the above example, if you wanted to refer to the symbol ‘y’ in the crate ‘x’, you would use `print extern x::y`.

- The Rust expression evaluator does not support “statement-like” expressions such as `if` or `match`, or lambda expressions.
- Tuple expressions are not implemented.
- The Rust expression evaluator does not currently implement the `Drop` trait. Objects that may be created by the evaluator will never be destroyed.
- GDB does not implement type inference for generics. In order to call generic functions or otherwise refer to generic items, you will have to specify the type parameters manually.
- In an expression context, completing a generic function name will give syntactically invalid results. This happens because Rust requires the ‘`::`’ operator between the function name and its generic arguments. For example, GDB might provide a completion like `crate::f<u32>`, where the parser would require `crate::f::<u32>`.
- As of this writing, the Rust compiler (version 1.8) has a few holes in the debugging information it generates. These holes prevent certain features from being implemented by GDB:
 - Method calls cannot be made via traits.
 - Operator overloading is not implemented.

- When debugging in a monomorphized function, you cannot use the generic type names.
- The type `Self` is not available.
- `use` statements are not available, so some names may not be available in the crate.

15.4.9 Modula-2

The extensions made to GDB to support Modula-2 only support output from the GNU Modula-2 compiler (which is currently being developed). Other Modula-2 compilers are not currently supported, and attempting to debug executables produced by them is most likely to give an error as GDB reads in the executable's symbol table.

15.4.9.1 Operators

Operators must be defined on values of specific types. For instance, `+` is defined on numbers, but not on structures. Operators are often defined on groups of types. For the purposes of Modula-2, the following definitions hold:

- *Integral types* consist of `INTEGER`, `CARDINAL`, and their subranges.
- *Character types* consist of `CHAR` and its subranges.
- *Floating-point types* consist of `REAL`.
- *Pointer types* consist of anything declared as `POINTER TO type`.
- *Scalar types* consist of all of the above.
- *Set types* consist of `SET` and `BITSET` types.
- *Boolean types* consist of `BOOLEAN`.

The following operators are supported, and appear in order of increasing precedence:

<code>,</code>	Function argument or array index separator.
<code>:=</code>	Assignment. The value of <code>var := value</code> is <code>value</code> .
<code><, ></code>	Less than, greater than on integral, floating-point, or enumerated types.
<code><=, >=</code>	Less than or equal to, greater than or equal to on integral, floating-point and enumerated types, or set inclusion on set types. Same precedence as <code><</code> .
<code>=, <>, #</code>	Equality and two ways of expressing inequality, valid on scalar types. Same precedence as <code><</code> . In GDB scripts, only <code><></code> is available for inequality, since <code>#</code> conflicts with the script comment character.
<code>IN</code>	Set membership. Defined on set types and the types of their members. Same precedence as <code><</code> .
<code>OR</code>	Boolean disjunction. Defined on boolean types.
<code>AND, &</code>	Boolean conjunction. Defined on boolean types.
<code>@</code>	The GDB “artificial array” operator (see Section 10.1 [Expressions], page 147).
<code>+, -</code>	Addition and subtraction on integral and floating-point types, or union and difference on set types.
<code>*</code>	Multiplication on integral and floating-point types, or set intersection on set types.

/	Division on floating-point types, or symmetric set difference on set types. Same precedence as <code>*</code> .
DIV, MOD	Integer division and remainder. Defined on integral types. Same precedence as <code>*</code> .
-	Negative. Defined on <code>INTEGER</code> and <code>REAL</code> data.
^	Pointer dereferencing. Defined on pointer types.
NOT	Boolean negation. Defined on boolean types. Same precedence as <code>^</code> .
.	<code>RECORD</code> field selector. Defined on <code>RECORD</code> data. Same precedence as <code>^</code> .
[]	Array indexing. Defined on <code>ARRAY</code> data. Same precedence as <code>^</code> .
()	Procedure argument list. Defined on <code>PROCEDURE</code> objects. Same precedence as <code>^</code> .
::, .	GDB and Modula-2 scope operators.

Warning: Set expressions and their operations are not yet supported, so GDB treats the use of the operator `IN`, or the use of operators `+`, `-`, `*`, `/`, `=`, `,`, `<>`, `#`, `<=`, and `>=` on sets as an error.

15.4.9.2 Built-in Functions and Procedures

Modula-2 also makes available several built-in procedures and functions. In describing these, the following metavariables are used:

<i>a</i>	represents an <code>ARRAY</code> variable.
<i>c</i>	represents a <code>CHAR</code> constant or variable.
<i>i</i>	represents a variable or constant of integral type.
<i>m</i>	represents an identifier that belongs to a set. Generally used in the same function with the metavariable <i>s</i> . The type of <i>s</i> should be <code>SET OF mtype</code> (where <i>mtype</i> is the type of <i>m</i>).
<i>n</i>	represents a variable or constant of integral or floating-point type.
<i>r</i>	represents a variable or constant of floating-point type.
<i>t</i>	represents a type.
<i>v</i>	represents a variable.
<i>x</i>	represents a variable or constant of one of many types. See the explanation of the function for details.

All Modula-2 built-in procedures also return a result, described below.

<code>ABS(<i>n</i>)</code>	Returns the absolute value of <i>n</i> .
<code>ADR(<i>n</i>)</code>	Returns the memory address of <i>n</i> .
<code>CAP(<i>c</i>)</code>	If <i>c</i> is a lower case letter, it returns its upper case equivalent, otherwise it returns its argument.
<code>CHR(<i>i</i>)</code>	Returns the character whose ordinal value is <i>i</i> .

<code>DEC(v)</code>	Decrements the value in the variable <i>v</i> by one. Returns the new value.
<code>DEC(v,i)</code>	Decrements the value in the variable <i>v</i> by <i>i</i> . Returns the new value.
<code>EXCL(m,s)</code>	Removes the element <i>m</i> from the set <i>s</i> . Returns the new set.
<code>FLOAT(i)</code>	Returns the floating point equivalent of the integer <i>i</i> .
<code>HIGH(a)</code>	Returns the index of the last member of <i>a</i> .
<code>INC(v)</code>	Increments the value in the variable <i>v</i> by one. Returns the new value.
<code>INC(v,i)</code>	Increments the value in the variable <i>v</i> by <i>i</i> . Returns the new value.
<code>INCL(m,s)</code>	Adds the element <i>m</i> to the set <i>s</i> if it is not already there. Returns the new set.
<code>MAX(t)</code>	Returns the maximum value of the type <i>t</i> .
<code>MIN(t)</code>	Returns the minimum value of the type <i>t</i> .
<code>ODD(i)</code>	Returns boolean TRUE if <i>i</i> is an odd number.
<code>ORD(x)</code>	Returns the ordinal value of its argument. For example, the ordinal value of a character is its ASCII value (on machines supporting the ASCII character set). The argument <i>x</i> must be of an ordered type, which include integral, character and enumerated types.
<code>SIZE(x)</code>	Returns the size of its argument. The argument <i>x</i> can be a variable or a type.
<code>TRUNC(r)</code>	Returns the integral part of <i>r</i> .
<code>TSIZE(x)</code>	Returns the size of its argument. The argument <i>x</i> can be a variable or a type.
<code>VAL(t,i)</code>	Returns the member of the type <i>t</i> whose ordinal value is <i>i</i> .

Warning: Sets and their operations are not yet supported, so GDB treats the use of procedures `INCL` and `EXCL` as an error.

15.4.9.3 Constants

GDB allows you to express the constants of Modula-2 in the following ways:

- Integer constants are simply a sequence of digits. When used in an expression, a constant is interpreted to be type-compatible with the rest of the expression. Hexadecimal integers are specified by a trailing ‘H’, and octal integers by a trailing ‘B’.
- Floating point constants appear as a sequence of digits, followed by a decimal point and another sequence of digits. An optional exponent can then be specified, in the form ‘E[+|-]nnn’, where ‘[+|-]nnn’ is the desired exponent. All of the digits of the floating point constant must be valid decimal (base 10) digits.
- Character constants consist of a single character enclosed by a pair of like quotes, either single (') or double ("). They may also be expressed by their ordinal value (their ASCII value, usually) followed by a ‘C’.
- String constants consist of a sequence of characters enclosed by a pair of like quotes, either single (') or double ("). Escape sequences in the style of C are also allowed. See Section 15.4.1.2 [C and C++ Constants], page 235, for a brief explanation of escape sequences.

- Enumerated constants consist of an enumerated identifier.
- Boolean constants consist of the identifiers `TRUE` and `FALSE`.
- Pointer constants consist of integral values only.
- Set constants are not yet supported.

15.4.9.4 Modula-2 Types

Currently GDB can print the following data types in Modula-2 syntax: array types, record types, set types, pointer types, procedure types, enumerated types, subrange types and base types. You can also print the contents of variables declared using these type. This section gives a number of simple source code examples together with sample GDB sessions.

The first example contains the following section of code:

```
VAR
  s: SET OF CHAR ;
  r: [20..40] ;
```

and you can request GDB to interrogate the type and value of `r` and `s`.

```
(gdb) print s
{'A'..'C', 'Z'}
(gdb) ptype s
SET OF CHAR
(gdb) print r
21
(gdb) ptype r
[20..40]
```

Likewise if your source code declares `s` as:

```
VAR
  s: SET ['A'..'Z'] ;
```

then you may query the type of `s` by:

```
(gdb) ptype s
type = SET ['A'..'Z']
```

Note that at present you cannot interactively manipulate set expressions using the debugger.

The following example shows how you might declare an array in Modula-2 and how you can interact with GDB to print its type and contents:

```
VAR
  s: ARRAY [-10..10] OF CHAR ;
(gdb) ptype s
ARRAY [-10..10] OF CHAR
```

Note that the array handling is not yet complete and although the type is printed correctly, expression handling still assumes that all arrays have a lower bound of zero and not `-10` as in the example above.

Here are some more type related Modula-2 examples:

```
TYPE
  colour = (blue, red, yellow, green) ;
  t = [blue..yellow] ;
VAR
  s: t ;
BEGIN
  s := blue ;
```

The GDB interaction shows how you can query the data type and value of a variable.

```
(gdb) print s
$1 = blue
(gdb) ptype t
type = [blue..yellow]
```

In this example a Modula-2 array is declared and its contents displayed. Observe that the contents are written in the same way as their C counterparts.

```
VAR
  s: ARRAY [1..5] OF CARDINAL ;
BEGIN
  s[1] := 1 ;

(gdb) print s
$1 = {1, 0, 0, 0, 0}
(gdb) ptype s
type = ARRAY [1..5] OF CARDINAL
```

The Modula-2 language interface to GDB also understands pointer types as shown in this example:

```
VAR
  s: POINTER TO ARRAY [1..5] OF CARDINAL ;
BEGIN
  NEW(s) ;
  s^[1] := 1 ;
```

and you can request that GDB describes the type of `s`.

```
(gdb) ptype s
type = POINTER TO ARRAY [1..5] OF CARDINAL
```

GDB handles compound types as we can see in this example. Here we combine array types, record types, pointer types and subrange types:

```
TYPE
  foo = RECORD
    f1: CARDINAL ;
    f2: CHAR ;
    f3: myarray ;
  END ;

  myarray = ARRAY myrange OF CARDINAL ;
  myrange = [-2..2] ;
VAR
  s: POINTER TO ARRAY myrange OF foo ;
```

and you can ask GDB to describe the type of `s` as shown below.

```
(gdb) ptype s
type = POINTER TO ARRAY [-2..2] OF foo = RECORD
  f1 : CARDINAL;
  f2 : CHAR;
  f3 : ARRAY [-2..2] OF CARDINAL;
END
```

15.4.9.5 Modula-2 Defaults

If type and range checking are set automatically by GDB, they both default to `on` whenever the working language changes to Modula-2. This happens regardless of whether you or GDB selected the working language.

If you allow GDB to set the language automatically, then entering code compiled from a file whose name ends with `.mod` sets the working language to Modula-2. See Section 15.1.3 [Having GDB Infer the Source Language], page 230, for further details.

15.4.9.6 Deviations from Standard Modula-2

A few changes have been made to make Modula-2 programs easier to debug. This is done primarily via loosening its type strictness:

- Unlike in standard Modula-2, pointer constants can be formed by integers. This allows you to modify pointer variables during debugging. (In standard Modula-2, the actual address contained in a pointer variable is hidden from you; it can only be modified through direct assignment to another pointer variable or expression that returned a pointer.)
- C escape sequences can be used in strings and characters to represent non-printable characters. GDB prints out strings with these escape sequences embedded. Single non-printable characters are printed using the ‘`CHR(nnn)`’ format.
- The assignment operator (`:=`) returns the value of its right-hand argument.
- All built-in procedures both modify *and* return their argument.

15.4.9.7 Modula-2 Type and Range Checks

Warning: in this release, GDB does not yet perform type or range checking.

GDB considers two Modula-2 variables type equivalent if:

- They are of types that have been declared equivalent via a `TYPE t1 = t2` statement
- They have been declared on the same line. (Note: This is true of the GNU Modula-2 compiler, but it may not be true of other compilers.)

As long as type checking is enabled, any attempt to combine variables whose types are not equivalent is an error.

Range checking is done on all mathematical operations, assignment, array index bounds, and all built-in functions and procedures.

15.4.9.8 The Scope Operators `::` and `.`

There are a few subtle differences between the Modula-2 scope operator (`.`) and the GDB scope operator (`::`). The two have similar syntax:

```
module . id
scope :: id
```

where *scope* is the name of a module or a procedure, *module* the name of a module, and *id* is any declared identifier within your program, except another module.

Using the `::` operator makes GDB search the scope specified by *scope* for the identifier *id*. If it is not found in the specified scope, then GDB searches all scopes enclosing the one specified by *scope*.

Using the `.` operator makes GDB search the current scope for the identifier specified by *id* that was imported from the definition module specified by *module*. With this operator, it is an error if the identifier *id* was not imported from definition module *module*, or if *id* is not an identifier in *module*.

15.4.9.9 GDB and Modula-2

Some GDB commands have little use when debugging Modula-2 programs. Five subcommands of `set print` and `show print` apply specifically to C and C++: `'vtbl'`, `'demangle'`, `'asm-demangle'`, `'object'`, and `'union'`. The first four apply to C++, and the last to the C `union` type, which has no direct analogue in Modula-2.

The `@` operator (see Section 10.1 [Expressions], page 147), while available with any language, is not useful with Modula-2. Its intent is to aid the debugging of *dynamic arrays*, which cannot be created in Modula-2 as they can in C or C++. However, because an address can be specified by an integral constant, the construct `'{type}adrexpr'` is still useful.

In GDB scripts, the Modula-2 inequality operator `#` is interpreted as the beginning of a comment. Use `<>` instead.

15.4.10 Ada

The extensions made to GDB for Ada only support output from the GNU Ada (GNAT) compiler. Other Ada compilers are not currently supported, and attempting to debug executables produced by them is most likely to be difficult.

15.4.10.1 Introduction

The Ada mode of GDB supports a fairly large subset of Ada expression syntax, with some extensions. The philosophy behind the design of this subset is

- That GDB should provide basic literals and access to operations for arithmetic, dereferencing, field selection, indexing, and subprogram calls, leaving more sophisticated computations to subprograms written into the program (which therefore may be called from GDB).
- That type safety and strict adherence to Ada language restrictions are not particularly important to the GDB user.
- That brevity is important to the GDB user.

Thus, for brevity, the debugger acts as if all names declared in user-written packages are directly visible, even if they are not visible according to Ada rules, thus making it unnecessary to fully qualify most names with their packages, regardless of context. Where this causes ambiguity, GDB asks the user's intent.

The debugger will start in Ada mode if it detects an Ada main program. As for other languages, it will enter Ada mode when stopped in a program that was translated from an Ada source file.

While in Ada mode, you may use `'--'` for comments. This is useful mostly for documenting command files. The standard GDB comment (`'#'`) still works at the beginning of a line in Ada mode, but not in the middle (to allow based literals).

15.4.10.2 Omissions from Ada

Here are the notable omissions from the subset:

- Only a subset of the attributes are supported:
 - `'First'`, `'Last'`, and `'Length'` on array objects (not on types and subtypes).
 - `'Min'` and `'Max'`.

- 'Pos and 'Val.
- 'Tag.
- 'Range on array objects (not subtypes), but only as the right operand of the membership (in) operator.
- 'Access, 'Unchecked_Access, and 'Unrestricted_Access (a GNAT extension).
- 'Address.
- 'Size is available for objects (not types).
- 'Object_Size is available, but not for indefinite types.
- The names in `Characters.Latin_1` are not available.
- Equality tests ('=' and '/=') on arrays test for bitwise equality of representations. They will generally work correctly for strings and arrays whose elements have integer or enumeration types. They may not work correctly for arrays whose element types have user-defined equality, for arrays of real values (in particular, IEEE-conformant floating point, because of negative zeroes and NaNs), and for arrays whose elements contain unused bits with indeterminate values.
- The other component-by-component array operations (`and`, `or`, `xor`, `not`, and relational tests other than equality) are not implemented.
- There is limited support for array and record aggregates. They are permitted only on the right sides of assignments, as in these examples:

```
(gdb) set An_Array := (1, 2, 3, 4, 5, 6)
(gdb) set An_Array := (1, others => 0)
(gdb) set An_Array := (0|4 => 1, 1..3 => 2, 5 => 6)
(gdb) set A_2D_Array := ((1, 2, 3), (4, 5, 6), (7, 8, 9))
(gdb) set A_Record := (1, "Peter", True);
(gdb) set A_Record := (Name => "Peter", Id => 1, Alive => True)
```

Changing a discriminant's value by assigning an aggregate has an undefined effect if that discriminant is used within the record. However, you can first modify discriminants by directly assigning to them (which normally would not be allowed in Ada), and then performing an aggregate assignment. For example, given a variable `A_Rec` declared to have a type such as:

```
type Rec (Len : Small_Integer := 0) is record
  Id : Integer;
  Vals : IntArray (1 .. Len);
end record;
```

you can assign a value with a different size of `Vals` with two assignments:

```
(gdb) set A_Rec.Len := 4
(gdb) set A_Rec := (Id => 42, Vals => (1, 2, 3, 4))
```

As this example also illustrates, GDB is very loose about the usual rules concerning aggregates. You may leave out some of the components of an array or record aggregate (such as the `Len` component in the assignment to `A_Rec` above); they will retain their original values upon assignment. You may freely use dynamic values as indices in component associations. You may even use overlapping or redundant component associations, although which component values are assigned in such cases is not defined.

- Calls to dispatching subprograms are not implemented.
- The overloading algorithm is much more limited (i.e., less selective) than that of real Ada. It makes only limited use of the context in which a subexpression appears to

resolve its meaning, and it is much looser in its rules for allowing type matches. As a result, some function calls will be ambiguous, and the user will be asked to choose the proper resolution.

- The **new** operator is not implemented.
- Entry calls are not implemented.
- Aside from printing, arithmetic operations on the native VAX floating-point formats are not supported.
- It is not possible to slice a packed array.
- The names **True** and **False**, when not part of a qualified name, are interpreted as if implicitly prefixed by **Standard**, regardless of context. Should your program redefine these names in a package or procedure (at best a dubious practice), you will have to use fully qualified names to access their new definitions.
- Based real literals are not implemented.

15.4.10.3 Additions to Ada

As it does for other languages, GDB makes certain generic extensions to Ada (see Section 10.1 [Expressions], page 147):

- If the expression *E* is a variable residing in memory (typically a local variable or array element) and *N* is a positive integer, then *E@N* displays the values of *E* and the *N*-1 adjacent variables following it in memory as an array. In Ada, this operator is generally not necessary, since its prime use is in displaying parts of an array, and slicing will usually do this in Ada. However, there are occasional uses when debugging programs in which certain debugging information has been optimized away.
- *B::var* means “the variable named *var* that appears in function or file *B*.” When *B* is a file name, you must typically surround it in single quotes.
- The expression *{type} addr* means “the variable of type *type* that appears at address *addr*.”
- A name starting with ‘\$’ is a convenience variable (see Section 10.12 [Convenience Vars], page 173) or a machine register (see Section 10.14 [Registers], page 181).

In addition, GDB provides a few other shortcuts and outright additions specific to Ada:

- The assignment statement is allowed as an expression, returning its right-hand operand as its value. Thus, you may enter


```
(gdb) set x := y + 3
(gdb) print A(tmp := y + 1)
```
- The semicolon is allowed as an “operator,” returning as its value the value of its right-hand operand. This allows, for example, complex conditional breaks:


```
(gdb) break f
(gdb) condition 1 (report(i); k += 1; A(k) > 100)
```
- An extension to based literals can be used to specify the exact byte contents of a floating-point literal. After the base, you can use from zero to two ‘l’ characters, followed by an ‘f’. The number of ‘l’ characters controls the width of the resulting real constant: zero means **Float** is used, one means **Long_Float**, and two means **Long_Long_Float**.


```
(gdb) print 16f#41b80000#
$1 = 23.0
```

- Rather than use catenation and symbolic character names to introduce special characters into strings, one may instead use a special bracket notation, which is also used to print strings. A sequence of characters of the form `["XX"]` within a string or character literal denotes the (single) character whose numeric encoding is `XX` in hexadecimal. The sequence of characters `[""]` also denotes a single quotation mark in strings. For example,

```
"One line.["0a"]Next line.["0a"]"
```

contains an ASCII newline character (`Ada.Characters.Latin_1.LF`) after each period.

- The subtype used as a prefix for the attributes `'Pos`, `'Min`, and `'Max` is optional (and is ignored in any case). For example, it is valid to write

```
(gdb) print 'max(x, y)
```

- When printing arrays, GDB uses positional notation when the array has a lower bound of 1, and uses a modified named notation otherwise. For example, a one-dimensional array of three integers with a lower bound of 3 might print as

```
(3 => 10, 17, 1)
```

That is, in contrast to valid Ada, only the first component has a `=>` clause.

- You may abbreviate attributes in expressions with any unique, multi-character subsequence of their names (an exact match gets preference). For example, you may use `a'len`, `a'gth`, or `a'lh` in place of `a'length`.
- Since Ada is case-insensitive, the debugger normally maps identifiers you type to lower case. The GNAT compiler uses upper-case characters for some of its internal identifiers, which are normally of no interest to users. For the rare occasions when you actually have to look at them, enclose them in angle brackets to avoid the lower-case mapping. For example,

```
(gdb) print <JMPBUF_SAVE>[0]
```

- Printing an object of class-wide type or dereferencing an access-to-class-wide value will display all the components of the object's specific type (as indicated by its run-time tag). Likewise, component selection on such a value will operate on the specific type of the object.

15.4.10.4 Overloading support for Ada

The debugger supports limited overloading. Given a subprogram call in which the function symbol has multiple definitions, it will use the number of actual parameters and some information about their types to attempt to narrow the set of definitions. It also makes very limited use of context, preferring procedures to functions in the context of the `call` command, and functions to procedures elsewhere.

If, after narrowing, the set of matching definitions still contains more than one definition, GDB will display a menu to query which one it should use, for instance:

```
(gdb) print f(1)
Multiple matches for f
[0] cancel
[1] foo.f (integer) return boolean at foo.adb:23
[2] foo.f (foo.new_integer) return boolean at foo.adb:28
>
```

In this case, just select one menu entry either to cancel expression evaluation (type `0` and press `RET`) or to continue evaluation with a specific instance (type the corresponding number and press `RET`).

Here are a couple of commands to customize GDB's behavior in this case:

set ada print-signatures

Control whether parameter types and return types are displayed in overloads selection menus. It is **on** by default. See Section 15.4.10.4 [Overloading support for Ada], page 255.

show ada print-signatures

Show the current setting for displaying parameter types and return types in overloads selection menu. See Section 15.4.10.4 [Overloading support for Ada], page 255.

15.4.10.5 Stopping at the Very Beginning

It is sometimes necessary to debug the program during elaboration, and before reaching the main procedure. As defined in the Ada Reference Manual, the elaboration code is invoked from a procedure called `adainit`. To run your program up to the beginning of elaboration, simply use the following two commands: `tbreak adainit` and `run`.

15.4.10.6 Ada Exceptions

A command is provided to list all Ada exceptions:

info exceptions

info exceptions *regexp*

The `info exceptions` command allows you to list all Ada exceptions defined within the program being debugged, as well as their addresses. With a regular expression, *regexp*, as argument, only those exceptions whose names match *regexp* are listed.

Below is a small example, showing how the command can be used, first without argument, and next with a regular expression passed as an argument.

```
(gdb) info exceptions
All defined Ada exceptions:
constraint_error: 0x613da0
program_error: 0x613d20
storage_error: 0x613ce0
tasking_error: 0x613ca0
const.aint_global_e: 0x613b00
(gdb) info exceptions const.aint
All Ada exceptions matching regular expression "const.aint":
constraint_error: 0x613da0
const.aint_global_e: 0x613b00
```

It is also possible to ask GDB to stop your program's execution when an exception is raised. For more details, see Section 5.1.3 [Set Catchpoints], page 71.

15.4.10.7 Extensions for Ada Tasks

Support for Ada tasks is analogous to that for threads (see Section 4.10 [Threads], page 48). GDB provides the following task-related commands:

info tasks

This command shows a list of current Ada tasks, as in the following example:

```
(gdb) info tasks
  ID      TID P-ID Pri State      Name
  1      8088000 0 15 Child Activation Wait main_task
  2      80a4000 1 15 Accept Statement    b
  3      809a800 1 15 Child Activation Wait a
* 4      80ae800 3 15 Runnable          c
```

In this listing, the asterisk before the last task indicates it to be the task currently being inspected.

ID Represents GDB's internal task number.

TID The Ada task ID.

P-ID The parent's task ID (GDB's internal task number).

Pri The base priority of the task.

State Current state of the task.

Unactivated

The task has been created but has not been activated. It cannot be executing.

Runnable The task is not blocked for any reason known to Ada. (It may be waiting for a mutex, though.) It is conceptually "executing" in normal mode.

Terminated

The task is terminated, in the sense of ARM 9.3 (5). Any dependents that were waiting on terminate alternatives have been awakened and have terminated themselves.

Child Activation Wait

The task is waiting for created tasks to complete activation.

Accept or Select Term

The task is waiting on an accept or selective wait statement.

Waiting on entry call

The task is waiting on an entry call.

Async Select Wait

The task is waiting to start the abortable part of an asynchronous select statement.

Delay Sleep

The task is waiting on a select statement with only a delay alternative open.

Child Termination Wait

The task is sleeping having completed a master within itself, and is waiting for the tasks dependent on that master to become terminated or waiting on a terminate Phase.

Wait Child in Term Alt

The task is sleeping waiting for tasks on terminate alternatives to finish terminating.

Asynchronous Hold

The task has been held by `Ada.Asynchronous_Task_Control.Hold_Task`.

Activating

The task has been created and is being made runnable.

Selective Wait

The task is waiting in a selective wait statement.

Suspension object sleep

The task is waiting in a call to `Suspend_Until_True` on a suspension object.

Accepting RV with *taskno*

The task is accepting a rendez-vous with the task *taskno*.

Waiting on RV with *taskno*

The task is waiting for a rendez-vous with the task *taskno*.

Name Name of the task in the program.

info task *taskno*

This command shows detailed information on the specified task, as in the following example:

```
(gdb) info tasks
  ID      TID P-ID Pri State      Name
  1    8077880   0  15 Child Activation Wait  main_task
*  2    807c468   1  15 Runnable      task_1
(gdb) info task 2
Ada Task: 0x807c468
Name: "task_1"
Thread: 0
LWP: 0x1fac
Parent: 1 ("main_task")
Base Priority: 15
State: Runnable
```

task This command prints the ID and name of the current task.

```
(gdb) info tasks
  ID      TID P-ID Pri State           Name
  1      8077870   0  15 Child Activation Wait  main_task
* 2      807c458   1  15 Runnable           some_task
(gdb) task
[Current task is 2 "some_task"]
```

task taskno

This command is like the `thread thread-id` command (see Section 4.10 [Threads], page 48). It switches the context of debugging from the current task to the given task.

```
(gdb) info tasks
  ID      TID P-ID Pri State           Name
  1      8077870   0  15 Child Activation Wait  main_task
* 2      807c458   1  15 Runnable           some_task
(gdb) task 1
[Switching to task 1 "main_task"]
#0 0x8067726 in pthread_cond_wait ()
(gdb) bt
#0 0x8067726 in pthread_cond_wait ()
#1 0x8056714 in system.os_interface.pthread_cond_wait ()
#2 0x805cb63 in system.task_primitives.operations.sleep ()
#3 0x806153e in system.tasking.stages.activate_tasks ()
#4 0x804aacc in un () at un.adb:5
```

task apply [task-id-list | all] [flag]... command

The `task apply` command is the Ada tasking analogue of `thread apply` (see Section 4.10 [Threads], page 48). It allows you to apply the named *command* to one or more tasks. Specify the tasks that you want affected using a list of task IDs, or specify `all` to apply to all tasks.

The *flag* arguments control what output to produce and how to handle errors raised when applying *command* to a task. *flag* must start with a - directly followed by one letter in `qcs`. If several flags are provided, they must be given individually, such as `-c -q`.

By default, GDB displays some task information before the output produced by *command*, and an error raised during the execution of a *command* will abort `task apply`. The following flags can be used to fine-tune this behavior:

- `-c` The flag `-c`, which stands for ‘continue’, causes any errors in *command* to be displayed, and the execution of `task apply` then continues.
- `-s` The flag `-s`, which stands for ‘silent’, causes any errors or empty output produced by a *command* to be silently ignored. That is, the execution continues, but the task information and errors are not printed.
- `-q` The flag `-q` (‘quiet’) disables printing the task information.

Flags `-c` and `-s` cannot be used together.

```
break locspec task taskno
break locspec task taskno if ...
```

These commands are like the `break ... thread ...` command (see Section 5.5 [Thread Stops], page 95). See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*.

Use the qualifier ‘`task taskno`’ with a breakpoint command to specify that you only want GDB to stop the program when a particular Ada task reaches this breakpoint. The *taskno* is one of the numeric task identifiers assigned by GDB, shown in the first column of the ‘`info tasks`’ display.

If you do not specify ‘`task taskno`’ when you set a breakpoint, the breakpoint applies to *all* tasks of your program.

You can use the `task` qualifier on conditional breakpoints as well; in this case, place ‘`task taskno`’ before the breakpoint condition (before the `if`).

For example,

```
(gdb) info tasks
  ID      TID P-ID Pri State      Name
  1 140022020 0 15 Child Activation Wait main_task
  2 140045060 1 15 Accept/Select Wait t2
  3 140044840 1 15 Runnable t1
* 4 140056040 1 15 Runnable t3
(gdb) b 15 task 2
Breakpoint 5 at 0x120044cb0: file test_task_debug.adb, line 15.
(gdb) cont
Continuing.
task # 1 running
task # 2 running

Breakpoint 5, test_task_debug () at test_task_debug.adb:15
15      flush;
(gdb) info tasks
  ID      TID P-ID Pri State      Name
  1 140022020 0 15 Child Activation Wait main_task
* 2 140045060 1 15 Runnable t2
  3 140044840 1 15 Runnable t1
  4 140056040 1 15 Delay Sleep t3
```

15.4.10.8 Tasking Support when Debugging Core Files

When inspecting a core file, as opposed to debugging a live program, tasking support may be limited or even unavailable, depending on the platform being used. For instance, on x86-linux, the list of tasks is available, but task switching is not supported.

On certain platforms, the debugger needs to perform some memory writes in order to provide Ada tasking support. When inspecting a core file, this means that the core file must be opened with read-write privileges, using the command ‘`set write on`’ (see Section 17.6 [Patching], page 287). Under these circumstances, you should make a backup copy of the core file before inspecting it with GDB.

15.4.10.9 Tasking Support when using the Ravenscar Profile

The *Ravenscar Profile* is a subset of the Ada tasking features, specifically designed for systems with safety-critical real-time requirements.

set ravenscar task-switching on

Allows task switching when debugging a program that uses the Ravenscar Profile. This is the default.

set ravenscar task-switching off

Turn off task switching when debugging a program that uses the Ravenscar Profile. This is mostly intended to disable the code that adds support for the Ravenscar Profile, in case a bug in either GDB or in the Ravenscar runtime is preventing GDB from working properly. To be effective, this command should be run before the program is started.

show ravenscar task-switching

Show whether it is possible to switch from task to task in a program using the Ravenscar Profile.

When Ravenscar task-switching is enabled, Ravenscar tasks are announced by GDB as if they were threads:

```
(gdb) continue
[New Ravenscar Thread 0x2b8f0]
```

Both Ravenscar tasks and the underlying CPU threads will show up in the output of **info threads**:

```
(gdb) info threads
  Id   Target Id               Frame
  1   Thread 1 (CPU#0 [running]) simple () at simple.adb:10
  2   Thread 2 (CPU#1 [running]) 0x0000000000003d34 in __gnat_initialize_cpu_devices ()
  3   Thread 3 (CPU#2 [running]) 0x0000000000003d28 in __gnat_initialize_cpu_devices ()
  4   Thread 4 (CPU#3 [halted ]) 0x000000000000c6ec in system.task_primitives.operations.idle ()
* 5   Ravenscar Thread 0x2b8f0    simple () at simple.adb:10
  6   Ravenscar Thread 0x2f150    0x000000000000c6ec in system.task_primitives.operations.idle ()
```

One known limitation of the Ravenscar support in GDB is that it isn't currently possible to single-step through the runtime initialization sequence. If you need to debug this code, you should use **set ravenscar task-switching off**.

15.4.10.10 Ada Source Character Set

The GNAT compiler supports a number of character sets for source files. See Section “Character Set Control” in *gnat_ugn*. GDB includes support for this as well.

set ada source-charset charset

Set the source character set for Ada. The character set must be supported by GNAT. Because this setting affects the decoding of symbols coming from the debug information in your program, the setting should be set as early as possible. The default is ISO-8859-1, because that is also GNAT's default.

show ada source-charset

Show the current source character set for Ada.

15.4.10.11 Known Peculiarities of Ada Mode

Besides the omissions listed previously (see Section 15.4.10.2 [Omissions from Ada], page 252), we know of several problems with and limitations of Ada mode in GDB, some of which will be fixed with planned future releases of the debugger and the GNU Ada compiler.

- Static constants that the compiler chooses not to materialize as objects in storage are invisible to the debugger.
- Named parameter associations in function argument lists are ignored (the argument lists are treated as positional).
- Many useful library packages are currently invisible to the debugger.
- Fixed-point arithmetic, conversions, input, and output is carried out using floating-point arithmetic, and may give results that only approximate those on the host machine.
- The GNAT compiler never generates the prefix **Standard** for any of the standard symbols defined by the Ada language. GDB knows about this: it will strip the prefix from names when you use it, and will never look for a name you have so qualified among local symbols, nor match against symbols in other packages or subprograms. If you have defined entities anywhere in your program other than parameters and local variables whose simple names match names in **Standard**, GNAT's lack of qualification here can cause confusion. When this happens, you can usually resolve the confusion by qualifying the problematic names with package **Standard** explicitly.

Older versions of the compiler sometimes generate erroneous debugging information, resulting in the debugger incorrectly printing the value of affected entities. In some cases, the debugger is able to work around an issue automatically. In other cases, the debugger is able to work around the issue, but the work-around has to be specifically enabled.

set ada trust-PAD-over-XVS on

Configure GDB to strictly follow the GNAT encoding when computing the value of Ada entities, particularly when **PAD** and **PAD__XVS** types are involved (see `ada/exp_debug.ads` in the GCC sources for a complete description of the encoding used by the GNAT compiler). This is the default.

set ada trust-PAD-over-XVS off

This is related to the encoding using by the GNAT compiler. If GDB sometimes prints the wrong value for certain entities, changing **ada trust-PAD-over-XVS** to **off** activates a work-around which may fix the issue. It is always safe to set **ada trust-PAD-over-XVS** to **off**, but this incurs a slight performance penalty, so it is recommended to leave this setting to **on** unless necessary.

Internally, the debugger also relies on the compiler following a number of conventions known as the ‘GNAT Encoding’, all documented in `gcc/ada/exp_debug.ads` in the GCC sources. This encoding describes how the debugging information should be generated for certain types. In particular, this convention makes use of *descriptive types*, which are artificial types generated purely to help the debugger.

These encodings were defined at a time when the debugging information format used was not powerful enough to describe some of the more complex types available in Ada. Since DWARF allows us to express nearly all Ada features, the long-term goal is to slowly replace

these descriptive types by their pure DWARF equivalent. To facilitate that transition, a new maintenance option is available to force the debugger to ignore those descriptive types. It allows the user to quickly evaluate how well GDB works without them.

maintenance ada set ignore-descriptive-types [on|off]

Control whether the debugger should ignore descriptive types. The default is not to ignore descriptives types (**off**).

maintenance ada show ignore-descriptive-types

Show if descriptive types are ignored by GDB.

15.5 Unsupported Languages

In addition to the other fully-supported programming languages, GDB also provides a pseudo-language, called **minimal**. It does not represent a real programming language, but provides a set of capabilities close to what the C or assembly languages provide. This should allow most simple operations to be performed while debugging an application that uses a language currently not supported by GDB.

If the language is set to **auto**, GDB will automatically select this language if the current frame corresponds to an unsupported language.

16 Examining the Symbol Table

The commands described in this chapter allow you to inquire about the symbols (names of variables, functions and types) defined in your program. This information is inherent in the text of your program and does not change as your program executes. GDB finds it in your program's symbol table, in the file indicated when you started GDB (see Section 2.1.1 [Choosing Files], page 12), or by one of the file-management commands (see Section 18.1 [Commands to Specify Files], page 293).

Occasionally, you may need to refer to symbols that contain unusual characters, which GDB ordinarily treats as word delimiters. The most frequent case is in referring to static variables in other source files (see Section 10.3 [Program Variables], page 149). File names are recorded in object files as debugging symbols, but GDB would ordinarily parse a typical file name, like `foo.c`, as the three words `'foo'` `'.'` `'c'`. To allow GDB to recognize `'foo.c'` as a single symbol, enclose it in single quotes; for example,

```
p 'foo.c':x
```

looks up the value of `x` in the scope of the file `foo.c`.

```
set case-sensitive on
set case-sensitive off
set case-sensitive auto
```

Normally, when GDB looks up symbols, it matches their names with case sensitivity determined by the current source language. Occasionally, you may wish to control that. The command `set case-sensitive` lets you do that by specifying `on` for case-sensitive matches or `off` for case-insensitive ones. If you specify `auto`, case sensitivity is reset to the default suitable for the source language. The default is case-sensitive matches for all languages except for Fortran, for which the default is case-insensitive matches.

```
show case-sensitive
```

This command shows the current setting of case sensitivity for symbols lookups.

```
set print type methods
set print type methods on
set print type methods off
```

Normally, when GDB prints a class, it displays any methods declared in that class. You can control this behavior either by passing the appropriate flag to `ptype`, or using `set print type methods`. Specifying `on` will cause GDB to display the methods; this is the default. Specifying `off` will cause GDB to omit the methods.

```
show print type methods
```

This command shows the current setting of method display when printing classes.

```
set print type nested-type-limit limit
set print type nested-type-limit unlimited
```

Set the limit of displayed nested types that the type printer will show. A *limit* of `unlimited` or `-1` will show all nested definitions. By default, the type printer will not show any nested types defined in classes.

show print type nested-type-limit

This command shows the current display limit of nested types when printing classes.

set print type typedefs

set print type typedefs on

set print type typedefs off

Normally, when GDB prints a class, it displays any typedefs defined in that class. You can control this behavior either by passing the appropriate flag to **p_{type}**, or using **set print type typedefs**. Specifying **on** will cause GDB to display the typedef definitions; this is the default. Specifying **off** will cause GDB to omit the typedef definitions. Note that this controls whether the typedef definition itself is printed, not whether typedef names are substituted when printing other types.

show print type typedefs

This command shows the current setting of typedef display when printing classes.

set print type hex

set print type hex on

set print type hex off

When GDB prints sizes and offsets of struct members, it can use either the decimal or hexadecimal notation. You can select one or the other either by passing the appropriate flag to **p_{type}**, or by using the **set print type hex** command.

show print type hex

This command shows whether the sizes and offsets of struct members are printed in decimal or hexadecimal notation.

info address *symbol*

Describe where the data for *symbol* is stored. For a register variable, this says which register it is kept in. For a non-register local variable, this prints the stack-frame offset at which the variable is always stored.

Note the contrast with '**print &*symbol***', which does not work at all for a register variable, and for a stack local variable prints the exact address of the current instantiation of the variable.

info symbol *addr*

Print the name of a symbol which is stored at the address *addr*. If no symbol is stored exactly at *addr*, GDB prints the nearest symbol and an offset from it:

```
(gdb) info symbol 0x54320
_initialize_vx + 396 in section .text
```

This is the opposite of the **info address** command. You can use it to find out the name of a variable or a function given its address.

For dynamically linked executables, the name of executable or shared library containing the symbol is also printed:

```
(gdb) info symbol 0x400225
_start + 5 in section .text of /tmp/a.out
```

```
(gdb) info symbol 0x2aaaac2811cf
__read_nocancel + 6 in section .text of /usr/lib64/libc.so.6
```

demangle [-l *language*] [--] *name*

Demangle *name*. If *language* is provided it is the name of the language to demangle *name* in. Otherwise *name* is demangled in the current language.

The ‘--’ option specifies the end of options, and is useful when *name* begins with a dash.

The parameter **demangle-style** specifies how to interpret the kind of mangling used. See Section 10.9 [Print Settings], page 159.

whatis [/flags] [*arg*]

Print the data type of *arg*, which can be either an expression or a name of a data type. With no argument, print the data type of \$, the last value in the value history.

If *arg* is an expression (see Section 10.1 [Expressions], page 147), it is not actually evaluated, and any side-effecting operations (such as assignments or function calls) inside it do not take place.

If *arg* is a variable or an expression, **whatis** prints its literal type as it is used in the source code. If the type was defined using a **typedef**, **whatis** will *not* print the data type underlying the **typedef**. If the type of the variable or the expression is a compound data type, such as **struct** or **class**, **whatis** never prints their fields or methods. It just prints the **struct/class** name (a.k.a. its *tag*). If you want to see the members of such a compound data type, use **ptype**.

If *arg* is a type name that was defined using **typedef**, **whatis** *unrolls* only one level of that **typedef**. Unrolling means that **whatis** will show the underlying type used in the **typedef** declaration of *arg*. However, if that underlying type is also a **typedef**, **whatis** will not unroll it.

For C code, the type names may also have the form ‘**class** *class-name*’, ‘**struct** *struct-tag*’, ‘**union** *union-tag*’ or ‘**enum** *enum-tag*’.

flags can be used to modify how the type is displayed. Available flags are:

- | | |
|----------|--|
| r | Display in “raw” form. Normally, GDB substitutes template parameters and typedefs defined in a class when printing the class’ members. The /r flag disables this. |
| m | Do not print methods defined in the class. |
| M | Print methods defined in the class. This is the default, but the flag exists in case you change the default with set print type methods . |
| t | Do not print typedefs defined in the class. Note that this controls whether the typedef definition itself is printed, not whether typedef names are substituted when printing other types. |
| T | Print typedefs defined in the class. This is the default, but the flag exists in case you change the default with set print type typedefs . |

- o Print the offsets and sizes of fields in a struct, similar to what the `pahole` tool does. This option implies the `/tm` flags.
- x Use hexadecimal notation when printing offsets and sizes of fields in a struct.
- d Use decimal notation when printing offsets and sizes of fields in a struct.

For example, given the following declarations:

```
struct tuv
{
    int a1;
    char *a2;
    int a3;
};

struct xyz
{
    int f1;
    char f2;
    void *f3;
    struct tuv f4;
};

union qwe
{
    struct tuv fff1;
    struct xyz fff2;
};

struct tyu
{
    int a1 : 1;
    int a2 : 3;
    int a3 : 23;
    char a4 : 2;
    int64_t a5;
    int a6 : 5;
    int64_t a7 : 3;
};
```

Issuing a `pptype /o struct tuv` command would print:

```
(gdb) pptype /o struct tuv
/* offset      |      size */  type = struct tuv {
/*    0        |      4 */    int a1;
/* XXX 4-byte hole */
/*    8        |      8 */    char *a2;
/*   16        |      4 */    int a3;
                                     /* total size (bytes):  24 */
}
```

Notice the format of the first column of comments. There, you can find two parts separated by the ‘|’ character: the *offset*, which indicates where the field is located inside the struct, in bytes, and the *size* of the field. Another interesting line is the marker of a *hole*

in the struct, indicating that it may be possible to pack the struct and make it use less space by reorganizing its fields.

It is also possible to print offsets inside an union:

```
(gdb) ptype /o union qwe
/* offset      |      size */  type = union qwe {
/*          0  |      4 */      struct tuv {
/*          0  |      4 */          int a1;
/* XXX 4-byte hole */
/*          8  |      8 */          char *a2;
/*          16 |      4 */          int a3;

/* total size (bytes): 24 */
} fff1;

/*          0  |      4 */      struct xyz {
/*          4  |      1 */          char f2;
/* XXX 3-byte hole */
/*          8  |      8 */          void *f3;
/*          16 |      24 */         struct tuv {
/*          16 |      4 */             int a1;
/* XXX 4-byte hole */
/*          24 |      8 */             char *a2;
/*          32 |      4 */             int a3;

/* total size (bytes): 24 */
} f4;

/* total size (bytes): 40 */
} fff2;

/* total size (bytes): 40 */
}
```

In this case, since `struct tuv` and `struct xyz` occupy the same space (because we are dealing with an union), the offset is not printed for them. However, you can still examine the offset of each of these structures' fields.

Another useful scenario is printing the offsets of a struct containing bitfields:

```
(gdb) ptype /o struct tyu
/* offset      |      size */  type = struct tyu {
/*      0:31   |      4 */      int a1 : 1;
/*      0:28   |      4 */      int a2 : 3;
/*      0: 5   |      4 */      int a3 : 23;
/*      3: 3   |      1 */      signed char a4 : 2;
/* XXX 3-bit hole */
/* XXX 4-byte hole */
/*          8  |      8 */      int64_t a5;
/*      16: 0   |      4 */      int a6 : 5;
/*      16: 5   |      8 */      int64_t a7 : 3;
/* XXX 7-byte padding */

/* total size (bytes): 24 */
}
```

Note how the offset information is now extended to also include the first bit of the bitfield.

`ptype[/flags] [arg]`

`ptype` accepts the same arguments as `whatism`, but prints a detailed description of the type, instead of just the name of the type. See Section 10.1 [Expressions], page 147.

Contrary to `whatism`, `ptype` always unrolls any `typedefs` in its argument declaration, whether the argument is a variable, expression, or a data type. This means that `ptype` of a variable or an expression will not print literally its type as present in the source code—use `whatism` for that. `typedefs` at the pointer or reference targets are also unrolled. Only `typedefs` of fields, methods and inner `class typedefs` of `structs`, `classes` and `unions` are not unrolled even with `ptype`.

For example, for this variable declaration:

```
typedef double real_t;
struct complex { real_t real; double imag; };
typedef struct complex complex_t;
complex_t var;
real_t *real_pointer_var;
```

the two commands give this output:

```
(gdb) whatism var
type = complex_t
(gdb) ptype var
type = struct complex {
    real_t real;
    double imag;
}
(gdb) whatism complex_t
type = struct complex
(gdb) whatism struct complex
type = struct complex
(gdb) ptype struct complex
type = struct complex {
    real_t real;
    double imag;
}
(gdb) whatism real_pointer_var
type = real_t *
(gdb) ptype real_pointer_var
type = double *
```

As with `whatism`, using `ptype` without an argument refers to the type of `$`, the last value in the value history.

Sometimes, programs use opaque data types or incomplete specifications of complex data structure. If the debug information included in the program does not allow GDB to display a full declaration of the data type, it will say ‘<incomplete type>’. For example, given these declarations:

```
struct foo;
struct foo *fooptr;
```

but no definition for `struct foo` itself, GDB will say:

```
(gdb) ptype foo
$1 = <incomplete type>
```

“Incomplete type” is C terminology for data types that are not completely specified.

Othertimes, information about a variable's type is completely absent from the debug information included in the program. This most often happens when the program or library where the variable is defined includes no debug information at all. GDB knows the variable exists from inspecting the linker/loader symbol table (e.g., the ELF dynamic symbol table), but such symbols do not contain type information. Inspecting the type of a (global) variable for which GDB has no type information shows:

```
(gdb) ptype var
type = <data variable, no debug info>
```

See Section 10.3 [Variables], page 149, for how to print the values of such variables.

`info types [-q] [regex]`

Print a brief description of all types whose names match the regular expression *regex* (or all types in your program, if you supply no argument). Each complete typename is matched as though it were a complete line; thus, '`i type value`' gives information on all types in your program whose names include the string *value*, but '`i type ^value$`' gives information only on types whose complete name is *value*.

In programs using different languages, GDB chooses the syntax to print the type description according to the '`set language`' value: using '`set language auto`' (see Section 15.1.3 [Set Language Automatically], page 230) means to use the language of the type, other values mean to use the manually specified language (see Section 15.1.2 [Set Language Manually], page 230).

This command differs from `ptype` in two ways: first, like `what is`, it does not print a detailed description; second, it lists all source files and line numbers where a type is defined.

The output from '`info types`' is preceded with a header line describing what types are being listed. The optional flag '`-q`', which stands for 'quiet', disables printing this header information.

`info type-printers`

Versions of GDB that ship with Python scripting enabled may have "type printers" available. When using `ptype` or `what is`, these printers are consulted when the name of a type is needed. See Section 23.3.2.9 [Type Printing API], page 435, for more information on writing type printers.

`info type-printers` displays all the available type printers.

`enable type-printer name...`

`disable type-printer name...`

These commands can be used to enable or disable type printers.

`info scope locspec`

List all the variables local to the lexical scope of the code location that results from resolving *locspec*. See Section 9.2 [Location Specifications], page 128, for details about supported forms of *locspec*. For example:

```
(gdb) info scope command_line_handler
Scope for command_line_handler:
Symbol rl is an argument at stack/frame offset 8, length 4.
```

```

Symbol linebuffer is in static storage at address 0x150a18, length 4.
Symbol linelength is in static storage at address 0x150a1c, length 4.
Symbol p is a local variable in register $esi, length 4.
Symbol p1 is a local variable in register $ebx, length 4.
Symbol nline is a local variable in register $edx, length 4.
Symbol repeat is a local variable at frame offset -8, length 4.

```

This command is especially useful for determining what data to collect during a *trace experiment*, see Section 13.1.6 [Tracepoint Actions], page 210.

info source

Show information about the current source file—that is, the source file for the function containing the current point of execution:

- the name of the source file, and the directory containing it,
- the directory it was compiled in,
- its length, in lines,
- which programming language it is written in,
- if the debug information provides it, the program that compiled the file (which may include, e.g., the compiler version and command line arguments),
- whether the executable includes debugging information for that file, and if so, what format the information is in (e.g., Dwarf 2), and
- whether the debugging information includes information about preprocessor macros.

info sources [-dirname | -basename] [--] [regexp]

With no options ‘**info sources**’ prints the names of all source files in your program for which there is debugging information. The source files are presented based on a list of object files (executables and libraries) currently loaded into GDB. For each object file all of the associated source files are listed.

Each source file will only be printed once for each object file, but a single source file can be repeated in the output if it is part of multiple object files.

If the optional *regexp* is provided, then only source files that match the regular expression will be printed. The matching is case-sensitive, except on operating systems that have case-insensitive filesystem (e.g., MS-Windows). ‘--’ can be used before *regexp* to prevent GDB interpreting *regexp* as a command option (e.g. if *regexp* starts with ‘-’).

By default, the *regexp* is used to match anywhere in the filename. If **-dirname**, only files having a *dirname* matching *regexp* are shown. If **-basename**, only files having a *basename* matching *regexp* are shown.

It is possible that an object file may be printed in the list with no associated source files. This can happen when either no source files match *regexp*, or, the object file was compiled without debug information and so GDB is unable to find any source file names.

info functions [-q] [-n]

Print the names and data types of all defined functions. Similarly to ‘**info types**’, this command groups its output by source files and annotates each function definition with its source line number.

In programs using different languages, GDB chooses the syntax to print the function name and type according to the ‘**set language**’ value: using ‘**set language auto**’ (see Section 15.1.3 [Set Language Automatically], page 230) means to use the language of the function, other values mean to use the manually specified language (see Section 15.1.2 [Set Language Manually], page 230).

The ‘**-n**’ flag excludes *non-debugging symbols* from the results. A non-debugging symbol is a symbol that comes from the executable’s symbol table, not from the debug information (for example, DWARF) associated with the executable.

The optional flag ‘**-q**’, which stands for ‘**quiet**’, disables printing header information and messages explaining why no functions have been printed.

info functions [-q] [-n] [-t *type_regex*] [*regex*]

Like ‘**info functions**’, but only print the names and data types of the functions selected with the provided *regex*(s).

If *regex* is provided, print only the functions whose names match the regular expression *regex*. Thus, ‘**info fun step**’ finds all functions whose names include **step**; ‘**info fun ^step**’ finds those whose names start with **step**. If a function name contains characters that conflict with the regular expression language (e.g. ‘**operator*()**’), they may be quoted with a backslash.

If *type_regex* is provided, print only the functions whose types, as printed by the **what is** command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes. Thus, ‘**info fun -t ^int (**’ finds the functions that return an integer; ‘**info fun -t ^(. *int. *)**’ finds the functions that have an argument type containing int; ‘**info fun -t ^int (^step**’ finds the functions whose names start with **step** and that return int.

If both *regex* and *type_regex* are provided, a function is printed only if its name matches *regex* and its type matches *type_regex*.

info variables [-q] [-n]

Print the names and data types of all variables that are defined outside of functions (i.e. excluding local variables). The printed variables are grouped by source files and annotated with their respective source line numbers.

In programs using different languages, GDB chooses the syntax to print the variable name and type according to the ‘**set language**’ value: using ‘**set language auto**’ (see Section 15.1.3 [Set Language Automatically], page 230) means to use the language of the variable, other values mean to use the manually specified language (see Section 15.1.2 [Set Language Manually], page 230).

The ‘**-n**’ flag excludes non-debugging symbols from the results.

The optional flag ‘**-q**’, which stands for ‘**quiet**’, disables printing header information and messages explaining why no variables have been printed.

info variables [-q] [-n] [-t *type_regex*] [*regex*]

Like **info variables**, but only print the variables selected with the provided *regex*(s).

If *regex* is provided, print only the variables whose names match the regular expression *regex*.

If *type_regex* is provided, print only the variables whose types, as printed by the **what**is command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes.

If both *regex* and *type_regex* are provided, an argument is printed only if its name matches *regex* and its type matches *type_regex*.

info modules [-q] [*regex*]

List all Fortran modules in the program, or all modules matching the optional regular expression *regex*.

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no modules have been printed.

info module functions [-q] [-m *module-regex*] [-t *type-regex*] [*regex*]

info module variables [-q] [-m *module-regex*] [-t *type-regex*] [*regex*]

List all functions or variables within all Fortran modules. The set of functions or variables listed can be limited by providing some or all of the optional regular expressions. If *module-regex* is provided, then only Fortran modules matching *module-regex* will be searched. Only functions or variables whose type matches the optional regular expression *type-regex* will be listed. And only functions or variables whose name matches the optional regular expression *regex* will be listed.

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no functions or variables have been printed.

info main Print the name of the starting function of the program. This serves primarily Fortran programs, which have a user-supplied name for the main subroutine.

info classes

info classes *regex*

Display all Objective-C classes in your program, or (with the *regex* argument) all those matching a particular regular expression.

info selectors

info selectors *regex*

Display all Objective-C selectors in your program, or (with the *regex* argument) all those matching a particular regular expression.

set opaque-type-resolution on

Tell GDB to resolve opaque types. An opaque type is a type declared as a pointer to a **struct**, **class**, or **union**—for example, **struct MyType ***—that is used in one source file although the full declaration of **struct MyType** is in another source file. The default is on.

A change in the setting of this subcommand may not take effect until the next time symbols for a file are loaded.

```
set opaque-type-resolution off
```

Tell GDB not to resolve opaque types. In this case, the type is printed as follows:

```
{<no data fields>}
```

```
show opaque-type-resolution
```

Show whether opaque types are resolved or not.

```
set print symbol-loading
```

```
set print symbol-loading full
```

```
set print symbol-loading brief
```

```
set print symbol-loading off
```

The `set print symbol-loading` command allows you to control the printing of messages when GDB loads symbol information. By default a message is printed for the executable and one for each shared library, and normally this is what you want. However, when debugging apps with large numbers of shared libraries these messages can be annoying. When set to `brief` a message is printed for each executable, and when GDB loads a collection of shared libraries at once it will only print one message regardless of the number of shared libraries. When set to `off` no messages are printed.

```
show print symbol-loading
```

Show whether messages will be printed when a GDB command entered from the keyboard causes symbol information to be loaded.

```
maint print symbols [-pc address] [filename]
```

```
maint print symbols [-objfile objfile] [-source source] [--] [filename]
```

```
maint print msymbols [-objfile objfile] [--] [filename]
```

Write a dump of debugging symbol data into the file *filename* or the terminal if *filename* is unspecified. If `-objfile objfile` is specified, only dump symbols for that objfile. If `-pc address` is specified, only dump symbols for the file with code at that address. Note that *address* may be a symbol like `main`. If `-source source` is specified, only dump symbols for that source file.

These commands are used to debug the GDB symbol-reading code. These commands do not modify internal GDB state, therefore ‘`maint print symbols`’ will only print symbols for already expanded symbol tables. You can use the command `info sources` to find out which files these are. ‘`maint print msymbols`’ dumps “minimal symbols”, e.g., “ELF symbols”.

See Section 18.1 [Commands to Specify Files], page 293, for a discussion of how GDB reads symbols (in the description of `symbol-file`).

```
maint info symtabs [ regexp ]
```

List the `struct symtab` structures whose names match *regexp*. If *regexp* is not given, list them all. The output includes expressions which you can copy into a GDB debugging this one to examine a particular structure in more detail. For example:

```
(gdb) maint info symtabs
{ objfile /home/gnu/build/binutils-gdb/gdb/a.out ((struct objfile *) 0x7d48d1c92b80)
  { ((struct compunit_symtab *) 0x7cd8d1be44b0)
    debugformat DWARF 5
    producer GNU C23 15.2.1 20260209 -mtune=generic -march=x86-64 -g3 -O0
```

```

name test.c
dirname /home/gnu/build/binutils-gdb/gdb
blockvector ((struct blockvector *) 0x7c48d1c1efc0)
user ((struct compunit_symtab *) (null))
{
  symtab test.c ((struct symtab *) 0x7df8d1d588a0)
  fullname /home/gnu/build/binutils-gdb/gdb/test.c
  linetable ((struct linetable *) 0x7df8d1d58cc0)
}
{
  symtab /usr/include/stdc-predef.h ((struct symtab *) 0x7df8d1d588e0)
  fullname (void)
  linetable ((struct linetable *) 0x0)
}
}
}

```

maint info line-table [regexp]

List the **struct linetable** from all **struct symtab** instances whose name matches *regexp*. If *regexp* is not given, list the **struct linetable** from all **struct symtab**. For example:

```

(gdb) maint info line-table
objfile: /home/gnu/build/a.out ((struct objfile *) 0x6120000e0d40)
compunit_symtab: simple.cpp ((struct compunit_symtab *) 0x6210000ff450)
symtab: /home/gnu/src/simple.cpp ((struct symtab *) 0x6210000ff4d0)
linetable: ((struct linetable *) 0x62100012b760):

```

INDEX	LINE	ADDRESS	IS-STMT	PROLOGUE-END	EPILOGUE-BEGIN
0	3	0x0000000000401110	Y		
1	4	0x0000000000401114	Y	Y	Y
2	9	0x0000000000401120	Y		
3	10	0x0000000000401124	Y	Y	
4	10	0x0000000000401129	Y		Y
5	15	0x0000000000401130	Y		
6	16	0x0000000000401134	Y	Y	
7	16	0x0000000000401139			
8	21	0x0000000000401140	Y		Y
9	22	0x000000000040114f	Y	Y	
10	22	0x0000000000401154			Y
11	END	0x000000000040115a	Y		

The ‘IS-STMT’ column indicates if the address is a recommended breakpoint location to represent a line or a statement. The ‘PROLOGUE-END’ column indicates that a given address is an adequate place to set a breakpoint at the first instruction following a function prologue. The ‘EPILOGUE-BEGIN’ column indicates that a given address marks the point where a block’s frame is destroyed, making local variables hard or impossible to find.

set always-read-ctf [on|off]

show always-read-ctf

When off, CTF debug info is only read if DWARF debug info is not present. When on, CTF debug info is read regardless of whether DWARF debug info is present. The default value is off.

maint set symbol-cache-size size

Set the size of the symbol cache to *size*. The default size is intended to be good enough for debugging most applications. This option exists to allow for experimenting with different sizes.

maint show symbol-cache-size
Show the size of the symbol cache.

maint print symbol-cache
Print the contents of the symbol cache. This is useful when debugging symbol cache issues.

maint print symbol-cache-statistics
Print symbol cache usage statistics. This helps determine how well the cache is being utilized.

maint flush symbol-cache
maint flush-symbol-cache
Flush the contents of the symbol cache, all entries are removed. This command is useful when debugging the symbol cache. It is also useful when collecting performance data. The command **maint flush-symbol-cache** is deprecated in favor of **maint flush symbol-cache**.

maint set ignore-prologue-end-flag [on|off]
Enable or disable the use of the 'PROLOGUE-END' flag from the line-table. When 'off' (the default), GDB uses the 'PROLOGUE-END' flag to place breakpoints past the end of a function prologue. When 'on', GDB ignores the flag and relies on prologue analyzers to skip function prologues.

maint show ignore-prologue-end-flag
Show whether GDB will ignore the 'PROLOGUE-END' flag.

17 Altering Execution

Once you think you have found an error in your program, you might want to find out for certain whether correcting the apparent error would lead to correct results in the rest of the run. You can find the answer by experiment, using the GDB features for altering execution of the program.

For example, you can store new values into variables or memory locations, give your program a signal, restart it at a different address, or even return prematurely from a function.

17.1 Assignment to Variables

To alter the value of a variable, evaluate an assignment expression. See Section 10.1 [Expressions], page 147. For example,

```
print x=4
```

stores the value 4 into the variable `x`, and then prints the value of the assignment expression (which is 4). See Chapter 15 [Using GDB with Different Languages], page 229, for more information on operators in supported languages.

If you are not interested in seeing the value of the assignment, use the `set` command instead of the `print` command. `set` is really the same as `print` except that the expression's value is not printed and is not put in the value history (see Section 10.11 [Value History], page 172). The expression is evaluated only for its effects.

If the beginning of the argument string of the `set` command appears identical to a `set` subcommand, use the `set variable` command instead of just `set`. This command is identical to `set` except for its lack of subcommands. For example, if your program has a variable `width`, you get an error if you try to set a new value with just '`set width=13`', because GDB has the command `set width`:

```
(gdb) whatis width
type = double
(gdb) p width
$4 = 13
(gdb) set width=47
Invalid syntax in expression.
```

The invalid expression, of course, is '`=47`'. In order to actually set the program's variable `width`, use

```
(gdb) set var width=47
```

Because the `set` command has many subcommands that can conflict with the names of program variables, it is a good idea to use the `set variable` command instead of just `set`. For example, if your program has a variable `g`, you run into problems if you try to set a new value with just '`set g=4`', because GDB has the command `set gnutarget`, abbreviated `set g`:

```

(gdb) whatis g
type = double
(gdb) p g
$1 = 1
(gdb) set g=4
(gdb) p g
$2 = 1
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/smith/cc_progs/a.out
"/home/smith/cc_progs/a.out": can't open to read symbols:
                                Invalid bfd target.

(gdb) show g
The current BFD target is "=4".

```

The program variable `g` did not change, and you silently set the `gnumtarget` to an invalid value. In order to set the variable `g`, use

```
(gdb) set var g=4
```

GDB allows more implicit conversions in assignments than C; you can freely store an integer value into a pointer variable or vice versa, and you can convert any structure to any other structure that is the same length or shorter.

To store values into arbitrary places in memory, use the `{...}` construct to generate a value of specified type at a specified address (see Section 10.1 [Expressions], page 147). For example, `{int}0x83040` refers to memory location `0x83040` as an integer (which implies a certain size and representation in memory), and

```
set {int}0x83040 = 4
```

stores the value 4 into that memory location.

17.2 Continuing at a Different Address

Ordinarily, when you continue your program, you do so at the place where it stopped, with the `continue` command. You can instead continue at an address of your own choosing, with the following commands:

`jump locspec`

`j locspec` Resume execution at the address of the code location that results from resolving *locspec*. See Section 9.2 [Location Specifications], page 128, for a description of the different forms of *locspec*. If *locspec* resolves to more than one address, those outside the current compilation unit are ignored. If considering just the addresses in the current compilation unit still doesn't yield a unique address, the command aborts before jumping. Execution stops again immediately if there is a breakpoint there. It is common practice to use the `tbreak` command in conjunction with `jump`. See Section 5.1.1 [Setting Breakpoints], page 60.

The `jump` command does not change the current stack frame, or the stack pointer, or the contents of any memory location or any register other than the program counter. If *locspec* resolves to an address in a different function from the one currently executing, the results may be bizarre if the two functions expect different patterns of arguments or of local variables. For this reason, the `jump` command requests confirmation if the jump address is not in the function

currently executing. However, even bizarre results are predictable if you are well acquainted with the machine-language code of your program.

On many systems, you can get much the same effect as the `jump` command by storing a new value into the register `$pc`. The difference is that this does not start your program running; it only changes the address of where it *will* run when you continue. For example,

```
set $pc = 0x485
```

makes the next `continue` command or stepping command execute at address `0x485`, rather than at the address where your program stopped. See Section 5.2 [Continuing and Stepping], page 85.

However, writing directly to `$pc` will only change the value of the program-counter register, while using `jump` will ensure that any additional auxiliary state is also updated. For example, on SPARC, `jump` will update both `$pc` and `$npc` registers prior to resuming execution. When using the approach of writing directly to `$pc` it is your job to also update the `$npc` register.

The most common occasion to use the `jump` command is to back up—perhaps with more breakpoints set—over a portion of a program that has already executed, in order to examine its execution in more detail.

17.3 Giving your Program a Signal

`signal signal`

Resume execution where your program is stopped, but immediately give it the signal *signal*. The *signal* can be the name or the number of a signal. For example, on many systems `signal 2` and `signal SIGINT` are both ways of sending an interrupt signal.

Alternatively, if *signal* is zero, continue execution without giving a signal. This is useful when your program stopped on account of a signal and would ordinarily see the signal when resumed with the `continue` command; ‘`signal 0`’ causes it to resume without a signal.

Note: When resuming a multi-threaded program, *signal* is delivered to the currently selected thread, not the thread that last reported a stop. This includes the situation where a thread was stopped due to a signal. So if you want to continue execution suppressing the signal that stopped a thread, you should select that same thread before issuing the ‘`signal 0`’ command. If you issue the ‘`signal 0`’ command with another thread as the selected one, GDB detects that and asks for confirmation.

Invoking the `signal` command is not the same as invoking the `kill` utility from the shell. Sending a signal with `kill` causes GDB to decide what to do with the signal depending on the signal handling tables (see Section 5.4 [Signals], page 92). The `signal` command passes the signal directly to your program.

`signal` does not repeat when you press `RET` a second time after executing the command.

`queue-signal signal`

Queue *signal* to be delivered immediately to the current thread when execution of the thread resumes. The *signal* can be the name or the number of a signal.

For example, on many systems **signal 2** and **signal SIGINT** are both ways of sending an interrupt signal. The handling of the signal must be set to pass the signal to the program, otherwise GDB will report an error. You can control the handling of signals from GDB with the **handle** command (see Section 5.4 [Signals], page 92).

Alternatively, if *signal* is zero, any currently queued signal for the current thread is discarded and when execution resumes no signal will be delivered. This is useful when your program stopped on account of a signal and would ordinarily see the signal when resumed with the **continue** command.

This command differs from the **signal** command in that the signal is just queued, execution is not resumed. And **queue-signal** cannot be used to pass a signal whose handling state has been set to **nopass** (see Section 5.4 [Signals], page 92).

See [stepping into signal handlers], page 94, for information on how stepping commands behave when the thread has a signal queued.

17.4 Returning from a Function

return

return expression

You can cancel execution of a function call with the **return** command. If you give an *expression* argument, its value is used as the function's return value.

When you use **return**, GDB discards the selected stack frame (and all frames within it). You can think of this as making the discarded frame return prematurely. If you wish to specify a value to be returned, give that value as the argument to **return**.

This pops the selected stack frame (see Section 8.3 [Selecting a Frame], page 119), and any other frames inside of it, leaving its caller as the innermost remaining frame. That frame becomes selected. The specified value is stored in the registers used for returning values of functions.

The **return** command does not resume execution; it leaves the program stopped in the state that would exist if the function had just returned. In contrast, the **finish** command (see Section 5.2 [Continuing and Stepping], page 85) resumes execution until the selected stack frame returns naturally.

GDB needs to know how the *expression* argument should be set for the inferior. The concrete registers assignment depends on the OS ABI and the type being returned by the selected stack frame. For example it is common for OS ABI to return floating point values in FPU registers while integer values in CPU registers. Still some ABIs return even floating point values in CPU registers. Larger integer widths (such as **long long int**) also have specific placement rules. GDB already knows the OS ABI from its current target so it needs to find out also the type being returned to make the assignment into the right register(s).

Normally, the selected stack frame has debug info. GDB will always use the debug info instead of the implicit type of *expression* when the debug info is available. For example, if you type **return -1**, and the function in the current stack frame is declared to return a **long long int**, GDB transparently converts the implicit **int** value of -1 into a **long long int**:

```

Breakpoint 1, func () at gdb.base/return-nodebug.c:29
29      return 31;
(gdb) return -1
Make func return now? (y or n) y
#0  0x004004f6 in main () at gdb.base/return-nodebug.c:43
43      printf ("result=%lld\n", func ());
(gdb)

```

However, if the selected stack frame does not have a debug info, e.g., if the function was compiled without debug info, GDB has to find out the type to return from user. Specifying a different type by mistake may set the value in different inferior registers than the caller code expects. For example, typing `return -1` with its implicit type `int` would set only a part of a `long long int` result for a debug info less function (on 32-bit architectures). Therefore the user is required to specify the return type by an appropriate cast explicitly:

```

Breakpoint 2, 0x0040050b in func ()
(gdb) return -1
Return value type not available for selected stack frame.
Please use an explicit cast of the value to return.
(gdb) return (long long int) -1
Make selected stack frame return now? (y or n) y
#0  0x00400526 in main ()
(gdb)

```

17.5 Calling Program Functions

`print expr`

Evaluate the expression `expr` and display the resulting value. The expression may include calls to functions in the program being debugged.

`call expr`

Evaluate the expression `expr` without displaying `void` returned values.

You can use this variant of the `print` command if you want to execute a function from your program that does not return anything (a.k.a. a *void function*), but without cluttering the output with `void` returned values that GDB will otherwise print. If the result is not `void`, it is printed and saved in the value history.

It is possible for the function you call via the `print` or `call` command to generate a signal (e.g., if there's a bug in the function, or if you passed it incorrect arguments). What happens in that case is controlled by the `set unwind-on-signal` command.

Similarly, with a C++ program it is possible for the function you call via the `print` or `call` command to generate an exception that is not handled due to the constraints of the dummy frame. In this case, any exception that is raised in the frame, but has an out-of-frame exception handler will not be found. GDB builds a dummy-frame for the inferior function call, and the unwinder cannot seek for exception handlers outside of this dummy-frame. What happens in that case is controlled by the `set unwind-on-terminating-exception` command.

`set unwind-on-signal`

Set unwinding of the stack if a signal is received while in a function that GDB called in the program being debugged. If set to on, GDB unwinds the stack it created for the call and restores the context to what it was before the call. If set to off (the default), GDB stops in the frame where the signal was received.

The command `set unwindonsignal` is an alias for this command, and is maintained for backward compatibility.

show unwind-on-signal

Show the current setting of stack unwinding in the functions called by GDB.

The command `show unwindonsignal` is an alias for this command, and is maintained for backward compatibility.

set unwind-on-terminating-exception

Set unwinding of the stack if a C++ exception is raised, but left unhandled while in a function that GDB called in the program being debugged. If set to `on` (the default), GDB unwinds the stack it created for the call and restores the context to what it was before the call. If set to `off`, GDB the exception is delivered to the default C++ exception handler and the inferior terminated.

show unwind-on-terminating-exception

Show the current setting of stack unwinding in the functions called by GDB.

set unwind-on-timeout

Set unwinding of the stack if a function called from GDB times out. If set to `off` (the default), GDB stops in the frame where the timeout occurred. If set to `on`, GDB unwinds the stack it created for the call and restores the context to what it was before the call.

show unwind-on-timeout

Show whether GDB will unwind the stack if a function called from GDB times out.

set may-call-functions

Set permission to call functions in the program. This controls whether GDB will attempt to call functions in the program, such as with expressions in the `print` command. It defaults to `on`.

To call a function in the program, GDB has to temporarily modify the state of the inferior. This has potentially undesired side effects. Also, having GDB call nested functions is likely to be erroneous and may even crash the program being debugged. You can avoid such hazards by forbidding GDB from calling functions in the program being debugged. If calling functions in the program is forbidden, GDB will throw an error when a command (such as printing an expression) starts a function call in the program.

show may-call-functions

Show permission to call functions in the program.

When calling a function within a program, it is possible that the program could enter a state from which the called function may never return. If this happens then it is possible to interrupt the function call by typing the interrupt character (often `Ctrl-c`).

If a called function is interrupted for any reason, including hitting a breakpoint, or triggering a watchpoint, and the stack is not unwound due to `set unwind-on-terminating-exception on`, `set unwind-on-timeout on`, or `set unwind-on-signal on` (see [stack unwind settings], page 283), then the dummy-frame, created by GDB to facilitate the call to

the program function, will be visible in the backtrace, for example frame #3 in the following backtrace:

```
(gdb) backtrace
#0 0x00007ffff7b3d1e7 in nanosleep () from /lib64/libc.so.6
#1 0x00007ffff7b3d11e in sleep () from /lib64/libc.so.6
#2 0x000000000040113f in deadlock () at test.cc:13
#3 <function called from gdb>
#4 breakpoint () at test.cc:20
#5 0x0000000000401151 in main () at test.cc:25
```

At this point it is possible to examine the state of the inferior just like any other stop.

Depending on why the function was interrupted then it may be possible to resume the inferior (using commands like `continue`, `step`, etc). In this case, when the inferior finally returns to the dummy-frame, GDB will once again halt the inferior.

On targets that support asynchronous execution (see Section 5.5.3 [Background Execution], page 98) GDB can place a timeout on any functions called from GDB. If the timeout expires and the function call is still ongoing, then GDB will interrupt the program.

If a function called from GDB is interrupted by a timeout, then by default the inferior is left in the frame where the timeout occurred, this behavior can be adjusted with ‘`set unwind-on-timeout`’ (see [set unwind-on-timeout], page 284).

For targets that don’t support asynchronous execution (see Section 5.5.3 [Background Execution], page 98) then timeouts for functions called from GDB are not supported, the timeout settings described below will be treated as `unlimited`, meaning GDB will wait indefinitely for function call to complete, unless interrupted by the user using `Ctrl-C`.

`set direct-call-timeout seconds`

Set the timeout used when calling functions in the program to *seconds*, which should be an integer greater than zero, or the special value `unlimited`, which indicates no timeout should be used. The default for this setting is `unlimited`.

This setting is used when the user calls a function directly from the command prompt, for example with a `call` or `print` command.

This setting only works for targets that support asynchronous execution (see Section 5.5.3 [Background Execution], page 98), for any other target the setting is treated as `unlimited`.

`show direct-call-timeout`

Show the timeout used when calling functions in the program with a `call` or `print` command.

It is also possible to call functions within the program from the condition of a conditional breakpoint (see Section 5.1.6 [Break Conditions], page 78). A different setting controls the timeout used for function calls made from a breakpoint condition.

`set indirect-call-timeout seconds`

Set the timeout used when calling functions in the program from a breakpoint or watchpoint condition to *seconds*, which should be an integer greater than zero, or the special value `unlimited`, which indicates no timeout should be used. The default for this setting is 30 seconds.

This setting only works for targets that support asynchronous execution (see Section 5.5.3 [Background Execution], page 98), for any other target the setting is treated as `unlimited`.

If a function called from a breakpoint or watchpoint condition times out, then GDB will stop at the point where the timeout occurred. The breakpoint condition evaluation will be abandoned.

`show indirect-call-timeout`

Show the timeout used when calling functions in the program from a breakpoint or watchpoint condition.

17.5.1 Calling functions with no debug info

Sometimes, a function you wish to call is missing debug information. In such case, GDB does not know the type of the function, including the types of the function's parameters. To avoid calling the inferior function incorrectly, which could result in the called function functioning erroneously and even crash, GDB refuses to call the function unless you tell it the type of the function.

For prototyped (i.e. ANSI/ISO style) functions, there are two ways to do that. The simplest is to cast the call to the function's declared return type. For example:

```
(gdb) p getenv ("PATH")
'getenv' has unknown return type; cast the call to its declared return type
(gdb) p (char *) getenv ("PATH")
$1 = 0x7fffffe7ba "/usr/local/bin:/..."
```

Casting the return type of a no-debug function is equivalent to casting the function to a pointer to a prototyped function that has a prototype that matches the types of the passed-in arguments, and calling that. I.e., the call above is equivalent to:

```
(gdb) p ((char * (*) (const char *)) getenv) ("PATH")
```

and given this prototyped C or C++ function with float parameters:

```
float multiply (float v1, float v2) { return v1 * v2; }
```

these calls are equivalent:

```
(gdb) p (float) multiply (2.0f, 3.0f)
(gdb) p ((float *) (float, float)) multiply (2.0f, 3.0f)
```

If the function you wish to call is declared as unprototyped (i.e. old K&R style), you must use the cast-to-function-pointer syntax, so that GDB knows that it needs to apply default argument promotions (promote float arguments to double). See Section 22.7 [ABI], page 380. For example, given this unprototyped C function with float parameters, and no debug info:

```
float
multiply_noproto (v1, v2)
    float v1, v2;
{
    return v1 * v2;
}
```

you call it like this:

```
(gdb) p ((float (*) ()) multiply_noproto) (2.0f, 3.0f)
```

17.6 Patching Programs

By default, GDB opens the file containing your program's executable code (or the corefile) read-only. This prevents accidental alterations to machine code; but it also prevents you from intentionally patching your program's binary.

If you'd like to be able to patch the binary, you can specify that explicitly with the **set write** command. For example, you might want to turn on internal debugging flags, or even to make emergency repairs.

```
set write on
set write off
```

If you specify '**set write on**', GDB opens executable and core files for both reading and writing; if you specify **set write off** (the default), GDB opens them read-only.

If you have already loaded a file, you must load it again (using the **exec-file** or **core-file** command) after changing **set write**, for your new setting to take effect.

```
show write
```

Display whether executable files and core files are opened for writing as well as reading.

17.7 Compiling and injecting code in GDB

GDB supports on-demand compilation and code injection into programs running under GDB. GCC 5.0 or higher built with **libgcc1.so** must be installed for this functionality to be enabled. This functionality is implemented with the following commands.

```
compile code source-code
compile code -raw -- source-code
```

Compile *source-code* with the compiler language found as the current language in GDB (see Chapter 15 [Languages], page 229). If compilation and injection is not supported with the current language specified in GDB, or the compiler does not support this feature, an error message will be printed. If *source-code* compiles and links successfully, GDB will load the object-code emitted, and execute it within the context of the currently selected inferior. It is important to note that the compiled code is executed immediately. After execution, the compiled code is removed from GDB and any new types or variables you have defined will be deleted.

The command allows you to specify *source-code* in two ways. The simplest method is to provide a single line of code to the command. E.g.:

```
compile code printf ("hello world\n");
```

If you specify options on the command line as well as source code, they may conflict. The '--' delimiter can be used to separate options from actual source code. E.g.:

```
compile code -r -- printf ("hello world\n");
```

Alternatively you can enter source code as multiple lines of text. To enter this mode, invoke the **compile code** command without any text following the

command. This will start the multiple-line editor and allow you to type as many lines of source code as required. When you have completed typing, enter ‘end’ on its own line to exit the editor.

```
compile code
>printf ("hello\n");
>printf ("world\n");
>end
```

Specifying ‘-raw’, prohibits GDB from wrapping the provided *source-code* in a callable scope. In this case, you must specify the entry point of the code by defining a function named `_gdb_expr_`. The ‘-raw’ code cannot access variables of the inferior. Using ‘-raw’ option may be needed for example when *source-code* requires ‘#include’ lines which may conflict with inferior symbols otherwise.

`compile file filename`

`compile file -raw filename`

Like `compile code`, but take the source code from *filename*.

```
compile file /home/user/example.c
```

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

`compile print [[options] --] expr`

`compile print [[options] --] /f expr`

Compile and execute *expr* with the compiler language found as the current language in GDB (see Chapter 15 [Languages], page 229). By default the value of *expr* is printed in a format appropriate to its data type; you can choose a different format by specifying ‘/f’, where *f* is a letter specifying the format; see Section 10.5 [Output Formats], page 152. The `compile print` command accepts the same options as the `print` command; see [print options], page 143.

`compile print [[options] --]`

`compile print [[options] --] /f`

Alternatively you can enter the expression (source code producing it) as multiple lines of text. To enter this mode, invoke the ‘`compile print`’ command without any text following the command. This will start the multiple-line editor.

The process of compiling and injecting the code can be inspected using:

`set debug compile`

Turns on or off display of GDB process of compiling and injecting the code. The default is off.

`show debug compile`

Displays the current state of displaying GDB process of compiling and injecting the code.

`set debug compile-cplusplus-types`

Turns on or off the display of C++ type conversion debugging information. The default is off.

show debug compile-cplus-types

Displays the current state of displaying debugging information for C++ type conversion.

17.7.1 Compilation options for the compile command

GDB needs to specify the right compilation options for the code to be injected, in part to make its ABI compatible with the inferior and in part to make the injected code compatible with GDB's injecting process.

The options used, in increasing precedence:

target architecture and OS options (**gdbarch**)

These options depend on target processor type and target operating system, usually they specify at least 32-bit (**-m32**) or 64-bit (**-m64**) compilation option.

compilation options recorded in the target

GCC (since version 4.7) stores the options used for compilation into **DW_AT_producer** part of DWARF debugging information according to the GCC option **-grecord-gcc-switches**. One has to explicitly specify **-g** during inferior compilation otherwise GCC produces no DWARF. This feature is only relevant for platforms where **-g** produces DWARF by default, otherwise one may try to enforce DWARF by using **-gdwarf-4**.

compilation options set by **set compile-args**

You can override compilation options using the following command:

set compile-args

Set compilation options used for compiling and injecting code with the **compile** commands. These options override any conflicting ones from the target architecture and/or options stored during inferior compilation.

show compile-args

Displays the current state of compilation options override. This does not show all the options actually used during compilation, use [set debug compile], page 288, for that.

17.7.2 Caveats when using the compile command

There are a few caveats to keep in mind when using the **compile** command. As the caveats are different per language, the table below highlights specific issues on a per language basis.

C code examples and caveats

When the language in GDB is set to 'C', the compiler will attempt to compile the source code with a 'C' compiler. The source code provided to the **compile** command will have much the same access to variables and types as it normally would if it were part of the program currently being debugged in GDB.

Below is a sample program that forms the basis of the examples that follow. This program has been compiled and loaded into GDB, much like any other normal debugging session.

```
void function1 (void)
{
    int i = 42;
```

```

    printf ("function 1\n");
}

void function2 (void)
{
    int j = 12;
    function1 ();
}

int main(void)
{
    int k = 6;
    int *p;
    function2 ();
    return 0;
}

```

For the purposes of the examples in this section, the program above has been compiled, loaded into GDB, stopped at the function `main`, and GDB is awaiting input from the user.

To access variables and types for any program in GDB, the program must be compiled and packaged with debug information. The `compile` command is not an exception to this rule. Without debug information, you can still use the `compile` command, but you will be very limited in what variables and types you can access.

So with that in mind, the example above has been compiled with debug information enabled. The `compile` command will have access to all variables and types (except those that may have been optimized out). Currently, as GDB has stopped the program in the `main` function, the `compile` command would have access to the variable `k`. You could invoke the `compile` command and type some source code to set the value of `k`. You can also read it, or do anything with that variable you would normally do in C. Be aware that changes to inferior variables in the `compile` command are persistent. In the following example:

```
compile code k = 3;
```

the variable `k` is now 3. It will retain that value until something else in the example program changes it, or another `compile` command changes it.

Normal scope and access rules apply to source code compiled and injected by the `compile` command. In the example, the variables `j` and `k` are not accessible yet, because the program is currently stopped in the `main` function, where these variables are not in scope. Therefore, the following command

```
compile code j = 3;
```

will result in a compilation error message.

Once the program is continued, execution will bring these variables in scope, and they will become accessible; then the code you specify via the `compile` command will be able to access them.

You can create variables and types with the `compile` command as part of your source code. Variables and types that are created as part of the `compile` command are not visible to the rest of the program for the duration of its run. This example is valid:

```
compile code int ff = 5; printf ("ff is %d\n", ff);
```

However, if you were to type the following into GDB after that command has completed:

```
compile code printf ("ff is %d\n", ff);
```

a compiler error would be raised as the variable `ff` no longer exists. Object code generated and injected by the `compile` command is removed when its execution ends. Caution is advised when assigning to program variables values of variables created by the code submitted to the `compile` command. This example is valid:

```
compile code int ff = 5; k = ff;
```

The value of the variable `ff` is assigned to `k`. The variable `k` does not require the existence of `ff` to maintain the value it has been assigned. However, pointers require particular care in assignment. If the source code compiled with the `compile` command changed the address of a pointer in the example program, perhaps to a variable created in the `compile` command, that pointer would point to an invalid location when the command exits. The following example would likely cause issues with your debugged program:

```
compile code int ff = 5; p = &ff;
```

In this example, `p` would point to `ff` when the `compile` command is executing the source code provided to it. However, as variables in the (example) program persist with their assigned values, the variable `p` would point to an invalid location when the command exits. A general rule should be followed in that you should either assign `NULL` to any assigned pointers, or restore a valid location to the pointer before the command exits.

Similar caution must be exercised with any structs, unions, and typedefs defined in `compile` command. Types defined in the `compile` command will no longer be available in the next `compile` command. Therefore, if you cast a variable to a type defined in the `compile` command, care must be taken to ensure that any future need to resolve the type can be achieved.

```
(gdb) compile code static struct a { int a; } v = { 42 }; argv = &v;
(gdb) compile code printf ("%d\n", ((struct a *) argv)->a);
gdb command line:1:36: error: dereferencing pointer to incomplete type struct a
Compilation failed.
(gdb) compile code struct a { int a; }; printf ("%d\n", ((struct a *) argv)-
>a);
42
```

Variables that have been optimized away by the compiler are not accessible to the code submitted to the `compile` command. Access to those variables will generate a compiler error which GDB will print to the console.

17.7.3 Compiler search for the compile command

GDB needs to find GCC for the inferior being debugged which may not be obvious for remote targets of different architecture than where GDB is running. Environment variable `PATH` on GDB host is searched for GCC binary matching the target architecture and operating system. This search can be overridden by `set compile-gcc` GDB command below. `PATH` is taken from shell that executed GDB, it is not the value set by GDB command `set environment`). See Section 4.4 [Environment], page 38.

Specifically `PATH` is searched for binaries matching regular expression `arch(-[^-]*)?-os-gcc` according to the inferior target being debugged. `arch` is processor name — multiarch is supported, so for example both `i386` and `x86_64` targets look for pattern `(x86_64|i.86)` and both `s390` and `s390x` targets look for pattern `s390x?`. `os` is currently supported only for pattern `linux(-gnu)?`.

On Posix hosts the compiler driver GDB needs to find also shared library `libcc1.so` from the compiler. It is searched in default shared library search path (overridable with usual environment variable `LD_LIBRARY_PATH`), unrelated to `PATH` or `set compile-gcc` settings. Contrary to it `libcc1plugin.so` is found according to the installation of the found compiler — as possibly specified by the `set compile-gcc` command.

set compile-gcc

Set compilation command used for compiling and injecting code with the `compile` commands. If this option is not set (it is set to an empty string), the search described above will occur — that is the default.

show compile-gcc

Displays the current compile command GCC driver filename. If set, it is the main command `gcc`, found usually for example under name `x86_64-linux-gnu-gcc`.

18 GDB Files

GDB needs to know the file name of the program to be debugged, both in order to read its symbol table and in order to start your program. To debug a core dump of a previous run, you must also tell GDB the name of the core dump file.

18.1 Commands to Specify Files

You may want to specify executable and core dump file names. The usual way to do this is at start-up time, using the arguments to GDB's start-up commands (see Chapter 2 [Getting In and Out of GDB], page 11).

Occasionally it is necessary to change to a different file during a GDB session. Or you may run GDB and forget to specify a file you want to use. Or you are debugging a remote target via `gdbserver` (see Section 20.3 [Using the `gdbserver` Program], page 322). In these situations the GDB commands to specify new files are useful.

`file filename`

Use *filename* as the program to be debugged. It is read for its symbols and for the contents of pure memory. It is also the program executed when you use the `run` command. If you do not specify a directory and the file is not found in the GDB working directory, GDB uses the environment variable `PATH` as a list of directories to search, just as the shell does when looking for a program to run. You can change the value of this variable, for both GDB and your program, using the `path` command.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

You can load unlinked object `.o` files into GDB using the `file` command. You will not be able to “run” an object file, but you can disassemble functions and inspect variables. Also, if the underlying BFD functionality supports it, you could use `gdb -write` to patch object files using this technique. Note that GDB can neither interpret nor modify relocations in this case, so branches and some initialized variables will appear to go to the wrong place. But this feature is still handy from time to time.

`file` `file` with no argument makes GDB discard any information it has on both executable file and the symbol table.

`exec-file [ filename ]`

Specify that the program to be run (but not the symbol table) is found in *filename*. GDB searches the environment variable `PATH` if necessary to locate your program. Omitting *filename* means to discard information on the executable file.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

`symbol-file [ filename [ -o offset ]]`

Read symbol table information from file *filename*. `PATH` is searched when necessary. Use the `file` command to get both symbol table and program to run from the same file.

If an optional *offset* is specified, it is added to the start address of each section in the symbol file. This is useful if the program is relocated at runtime, such as the Linux kernel with kASLR enabled.

symbol-file with no argument clears out GDB information on your program's symbol table.

The **symbol-file** command causes GDB to forget the contents of some breakpoints and auto-display expressions. This is because they may contain pointers to the internal data recording symbols and data types, which are part of the old symbol table data being discarded inside GDB.

symbol-file does not repeat if you press RET again after executing it once.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

When GDB is configured for a particular environment, it understands debugging information in whatever format is the standard generated for that environment; you may use either a GNU compiler, or other compilers that adhere to the local conventions. Best results are usually obtained from GNU compilers; for example, using GCC you can generate debugging information for optimized code.

For most kinds of object files, with the exception of old SVR3 systems using COFF, the **symbol-file** command does not normally read the symbol table in full right away. Instead, it scans the symbol table quickly to find which source files and which symbols are present. The details are read later, one source file at a time, as they are needed.

The purpose of this two-stage reading strategy is to make GDB start up faster. For the most part, it is invisible except for occasional pauses while the symbol table details for a particular source file are being read. (The **set verbose** command can turn these pauses into messages if desired. See Section 22.9 [Optional Warnings and Messages], page 386.)

We have not implemented the two-stage strategy for COFF yet. When the symbol table is stored in COFF format, **symbol-file** reads the symbol table data in full right away.

```
symbol-file [-readnow] filename
```

```
file [-readnow] filename
```

You can override the GDB two-stage strategy for reading symbol tables by using the **'-readnow'** option with any of the commands that load symbol table information, if you want to be sure GDB has the entire symbol table available.

```
symbol-file [-readnever] filename
```

```
file [-readnever] filename
```

You can instruct GDB to never read the symbolic information contained in *filename* by using the **'-readnever'** option. See [-readnever], page 13.

```
core-file [filename]
```

core Specify the whereabouts of a core dump file to be used as the “contents of memory”. Traditionally, core files contain only some parts of the address space of the process that generated them; GDB can access the executable file itself for other parts.

`core-file` with no argument specifies that no core file is to be used.

Note that the core file is ignored when your program is actually running under GDB. So, if you have been running your program and you wish to debug a core file instead, you must kill the subprocess in which the program is running. To do this, use the `kill` command (see Section 4.8 [Killing the Child Process], page 42).

To see which core files are loaded into which inferiors, use the `info inferiors` command (see [info inferiors command], page 43).

```
add-symbol-file filename [ -readnow | -readnever ] [ -o offset ] [ textaddress ] [
-s section address ... ]
```

The `add-symbol-file` command reads additional symbol table information from the file *filename*. You would use this command when *filename* has been dynamically loaded (by some other means) into the program that is running. The *textaddress* parameter gives the memory address at which the file's text section has been loaded. You can additionally specify the base address of other sections using an arbitrary number of '`-s section address`' pairs. If a section is omitted, GDB will use its default addresses as found in *filename*. Any *address* or *textaddress* can be given as an expression.

If an optional *offset* is specified, it is added to the start address of each section, except those for which the address was specified explicitly.

The symbol table of the file *filename* is added to the symbol table originally read with the `symbol-file` command. You can use the `add-symbol-file` command any number of times; the new symbol data thus read is kept in addition to the old.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

Changes can be reverted using the command `remove-symbol-file`.

Although *filename* is typically a shared library file, an executable file, or some other object file which has been fully relocated for loading into a process, you can also load symbolic information from relocatable `.o` files, as long as:

- the file's symbolic information refers only to linker symbols defined in that file, not to symbols defined by other object files,
- every section the file's symbolic information refers to has actually been loaded into the inferior, as it appears in the file, and
- you can determine the address at which every section was loaded, and provide these to the `add-symbol-file` command.

Some embedded operating systems, like Sun Chorus and VxWorks, can load relocatable files into an already running program; such systems typically make the requirements above easy to meet. However, it's important to recognize that many native systems use complex link procedures (`.linkonce` section factoring and C++ constructor table assembly, for example) that make the requirements difficult to meet. In general, one cannot assume that using `add-symbol-file` to read a relocatable object file's symbolic information will have the same effect as linking the relocatable object file into the program in the normal way.

`add-symbol-file` does not repeat if you press RET after using it.

`remove-symbol-file filename`

`remove-symbol-file -a address`

Remove a symbol file added via the `add-symbol-file` command. The file to remove can be identified by its *filename* or by an *address* that lies within the boundaries of this symbol file in memory. Example:

```
(gdb) add-symbol-file /home/user/gdb/mylib.so 0x7ffff7ff9480
add symbol table from file "/home/user/gdb/mylib.so" at
      .text_addr = 0x7ffff7ff9480
(y or n) y
Reading symbols from /home/user/gdb/mylib.so...
(gdb) remove-symbol-file -a 0x7ffff7ff9480
Remove symbol table from file "/home/user/gdb/mylib.so"? (y or n) y
(gdb)
```

The *address* can be any expression which evaluates to an address.

`remove-symbol-file` does not repeat if you press RET after using it.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

`add-symbol-file-from-memory address`

Load symbols from the given *address* in a dynamically loaded object file whose image is mapped directly into the inferior's memory. For example, the Linux kernel maps a `syscall` DSO into each process's address space; this DSO provides kernel-specific code for some system calls. The argument can be any expression whose evaluation yields the address of the file's shared object file header. For this command to work, you must have used `symbol-file` or `exec-file` commands in advance.

`section section addr`

The `section` command changes the base address of the named *section* of the exec file to *addr*. This can be used if the exec file does not contain section addresses, (such as in the `a.out` format), or when the addresses specified in the file itself are wrong. Each section must be changed separately. The `info files` command, described below, lists all the sections and their addresses.

`info files`

`info target`

`info files` and `info target` are synonymous; both print the current target (see Chapter 19 [Specifying a Debugging Target], page 313), including the names of the executable and core dump files currently in use by GDB, and the files from which symbols were loaded. The command `help target` lists all possible targets rather than current ones.

`maint info sections [-all-objects] [filter-list]`

Another command that can give you extra information about program sections is `maint info sections`. In addition to the section information displayed by `info files`, this command displays the flags and file offset of each section in the executable and core dump files.

When `'-all-objects'` is passed then sections from all loaded object files, including shared libraries, are printed.

The optional *filter-list* is a space separated list of filter keywords. Sections that match any one of the filter criteria will be printed. There are two types of filter:

section-name

Display information about any section named *section-name*.

section-flag

Display information for any section with *section-flag*. The section flags that GDB currently knows about are:

ALLOC Section will have space allocated in the process when loaded. Set for all sections except those containing debug information.

LOAD Section will be loaded from the file into the child process memory. Set for pre-initialized code and data, clear for **.bss** sections.

RELOC Section needs to be relocated before loading.

READONLY Section cannot be modified by the child process.

CODE Section contains executable code only.

DATA Section contains data only (no executable code).

ROM Section will reside in ROM.

CONSTRUCTOR

Section contains data for constructor/destructor lists.

HAS_CONTENTS

Section is not empty.

NEVER_LOAD

An instruction to the linker to not output the section.

COFF_SHARED_LIBRARY

A notification to the linker that the section contains COFF shared library information.

IS_COMMON

Section contains common symbols.

maint info target-sections

This command prints GDB's internal section table. For each target GDB maintains a table containing the allocatable sections from all currently mapped objects, along with information about where the section is mapped.

set trust-readonly-sections on

Tell GDB that readonly sections in your object file really are read-only (i.e. that their contents will not change). In that case, GDB can fetch values from these sections out of the object file, rather than from the target program. For some targets (notably embedded ones), this can be a significant enhancement to debugging performance.

The default is off.

set trust-readonly-sections off

Tell GDB not to trust readonly sections. This means that the contents of the section might change while the program is running, and must therefore be fetched from the target when needed.

show trust-readonly-sections

Show the current setting of trusting readonly sections.

All file-specifying commands allow both absolute and relative file names as arguments. GDB always converts the file name to an absolute file name and remembers it that way.

GDB supports GNU/Linux, MS-Windows, SunOS, Darwin/Mach-O, SVr4, IBM RS/6000 AIX, FDPIC (FR-V), and DSBT (TIC6X) shared libraries.

On MS-Windows GDB must be linked with the Expat library to support shared libraries. See [Expat], page 768.

GDB automatically loads symbol definitions from shared libraries when you use the **run** command, or when you examine a core file. (Before you issue the **run** command, GDB does not understand references to a function in a shared library, however—unless you are debugging a core file).

There are times, however, when you may wish to not automatically load symbol definitions from shared libraries, such as when they are particularly large or there are many of them.

To control the automatic loading of shared library symbols, use the commands:

set auto-solib-add mode

If *mode* is **on**, symbols from all shared object libraries will be loaded automatically when the inferior begins execution, you attach to an independently started inferior, or when the dynamic linker informs GDB that a new library has been loaded. If *mode* is **off**, symbols must be loaded manually, using the **sharedlibrary** command. The default value is **on**.

If your program uses lots of shared libraries with debug info that takes large amounts of memory, you can decrease the GDB memory footprint by preventing it from automatically loading the symbols from shared libraries. To that end, type **set auto-solib-add off** before running the inferior, then load each library whose debug symbols you do need with **sharedlibrary regex**, where *regex* is a regular expression that matches the libraries whose symbols you want to be loaded.

show auto-solib-add

Display the current autoloading mode.

To explicitly load shared library symbols, use the **sharedlibrary** command:

info share regex

info sharedlibrary regex

Print the names of the shared libraries which are currently loaded that match *regex*. If *regex* is omitted then print all shared libraries that are loaded.

For each library, GDB also lists the address range allocated to that library if it can be determined. If the address range cannot be determined then the address

range for the `.text` section from the library will be listed. If the `.text` section cannot be found then no addresses will be listed.

On systems that support linker namespaces, the output includes an additional column `NS` if the inferior has more than one active namespace when the command is used. This column the linker namespace that the shared library was loaded into.

Linker namespaces are a feature of some standard libraries, that allow shared objects to be loaded in isolated environment, eliminating the possibility that those objects may cross-talk. Each set of isolated shared objects is said to belong to a “namespace”, and linker related actions such as relocations do not cross namespace boundaries.

info dll regex

This is an alias of **info sharedlibrary**.

sharedlibrary regex

share regex

Load shared object library symbols for files matching a Unix regular expression. As with files loaded automatically, it only loads shared libraries required by your program for a core file or after typing **run**. If *regex* is omitted all shared libraries required by your program are loaded.

nosharedlibrary

Unload all shared object library symbols. This discards all symbols that have been loaded from all shared libraries. Symbols from shared libraries that were loaded by explicit user requests are not discarded.

Sometimes you may wish that GDB stops and gives you control when any of shared library events happen. The best way to do this is to use **catch load** and **catch unload** (see Section 5.1.3 [Set Catchpoints], page 71).

GDB also supports the **set stop-on-solib-events** command for this. This command exists for historical reasons. It is less useful than setting a catchpoint, because it does not allow for conditions or commands as a catchpoint does.

info linker-namespaces

info linker-namespaces [[n]]

With no argument, print the number of linker namespaces which are currently active — that is, namespaces that have libraries loaded into them. Then, it prints the number of libraries loaded into each namespace, and all the libraries loaded into them, in the same way as **info sharedlibrary**.

If an argument **[[n]]** is provided, only prints the library count and the libraries for the provided namespace *n*. The surrounding square brackets are optional.

set stop-on-solib-events

This command controls whether GDB should give you control when the dynamic linker notifies it about some shared library event. The most common event of interest is loading or unloading of a new shared library.

show stop-on-solib-events

Show whether GDB stops and gives you control when shared library events happen.

Shared libraries are also supported in many cross or remote debugging configurations. GDB needs to have access to the target's libraries; this can be accomplished either by providing copies of the libraries on the host system, or by asking GDB to automatically retrieve the libraries from the target. If copies of the target libraries are provided, they need to be the same as the target libraries, although the copies on the target can be stripped as long as the copies on the host are not.

For remote debugging, you need to tell GDB where the target libraries are, so that it can load the correct copies—otherwise, it may try to load the host's libraries. GDB has two variables to specify the search directories for target libraries.

set sysroot *path*

Use *path* as the system root for the program being debugged. Any absolute shared library paths will be prefixed with *path*; many runtime loaders store the absolute paths to the shared library in the target program's memory. When starting processes remotely, and when attaching to already-running processes (local or remote), their executable filenames will be prefixed with *path* if reported to GDB as absolute by the operating system. If you use **set sysroot** to find executables and shared libraries, they need to be laid out in the same way that they are on the target, with e.g. a */bin*, */lib* and */usr/lib* hierarchy under *path*.

If *path* starts with the sequence **target:** and the target system is remote then GDB will retrieve the target binaries from the remote system. This is only supported when using a remote target that supports the **remote get** command (see Section 20.2 [Sending files to a remote system], page 322). The part of *path* following the initial **target:** (if present) is used as system root prefix on the remote file system. If *path* starts with the sequence **remote:** this is converted to the sequence **target:** by **set sysroot**¹. If you want to specify a local system root using a directory that happens to be named **target:** or **remote:**, you need to use some equivalent variant of the name like *./target:*.

For targets with an MS-DOS based filesystem, such as MS-Windows, GDB tries prefixing a few variants of the target absolute file name with *path*. But first, on Unix hosts, GDB converts all backslash directory separators into forward slashes, because the backslash is not a directory separator on Unix:

```
c:\foo\bar.dll ⇒ c:/foo/bar.dll
```

Then, GDB attempts prefixing the target file name with *path*, and looks for the resulting file name in the host file system:

```
c:/foo/bar.dll ⇒ /path/to/sysroot/c:/foo/bar.dll
```

If that does not find the binary, GDB tries removing the *:* character from the drive spec, both for convenience, and, for the case of the host file system not supporting file names with colons:

```
c:/foo/bar.dll ⇒ /path/to/sysroot/c/foo/bar.dll
```

This makes it possible to have a system root that mirrors a target with more than one drive. E.g., you may want to setup your local copies of the target system shared libraries like so (note *'c'* vs *'z'*):

¹ Historically the functionality to retrieve binaries from the remote system was provided by prefixing *path* with **remote:**

```

/path/to/sysroot/c/sys/bin/foo.dll
/path/to/sysroot/c/sys/bin/bar.dll
/path/to/sysroot/z/sys/bin/bar.dll

```

and point the system root at `/path/to/sysroot`, so that GDB can find the correct copies of both `c:\sys\bin\foo.dll`, and `z:\sys\bin\bar.dll`.

If that still does not find the binary, GDB tries removing the whole drive spec from the target file name:

```
c:/foo/bar.dll ⇒ /path/to/sysroot/foo/bar.dll
```

This last lookup makes it possible to not care about the drive name, if you don't want or need to.

The `set solib-absolute-prefix` command is an alias for `set sysroot`.

You can set the default system root by using the configure-time `'--with-sysroot'` option. If the system root is inside GDB's configured binary prefix (set with `'--prefix'` or `'--exec-prefix'`), then the default system root will be updated automatically if the installed GDB is moved to a new location.

`show sysroot`

Display the current executable and shared library prefix.

`set solib-search-path path`

If this variable is set, *path* is a colon-separated list of directories to search for shared libraries. `'solib-search-path'` is used after `'sysroot'` fails to locate the library, or if the path to the library is relative instead of absolute. If you want to use `'solib-search-path'` instead of `'sysroot'`, be sure to set `'sysroot'` to a nonexistent directory to prevent GDB from finding your host's libraries. `'sysroot'` is preferred; setting it to a nonexistent directory may interfere with automatic loading of shared library symbols.

`show solib-search-path`

Display the current shared library search path.

`set target-file-system-kind kind`

Set assumed file system kind for target reported file names.

Shared library file names as reported by the target system may not make sense as is on the system GDB is running on. For example, when remote debugging a target that has MS-DOS based file system semantics, from a Unix host, the target may be reporting to GDB a list of loaded shared libraries with file names such as `c:\Windows\kernel32.dll`. On Unix hosts, there's no concept of drive letters, so the `'c:\'` prefix is not normally understood as indicating an absolute file name, and neither is the backslash normally considered a directory separator character. In that case, the native file system would interpret this whole absolute file name as a relative file name with no directory components. This would make it impossible to point GDB at a copy of the remote target's shared libraries on the host using `set sysroot`, and impractical with `set solib-search-path`. Setting `target-file-system-kind` to `dos-based` tells GDB to interpret such file names similarly to how the target would, and to map them to file names valid on GDB's native file system semantics. The value of *kind* can be `"auto"`, in addition to one of the supported file system kinds. In that case, GDB tries

to determine the appropriate file system variant based on the current target's operating system (see Section 22.7 [Configuring the Current ABI], page 380). The supported file system settings are:

- unix** Instruct GDB to assume the target file system is of Unix kind. Only file names starting the forward slash ('/') character are considered absolute, and the directory separator character is also the forward slash.
- dos-based** Instruct GDB to assume the target file system is DOS based. File names starting with either a forward slash, or a drive letter followed by a colon (e.g., 'c:'), are considered absolute, and both the slash ('/') and the backslash ('\') characters are considered directory separators.
- auto** Instruct GDB to use the file system kind associated with the target operating system (see Section 22.7 [Configuring the Current ABI], page 380). This is the default.

When processing file names provided by the user, GDB frequently needs to compare them to the file names recorded in the program's debug info. Normally, GDB compares just the *base names* of the files as strings, which is reasonably fast even for very large programs. (The base name of a file is the last portion of its name, after stripping all the leading directories.) This shortcut in comparison is based upon the assumption that files cannot have more than one base name. This is usually true, but references to files that use symlinks or similar filesystem facilities violate that assumption. If your program records files using such facilities, or if you provide file names to GDB using symlinks etc., you can set **basenames-may-differ** to **true** to instruct GDB to completely canonicalize each pair of file names it needs to compare. This will make file-name comparisons accurate, but at a price of a significant slowdown.

set basenames-may-differ

Set whether a source file may have multiple base names.

show basenames-may-differ

Show whether a source file may have multiple base names.

18.2 File Caching

To speed up file loading, and reduce memory usage, GDB will reuse the **bfd** objects used to track open files. See Section “BFD” in *The Binary File Descriptor Library*. The following commands allow visibility and control of the caching behavior.

maint info bfd

This prints information about each **bfd** object that is known to GDB.

maint set bfd-sharing

maint show bfd-sharing

Control whether **bfd** objects can be shared. When sharing is enabled GDB reuses already open **bfd** objects rather than reopening the same file. Turning sharing off does not cause already shared **bfd** objects to be unshared, but all

future files that are opened will create a new `bfd` object. Similarly, re-enabling sharing does not cause multiple existing `bfd` objects to be collapsed into a single shared `bfd` object.

`set debug bfd-cache level`

Turns on debugging of the `bfd` cache, setting the level to *level*.

`show debug bfd-cache`

Show the current debugging level of the `bfd` cache.

18.3 Debugging Information in Separate Files

GDB allows you to put a program’s debugging information in a file separate from the executable itself, in a way that allows GDB to find and load the debugging information automatically. Since debugging information can be very large—sometimes larger than the executable code itself—some systems distribute debugging information for their executables in separate files, which users can install only when they need to debug a problem.

GDB supports two ways of specifying the separate debug info file:

- The executable contains a *debug link* that specifies the name of the separate debug info file. The separate debug file’s name is usually `executable.debug`, where *executable* is the name of the corresponding executable file without leading directories (e.g., `ls.debug` for `/usr/bin/ls`). In addition, the debug link specifies a 32-bit *Cyclic Redundancy Check* (CRC) checksum for the debug file, which GDB uses to validate that the executable and the debug file came from the same build.
- The executable contains a *build ID*, a unique bit string that is also present in the corresponding debug info file. (This is supported only on some operating systems, when using the ELF or PE file formats for binary files and the GNU Binutils.) For more details about this feature, see the description of the `--build-id` command-line option in Section “Command Line Options” in *The GNU Linker*. The debug info file’s name is not specified explicitly by the build ID, but can be computed from the build ID, see below.

Depending on the way the debug info file is specified, GDB uses two different methods of looking for the debug file:

- For the “debug link” method, GDB looks up the named file in the directory of the executable file, then in a subdirectory of that directory named `.debug`, and finally under each one of the global debug directories, in a subdirectory whose name is identical to the leading directories of the executable’s absolute file name. (On MS-Windows/MS-DOS, the drive letter of the executable’s leading directories is converted to a one-letter subdirectory, i.e. `d:/usr/bin/` is converted to `/d/usr/bin/`, because Windows filesystems disallow colons in file names.)
- For the “build ID” method, GDB looks in the `.build-id` subdirectory of each one of the global debug directories for a file named `nn/nnnnnnnn.debug`, where *nn* are the first 2 hex characters of the build ID bit string, and *nnnnnnnn* are the rest of the bit string. (Real build ID strings are 32 or more hex characters, not 10.) GDB can automatically query `debuginfod` servers using build IDs in order to download separate debug files that cannot be found locally. For more information see Appendix K [Debuginfod], page 913.

So, for example, suppose you ask GDB to debug `/usr/bin/ls`, which has a debug link that specifies the file `ls.debug`, and a build ID whose value in hex is `abcdef1234`. If the list of the global debug directories includes `/usr/lib/debug`, then GDB will look for the following debug information files, in the indicated order:

- `/usr/lib/debug/.build-id/ab/cdef1234.debug`
- `/usr/bin/ls.debug`
- `/usr/bin/.debug/ls.debug`
- `/usr/lib/debug/usr/bin/ls.debug`.

If the debug file still has not been found and `debuginfod` (see Appendix K [Debuginfod], page 913) is enabled, GDB will attempt to download the file from `debuginfod` servers.

Global debugging info directories default to what is set by GDB configure option `--with-separate-debug-dir` and augmented by the colon-separated list of directories provided via GDB configure option `--additional-debug-dirs`. During GDB run you can also set the global debugging info directories, and view the list GDB is currently using.

`set debug-file-directory directories`

Set the directories which GDB searches for separate debugging information files to *directory*. Multiple path components can be set concatenating them by a path separator.

`show debug-file-directory`

Show the directories GDB searches for separate debugging information files.

A debug link is a special section of the executable file named `.gnu_debuglink`. The section must contain:

- A filename, with any leading directory components removed, followed by a zero byte,
- zero to three bytes of padding, as needed to reach the next four-byte boundary within the section, and
- a four-byte CRC checksum, stored in the same endianness used for the executable file itself. The checksum is computed on the debugging information file's full contents by the function given below, passing zero as the *crc* argument.

Any executable file format can carry a debug link, as long as it can contain a section named `.gnu_debuglink` with the contents described above.

The build ID is a special section in the executable file (and in other ELF binary files that GDB may consider). This section is often named `.note.gnu.build-id`, but that name is not mandatory. It contains unique identification for the built files—the ID remains the same across multiple builds of the same build tree. The default algorithm SHA1 produces 160 bits (40 hexadecimal characters) of the content for the build ID string. The same section with an identical value is present in the original built binary with symbols, in its stripped variant, and in the separate debugging information file.

The debugging information file itself should be an ordinary executable, containing a full set of linker symbols, sections, and debugging information. The sections of the debugging information file should have the same names, addresses, and sizes as the original file, but they need not contain any data—much like a `.bss` section in an ordinary executable.

The GNU binary utilities (Binutils) package includes the ‘`objcopy`’ utility that can produce the separated executable / debugging information file pairs using the following commands:

```
objcopy --only-keep-debug foo foo.debug
strip -g foo
```

These commands remove the debugging information from the executable file `foo` and place it in the file `foo.debug`. You can use the first, second or both methods to link the two files:

- The debug link method needs the following additional command to also leave behind a debug link in `foo`:

```
objcopy --add-gnu-debuglink=foo.debug foo
```

Ulrich Drepper’s `elfutils` package, starting with version 0.53, contains a version of the `strip` command such that the command `strip foo -f foo.debug` has the same functionality as the two `objcopy` commands and the `ln -s` command above, together.

- Build ID gets embedded into the main executable using `ld --build-id` or the GCC counterpart `gcc -Wl,--build-id`. Build ID support plus compatibility fixes for debug files separation are present in GNU binary utilities (Binutils) package since version 2.18.

The CRC used in `.gnu_debuglink` is the CRC-32 defined in IEEE 802.3 using the polynomial:

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

The function is computed byte at a time, taking the least significant bit of each byte first. The initial pattern `0xffffffff` is used, to ensure leading zeros affect the CRC and the final result is inverted to ensure trailing zeros also affect the CRC.

Note: This is the same CRC polynomial as used in handling the *Remote Serial Protocol* qCRC packet (see [qCRC packet], page 812). However in the case of the Remote Serial Protocol, the CRC is computed *most* significant bit first, and the result is not inverted, so trailing zeros have no effect on the CRC value.

To complete the description, we show below the code of the function which produces the CRC used in `.gnu_debuglink`. Inverting the initially supplied `crc` argument means that an initial call to this function passing in zero will start computing the CRC using `0xffffffff`.

```
unsigned long
gnu_debuglink_crc32 (unsigned long crc,
                    unsigned char *buf, size_t len)
{
    static const unsigned long crc32_table[256] =
    {
        0x00000000, 0x77073096, 0xee0e612c, 0x990951ba, 0x076dc419,
        0x706af48f, 0xe963a535, 0x9e6495a3, 0x0edb8832, 0x79dcb8a4,
        0xe0d5e91e, 0x97d2d988, 0x09b64c2b, 0x7eb17cbd, 0xe7b82d07,
        0x90bf1d91, 0x1dbf1064, 0x6ab020f2, 0xf3b97148, 0x84be41de,
        0x1adad47d, 0x6ddde4eb, 0xf4d4b551, 0x83d385c7, 0x136c9856,
        0x646ba8c0, 0xfd62f97a, 0x8a65c9ec, 0x14015c4f, 0x63066cd9,
        0xfa0f3d63, 0x8d080df5, 0x3b6e20c8, 0x4c69105e, 0xd56041e4,
        0xa2677172, 0x3c03e4d1, 0x4b04d447, 0xd20d85fd, 0xa50ab56b,
        0x32b5a8fa, 0x45b2986c, 0xdbbbc9d6, 0xacbcf940, 0x32d86ce3,
        0x45df5c75, 0xdcd60dcf, 0xabd13d59, 0x26d930ac, 0x51de003a,
        0xc8d75180, 0xbfd06116, 0x21b4f4b5, 0x56b3c423, 0xcfba9599,
        0xb8bda50f, 0x2802b89e, 0x5f058808, 0xc60cd9b2, 0xb10be924,
```

```

0x2f6f7c87, 0x58684c11, 0xc1611dab, 0xb6662d3d, 0x76dc4190,
0x01db7106, 0x98d220bc, 0xefd5102a, 0x71b18589, 0x06b6b51f,
0x9fbfe4a5, 0xe8b8d433, 0x7807c9a2, 0x0f00f934, 0x9609a88e,
0xe10e9818, 0x7f6a0dbb, 0x086d3d2d, 0x91646c97, 0xe6635c01,
0x6b6b51f4, 0x1c6c6162, 0x856530d8, 0xf262004e, 0x6c0695ed,
0x1b01a57b, 0x8208f4c1, 0xf50fc457, 0x65b0d9c6, 0x12b7e950,
0x8bbeb8ea, 0xfcb9887c, 0x62dd1ddf, 0x15da2d49, 0x8cd37cf3,
0xfbd44c65, 0x4db26158, 0x3ab551ce, 0xa3bc0074, 0xd4bb30e2,
0x4adfa541, 0x3dd895d7, 0xa4d1c46d, 0xd3d6f4fb, 0x4369e96a,
0x346ed9fc, 0xad678846, 0xda60b8d0, 0x44042d73, 0x33031de5,
0xaa0a4c5f, 0xdd0d7cc9, 0x5005713c, 0x270241aa, 0xbe0b1010,
0xc90c2086, 0x5768b525, 0x206f85b3, 0xb966d409, 0xce61e49f,
0x5edef90e, 0x29d9c998, 0xb0d09822, 0xc7d7a8b4, 0x59b33d17,
0x2eb40d81, 0xb7bd5c3b, 0xc0ba6cad, 0xedb88320, 0x9abfb3b6,
0x03b6e20c, 0x74b1d29a, 0xead54739, 0x9dd277af, 0x04db2615,
0x73dc1683, 0xe3630b12, 0x94643b84, 0xd6d6a3e, 0x7a6a5aa8,
0xe4ecf0b, 0x9309ff9d, 0xa00ae27, 0x7d079eb1, 0xf00f9344,
0x8708a3d2, 0x1e01f268, 0x6906c2fe, 0xf762575d, 0x806567cb,
0x196c3671, 0x6e6b06e7, 0xfed41b76, 0x89d32be0, 0x10da7a5a,
0x67dd4acc, 0xf9b9df6f, 0x8ebeeff9, 0x17b7be43, 0x60b08ed5,
0xd6d6a3e8, 0xa1d1937e, 0x38d8c2c4, 0x4fdff252, 0xd1bb67f1,
0xa6bc5767, 0x3fb506dd, 0x48b2364b, 0xd80d2bda, 0xaf0a1b4c,
0x36034af6, 0x41047a60, 0xdf60efc3, 0xa867df55, 0x316e8eef,
0x4669be79, 0xcb61b38c, 0xbc66831a, 0x256fd2a0, 0x5268e236,
0xcc0c7795, 0xbb0b4703, 0x220216b9, 0x5505262f, 0xc5ba3bbe,
0xb2bd0b28, 0x2bb45a92, 0x5cb36a04, 0xc2d7ffa7, 0xb5d0cf31,
0x2cd99e8b, 0x5bdeae1d, 0x9b64c2b0, 0xec63f226, 0x756aa39c,
0x026d930a, 0x9c0906a9, 0xeb0e363f, 0x72076785, 0x05005713,
0x95bf4a82, 0xe2b87a14, 0x7bb12bae, 0x0cb61b38, 0x92d28e9b,
0xe5d5be0d, 0x7cdcefb7, 0x0bdbdf21, 0x86d3d2d4, 0xf1d4e242,
0x68ddb3f8, 0x1fda836e, 0x81be16cd, 0xf6b9265b, 0x6fb077e1,
0x18b74777, 0x88085ae6, 0xff0f6a70, 0x66063bca, 0x11010b5c,
0x8f659eff, 0xf862ae69, 0x616bffd3, 0x166ccf45, 0xa00ae278,
0xd70dd2ee, 0x4e048354, 0x3903b3c2, 0xa7672661, 0xd06016f7,
0x4969474d, 0x3e6e77db, 0xaed16a4a, 0xd9d65adc, 0x40df0b66,
0x37d83bf0, 0xa9bcae53, 0xdeb9ec5, 0x47b2cf7f, 0x30b5ffe9,
0xbdbdf21c, 0xcabac28a, 0x53b39330, 0x24b4a3a6, 0xbad03605,
0xcdd70693, 0x54de5729, 0x23d967bf, 0xb3667a2e, 0xc4614ab8,
0x5d681b02, 0x2a6f2b94, 0xb40bbe37, 0xc30c8ea1, 0x5a05df1b,
0x2d02ef8d
};
unsigned char *end;

crc = ~crc & 0xffffffff;
for (end = buf + len; buf < end; ++buf)
    crc = crc32_table[(crc ^ *buf) & 0xff] ^ (crc >> 8);
return ~crc & 0xffffffff;
}

```

This computation does not apply to the “build ID” method.

18.4 Debugging information in a special section

Some systems ship pre-built executables and libraries that have a special ‘.gnu_debugdata’ section. This feature is called *MiniDebugInfo*. This section holds an LZMA-compressed object and is used to supply extra symbols for backtraces.

The intent of this section is to provide extra minimal debugging information for use in simple backtraces. It is not intended to be a replacement for full separate debugging information (see Section 18.3 [Separate Debug Files], page 303). The example below shows the intended use; however, GDB does not currently put restrictions on what sort of debugging information might be included in the section.

GDB has support for this extension. If the section exists, then it is used provided that no other source of debugging information can be found, and that GDB was configured with LZMA support.

This section can be easily created using `objcopy` and other standard utilities:

```
# Extract the dynamic symbols from the main binary, there is no need
# to also have these in the normal symbol table.
nm -D binary --format=posix --defined-only \
    | awk '{ print $1 }' | sort > dynsyms

# Extract all the text (i.e. function) symbols from the debuginfo.
# (Note that we actually also accept "D" symbols, for the benefit
# of platforms like PowerPC64 that use function descriptors.)
nm binary --format=posix --defined-only \
    | awk '{ if ($2 == "T" || $2 == "t" || $2 == "D") print $1 }' \
    | sort > funcsyms

# Keep all the function symbols not already in the dynamic symbol
# table.
comm -13 dynsyms funcsyms > keep_symbols

# Separate full debug info into debug binary.
objcopy --only-keep-debug binary debug

# Copy the full debuginfo, keeping only a minimal set of symbols and
# removing some unnecessary sections.
objcopy -S --remove-section .gdb_index --remove-section .comment \
    --keep-symbols=keep_symbols debug mini_debuginfo

# Drop the full debug info from the original binary.
strip --strip-all -R .comment binary

# Inject the compressed data into the .gnu_debugdata section of the
# original binary.
xz mini_debuginfo
objcopy --add-section .gnu_debugdata=mini_debuginfo.xz binary
```

18.5 Index Files Speed Up GDB

When GDB finds a symbol file, it scans the symbols in the file in order to construct an internal symbol table. This lets most GDB operations work quickly—at the cost of a delay early on. For large programs, this delay can be quite lengthy, so GDB provides a way to build an index, which speeds up startup.

For convenience, GDB comes with a program, `gdb-add-index`, which can be used to add the index to a symbol file. It takes the symbol file as its only argument:

```
$ gdb-add-index symfile
```

See [gdb-add-index], page 924.

It is also possible to do the work manually. Here is what `gdb-add-index` does behind the curtains.

The index is stored as a section in the symbol file. GDB can write the index to a file, then you can put it into the symbol file using `objcopy`.

To create an index file, use the `save gdb-index` command:

```
save gdb-index [-dwarf-5] directory
```

Create index files for all symbol files currently known by GDB. For each known *symbol-file*, this command by default produces a single file *symbol-file.gdb-index*. If you invoke this command with the `-dwarf-5` option, it produces 2 files: *symbol-file.debug_names* and *symbol-file.debug_str*. The files are created in the given *directory*.

The *directory* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

Once you have created an index file you can merge it into your symbol file, here named *symfile*, using `objcopy`:

```
$ objcopy --add-section .gdb_index=symfile.gdb-index \
  --set-section-flags .gdb_index=readonly symfile symfile
```

Or for `-dwarf-5`:

```
$ objcopy --dump-section .debug_str=symfile.debug_str.new symfile
$ cat symfile.debug_str >>symfile.debug_str.new
$ objcopy --add-section .debug_names=symfile.gdb-index \
  --set-section-flags .debug_names=readonly \
  --update-section .debug_str=symfile.debug_str.new symfile symfile
```

GDB will normally ignore older versions of `.gdb_index` sections that have been deprecated. Usually they are deprecated because they are missing a new feature or have performance issues. To tell GDB to use a deprecated index section anyway specify `set use-deprecated-index-sections on`. The default is `off`. This can speed up startup, but may result in some functionality being lost. See Appendix J [Index Section Format], page 909.

Warning: Setting `use-deprecated-index-sections` to `on` must be done before `gdb` reads the file. The following will not work:

```
$ gdb -ex "set use-deprecated-index-sections on" <program>
```

Instead you must do, for example,

```
$ gdb -iex "set use-deprecated-index-sections on" <program>
```

Indices only work when using DWARF debugging information.

18.5.1 Automatic symbol index cache

It is possible for GDB to automatically save a copy of this index in a cache on disk and retrieve it from there when loading the same binary in the future. This feature can be turned on with `set index-cache enabled on`. Note that the cache will use the ELF build ID to identify the cached indices, so files without build ID will not have their index cached. The following commands can be used to tweak the behavior of the index cache.

```
set index-cache enabled on
```

```
set index-cache enabled off
```

Enable or disable the use of the symbol index cache.

```
set index-cache directory directory
show index-cache directory
```

Set/show the directory where index files will be saved.

The default value for this directory depends on the host platform. On most systems, the index is cached in the `gdb` subdirectory of the directory pointed to by the `XDGCACHEHOME` environment variable, if it is defined, else in the `.cache/gdb` subdirectory of your home directory. However, on some systems, the default may differ according to local convention.

There is no limit on the disk space used by index cache. It is perfectly safe to delete the content of that directory to free up disk space.

```
show index-cache stats
```

Print the number of cache hits and misses since the launch of GDB.

18.5.2 Building the index with GNU gold

The GNU `gold` linker can write the index at link time into the resulting ELF file, by passing the `--gdb-index` flag to `gold`.

This is especially helpful if you intend to build a program and immediately run it under GDB. You may find that it is faster overall for the linker to write the index while it has the relevant information in memory anyways, rather than re-reloading the data from disk with `gdb-add-index`, or debugging the program without an index at all, especially for large programs.

18.6 Extensions to ‘.debug_names’

The DWARF specification documents an optional index section called ‘.debug_names’. GDB can both read and create this section. However, in order to work with GDB, some extensions were necessary.

GDB uses an augmentation string to specify which extensions are in use and to allow support of backwards-incompatible changes in this functionality. The augmentation string has the form ‘GDB n ’, where n is an integral version number of the extensions, which is incremented when the extensions are added or modified. The smallest n is 2; earlier versions of GDB used just ‘GDB’ with no version number, but these versions of the index are no longer supported.

Here is a list of augmentation string versions along with the changes introduced with each version, compared to the previous version.

- ‘GDB2’ Specifies the use of attributes `DW_IDX_GNU_internal`, `DW_IDX_GNU_main`, `DW_IDX_GNU_language` and `DW_IDX_GNU_linkage_name`, described below.
- ‘GDB3’ Changes the semantic of the `DW_IDX_parent` attribute. With ‘GDB2’, `DW_IDX_parent` provided an offset into the name table. With ‘GDB3’, it now provides an offset to the index entry of the parent, relative to the start of the entry pool region.

GDB produces indexes with the augmentation string ‘GDB3’.

GDB can read indexes with augmentation strings ‘GDB2’ or ‘GDB3’. GDB does not support reading indexes with any other augmentation strings.

GDB does not use the specified hash table. Therefore, because this hash table is optional, GDB also does not write it.

GDB generates and uses the following non-standard index attributes:

DW_IDX_GNU_internal

This has the value ‘0x2000’. It is a flag that, when set, indicates that the associated entry has **static** linkage.

DW_IDX_GNU_main

This has the value ‘0x2002’. It is a flag that, when set, indicates that the associated entry is the program’s **main**.

DW_IDX_GNU_language

This has the value ‘0x2003’. It is a ‘DW_LANG_’ constant, indicating the language of the associated entry.

DW_IDX_GNU_linkage_name

This has the value ‘0x2004’. It is a flag that, when set, indicates that the associated entry is a linkage name, and not a source name.

GDB also has some special handling for cases not considered in the DWARF specification.

- The **DW_IDX_parent** for a C-style enumerator does not point at the entry for **enum** itself, but rather the parent of the type. The reason for this is that C-style enumerators are injected into the containing scope, and so their name is not qualified by the **enum**; and furthermore there is no way to distinguish between C-style enumerators and **enum class**-style enumerators in ‘.debug_names’.
- Similarly, **DW_IDX_parent** is omitted for any linkage name entries that are written.
- Definitions in partial units are handled differently. These most typically are seen in the output of **dwz**.

In general, a DWARF partial unit cannot be read in isolation, but only by reading it in the context of some other unit that references it via **DW_TAG_imported_unit**.

Therefore, an ordinary definition in a partial unit is attributed to one of the outermost containing units. This is done by referencing this containing CU in the **DW_IDX_compile_unit** attribute.

A further special case applies to **DW_TAG_inlined_subroutine** entries. An inlined subroutine appearing in a partial unit may be inlined in all of the outermost compilation units that directly or indirectly include the partial unit. Therefore, in this case, GDB will emit a separate index entry for the entry, once for each such containing unit.

18.7 Errors Reading Symbol Files

While reading a symbol file, GDB occasionally encounters problems, such as symbol types it does not recognize, or known bugs in compiler output. By default, GDB does not notify you of such problems, since they are relatively common and primarily of interest to people debugging compilers. If you are interested in seeing information about ill-constructed symbol tables, you can either ask GDB to print only one message about each such type of problem, no matter how many times the problem occurs; or you can ask GDB to print more messages, to see how many times the problems occur, with the **set complaints** command (see Section 22.9 [Optional Warnings and Messages], page 386).

The messages currently printed, and their meanings, include:

inner block not inside outer block in *symbol*

The symbol information shows where symbol scopes begin and end (such as at the start of a function or a block of statements). This error indicates that an inner scope block is not fully contained in its outer scope blocks.

GDB circumvents the problem by treating the inner block as if it had the same scope as the outer block. In the error message, *symbol* may be shown as “(don’t know)” if the outer block is not a function.

block at *address* out of order

The symbol information for symbol scope blocks should occur in order of increasing addresses. This error indicates that it does not do so.

GDB does not circumvent this problem, and has trouble locating symbols in the source file whose symbols it is reading. (You can often determine what source file is affected by specifying **set verbose on**. See Section 22.9 [Optional Warnings and Messages], page 386.)

bad block start address patched

The symbol information for a symbol scope block has a start address smaller than the address of the preceding source line. This is known to occur in the SunOS 4.1.1 (and earlier) C compiler.

GDB circumvents the problem by treating the symbol scope block as starting on the previous source line.

bad string table offset in symbol *n*

Symbol number *n* contains a pointer into the string table which is larger than the size of the string table.

GDB circumvents the problem by considering the symbol to have the name **foo**, which may cause other problems if many symbols end up with this name.

unknown symbol type *0xnn*

The symbol information contains new data types that GDB does not yet know how to read. *0xnn* is the symbol type of the uncomprehended information, in hexadecimal.

GDB circumvents the error by ignoring this symbol information. This usually allows you to debug your program, though certain symbols are not accessible. If you encounter such a problem and feel like debugging it, you can debug **gdb** with itself, breakpoint on **complain**, then go up to the function **read_dbx_symtab** and examine ***bufp** to see the symbol.

stub type has NULL name

GDB could not find the full definition for a struct or class.

const/volatile indicator missing (ok if using g++ v1.x), got...

The symbol information for a C++ member function is missing some information that recent versions of the compiler should have output for it.

info mismatch between compiler and debugger

GDB could not parse a type specification output by the compiler.

18.8 GDB Data Files

GDB will sometimes read an auxiliary data file. These files are kept in a directory known as the *data directory*.

You can set the data directory's name, and view the name GDB is currently using.

set data-directory *directory*

Set the directory which GDB searches for auxiliary data files to *directory*.

show data-directory

Show the directory GDB searches for auxiliary data files.

You can set the default data directory by using the configure-time '`--with-gdb-datadir`' option. If the data directory is inside GDB's configured binary prefix (set with '`--prefix`' or '`--exec-prefix`'), then the default data directory will be updated automatically if the installed GDB is moved to a new location.

The data directory may also be specified with the `--data-directory` command line option. See Section 2.1.2 [Mode Options], page 13.

19 Specifying a Debugging Target

A *target* is the execution environment occupied by your program.

Often, GDB runs in the same host environment as your program; in that case, the debugging target is specified as a side effect when you use the `file` or `core` commands. When you need more flexibility—for example, running GDB on a physically separate host, or controlling a standalone system over a serial port or a realtime system over a TCP/IP connection—you can use the `target` command to specify one of the target types configured for GDB (see Section 19.2 [Commands for Managing Targets], page 313).

It is possible to build GDB for several different *target architectures*. When GDB is built like that, you can choose one of the available architectures with the `set architecture` command.

`set architecture arch`

This command sets the current target architecture to *arch*. The value of *arch* can be "auto", in addition to one of the supported architectures.

`show architecture`

Show the current target architecture.

`set processor`

`processor`

These are alias commands for, respectively, `set architecture` and `show architecture`.

19.1 Active Targets

There are multiple classes of targets such as: processes, executable files or recording sessions. Core files belong to the process class, making core file and process mutually exclusive. Otherwise, GDB can work concurrently on multiple active targets, one in each class. This allows you to (for example) start a process and inspect its activity, while still having access to the executable file after the process finishes. Or if you start process recording (see Chapter 6 [Reverse Execution], page 103) and `reverse-step` there, you are presented a virtual layer of the recording target, while the process target remains stopped at the chronologically last point of the process execution.

Use the `core-file` and `exec-file` commands to select a new core file or executable target (see Section 18.1 [Commands to Specify Files], page 293). To specify as a target a process that is already running, use the `attach` command (see Section 4.7 [Debugging an Already-running Process], page 41).

19.2 Commands for Managing Targets

`target type parameters`

Connects the GDB host environment to a target machine or process. A target is typically a protocol for talking to debugging facilities. You use the argument *type* to specify the type or protocol of the target machine.

Further *parameters* are interpreted by the target protocol, but typically include things like device names or host names to connect with, process numbers, and baud rates.

The **target** command does not repeat if you press **RET** again after executing the command.

help target

Displays the names of all targets available. To display targets currently selected, use either **info target** or **info files** (see Section 18.1 [Commands to Specify Files], page 293).

help target name

Describe a particular target, including any parameters necessary to select it.

set gnutarget args

GDB uses its own library BFD to read your files. GDB knows whether it is reading an *executable*, a *core*, or a *.o* file; however, you can specify the file format with the **set gnutarget** command. Unlike most **target** commands, with **gnutarget** the **target** refers to a program, not a machine.

Warning: To specify a file format with **set gnutarget**, you must know the actual BFD name.

See Section 18.1 [Commands to Specify Files], page 293.

show gnutarget

Use the **show gnutarget** command to display what file format **gnutarget** is set to read. If you have not set **gnutarget**, GDB will determine the file format for each file automatically, and **show gnutarget** displays ‘The current BFD target is “auto”’.

Here are some common targets (available, or not, depending on the GDB configuration):

target exec program

An executable file. ‘**target exec program**’ is the same as ‘**exec-file program**’. The *program* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

target core filename

A core dump file. ‘**target core filename**’ is the same as ‘**core-file filename**’.

The *filename* argument supports escaping and quoting, see Section 3.4 [File-names As Command Arguments], page 27.

target remote medium

A remote system connected to GDB via a serial line or network connection. This command tells GDB to use its own remote protocol over *medium* for debugging. See Chapter 20 [Remote Debugging], page 317.

For example, if you have a board connected to **/dev/ttya** on the machine running GDB, you could say:

```
target remote /dev/ttya
```

target remote supports the **load** command. This is only useful if you have some other way of getting the stub to the target system, and you can put it somewhere in memory where it won’t get clobbered by the download.

target sim [*simargs*] ...

Builtin CPU simulator. GDB includes simulators for most architectures. In general,

```
target sim
load
run
```

works; however, you cannot assume that a specific memory map, device drivers, or even basic I/O is available, although some simulators do provide these. For info about any processor-specific simulator details, see the appropriate section in Section 21.3 [Embedded Processors], page 350.

target native

Setup for local/native process debugging. Useful to make the **run** command spawn native processes (likewise **attach**, etc.) even when **set auto-connect-native-target** is **off** (see [set auto-connect-native-target], page 36).

Different targets are available on different configurations of GDB; your configuration may have more or fewer targets.

Many remote targets require you to download the executable's code once you've successfully established a connection. You may wish to control various aspects of this process.

set hash This command controls whether a hash mark '#' is displayed while downloading a file to the remote monitor. If on, a hash mark is displayed after each S-record is successfully downloaded to the monitor.

show hash Show the current status of displaying the hash mark.

set debug monitor

Enable or disable display of communications messages between GDB and the remote monitor.

show debug monitor

Show the current status of displaying communications between GDB and the remote monitor.

load filename offset

Depending on what remote debugging facilities are configured into GDB, the **load** command may be available. Where it exists, it is meant to make *filename* (an executable) available for debugging on the remote system—by downloading, or dynamic linking, for example. **load** also records the *filename* symbol table in GDB, like the **add-symbol-file** command.

If your GDB does not have a **load** command, attempting to execute it gets the error message "You can't do that when your target is ..."

The file is loaded at whatever address is specified in the executable. For some object file formats, you can specify the load address when you link the program; for other formats, like a.out, the object file format specifies a fixed address.

It is also possible to tell GDB to load the executable file at a specific offset described by the optional argument *offset*. When *offset* is provided, *filename* must also be provided.

Depending on the remote side capabilities, GDB may be able to load programs into flash memory.

`load` does not repeat if you press `RET` again after using it.

flash-erase

Erases all known flash memory regions on the target.

19.3 Choosing Target Byte Order

Some types of processors, such as the MIPS, PowerPC, and Renesas SH, offer the ability to run either big-endian or little-endian byte orders. Usually the executable or symbol will include a bit to designate the endian-ness, and you will not need to worry about which to use. However, you may still find it useful to adjust GDB's idea of processor endian-ness manually.

set endian big

Instruct GDB to assume the target is big-endian.

set endian little

Instruct GDB to assume the target is little-endian.

set endian auto

Instruct GDB to use the byte order associated with the executable.

show endian

Display GDB's current idea of the target byte order.

If the **set endian auto** mode is in effect and no executable has been selected, then the endianness used is the last one chosen either by one of the **set endian big** and **set endian little** commands or by inferring from the last executable used. If no endianness has been previously chosen, then the default for this mode is inferred from the target GDB has been built for, and is **little** if the name of the target CPU has an **el** suffix and **big** otherwise.

Note that these commands merely adjust interpretation of symbolic data on the host, and that they have absolutely no effect on the target system.

20 Debugging Remote Programs

If you are trying to debug a program running on a machine that cannot run GDB in the usual way, it is often useful to use remote debugging. For example, you might use remote debugging on an operating system kernel, or on a small system which does not have a general purpose operating system powerful enough to run a full-featured debugger.

Some configurations of GDB have special serial or TCP/IP interfaces to make this work with particular debugging targets. In addition, GDB comes with a generic serial protocol (specific to GDB, but not specific to any particular target system) which you can use if you write the remote stubs—the code that runs on the remote system to communicate with GDB.

Other remote targets may be available in your configuration of GDB; use `help target` to list them.

20.1 Connecting to a Remote Target

This section describes how to connect to a remote target, including the types of connections and their differences, how to set up executable and symbol files on the host and target, and the commands used for connecting to and disconnecting from the remote target.

20.1.1 Types of Remote Connections

GDB supports two types of remote connections, `target remote` mode and `target extended-remote` mode. Note that many remote targets support only `target remote` mode. There are several major differences between the two types of connections, enumerated here:

Result of detach or program exit

With `target remote` mode: When the debugged program exits or you detach from it, GDB disconnects from the target. When using `gdbserver`, `gdbserver` will exit.

With `target extended-remote` mode: When the debugged program exits or you detach from it, GDB remains connected to the target, even though no program is running. You can rerun the program, attach to a running program, or use `monitor` commands specific to the target.

When using `gdbserver` in this case, it does not exit unless it was invoked using the `--once` option. If the `--once` option was not used, you can ask `gdbserver` to exit using the `monitor exit` command (see [Monitor Commands for `gdbserver`], page 326).

Specifying the program to debug

For both connection types you use the `file` command to specify the program on the host system. If you are using `gdbserver` there are some differences in how to specify the location of the program on the target.

With `target remote` mode: You must either specify the program to debug on the `gdbserver` command line or use the `--attach` option (see [Attaching to a Running Program], page 323).

With target extended-remote mode: You may specify the program to debug on the `gdbserver` command line, or you can load the program or attach to it using GDB commands after connecting to `gdbserver`.

You can start `gdbserver` without supplying an initial command to run or process ID to attach. To do this, use the `--multi` command line option. Then you can connect using `target extended-remote` and start the program you want to debug (see below for details on using the `run` command in this scenario). Note that the conditions under which `gdbserver` terminates depend on how GDB connects to it (`target remote` or `target extended-remote`). The `--multi` option to `gdbserver` has no influence on that.

The `run` command

With target remote mode: The `run` command is not supported. Once a connection has been established, you can use all the usual GDB commands to examine and change data. The remote program is already running, so you can use commands like `step` and `continue`.

With target extended-remote mode: The `run` command is supported. The `run` command uses the value set by `set remote exec-file` (see [set remote exec-file], page 329) to select the program to run. Command line arguments are supported, except for wildcard expansion and I/O redirection (see Section 4.3 [Arguments], page 38).

If you specify the program to debug on the command line, then the `run` command is not required to start execution, and you can resume using commands like `step` and `continue` as with `target remote` mode.

Attaching

With target remote mode: The GDB command `attach` is not supported. To attach to a running program using `gdbserver`, you must use the `--attach` option (see [Running gdbserver], page 323).

With target extended-remote mode: To attach to a running program, you may use the `attach` command after the connection has been established. If you are using `gdbserver`, you may also invoke `gdbserver` using the `--attach` option (see [Running gdbserver], page 323).

Some remote targets allow GDB to determine the executable file running in the process the debugger is attaching to. In such a case, GDB uses the value of `exec-file-mismatch` to handle a possible mismatch between the executable file name running in the process and the name of the current `exec-file` loaded by GDB (see [set exec-file-mismatch], page 42).

20.1.2 Host and Target Files

GDB, running on the host, needs access to symbol and debugging information for your program running on the target. This requires access to an unstripped copy of your program, and possibly any associated symbol files. Note that this section applies equally to both `target remote` mode and `target extended-remote` mode.

Some remote targets (see [qXfer executable filename read], page 834, and see Section E.8 [Host I/O Packets], page 847) allow GDB to access program files over the same connection used to communicate with GDB. With such a target, if the remote program is unstripped, the only command you need is `target remote` (or `target extended-remote`).

If the remote program is stripped, or the target does not support remote program file access, start up GDB using the name of the local unstripped copy of your program as the first argument, or use the `file` command. Use `set sysroot` to specify the location (on the host) of target libraries (unless your GDB was compiled with the correct sysroot using `--with-sysroot`). Alternatively, you may use `set solib-search-path` to specify how GDB locates target libraries.

The symbol file and target libraries must exactly match the executable and libraries on the target, with one exception: the files on the host system should not be stripped, even if the files on the target system are. Mismatched or missing files will lead to confusing results during debugging. On GNU/Linux targets, mismatched or missing files may also prevent `gdbserver` from debugging multi-threaded programs.

20.1.3 Remote Connection Commands

GDB can communicate with the target over a serial line, a local Unix domain socket, or over an IP network using TCP or UDP. In each case, GDB uses the same protocol for debugging your program; only the medium carrying the debugging packets varies. The `target remote` and `target extended-remote` commands establish a connection to the target. Both commands accept the same arguments, which indicate the medium to use:

```
target remote serial-device
target extended-remote serial-device
```

Use *serial-device* to communicate with the target. For example, to use a serial line connected to the device named `/dev/ttyb`:

```
target remote /dev/ttyb
```

If you're using a serial line, you may want to give GDB the `--baud` option, or use the `set serial baud` command (see Section 20.4 [Remote Configuration], page 328) before the `target` command.

```
target remote local-socket
target extended-remote local-socket
```

Use *local-socket* to communicate with the target. For example, to use a local Unix domain socket bound to the file system entry `/tmp/gdb-socket0`:

```
target remote /tmp/gdb-socket0
```

Note that this command has the same form as the command to connect to a serial line. GDB will automatically determine which kind of file you have specified and will make the appropriate kind of connection. This feature is not available if the host system does not support Unix domain sockets.

```

target remote host:port
target remote [host]:port
target remote tcp:host:port
target remote tcp:[host]:port
target remote tcp4:host:port
target remote tcp6:host:port
target remote tcp6:[host]:port
target extended-remote host:port
target extended-remote [host]:port
target extended-remote tcp:host:port
target extended-remote tcp:[host]:port
target extended-remote tcp4:host:port
target extended-remote tcp6:host:port
target extended-remote tcp6:[host]:port

```

Debug using a TCP connection to *port* on *host*. The *host* may be either a host name, a numeric IPv4 address, or a numeric IPv6 address (with or without the square brackets to separate the address from the port); *port* must be a decimal number. The *host* could be the target machine itself, if it is directly connected to the net, or it might be a terminal server which in turn has a serial line to the target.

For example, to connect to port 2828 on a terminal server named **manyfarms**:

```
target remote manyfarms:2828
```

To connect to port 2828 on a terminal server whose address is 2001:0db8:85a3:0000:0000:8a2e:0370:7334, you can either use the square bracket syntax:

```
target remote [2001:0db8:85a3:0000:0000:8a2e:0370:7334]:2828
```

or explicitly specify the IPv6 protocol:

```
target remote tcp6:2001:0db8:85a3:0000:0000:8a2e:0370:7334:2828
```

This last example may be confusing to the reader, because there is no visible separation between the hostname and the port number. Therefore, we recommend the user to provide IPv6 addresses using square brackets for clarity. However, it is important to mention that for GDB there is no ambiguity: the number after the last colon is considered to be the port number.

If your remote target is actually running on the same machine as your debugger session (e.g. a simulator for your target running on the same host), you can omit the hostname. For example, to connect to port 1234 on your local machine:

```
target remote :1234
```

Note that the colon is still required here.


```
target remote udp:host:port
target remote udp:[host]:port
target remote udp4:host:port
target remote udp6:[host]:port
target extended-remote udp:host:port
target extended-remote udp:[host]:port
target extended-remote udp4:host:port
target extended-remote udp6:host:port
target extended-remote udp6:[host]:port
```

Debug using UDP packets to *port* on *host*. For example, to connect to UDP port 2828 on a terminal server named *manyfarms*:

```
target remote udp:manyfarms:2828
```

When using a UDP connection for remote debugging, you should keep in mind that the ‘U’ stands for “Unreliable”. UDP can silently drop packets on busy or unreliable networks, which will cause havoc with your debugging session.

```
target remote | command
target extended-remote | command
```

Run *command* in the background and communicate with it using a pipe. The *command* is a shell command, to be parsed and expanded by the system’s command shell, */bin/sh*; it should expect remote protocol packets on its standard input, and send replies on its standard output. You could use this to run a stand-alone simulator that speaks the remote debugging protocol, to make net connections using programs like *ssh*, or for other similar tricks.

If *command* closes its standard output (perhaps by exiting), GDB will try to send it a *SIGTERM* signal. (If the program has already exited, this will have no effect.)

Whenever GDB is waiting for the remote program, if you type the interrupt character (often *Ctrl-c*), GDB attempts to stop the program. This may or may not succeed, depending in part on the hardware and the serial drivers the remote system uses. If you type the interrupt character once again, GDB displays this prompt:

```
Interrupted while waiting for the program.
Give up (and stop debugging it)? (y or n)
```

In *target remote* mode, if you type *y*, GDB abandons the remote debugging session. (If you decide you want to try again later, you can use *target remote* again to connect once more.) If you type *n*, GDB goes back to waiting.

In *target extended-remote* mode, typing *n* will leave GDB connected to the target.

detach When you have finished debugging the remote program, you can use the **detach** command to release it from GDB control. Detaching from the target normally resumes its execution, but the results will depend on your particular remote stub. After the **detach** command in *target remote* mode, GDB is free to connect to another target. In *target extended-remote* mode, GDB is still connected to the target.

disconnect

The **disconnect** command closes the connection to the target, and the target is generally not resumed. It will wait for GDB (this instance or another one) to connect and continue debugging. After the **disconnect** command, GDB is again free to connect to another target.

monitor cmd

This command allows you to send arbitrary commands directly to the remote monitor. Since GDB doesn't care about the commands it sends like this, this command is the way to extend GDB—you can add new commands that only the external monitor will understand and implement.

20.2 Sending files to a remote system

Some remote targets offer the ability to transfer files over the same connection used to communicate with GDB. This is convenient for targets accessible through other means, e.g. GNU/Linux systems running **gdbserver** over a network interface. For other targets, e.g. embedded devices with only a single serial port, this may be the only way to upload or download files.

Not all remote targets support these commands.

remote put *hostfile targetfile*

Copy file *hostfile* from the host system (the machine running GDB) to *targetfile* on the target system.

remote get *targetfile hostfile*

Copy file *targetfile* from the target system to *hostfile* on the host system.

remote delete *targetfile*

Delete *targetfile* from the target system.

20.3 Using the gdbserver Program

gdbserver is a control program for Unix-like systems, which allows you to connect your program with a remote GDB via **target remote** or **target extended-remote**—but without linking in the usual debugging stub.

gdbserver is not a complete replacement for the debugging stubs, because it requires essentially the same operating-system facilities that GDB itself does. In fact, a system that can run **gdbserver** to connect to a remote GDB could also run GDB locally! **gdbserver** is sometimes useful nevertheless, because it is a much smaller program than GDB itself. It is also easier to port than all of GDB, so you may be able to get started more quickly on a new system by using **gdbserver**. Finally, if you develop code for real-time systems, you may find that the tradeoffs involved in real-time operation make it more convenient to do as much development work as possible on another system, for example by cross-compiling. You can use **gdbserver** to make a similar choice for debugging.

GDB and **gdbserver** communicate via either a serial line or a TCP connection, using the standard GDB remote serial protocol.

Warning: **gdbserver** does not have any built-in security. Do not run **gdbserver** connected to any public network; a GDB connection to **gdbserver**

provides access to the target system with the same privileges as the user running `gdbserver`.

20.3.1 Running `gdbserver`

Run `gdbserver` on the target system. You need a copy of the program you want to debug, including any libraries it requires. `gdbserver` does not need your program's symbol table, so you can strip the program if necessary to save space. GDB on the host system does all the symbol handling.

To use the server, you must tell it how to communicate with GDB; the name of your program; and the arguments for your program. The usual syntax is:

```
target> gdbserver comm program [ args ... ]
```

`comm` is either a device name (to use a serial line), or a TCP hostname and portnumber, or `-` or `stdio` to use `stdin/stdout` of `gdbserver`. For example, to debug Emacs with the argument `'foo.txt'` and communicate with GDB over the serial port `/dev/com1`:

```
target> gdbserver /dev/com1 emacs foo.txt
```

`gdbserver` waits passively for the host GDB to communicate with it.

To use a TCP connection instead of a serial line:

```
target> gdbserver host:2345 emacs foo.txt
```

The only difference from the previous example is the first argument, specifying that you are communicating with the host GDB via TCP. The `'host:2345'` argument means that `gdbserver` is to expect a TCP connection from machine `'host'` to local TCP port 2345. (Currently, the `'host'` part is ignored.) You can choose any number you want for the port number as long as it does not conflict with any TCP ports already in use on the target system (for example, 23 is reserved for `telnet`).¹ You must use the same port number with the host GDB `target remote` command.

The `stdio` connection is useful when starting `gdbserver` with `ssh`:

```
(gdb) target remote | ssh -T hostname gdbserver - hello
```

The `'-T'` option to `ssh` is provided because we don't need a remote pty, and we don't want escape-character handling. `Ssh` does this by default when a command is provided, the flag is provided to make it explicit. You could elide it if you want to.

Programs started with `stdio`-connected `gdbserver` have `/dev/null` for `stdin`, and `stdout`, `stderr` are sent back to `gdb` for display through a pipe connected to `gdbserver`. Both `stdout` and `stderr` use the same pipe.

20.3.1.1 Attaching to a Running Program

On some targets, `gdbserver` can also attach to running programs. This is accomplished via the `--attach` argument. The syntax is:

```
target> gdbserver --attach comm pid
```

`pid` is the process ID of a currently running process. It isn't necessary to point `gdbserver` at a binary for the running process.

In `target extended-remote` mode, you can also attach using the GDB `attach` command (see [Attaching in Types of Remote Connections], page 318).

¹ If you choose a port number that conflicts with another service, `gdbserver` prints an error message and exits.

You can debug processes by name instead of process ID if your target has the `pidof` utility:

```
target> gdbserver --attach comm 'pidof program'
```

In case more than one copy of *program* is running, or *program* has multiple threads, most versions of `pidof` support the `-s` option to only return the first process ID.

20.3.1.2 TCP port allocation lifecycle of gdbserver

This section applies only when `gdbserver` is run to listen on a TCP port.

`gdbserver` normally terminates after all of its debugged processes have terminated in *target remote* mode. On the other hand, for *target extended-remote*, `gdbserver` stays running even with no processes left. GDB normally terminates the spawned debugged process on its exit, which normally also terminates `gdbserver` in the *target remote* mode. Therefore, when the connection drops unexpectedly, and GDB cannot ask `gdbserver` to kill its debugged processes, `gdbserver` stays running even in the *target remote* mode.

When `gdbserver` stays running, GDB can connect to it again later. Such reconnecting is useful for features like [disconnected tracing], page 214. For completeness, at most one GDB can be connected at a time.

By default, `gdbserver` keeps the listening TCP port open, so that subsequent connections are possible. However, if you start `gdbserver` with the `--once` option, it will stop listening for any further connection attempts after connecting to the first GDB session. This means no further connections to `gdbserver` will be possible after the first one. It also means `gdbserver` will terminate after the first connection with remote GDB has closed, even for unexpectedly closed connections and even in the *target extended-remote* mode. The `--once` option allows reusing the same port number for connecting to multiple instances of `gdbserver` running on the same host, since each instance closes its port after the first connection.

20.3.1.3 Other Command-Line Arguments for gdbserver

You can use the `--multi` option to start `gdbserver` without specifying a program to debug or a process to attach to. Then you can attach in *target extended-remote* mode and run or attach to a program. For more information, see [–multi Option in Types of Remote Connections], page 318.

The `--debug[=option1,option2,...]` option tells `gdbserver` to display extra diagnostic information about the debugging process. The options (*option1*, *option2*, etc) control for which areas of `gdbserver` additional information will be displayed, possible values are:

- all** This enables all available diagnostic output.
- threads** This enables diagnostic output related to threading. Currently other general diagnostic output is included in this category, but this could change in future releases of `gdbserver`.
- event-loop** This enables event-loop specific diagnostic output.
- remote** This enables diagnostic output related to the transfer of remote protocol packets too and from the debugger.

If no options are passed to `--debug` then this is treated as equivalent to `--debug=threads`. This could change in future releases of `gdbserver`. The options passed to `--debug` are processed left to right, and individual options can be prefixed with the `-` (minus) character to disable diagnostic output from this area, so it is possible to use:

```
target> gdbserver --debug=all,-event-loop
```

In order to enable all diagnostic output except that for the event-loop.

The `--debug-file=filename` option tells `gdbserver` to write any debug output to the given *filename*. These options are intended for `gdbserver` development and for bug reports to the developers.

The `--debug-format=option1[,option2,...]` option tells `gdbserver` to include additional information in each output. Possible options are:

none Turn off all extra information in debugging output.

all Turn on all extra information in debugging output.

timestamps
 Include a timestamp in each line of debugging output.

Options are processed in order. Thus, for example, if **none** appears last then no additional information is added to debugging output.

The `--wrapper` option specifies a wrapper to launch programs for debugging. The option should be followed by the name of the wrapper, then any command-line arguments to pass to the wrapper, then `--` indicating the end of the wrapper arguments.

`gdbserver` runs the specified wrapper program with a combined command line including the wrapper arguments, then the name of the program to debug, then any arguments to the program. The wrapper runs until it executes your program, and then GDB gains control.

You can use any program that eventually calls `execve` with its arguments as a wrapper. Several standard Unix utilities do this, e.g. `env` and `nohup`. Any Unix shell script ending with `exec "$@"` will also work.

For example, you can use `env` to pass an environment variable to the debugged program, without setting the variable in `gdbserver`'s environment:

```
$ gdbserver --wrapper env LD_PRELOAD=libtest.so -- :2222 ./testprog
```

The `--selftest` option runs the self tests in `gdbserver`:

```
$ gdbserver --selftest
Ran 2 unit tests, 0 failed
```

These tests are disabled in release.

20.3.2 Connecting to gdbserver

The basic procedure for connecting to the remote target is:

- Run GDB on the host system.
- Make sure you have the necessary symbol files (see [Host and target files], page 318). Load symbols for your application using the `file` command before you connect. Use `set sysroot` to locate target libraries (unless your GDB was compiled with the correct `sysroot` using `--with-sysroot`).

- Connect to your target (see Section 20.1 [Connecting to a Remote Target], page 317). For TCP connections, you must start up `gdbserver` prior to using the `target` command. Otherwise you may get an error whose text depends on the host system, but which usually looks something like ‘`Connection refused`’. Don’t use the `load` command in GDB when using `target remote` mode, since the program is already on the target.

20.3.3 Monitor Commands for `gdbserver`

During a GDB session using `gdbserver`, you can use the `monitor` command to send special requests to `gdbserver`. Here are the available commands.

`monitor help`

List the available monitor commands.

`monitor set debug off`

Disable all internal logging from `gdbserver`.

`monitor set debug on`

Enable some general logging from within `gdbserver`. Currently this is equivalent to `monitor set debug threads on`, but this might change in future releases of `gdbserver`.

`monitor set debug threads off`

`monitor set debug threads on`

Disable or enable specific logging messages associated with thread handling in `gdbserver`. Currently this category also includes additional output not specifically related to thread handling, this could change in future releases of `gdbserver`.

`monitor set debug remote off`

`monitor set debug remote on`

Disable or enable specific logging messages associated with the remote protocol (see Appendix E [Remote Protocol], page 795).

`monitor set debug event-loop off`

`monitor set debug event-loop on`

Disable or enable specific logging messages associated with `gdbserver`’s event-loop.

`monitor set debug-file filename`

`monitor set debug-file`

Send any debug output to the given file, or to `stderr`.

`monitor set debug-format option1[,option2,...]`

Specify additional text to add to debugging messages. Possible options are:

`none` Turn off all extra information in debugging output.

`all` Turn on all extra information in debugging output.

`timestamps`

Include a timestamp in each line of debugging output.

Options are processed in order. Thus, for example, if **none** appears last then no additional information is added to debugging output.

monitor set libthread-db-search-path [PATH]

When this command is issued, *path* is a colon-separated list of directories to search for **libthread_db** (see Section 4.10 [set libthread-db-search-path], page 48). If you omit *path*, ‘libthread-db-search-path’ will be reset to its default value.

The special entry ‘\$pdir’ for ‘libthread-db-search-path’ is not supported in **gdbserver**.

monitor exit

Tell **gdbserver** to exit immediately. This command should be followed by **disconnect** to close the debugging session. **gdbserver** will detach from any attached processes and kill any processes it created. Use **monitor exit** to terminate **gdbserver** at the end of a multi-process mode debug session.

20.3.4 Tracepoints support in gdbserver

On some targets, **gdbserver** supports tracepoints and fast tracepoints.

For fast tracepoints to work, a special library called the *in-process agent* (IPA), must be loaded in the inferior process. This library is built and distributed as an integral part of **gdbserver**.

There are several ways to load the in-process agent in your program:

Specifying it as dependency at link time

You can link your program dynamically with the in-process agent library. On most systems, this is accomplished by adding **-linproctrace** to the link command.

Using the system’s preloading mechanisms

You can force loading the in-process agent at startup time by using your system’s support for preloading shared libraries. Many Unixes support the concept of preloading user defined libraries. In most cases, you do that by specifying **LD_PRELOAD=libinproctrace.so** in the environment. See also the description of **gdbserver**’s **--wrapper** command line option.

Using GDB to force loading the agent at run time

On some systems, you can force the inferior to load a shared library, by calling a dynamic loader function in the inferior that takes care of dynamically looking up and loading a shared library. On most Unix systems, the function is **dlopen**. You’ll use the **call** command for that. For example:

```
(gdb) call dlopen ("libinproctrace.so", ...)
```

Note that on most Unix systems, for the **dlopen** function to be available, the program needs to be linked with **-ldl**.

On systems that have a userspace dynamic loader, like most Unix systems, when you connect to **gdbserver** using **target remote**, you’ll find that the program is stopped at the dynamic loader’s entry point, and no shared library has been loaded in the program’s address space yet, including the in-process agent. In that case, before being able to use any

of the fast or static tracepoints features, you need to let the loader run and load the shared libraries. The simplest way to do that is to run the program to the main procedure. E.g., if debugging a C or C++ program, start `gdbserver` like so:

```
$ gdbserver :9999 myprogram
```

Start GDB and connect to `gdbserver` like so, and run to main:

```
$ gdb myprogram
(gdb) target remote myhost:9999
0x00007f215893ba60 in ?? () from /lib64/ld-linux-x86-64.so.2
(gdb) b main
(gdb) continue
```

The in-process tracing agent library should now be loaded into the process; you can confirm it with the `info sharedlibrary` command, which will list `libinproctrace.so` as loaded in the process. You are now ready to install fast tracepoints, list static tracepoint markers, probe static tracepoints markers, and start tracing.

20.4 Remote Configuration

This section documents the configuration options available when debugging remote programs. For the options related to the File I/O extensions of the remote protocol, see [system], page 861.

set remoteaddresssize *bits*

Set the maximum size of address in a memory packet to the specified number of bits. GDB will mask off the address bits above that number, when it passes addresses to the remote target. The default value is the number of bits in the target's address.

show remoteaddresssize

Show the current value of remote address size in bits.

set serial baud *n*

Set the baud rate for the remote serial I/O to *n* baud. The value is used to set the speed of the serial port used for debugging remote targets.

show serial baud

Show the current speed of the remote connection.

set serial parity *parity*

Set the parity for the remote serial I/O. Supported values of *parity* are: `even`, `none`, and `odd`. The default is `none`.

show serial parity

Show the current parity of the serial port.

set remotebreak

If set to on, GDB sends a `BREAK` signal to the remote when you type `Ctrl-c` to interrupt the program running on the remote. If set to off, GDB sends the 'Ctrl-C' character instead. The default is off, since most remote systems expect to see 'Ctrl-C' as the interrupt signal.

show remotebreak

Show whether GDB sends `BREAK` or 'Ctrl-C' to interrupt the remote program.

`set remoteflow on`
`set remoteflow off`
Enable or disable hardware flow control (RTS/CTS) on the serial port used to communicate to the remote target.

`show remoteflow`
Show the current setting of hardware flow control.

`set remotelogbase base`
Set the base (a.k.a. radix) of logging serial protocol communications to *base*. Supported values of *base* are: `ascii`, `octal`, and `hex`. The default is `ascii`.

`show remotelogbase`
Show the current setting of the radix for logging remote serial protocol.

`set remotelogfile file`
Record remote serial communications on the named *file*. The default is not to record at all.

`show remotelogfile`
Show the current setting of the file name on which to record the serial communications.

`set remotetimeout num`
Set the timeout limit to wait for the remote target to respond to *num* seconds. The default is 2 seconds.

`show remotetimeout`
Show the current number of seconds to wait for the remote target responses.

`set remote hardware-watchpoint-limit limit`
`set remote hardware-breakpoint-limit limit`
Restrict GDB to using *limit* remote hardware watchpoints or breakpoints. The *limit* can be set to 0 to disable hardware watchpoints or breakpoints, and `unlimited` for unlimited watchpoints or breakpoints.

`show remote hardware-watchpoint-limit`
`show remote hardware-breakpoint-limit`
Show the current limit for the number of hardware watchpoints or breakpoints that GDB can use.

`set remote hardware-watchpoint-length-limit limit`
Restrict GDB to using *limit* bytes for the maximum length of a remote hardware watchpoint. A *limit* of 0 disables hardware watchpoints and `unlimited` allows watchpoints of any length.

`show remote hardware-watchpoint-length-limit`
Show the current limit (in bytes) of the maximum length of a remote hardware watchpoint.

`set remote exec-file filename`
`show remote exec-file`
Select the file used for `run` with `target extended-remote`. This should be set to a filename valid on the target system. If it is not set, the target will use

a default filename (e.g. the last program run, or a filename derived from the current executable if GDB and the remote can see the same file system).

When connecting to a remote system, with *target extended-remote* or *target remote*, if the remote server supports the `qExecAndArgs` packet (see [qExecAndArgs], page 838), then the *remote exec-file* will be updated to reflect the executable currently running on the remote system.

`set remote interrupt-sequence`

Allow the user to select one of ‘Ctrl-C’, a `BREAK` or ‘`BREAK-g`’ as the sequence to the remote target in order to interrupt the execution. ‘Ctrl-C’ is a default. Some system prefers `BREAK` which is high level of serial line for some certain time. Linux kernel prefers ‘`BREAK-g`’, a.k.a Magic SysRq `g`. It is `BREAK` signal followed by character `g`.

`show remote interrupt-sequence`

Show which of ‘Ctrl-C’, `BREAK` or `BREAK-g` is sent by GDB to interrupt the remote program. `BREAK-g` is `BREAK` signal followed by `g` and also known as Magic SysRq `g`.

`set remote interrupt-on-connect`

Specify whether interrupt-sequence is sent to remote target when GDB connects to it. This is mostly needed when you debug Linux kernel. Linux kernel expects `BREAK` followed by `g` which is known as Magic SysRq `g` in order to connect GDB.

`show remote interrupt-on-connect`

Show whether interrupt-sequence is sent to remote target when GDB connects to it.

`set tcp auto-retry on`

Enable auto-retry for remote TCP connections. This is useful if the remote debugging agent is launched in parallel with GDB; there is a race condition because the agent may not become ready to accept the connection before GDB attempts to connect. When auto-retry is enabled, if the initial attempt to connect fails, GDB reattempts to establish the connection using the timeout specified by `set tcp connect-timeout`.

`set tcp auto-retry off`

Do not auto-retry failed TCP connections.

`show tcp auto-retry`

Show the current auto-retry setting.

`set tcp connect-timeout seconds`

`set tcp connect-timeout unlimited`

Set the timeout for establishing a TCP connection to the remote target to *seconds*. The timeout affects both polling to retry failed connections (enabled by `set tcp auto-retry on`) and waiting for connections that are merely slow to complete, and represents an approximate cumulative value. If *seconds* is `unlimited`, there is no timeout and GDB will keep attempting to establish a connection forever, unless interrupted with `Ctrl-c`. The default is 15 seconds.

show tcp connect-timeout

Show the current connection timeout setting.

The GDB remote protocol autodetects the packets supported by your debugging stub. If you need to override the autodetection, you can use these commands to enable or disable individual packets. Each packet can be set to ‘on’ (the remote target supports this packet), ‘off’ (the remote target does not support this packet), or ‘auto’ (detect remote target support for this packet). They all default to ‘auto’. For more information about each packet, see Appendix E [Remote Protocol], page 795.

During normal use, you should not have to use any of these commands. If you do, that may be a bug in your remote debugging stub, or a bug in GDB. You may want to report the problem to the GDB developers.

For each packet *name*, the command to enable or disable the packet is **set remote name-packet**. If you configure a packet, the configuration will apply for all future remote targets if no target is selected. In case there is a target selected, only the configuration of the current target is changed. All other existing remote targets’ features are not affected. The command to print the current configuration of a packet is **show remote name-packet**. It displays the current remote target’s configuration. If no remote target is selected, the default configuration for future connections is shown. The available settings are:

Command Name	Remote Packet	Related Features
fetch-register	p	info registers
set-register	P	set
binary-download	X	load, set
read-aux-vector	qXfer:auxv:read	info auxv
symbol-lookup	qSymbol	Detecting multiple threads
attach	vAttach	attach
verbose-resume	vCont	Stepping or resuming multiple threads
run	vRun	run
software-breakpoint	Z0	break
hardware-breakpoint	Z1	hbreak
write-watchpoint	Z2	watch
read-watchpoint	Z3	rwatch

access-watchpoint	Z4	awatch
pid-to-exec-file	qXfer:exec-file:read	attach, run
target-features	qXfer:features:read	set architecture
library-info	qXfer:libraries:read	info sharedlibrary
memory-map	qXfer:memory-map:read	info mem
read-sdata-object	qXfer:sdata:read	print \$_sdata
read-siginfo-object	qXfer:siginfo:read	print \$_siginfo
write-siginfo-object	qXfer:siginfo:write	set \$_siginfo
threads	qXfer:threads:read	info threads
get-thread-local-storage-address	qGetTLSAddr	Displaying __thread variables
get-thread-information-block-address	qGetTIBAddr	Display MS- Windows Thread Information Block.
search-memory	qSearch:memory	find
supported-packets	qSupported	Remote com- munications parameters
catch-syscalls	QCatchSyscalls	catch syscall
pass-signals	QPassSignals	handle <i>signal</i>
program-signals	QProgramSignals	handle <i>signal</i>
hostio-close-packet	vFile:close	remote get, remote put
hostio-open-packet	vFile:open	remote get, remote put
hostio-pread-packet	vFile:pread	remote get, remote put

hostio-pwrite-packet	vFile:pwrite	remote get, remote put
hostio-unlink-packet	vFile:unlink	remote delete
hostio-readlink-packet	vFile:readlink	Host I/O
hostio-fstat-packet	vFile:fstat	Host I/O
hostio-stat-packet	vFile:stat	Host I/O
hostio-lstat-packet	vFile:lstat	Host I/O
hostio-setfs-packet	vFile:setfs	Host I/O
noack-packet	QStartNoAckMode	Packet acknowledgment
osdata	qXfer:osdata:read	info os
query-attached	qAttached	Querying process state. remote attach
trace-buffer-size	QTBuffer:size	set trace-buffer-size
trace-status	qTStatus	tstatus
traceframe-info	qXfer:traceframe-info:read	Traceframe info
install-in-trace	InstallInTrace	Install tracepoint in tracing
disable-randomization	QDisableRandomization	set disable-randomization
startup-with-shell	QStartupWithShell	set startup-with-shell
environment-hex-encoded	QEnvironmentHexEncoded	set environment
environment-unset	QEnvironmentUnset	unset environment

environment-reset	QEnvironmentReset	Reset the inferior environment (i.e., unset user-set variables)
set-working-dir	QSetWorkingDir	set cwd
conditional-breakpointsZ0 and Z1 packet		Support for target-side breakpoint condition evaluation
multiprocess-extensions	multiprocess extensions	Debug multiple processes and remote process PID awareness
swbreak-feature	swbreak stop reason	break
hwbreak-feature	hwbreak stop reason	hbreak
fork-event-feature	fork stop reason	fork
vfork-event-feature	vfork stop reason	vfork
exec-event-feature	exec stop reason	exec
thread-events	QThreadEvents	Tracking thread life-time.
thread-options	QThreadOptions	Set thread event reporting options.
no-resumed-stop-reply	no resumed thread left stop reply	Tracking thread life-time.
multiple-watchpoint-addresses	multiple watchpoint stop reasons	Allow multiple, ambiguous, watchpoint addresses in 'T' stop reply.

The number of bytes per memory-read or memory-write packet for a remote target can be configured using the commands `set remote memory-read-packet-size` and `set remote memory-write-packet-size`. If set to '0' (zero) the default packet size will

be used. The actual limit is further reduced depending on the target. Specify ‘**fixed**’ to disable the target-dependent restriction and ‘**limit**’ to enable it. Similar to the enabling and disabling of remote packets, the command applies to the currently selected target (if available). If no remote target is selected, it applies to all future remote connections. The configuration of the selected target can be displayed using the commands **show remote memory-read-packet-size** and **show remote memory-write-packet-size**. If no remote target is selected, the default configuration for future connections is shown.

20.5 Implementing a Remote Stub

The stub files provided with GDB implement the target side of the communication protocol, and the GDB side is implemented in the GDB source file **remote.c**. Normally, you can simply allow these subroutines to communicate, and ignore the details. (If you’re implementing your own stub file, you can still ignore the details: start with one of the existing stub files. **sparc-stub.c** is the best organized, and therefore the easiest to read.)

To debug a program running on another machine (the debugging *target* machine), you must first arrange for all the usual prerequisites for the program to run by itself. For example, for a C program, you need:

1. A startup routine to set up the C runtime environment; these usually have a name like **crt0**. The startup routine may be supplied by your hardware supplier, or you may have to write your own.
2. A C subroutine library to support your program’s subroutine calls, notably managing input and output.
3. A way of getting your program to the other machine—for example, a download program. These are often supplied by the hardware manufacturer, but you may have to write your own from hardware documentation.

The next step is to arrange for your program to use a serial port to communicate with the machine where GDB is running (the *host* machine). In general terms, the scheme looks like this:

On the host,

GDB already understands how to use this protocol; when everything else is set up, you can simply use the ‘**target remote**’ command (see Chapter 19 [Specifying a Debugging Target], page 313).

On the target,

you must link with your program a few special-purpose subroutines that implement the GDB remote serial protocol. The file containing these subroutines is called a *debugging stub*.

On certain remote targets, you can use an auxiliary program **gdbserver** instead of linking a stub into your program. See Section 20.3 [Using the **gdbserver** Program], page 322, for details.

The debugging stub is specific to the architecture of the remote machine; for example, use **sparc-stub.c** to debug programs on SPARC boards.

These working remote stubs are distributed with GDB:

i386-stub.c

For Intel 386 and compatible architectures.

`m68k-stub.c`

For Motorola 680x0 architectures.

`sh-stub.c`

For Renesas SH architectures.

`sparc-stub.c`

For SPARC architectures.

`sparcl-stub.c`

For Fujitsu SPARCLITE architectures.

The README file in the GDB distribution may list other recently added stubs.

20.5.1 What the Stub Can Do for You

The debugging stub for your architecture supplies these three subroutines:

set_debug_traps

This routine arranges for **handle_exception** to run when your program stops. You must call this subroutine explicitly in your program's startup code.

handle_exception

This is the central workhorse, but your program never calls it explicitly—the setup code arranges for **handle_exception** to run when a trap is triggered.

handle_exception takes control when your program stops during execution (for example, on a breakpoint), and mediates communications with GDB on the host machine. This is where the communications protocol is implemented; **handle_exception** acts as the GDB representative on the target machine. It begins by sending summary information on the state of your program, then continues to execute, retrieving and transmitting any information GDB needs, until you execute a GDB command that makes your program resume; at that point, **handle_exception** returns control to your own code on the target machine.

breakpoint

Use this auxiliary subroutine to make your program contain a breakpoint. Depending on the particular situation, this may be the only way for GDB to get control. For instance, if your target machine has some sort of interrupt button, you won't need to call this; pressing the interrupt button transfers control to **handle_exception**—in effect, to GDB. On some machines, simply receiving characters on the serial port may also trigger a trap; again, in that situation, you don't need to call **breakpoint** from your own program—simply running 'target remote' from the host GDB session gets control.

Call **breakpoint** if none of these is true, or if you simply want to make certain your program stops at a predetermined point for the start of your debugging session.

20.5.2 What You Must Do for the Stub

The debugging stubs that come with GDB are set up for a particular chip architecture, but they have no information about the rest of your debugging target machine.

First of all you need to tell the stub how to communicate with the serial port.

int getDebugChar()

Write this subroutine to read a single character from the serial port. It may be identical to `getchar` for your target system; a different name is used to allow you to distinguish the two if you wish.

void putDebugChar(int)

Write this subroutine to write a single character to the serial port. It may be identical to `putchar` for your target system; a different name is used to allow you to distinguish the two if you wish.

If you want GDB to be able to stop your program while it is running, you need to use an interrupt-driven serial driver, and arrange for it to stop when it receives a `^C` (`'\003'`, the control-C character). That is the character which GDB uses to tell the remote system to stop.

Getting the debugging target to return the proper status to GDB probably requires changes to the standard stub; one quick and dirty way is to just execute a breakpoint instruction (the “dirty” part is that GDB reports a `SIGTRAP` instead of a `SIGINT`).

Other routines you need to supply are:

void exceptionHandler (int exception_number, void *exception_address)

Write this function to install `exception_address` in the exception handling tables. You need to do this because the stub does not have any way of knowing what the exception handling tables on your target system are like (for example, the processor’s table might be in ROM, containing entries which point to a table in RAM). The `exception_number` specifies the exception which should be changed; its meaning is architecture-dependent (for example, different numbers might represent divide by zero, misaligned access, etc). When this exception occurs, control should be transferred directly to `exception_address`, and the processor state (stack, registers, and so on) should be just as it is when a processor exception occurs. So if you want to use a jump instruction to reach `exception_address`, it should be a simple jump, not a jump to subroutine.

For the 386, `exception_address` should be installed as an interrupt gate so that interrupts are masked while the handler runs. The gate should be at privilege level 0 (the most privileged level). The SPARC and 68k stubs are able to mask interrupts themselves without help from `exceptionHandler`.

void flush_i_cache()

On SPARC and SPARCLITE only, write this subroutine to flush the instruction cache, if any, on your target machine. If there is no instruction cache, this subroutine may be a no-op.

On target machines that have instruction caches, GDB requires this function to make certain that the state of your program is stable.

You must also make sure this library routine is available:

void *memset(void *, int, int)

This is the standard library function `memset` that sets an area of memory to a known value. If you have one of the free versions of `libc.a`, `memset` can be found

there; otherwise, you must either obtain it from your hardware manufacturer, or write your own.

If you do not use the GNU C compiler, you may need other standard library subroutines as well; this varies from one stub to another, but in general the stubs are likely to use any of the common library subroutines which GCC generates as inline code.

20.5.3 Putting it All Together

In summary, when your program is ready to debug, you must follow these steps.

1. Make sure you have defined the supporting low-level routines (see Section 20.5.2 [What You Must Do for the Stub], page 336):

```
getDebugChar, putDebugChar,
flush_i_cache, memset, exceptionHandler.
```

2. Insert these lines in your program's startup code, before the main procedure is called:

```
set_debug_traps();
breakpoint();
```

On some machines, when a breakpoint trap is raised, the hardware automatically makes the PC point to the instruction after the breakpoint. If your machine doesn't do that, you may need to adjust `handle_exception` to arrange for it to return to the instruction after the breakpoint on this first invocation, so that your program doesn't keep hitting the initial breakpoint instead of making progress.

3. For the 680x0 stub only, you need to provide a variable called `exceptionHook`. Normally you just use:

```
void (*exceptionHook)() = 0;
```

but if before calling `set_debug_traps`, you set it to point to a function in your program, that function is called when GDB continues after stopping on a trap (for example, bus error). The function indicated by `exceptionHook` is called with one parameter: an `int` which is the exception number.

4. Compile and link together: your program, the GDB debugging stub for your target architecture, and the supporting subroutines.
5. Make sure you have a serial connection between your target machine and the GDB host, and identify the serial port on the host.
6. Download your program to your target machine (or get it there by whatever means the manufacturer provides), and start it.
7. Start GDB on the host, and connect to the target (see Section 20.1 [Connecting to a Remote Target], page 317).

21 Configuration-Specific Information

While nearly all GDB commands are available for all native and cross versions of the debugger, there are some exceptions. This chapter describes things that are only available in certain configurations.

There are three major categories of configurations: native configurations, where the host and target are the same, embedded operating system configurations, which are usually the same for several different processor architectures, and bare embedded processors, which are quite different from each other.

21.1 Native

This section describes details specific to particular native configurations.

21.1.1 BSD libkvm Interface

BSD-derived systems (FreeBSD/NetBSD/OpenBSD) have a kernel memory interface that provides a uniform interface for accessing kernel virtual memory images, including live systems and crash dumps. GDB uses this interface to allow you to debug live kernels and kernel crash dumps on many native BSD configurations. This is implemented as a special **kvm** debugging target. For debugging a live system, load the currently running kernel into GDB and connect to the **kvm** target:

```
(gdb) target kvm
```

For debugging crash dumps, provide the file name of the crash dump as an argument:

```
(gdb) target kvm /var/crash/bsd.0
```

Once connected to the **kvm** target, the following commands are available:

kvm pcb Set current context from the *Process Control Block* (PCB) address.

kvm proc Set current context from proc address. This command isn't available on modern FreeBSD systems.

21.1.2 Process Information

Some operating systems provide interfaces to fetch additional information about running processes beyond memory and per-thread register state. If GDB is configured for an operating system with a supported interface, the command **info proc** is available to report information about the process running your program, or about any process running on your system.

One supported interface is a facility called **/proc** that can be used to examine the image of a running process using file-system subroutines. This facility is supported on GNU/Linux and Solaris systems.

On FreeBSD and NetBSD systems, system control nodes are used to query process information.

In addition, some systems may provide additional process information in core files. Note that a core file may include a subset of the information available from a live process. Process information is currently available from cores created on GNU/Linux and FreeBSD systems.

info proc

info proc *process-id*

Summarize available information about a process. If a process ID is specified by *process-id*, display information about that process; otherwise display information about the program being debugged. The summary includes the debugged process ID, the command line used to invoke it, its current working directory, and its executable file's absolute file name.

On some systems, *process-id* can be of the form '*[pid]/tid*' which specifies a certain thread ID within a process. If the optional *pid* part is missing, it means a thread from the process being debugged (the leading '/' still needs to be present, or else GDB will interpret the number as a process ID rather than a thread ID).

info proc cmdline

Show the original command line of the process. This command is supported on GNU/Linux, FreeBSD and NetBSD.

info proc cwd

Show the current working directory of the process. This command is supported on GNU/Linux, FreeBSD and NetBSD.

info proc environ

Show the initial environment variables of the process. This command is supported on GNU/Linux. Note that the information printed by **info proc environ** is not necessarily the same as **show environment**, which lists the environment variables that will be given to the program next time it is started by GDB. The two information may differ in particular when GDB attached to a running program that was started in an environment different than GDB's.

info proc exe

Show the name of executable of the process. This command is supported on GNU/Linux, FreeBSD and NetBSD.

info proc files

Show the file descriptors open by the process. For each open file descriptor, GDB shows its number, type (file, directory, character device, socket), file pointer offset, and the name of the resource open on the descriptor. The resource name can be a file name (for files, directories, and devices) or a protocol followed by socket address (for network connections). This command is supported on FreeBSD.

This example shows the open file descriptors for a process using a tty for standard input and output as well as two network sockets:

```
(gdb) info proc files 22136
process 22136
Open files:
```

FD	Type	Offset	Flags	Name
text	file	-	r-----	/usr/bin/ssh
ctty	chr	-	rw-----	/dev/pts/20
cwd	dir	-	r-----	/usr/home/john
root	dir	-	r-----	/

```

0   chr  0x32933a4 rw----- /dev/pts/20
1   chr  0x32933a4 rw----- /dev/pts/20
2   chr  0x32933a4 rw----- /dev/pts/20
3  socket      0x0 rw----n-- tcp4 10.0.1.2:53014 -> 10.0.1.10:22
4  socket      0x0 rw----- unix stream:/tmp/ssh-FIt89oAz0n5f/agent.2456

```

info proc mappings

Report the memory address space ranges accessible in a process. On Solaris, FreeBSD and NetBSD systems, each memory range includes information on whether the process has read, write, or execute access rights to each range. On GNU/Linux, FreeBSD and NetBSD systems, each memory range includes the object file which is mapped to that range.

info proc stat**info proc status**

Show additional process-related information, including the user ID and group ID; virtual memory usage; the signals that are pending, blocked, and ignored; its TTY; its consumption of system and user time; its stack size; its ‘nice’ value; etc. These commands are supported on GNU/Linux, FreeBSD and NetBSD.

For GNU/Linux systems, see the ‘proc’ man page for more information (type *man 5 proc* from your shell prompt).

For FreeBSD and NetBSD systems, **info proc stat** is an alias for **info proc status**.

info proc all

Show all the information about the process described under all of the above **info proc** subcommands.

set procfs-trace

This command enables and disables tracing of **procfs** API calls.

show procfs-trace

Show the current state of **procfs** API call tracing.

set procfs-file file

Tell GDB to write **procfs** API trace to the named *file*. GDB appends the trace info to the previous contents of the file. The default is to display the trace on the standard output.

show procfs-file

Show the file to which **procfs** API trace is written.

proc-trace-entry**proc-trace-exit****proc-untrace-entry****proc-untrace-exit**

These commands enable and disable tracing of entries into and exits from the **syscall** interface.

21.1.3 Features for Debugging DJGPP Programs

DJGPP is a port of the GNU development tools to MS-DOS and MS-Windows. DJGPP programs are 32-bit protected-mode programs that use the *DPMI* (DOS Protected-Mode Interface) API to run on top of real-mode DOS systems and their emulations.

GDB supports native debugging of DJGPP programs, and defines a few commands specific to the DJGPP port. This subsection describes those commands.

info dos This is a prefix of DJGPP-specific commands which print information about the target system and important OS structures.

info dos sysinfo

This command displays assorted information about the underlying platform: the CPU type and features, the OS version and flavor, the DPMI version, and the available conventional and DPMI memory.

info dos gdt

info dos ldt

info dos idt

These 3 commands display entries from, respectively, Global, Local, and Interrupt Descriptor Tables (GDT, LDT, and IDT). The descriptor tables are data structures which store a descriptor for each segment that is currently in use. The segment's selector is an index into a descriptor table; the table entry for that index holds the descriptor's base address and limit, and its attributes and access rights.

A typical DJGPP program uses 3 segments: a code segment, a data segment (used for both data and the stack), and a DOS segment (which allows access to DOS/BIOS data structures and absolute addresses in conventional memory). However, the DPMI host will usually define additional segments in order to support the DPMI environment.

These commands allow to display entries from the descriptor tables. Without an argument, all entries from the specified table are displayed. An argument, which should be an integer expression, means display a single entry whose index is given by the argument. For example, here's a convenient way to display information about the debugged program's data segment:

```
(gdb) info dos ldt $ds
0x13f: base=0x11970000 limit=0x0009ffff 32-Bit Data (Read/Write, Exp-up)
```

This comes in handy when you want to see whether a pointer is outside the data segment's limit (i.e. *garbled*).

info dos pde

info dos pte

These two commands display entries from, respectively, the Page Directory and the Page Tables. Page Directories and Page Tables are data structures which control how virtual memory addresses are mapped into physical addresses. A Page Table includes an entry for every page of memory that is mapped into the program's address space; there may be several Page Tables, each one holding up to 4096 entries. A Page Directory has up to 4096 entries, one each for every Page Table that is currently in use.

Without an argument, **info dos pde** displays the entire Page Directory, and **info dos pte** displays all the entries in all of the Page Tables. An argument, an integer expression, given to the **info dos pde** command means display only that entry from the Page Directory table. An argument given to the **info dos**

pte command means display entries from a single Page Table, the one pointed to by the specified entry in the Page Directory.

These commands are useful when your program uses *DMA* (Direct Memory Access), which needs physical addresses to program the DMA controller.

These commands are supported only with some DPMI servers.

info dos address-pte *addr*

This command displays the Page Table entry for a specified linear address. The argument *addr* is a linear address which should already have the appropriate segment's base address added to it, because this command accepts addresses which may belong to *any* segment. For example, here's how to display the Page Table entry for the page where a variable *i* is stored:

```
(gdb) info dos address-pte __djgpp_base_address + (char *)&i
Page Table entry for address 0x11a00d30:
Base=0x02698000 Dirty Acc. Not-Cached Write-Back Usr Read-Write +0xd30
```

This says that *i* is stored at offset 0xd30 from the page whose physical base address is 0x02698000, and shows all the attributes of that page.

Note that you must cast the addresses of variables to a **char ***, since otherwise the value of `__djgpp_base_address`, the base address of all variables and functions in a DJGPP program, will be added using the rules of C pointer arithmetic: if *i* is declared an **int**, GDB will add 4 times the value of `__djgpp_base_address` to the address of *i*.

Here's another example, it displays the Page Table entry for the transfer buffer:

```
(gdb) info dos address-pte *((unsigned *)&_go32_info_block + 3)
Page Table entry for address 0x29110:
Base=0x00029000 Dirty Acc. Not-Cached Write-Back Usr Read-Write +0x110
```

(The + 3 offset is because the transfer buffer's address is the 3rd member of the `_go32_info_block` structure.) The output clearly shows that this DPMI server maps the addresses in conventional memory 1:1, i.e. the physical (0x00029000 + 0x110) and linear (0x29110) addresses are identical.

This command is supported only with some DPMI servers.

In addition to native debugging, the DJGPP port supports remote debugging via a serial data link. The following commands are specific to remote serial debugging in the DJGPP port of GDB.

set com1base *addr*

This command sets the base I/O port address of the COM1 serial port.

set com1irq *irq*

This command sets the *Interrupt Request* (IRQ) line to use for the COM1 serial port.

There are similar commands 'set com2base', 'set com3irq', etc. for setting the port address and the IRQ lines for the other 3 COM ports.

The related commands 'show com1base', 'show com1irq' etc. display the current settings of the base address and the IRQ lines used by the COM ports.

info serial

This command prints the status of the 4 DOS serial ports. For each port, it prints whether it's active or not, its I/O base address and IRQ number,

whether it uses a 16550-style FIFO, its baudrate, and the counts of various errors encountered so far.

21.1.4 Features for Debugging MS Windows PE Executables

GDB supports native debugging of MS Windows programs, including DLLs with and without symbolic debugging information.

MS-Windows programs that call `SetConsoleMode` to switch off the special meaning of the ‘`Ctrl-C`’ keystroke cannot be interrupted by typing `C-c`. For this reason, GDB on MS-Windows supports `C-BREAK` as an alternative interrupt key sequence, which can be used to interrupt the debuggee even if it ignores `C-c`.

There are various additional Cygwin-specific commands, described in this section. Working with DLLs that have no debugging symbols is described in Section 21.1.4.1 [Non-debug DLL Symbols], page 345.

info w32 This is a prefix of MS Windows-specific commands which print information about the target system and important OS structures.

info w32 selector

This command displays information returned by the Win32 API `GetThreadSelectorEntry` function. It takes an optional argument that is evaluated to a long value to give the information about this given selector. Without argument, this command displays information about the six segment registers.

info w32 thread-information-block

This command displays thread specific information stored in the Thread Information Block (readable on the X86 CPU family using `$fs` selector for 32-bit programs and `$gs` for 64-bit programs).

signal-event id

This command signals an event with user-provided *id*. Used to resume crashing process when attached to it using MS-Windows JIT debugging (AeDebug).

To use it, create or edit the following keys in `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AeDebug` and/or `HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows NT\CurrentVersion\AeDebug` (for x86_64 versions):

- **Debugger** (REG_SZ) — a command to launch the debugger. Suggested command is: *fully-qualified-path-to-gdb.exe -ex "attach %ld" -ex "signal-event %ld" -ex "continue"*.

The first `%ld` will be replaced by the process ID of the crashing process, the second `%ld` will be replaced by the ID of the event that blocks the crashing process, waiting for GDB to attach.

- **Auto** (REG_SZ) — either 1 or 0. 1 will make the system run debugger specified by the Debugger key automatically, 0 will cause a dialog box with “OK” and “Cancel” buttons to appear, which allows the user to either terminate the crashing process (OK) or debug it (Cancel).

set cygwin-exceptions mode

If *mode* is **on**, GDB will break on exceptions that happen inside the Cygwin DLL. If *mode* is **off**, GDB will delay recognition of exceptions, and may ignore some

exceptions which seem to be caused by internal Cygwin DLL “bookkeeping”. This option is meant primarily for debugging the Cygwin DLL itself; the default value is `off` to avoid annoying GDB users with false `SIGSEGV` signals.

show cygwin-exceptions

Displays whether GDB will break on exceptions that happen inside the Cygwin DLL itself.

set new-console *mode*

If *mode* is `on` the debuggee will be started in a new console on next start. If *mode* is `off`, the debuggee will be started in the same console as the debugger.

show new-console

Displays whether a new console is used when the debuggee is started.

set new-group *mode*

This boolean value controls whether the debuggee should start a new group or stay in the same group as the debugger. This affects the way the Windows OS handles ‘Ctrl-C’.

show new-group

Displays current value of new-group boolean.

set debugevents

This boolean value adds debug output concerning kernel events related to the debuggee seen by the debugger. This includes events that signal thread and process creation and exit, DLL loading and unloading, console interrupts, and debugging messages produced by the Windows `OutputDebugString` API call.

set debugexec

This boolean value adds debug output concerning execute events (such as resume thread) seen by the debugger.

set debugexceptions

This boolean value adds debug output concerning exceptions in the debuggee seen by the debugger.

set debugmemory

This boolean value adds debug output concerning debuggee memory reads and writes by the debugger.

set shell This boolean value specifies whether the debuggee is called via a shell or directly (default value is `on`).

show shell

Displays if the debuggee will be started with a shell.

21.1.4.1 Support for DLLs without Debugging Symbols

Very often on windows, some of the DLLs that your program relies on do not include symbolic debugging information (for example, `kernel32.dll`). When GDB doesn’t recognize any debugging symbols in a DLL, it relies on the minimal amount of symbolic information contained in the DLL’s export table. This section describes working with such symbols, known internally to GDB as “minimal symbols”.

Note that before the debugged program has started execution, no DLLs will have been loaded. The easiest way around this problem is simply to start the program — either by setting a breakpoint or letting the program run once to completion.

21.1.4.2 DLL Name Prefixes

In keeping with the naming conventions used by the Microsoft debugging tools, DLL export symbols are made available with a prefix based on the DLL name, for instance `KERNEL32!CreateFileA`. The plain name is also entered into the symbol table, so `CreateFileA` is often sufficient. In some cases there will be name clashes within a program (particularly if the executable itself includes full debugging symbols) necessitating the use of the fully qualified name when referring to the contents of the DLL. Use single-quotes around the name to avoid the exclamation mark (“!”) being interpreted as a language operator.

Note that the internal name of the DLL may be all upper-case, even though the file name of the DLL is lower-case, or vice-versa. Since symbols within GDB are *case-sensitive* this may cause some confusion. If in doubt, try the `info functions` and `info variables` commands or even `maint print msymbols` (see Chapter 16 [Symbols], page 265). Here’s an example:

```
(gdb) info function CreateFileA
All functions matching regular expression "CreateFileA":

Non-debugging symbols:
0x77e885f4  CreateFileA
0x77e885f4  KERNEL32!CreateFileA

(gdb) info function !
All functions matching regular expression "!":

Non-debugging symbols:
0x6100114c  cygwin1!__assert
0x61004034  cygwin1!_dll_crt0@0
0x61004240  cygwin1!dll_crt0(per_process *)
[etc...]
```

21.1.4.3 Working with Minimal Symbols

Symbols extracted from a DLL’s export table do not contain very much type information. All that GDB can do is guess whether a symbol refers to a function or variable depending on the linker section that contains the symbol. Also note that the actual contents of the memory contained in a DLL are not available unless the program is running. This means that you cannot examine the contents of a variable or disassemble a function within a DLL without a running program.

Variables are generally treated as pointers and dereferenced automatically. For this reason, it is often necessary to prefix a variable name with the address-of operator (“&”) and provide explicit type information in the command. Here’s an example of the type of problem:

```
(gdb) print 'cygwin1!__argv'
'cygwin1!__argv' has unknown type; cast it to its declared type

(gdb) x 'cygwin1!__argv'
'cygwin1!__argv' has unknown type; cast it to its declared type
```

And two possible solutions:

```
(gdb) print ((char **)'cygwin1!__argv')[0]
$2 = 0x22fd98 "/cygdrive/c/mydirectory/myprogram"
(gdb) x/2x &'cygwin1!__argv'
0x610c0aa8 <cygwin1!__argv>:    0x10021608    0x00000000
(gdb) x/x 0x10021608
0x10021608:    0x0022fd98
(gdb) x/s 0x0022fd98
0x22fd98:    "/cygdrive/c/mydirectory/myprogram"
```

Setting a break point within a DLL is possible even before the program starts execution. However, under these circumstances, GDB can't examine the initial instructions of the function in order to skip the function's frame set-up code. You can work around this by using “*&” to set the breakpoint at a raw memory address:

```
(gdb) break *&'python22!PyOS_Readline'
Breakpoint 1 at 0x1e04eff0
```

The author of these extensions is not entirely convinced that setting a break point within a shared DLL like `kernel32.dll` is completely safe.

21.1.5 Commands Specific to GNU Hurd Systems

This subsection describes GDB commands specific to the GNU Hurd native debugging.

set signals

set sigs This command toggles the state of inferior signal interception by GDB. Mach exceptions, such as breakpoint traps, are not affected by this command. **sigs** is a shorthand alias for **signals**.

show signals

show sigs Show the current state of intercepting inferior's signals.

set signal-thread

set sigthread

This command tells GDB which thread is the `libc` signal thread. That thread is run when a signal is delivered to a running process. **set sigthread** is the shorthand alias of **set signal-thread**.

show signal-thread

show sigthread

These two commands show which thread will run when the inferior is delivered a signal.

set stopped

This command tells GDB that the inferior process is stopped, as with the `SIGSTOP` signal. The stopped process can be continued by delivering a signal to it.

show stopped

This command shows whether GDB thinks the debuggee is stopped.

set exceptions

Use this command to turn off trapping of exceptions in the inferior. When exception trapping is off, neither breakpoints nor single-stepping will work. To restore the default, set exception trapping on.

show exceptions

Show the current state of trapping exceptions in the inferior.

set task pause

This command toggles task suspension when GDB has control. Setting it to on takes effect immediately, and the task is suspended whenever GDB gets control. Setting it to off will take effect the next time the inferior is continued. If this option is set to off, you can use **set thread default pause on** or **set thread pause on** (see below) to pause individual threads.

show task pause

Show the current state of task suspension.

set task detach-suspend-count

This command sets the suspend count the task will be left with when GDB detaches from it.

show task detach-suspend-count

Show the suspend count the task will be left with when detaching.

set task exception-port**set task excp**

This command sets the task exception port to which GDB will forward exceptions. The argument should be the value of the *send rights* of the task. **set task excp** is a shorthand alias.

set noninvasive

This command switches GDB to a mode that is the least invasive as far as interfering with the inferior is concerned. This is the same as using **set task pause**, **set exceptions**, and **set signals** to values opposite to the defaults.

info send-rights**info receive-rights****info port-rights****info port-sets****info dead-names****info ports****info psets**

These commands display information about, respectively, send rights, receive rights, port rights, port sets, and dead names of a task. There are also shorthand aliases: **info ports** for **info port-rights** and **info psets** for **info port-sets**.

set thread pause

This command toggles current thread suspension when GDB has control. Setting it to on takes effect immediately, and the current thread is suspended whenever GDB gets control. Setting it to off will take effect the next time the inferior is continued. Normally, this command has no effect, since when GDB has control, the whole task is suspended. However, if you used **set task pause off** (see above), this command comes in handy to suspend only the current thread.

show thread pause

This command shows the state of current thread suspension.

set thread run

This command sets whether the current thread is allowed to run.

show thread run

Show whether the current thread is allowed to run.

set thread detach-suspend-count

This command sets the suspend count GDB will leave on a thread when detaching. This number is relative to the suspend count found by GDB when it notices the thread; use **set thread takeover-suspend-count** to force it to an absolute value.

show thread detach-suspend-count

Show the suspend count GDB will leave on the thread when detaching.

set thread exception-port

set thread excp

Set the thread exception port to which to forward exceptions. This overrides the port set by **set task exception-port** (see above). **set thread excp** is the shorthand alias.

set thread takeover-suspend-count

Normally, GDB's thread suspend counts are relative to the value GDB finds when it notices each thread. This command changes the suspend counts to be absolute instead.

set thread default

show thread default

Each of the above **set thread** commands has a **set thread default** counterpart (e.g., **set thread default pause**, **set thread default exception-port**, etc.). The **thread default** variety of commands sets the default thread properties for all threads; you can then change the properties of individual threads with the non-default commands.

21.1.6 Darwin

GDB provides the following commands specific to the Darwin target:

set debug darwin num

When set to a non zero value, enables debugging messages specific to the Darwin support. Higher values produce more verbose output.

show debug darwin

Show the current state of Darwin messages.

set debug mach-o num

When set to a non zero value, enables debugging messages while GDB is reading Darwin object files. (*Mach-O* is the file format used on Darwin for object and executable files.) Higher values produce more verbose output. This is a command to diagnose problems internal to GDB and should not be needed in normal usage.

show debug mach-o

Show the current state of Mach-O file messages.

```
set mach-exceptions on
set mach-exceptions off
```

On Darwin, faults are first reported as a Mach exception and are then mapped to a Posix signal. Use this command to turn on trapping of Mach exceptions in the inferior. This might be sometimes useful to better understand the cause of a fault. The default is off.

```
show mach-exceptions
```

Show the current state of exceptions trapping.

21.1.7 FreeBSD

When the ABI of a system call is changed in the FreeBSD kernel, this is implemented by leaving a compatibility system call using the old ABI at the existing number and allocating a new system call number for the version using the new ABI. As a convenience, when a system call is caught by name (see [catch syscall], page 72), compatibility system calls are also caught.

For example, FreeBSD 12 introduced a new variant of the `kevent` system call and catching the `kevent` system call by name catches both variants:

```
(gdb) catch syscall kevent
Catchpoint 1 (syscalls 'freebsd11_kevent' [363] 'kevent' [560])
(gdb)
```

21.2 Embedded Operating Systems

This section describes configurations involving the debugging of embedded operating systems that are available for several different architectures.

GDB includes the ability to debug programs running on various real-time operating systems.

21.3 Embedded Processors

This section goes into details specific to particular embedded configurations.

Whenever a specific embedded processor has a simulator, GDB allows to send an arbitrary command to the simulator.

```
sim command
```

Send an arbitrary *command* string to the simulator. Consult the documentation for the specific simulator in use for information about acceptable commands.

21.3.1 Synopsys ARC

GDB provides the following ARC-specific commands:

```
set debug arc
```

Control the level of ARC specific debug messages. Use 0 for no messages (the default), 1 for debug messages, and 2 for even more debug messages.

```
show debug arc
```

Show the level of ARC specific debugging in operation.

```
maint print arc arc-instruction address
```

Print internal disassembler information about instruction at a given address.

21.3.2 ARM

GDB provides the following ARM-specific commands:

set arm disassembler

This command selects from a list of disassembly styles. The "**std**" style is the standard style.

show arm disassembler

Show the current disassembly style.

set arm apcs32

This command toggles ARM operation mode between 32-bit and 26-bit.

show arm apcs32

Display the current usage of the ARM 32-bit mode.

set arm fpu *fputype*

This command sets the ARM floating-point unit (FPU) type. The argument *fputype* can be one of these:

auto Determine the FPU type by querying the OS ABI.

softfpa Software FPU, with mixed-endian doubles on little-endian ARM processors.

fpa GCC-compiled FPA co-processor.

softvfp Software FPU with pure-endian doubles.

vfp VFP co-processor.

show arm fpu

Show the current type of the FPU.

set arm abi

This command forces GDB to use the specified ABI.

show arm abi

Show the currently used ABI.

set arm fallback-mode (arm|thumb|auto)

GDB uses the symbol table, when available, to determine whether instructions are ARM or Thumb. This command controls GDB's default behavior when the symbol table is not available. The default is '**auto**', which causes GDB to use the current execution mode (from the T bit in the CPSR register).

show arm fallback-mode

Show the current fallback instruction mode.

set arm force-mode (arm|thumb|auto)

This command overrides use of the symbol table to determine whether instructions are ARM or Thumb. The default is '**auto**', which causes GDB to use the symbol table and then the setting of '**set arm fallback-mode**'.

show arm force-mode

Show the current forced instruction mode.

set arm unwind-secure-frames

This command enables unwinding from Non-secure to Secure mode on Cortex-M with Security extension. This can trigger security exceptions when unwinding the exception stack. It is enabled by default.

show arm unwind-secure-frames

Show whether unwinding from Non-secure to Secure mode is enabled.

set debug arm

Toggle whether to display ARM-specific debugging messages from the ARM target support subsystem.

show debug arm

Show whether ARM-specific debugging messages are enabled.

target sim [simargs] ...

The GDB ARM simulator accepts the following optional arguments.

--swi-support=type

Tell the simulator which SWI interfaces to support. The argument *type* may be a comma separated list of the following values. The default value is **all**.

none

demon

angel

redboot

all

21.3.3 BPF**target sim [simargs] ...**

The GDB BPF simulator accepts the following optional arguments.

--skb-data-offset=offset

Tell the simulator the offset, measured in bytes, of the **skb_data** field in the kernel **struct sk_buff** structure. This offset is used by some BPF specific-purpose load/store instructions. Defaults to 0.

21.3.4 M68k

The Motorola m68k configuration includes ColdFire support.

21.3.5 MicroBlaze

The MicroBlaze is a soft-core processor supported on various Xilinx FPGAs, such as Spartan or Virtex series. Boards with these processors usually have JTAG ports which connect to a host system running the Xilinx Embedded Development Kit (EDK) or Software Development Kit (SDK). This host system is used to download the configuration bitstream to the target FPGA. The Xilinx Microprocessor Debugger (XMD) program communicates with the target board using the JTAG interface and presents a **gdbserver** interface to the

board. By default `xmd` uses port 1234. (While it is possible to change this default port, it requires the use of undocumented `xmd` commands. Contact Xilinx support if you need to do this.)

Use these GDB commands to connect to the MicroBlaze target processor.

target remote :1234

Use this command to connect to the target if you are running GDB on the same system as `xmd`.

target remote *xmd-host*:1234

Use this command to connect to the target if it is connected to `xmd` running on a different system named *xmd-host*.

load Use this command to download a program to the MicroBlaze target.

set debug microblaze *n*

Enable MicroBlaze-specific debugging messages if non-zero.

show debug microblaze *n*

Show MicroBlaze-specific debugging level.

21.3.6 MIPS Embedded

GDB supports these special commands for MIPS targets:

set mipsfpu double

set mipsfpu single

set mipsfpu none

set mipsfpu auto

show mipsfpu

If your target board does not support the MIPS floating point coprocessor, you should use the command '**set mipsfpu none**' (if you need this, you may wish to put the command in your GDB init file). This tells GDB how to find the return value of functions which return floating point values. It also allows GDB to avoid saving the floating point registers when calling functions on the board. If you are using a floating point coprocessor with only single precision floating point support, as on the R4650 processor, use the command '**set mipsfpu single**'. The default double precision floating point coprocessor may be selected using '**set mipsfpu double**'.

In previous versions the only choices were double precision or no floating point, so '**set mipsfpu on**' will select double precision and '**set mipsfpu off**' will select no floating point.

As usual, you can inquire about the `mipsfpu` variable with '**show mipsfpu**'.

21.3.7 OpenRISC 1000

The OpenRISC 1000 provides a free RISC instruction set architecture. It is mainly provided as a soft-core which can run on Xilinx, Altera and other FPGA's.

GDB for OpenRISC supports the below commands when connecting to a target:

target sim

Runs the builtin CPU simulator which can run very basic programs but does not support most hardware functions like MMU. For more complex use cases the user is advised to run an external target, and connect using ‘**target remote**’.

Example: **target sim**

set debug or1k

Toggle whether to display OpenRISC-specific debugging messages from the OpenRISC target support subsystem.

show debug or1k

Show whether OpenRISC-specific debugging messages are enabled.

21.3.8 PowerPC Embedded

GDB supports using the DVC (Data Value Compare) register to implement in hardware simple hardware watchpoint conditions of the form:

```
(gdb) watch address|variable \
      if address|variable == constant expression
```

The DVC register will be automatically used when GDB detects such pattern in a condition expression, and the created watchpoint uses one debug register (either the **exact-watchpoints** option is on and the variable is scalar, or the variable has a length of one byte). This feature is available in native GDB running on a Linux kernel version 2.6.34 or newer.

When running on PowerPC embedded processors, GDB automatically uses ranged hardware watchpoints, unless the **exact-watchpoints** option is on, in which case watchpoints using only one debug register are created when watching variables of scalar types.

You can create an artificial array to watch an arbitrary memory region using one of the following commands (see Section 10.1 [Expressions], page 147):

```
(gdb) watch *((char *) address)@length
(gdb) watch {char[length]} address
```

PowerPC embedded processors support masked watchpoints. See the discussion about the **mask** argument in Section 5.1.2 [Set Watchpoints], page 68.

PowerPC embedded processors support hardware accelerated *ranged breakpoints*. A ranged breakpoint stops execution of the inferior whenever it executes an instruction at any address within the range it was set at. To set a ranged breakpoint in GDB, use the **break-range** command.

GDB provides the following PowerPC-specific commands:

break-range start-locs, end-locs

Set a breakpoint for an address range given by *start-locs* and *end-locs*, which are location specs. See Section 9.2 [Location Specifications], page 128, for a list of all the possible forms of location specs. GDB resolves both *start-locs* and *end-locs*, and uses the addresses of the resolved code locations as start and end addresses of the range to break at. The breakpoint will stop execution of the inferior whenever it executes an instruction at any address between the start and end addresses, inclusive. If either *start-locs* or *end-locs* resolve to multiple code locations in the program, then the command aborts with an error without creating a breakpoint.

set powerpc soft-float

show powerpc soft-float

Force GDB to use (or not use) a software floating point calling convention. By default, GDB selects the calling convention based on the selected architecture and the provided executable file.

set powerpc vector-abi

show powerpc vector-abi

Force GDB to use the specified calling convention for vector arguments and return values. The valid options are ‘**auto**’; ‘**generic**’, to avoid vector registers even if they are present; ‘**altivec**’, to use AltiVec registers; and ‘**spe**’ to use SPE registers. By default, GDB selects the calling convention based on the selected architecture and the provided executable file.

set powerpc exact-watchpoints

show powerpc exact-watchpoints

Allow GDB to use only one debug register when watching a variable of scalar type, thus assuming that the variable is accessed through the address of its first byte.

21.3.9 Atmel AVR

When configured for debugging the Atmel AVR, GDB supports the following AVR-specific commands:

info io_registers

This command displays information about the AVR I/O registers. For each register, GDB prints its number and value.

21.3.10 CRIS

When configured for debugging CRIS, GDB provides the following CRIS-specific commands:

set cris-version ver

Set the current CRIS version to *ver*, either ‘10’ or ‘32’. The CRIS version affects register names and sizes. This command is useful in case autodetection of the CRIS version fails.

show cris-version

Show the current CRIS version.

set cris-dwarf2-cfi

Set the usage of DWARF-2 CFI for CRIS debugging. The default is ‘on’. Change to ‘off’ when using `gcc-cris` whose version is below R59.

show cris-dwarf2-cfi

Show the current state of using DWARF-2 CFI.

set cris-mode mode

Set the current CRIS mode to *mode*. It should only be changed when debugging in guru mode, in which case it should be set to ‘guru’ (the default is ‘normal’).

show cris-mode

Show the current CRIS mode.

21.3.11 Renesas Super-H

For the Renesas Super-H processor, GDB provides these commands:

set sh calling-convention *convention*

Set the calling-convention used when calling functions from GDB. Allowed values are ‘gcc’, which is the default setting, and ‘renesas’. With the ‘gcc’ setting, functions are called using the GCC calling convention. If the DWARF-2 information of the called function specifies that the function follows the Renesas calling convention, the function is called using the Renesas calling convention. If the calling convention is set to ‘renesas’, the Renesas calling convention is always used, regardless of the DWARF-2 information. This can be used to override the default of ‘gcc’ if debug information is missing, or the compiler does not emit the DWARF-2 calling convention entry for a function.

show sh calling-convention

Show the current calling convention setting.

21.4 Architectures

This section describes characteristics of architectures that affect all uses of GDB with the architecture, both native and cross.

21.4.1 AArch64

When GDB is debugging the AArch64 architecture, it provides the following special commands:

set debug aarch64

This command determines whether AArch64 architecture-specific debugging messages are to be displayed.

show debug aarch64

Show whether AArch64 debugging messages are displayed.

21.4.1.1 AArch64 Scalable Vector Extension

When GDB is debugging the AArch64 architecture, if the Scalable Vector Extension (SVE) is present, then GDB will provide the vector registers **\$z0** through **\$z31**, vector predicate registers **\$p0** through **\$p15**, and the **\$ffr** register. In addition, the pseudo register **\$vg** will be provided. This is the vector granule for the current thread and represents the number of 64-bit chunks in an SVE **z** register.

If the vector length changes, then the **\$vg** register will be updated, but the lengths of the **z** and **p** registers will not change. This is a known limitation of GDB and does not affect the execution of the target process.

For SVE, the following definitions are used throughout GDB’s source code and in this document:

- **vl**: The vector length, in bytes. It defines the size of each **Z** register.
- **vg**: The number of 128 bit units in **vl**. This is mostly used internally by GDB and the Linux Kernel.
- **vg**: The number of 64 bit units in **vl**. This is mostly used internally by GDB and the Linux Kernel.

21.4.1.2 AArch64 Scalable Matrix Extension

The Scalable Matrix Extension (SME (<https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/scalable-matrix-extension-armv9-a-architecture>)) is an AArch64 architecture extension that expands on the concept of the Scalable Vector Extension (SVE (<https://developer.arm.com/documentation/101726/4-0/Learn-about-the-Scalable-Vector-Extension--SVE-/What-is-the-Scalable-Vector-Extension->)) by providing a 2-dimensional register ZA, which is a square matrix of variable size, just like SVE provides a group of vector registers of variable size.

Similarly to SVE, where the size of each Z register is directly related to the vector length (*vl* for short), the SME ZA matrix register's size is directly related to the streaming vector length (*svl* for short). See [*vl*], page 356. See [*svl*], page 357.

The ZA register state can be either active or inactive, if it is not in use.

SME also introduces a new execution mode called streaming SVE mode (streaming mode for short). When streaming mode is enabled, the program supports execution of SVE2 instructions and the SVE registers will have vector length *svl*. When streaming mode is disabled, the SVE registers have vector length *vl*.

For more information about SME and SVE, please refer to official architecture documentation (<https://developer.arm.com/documentation/ddi0487/latest>).

The following definitions are used throughout GDB's source code and in this document:

- *svl*: The streaming vector length, in bytes. It defines the size of each dimension of the 2-dimensional square ZA matrix. The total size of ZA is therefore *svl* by *svl*.

When streaming mode is enabled, it defines the size of the SVE registers as well.

- *svq*: The number of 128 bit units in *svl*, also known as streaming vector granule. This is mostly used internally by GDB and the Linux Kernel.
- *svg*: The number of 64 bit units in *svl*. This is mostly used internally by GDB and the Linux Kernel.

When GDB is debugging the AArch64 architecture, if the Scalable Matrix Extension (SME) is present, then GDB will make the ZA register available. GDB will also make the SVG register and SVCR pseudo-register available.

The ZA register is a 2-dimensional square *svl* by *svl* matrix of bytes. To simplify the representation and access to the ZA register in GDB, it is defined as a vector of *svl* × *svl* bytes.

If the user wants to index the ZA register as a matrix, it is possible to reference ZA as `ZA[i][j]`, where *i* is the row number and *j* is the column number.

The SVG register always contains the streaming vector granule (*svg*) for the current thread. From the value of register SVG we can easily derive the *svl* value.

The SVCR pseudo-register (streaming vector control register) is a status register that holds two state bits: SM in bit 0 and ZA in bit 1.

If the SM bit is 1, it means the current thread is in streaming mode, and the SVE registers will use *svl* for their sizes. If the SM bit is 0, the current thread is not in streaming mode, and the SVE registers will use *vl* for their sizes. See [*vl*], page 356.

If the ZA bit is 1, it means the ZA register is being used and has meaningful contents. If the ZA bit is 0, the ZA register is unavailable and its contents are undefined.

For convenience and simplicity, if the ZA bit is 0, the ZA register and all of its pseudo-registers will read as zero.

If *svl* changes during the execution of a program, then the ZA register size and the bits in the SVCR pseudo-register will be updated to reflect it.

It is possible for users to change *svl* during the execution of a program by modifying the SVG register value.

Whenever the SVG register is modified with a new value, the following will be observed:

- The ZA and SM bits will be cleared in the SVCR pseudo-register.
- The ZA register will have a new size and its state will be cleared, forcing its contents and the contents of all of its pseudo-registers back to zero.
- If the SM bit was 1, the SVE registers will be reset to having their sizes based on *vl* as opposed to *svl*. If the SM bit was 0 prior to modifying the SVG register, there will be no observable effect on the SVE registers.

The possible values for the SVG register are 2, 4, 8, 16, 32. These numbers correspond to streaming vector length (*svl*) values of 16 bytes, 32 bytes, 64 bytes, 128 bytes and 256 bytes respectively.

The minimum size of the ZA register is 16 x 16 (256) bytes, and the maximum size is 256 x 256 (65536) bytes. In streaming mode, with bit SM set, the size of the ZA register is the size of all the SVE Z registers combined.

The ZA register can also be accessed using tiles and tile slices.

Tile pseudo-registers are square, 2-dimensional sub-arrays of elements within the ZA register.

The tile pseudo-registers have the following naming pattern: **ZA<tile number><qualifier>**.

There is a total of 31 ZA tile pseudo-registers. They are ZA0B, ZA0H through ZA1H, ZA0S through ZA3S, ZA0D through ZA7D and ZA0Q through ZA15Q.

Tile slice pseudo-registers are vectors of horizontally or vertically contiguous elements within the ZA register.

The tile slice pseudo-registers have the following naming pattern: **ZA<tile number><direction><qualifier> <slice number>**.

There are up to 16 tiles (0 ~ 15), the direction can be either **v** (vertical) or **h** (horizontal), the qualifiers can be **b** (byte), **h** (halfword), **s** (word), **d** (doubleword) and **q** (quadword) and there are up to 256 slices (0 ~ 255) depending on the value of *svl*. The number of slices is the same as the value of *svl*.

The number of available tile slice pseudo-registers can be large. For a minimum *svl* of 16 bytes, there are 5 (number of qualifiers) x 2 (number of directions) x 16 (*svl*) pseudo-registers. For the maximum *svl* of 256 bytes, there are 5 x 2 x 256 pseudo-registers.

When listing all the available registers, users will see the currently-available ZA pseudo-registers. Pseudo-registers that don't exist for a given *svl* value will not be displayed.

For more information on SME and its terminology, please refer to the Arm Architecture Reference Manual Supplement (<https://developer.arm.com/documentation/ddi0616/aa/>), The Scalable Matrix Extension (SME), for Armv9-A.

Some features are still under development and rely on ACLE (<https://github.com/ARM-software/acle/releases/latest>) and ABI (<https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst>) definitions, so there are known limitations to the current SME support in GDB.

One such example is calling functions in the program being debugged by GDB. Such calls are not SME-aware and thus don't take into account the `SVCR` pseudo-register bits nor the `ZA` register contents. See Section 17.5 [Calling], page 283.

The lazy saving scheme (<https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst#the-za-lazy-saving-scheme>) involving the `TPIDR2` register is not yet supported by GDB, though the `TPIDR2` register is known and supported by GDB.

Lastly, an important limitation for `gdbserver` is its inability to communicate `svl` changes to GDB. This means `gdbserver`, even though it is capable of adjusting its internal caches to reflect a change in the value of `svl` mid-execution, will operate with a potentially different `svl` value compared to GDB. This can lead to GDB showing incorrect values for the `ZA` register and incorrect values for SVE registers (when in streaming mode).

This is the same limitation we have for the SVE registers, and there are plans to address this limitation going forward.

21.4.1.3 AArch64 Scalable Matrix Extension 2

The Scalable Matrix Extension 2 is an AArch64 architecture extension that further expands the SME extension with the following:

- The ability to address the `ZA` array through groups of one-dimensional `ZA` array vectors, as opposed to `ZA` tiles with 2 dimensions.
- Instructions to operate on groups of SVE `Z` registers and `ZA` array vectors.
- A new 512 bit `ZT0` lookup table register, for data decompression.

When GDB is debugging the AArch64 architecture, if the Scalable Matrix Extension 2 (SME2) is present, then GDB will make the `ZT0` register available.

The `ZT0` register is only considered active when the `ZA` register state is active, therefore when the `ZA` bit of the `SVCR` is 1.

When the `ZA` bit of `SVCR` is 0, that means the `ZA` register state is not active, which means the `ZT0` register state is also not active.

When `ZT0` is not active, it is comprised of zeroes, just like `ZA`.

Similarly to the `ZA` register, if the `ZT0` state is not active and the user attempts to modify its value such that any of its bytes is non-zero, then GDB will initialize the `ZA` register state as well, which means the `SVCR` `ZA` bit gets set to 1.

For more information about SME2, please refer to the official architecture documentation (<https://developer.arm.com/documentation/ddi0487/latest>).

21.4.1.4 AArch64 Pointer Authentication

When GDB is debugging the AArch64 architecture, and the program is using the v8.3-A feature Pointer Authentication (PAC), then whenever the link register `$lr` is pointing to an PAC function its value will be masked. When GDB prints a backtrace, any addresses that required unmasking will be postfixed with the marker [PAC]. When using the MI, this is printed as part of the `addr_flags` field.

21.4.1.5 AArch64 Memory Tagging Extension

When GDB is debugging the AArch64 architecture, the program is using the v8.5-A feature Memory Tagging Extension (MTE) and there is support in the kernel for MTE, GDB will make memory tagging functionality available for inspection and editing of logical and allocation tags. See Section 10.7 [Memory Tagging], page 157.

To aid debugging, GDB will output additional information when SIGSEGV signals are generated as a result of memory tag failures.

If the tag violation is synchronous, the following will be shown:

```
Program received signal SIGSEGV, Segmentation fault
Memory tag violation while accessing address 0x0500ffff7ff8000
Allocation tag 0x1
Logical tag 0x5.
```

If the tag violation is asynchronous, the fault address is not available. In this case GDB will show the following:

```
Program received signal SIGSEGV, Segmentation fault
Memory tag violation
Fault address unavailable.
```

A special register, `tag_ctl`, is made available through the `org.gnu.gdb.aarch64.mte` feature. This register exposes some options that can be controlled at runtime and emulates the `prctl` option `PR_SET_TAGGED_ADDR_CTRL`. For further information, see the documentation in the Linux kernel.

GDB supports dumping memory tag data to core files through the `gcore` command and reading memory tag data from core files generated by the `gcore` command or the Linux kernel.

When a process uses memory-mapped pages protected by memory tags (for example, AArch64 MTE), this additional information will be recorded in the core file in the event of a crash or if GDB generates a core file from the current process state.

The memory tag data will be used so developers can display the memory tags from a particular memory region (using the ‘m’ modifier to the `x` command, using the `print` command or using the various `memory-tag` subcommands).

In the case of a crash, GDB will attempt to retrieve the memory tag information automatically from the core file, and will show one of the above messages depending on whether the synchronous or asynchronous mode is selected. See Section 10.7 [Memory Tagging], page 157. See Section 10.6 [Memory], page 154.

21.4.1.6 AArch64 Guarded Control Stack

When GDB is debugging the AArch64 architecture, the program is using the feature Guarded Control Stack (GCS), the operating system kernel is Linux and it supports GCS, GDB will make a couple of special registers — `gcs_features_enabled` and `gcs_features_locked` — available through the `org.gnu.gdb.aarch64.gcs.linux` feature. These registers expose some options that can be controlled at runtime and emulate the `prctl` option `PR_SET_SHADOW_STACK_STATUS`. For further information, see the documentation in the Linux kernel.

Naturally the Guarded Control Stack pointer at EL0 is also available, as the `gcspr` register.

To aid debugging, GDB will note when SIGSEGV signals are generated as a result of a Guarded Control Stack error:

```
Program received signal SIGSEGV, Segmentation fault
Guarded Control Stack error.
```

21.4.1.7 AArch64 Floating Point Mode Register

When GDB is debugging the AArch64 architecture and the Floating Point Mode Register (FPMR) is available, then GDB will make the `fpmr` available. This register determines the behavior of FP8 instructions.

To aid debugging, GDB interprets the mode or value of each field in `fpmr`.

```
p $fpmr
$1 = [ F8S1=E5M2 F8S2=E5M2 F8D=E5M2 OSM=Inf OSC=Inf/NaN LSCALE=0 NSCALE=0 LSCALE2=0 ]
```

For more information about FPMR, please refer to the official architecture registers documentation.

21.4.2 x86

`set struct-convention mode`

Set the convention used by the inferior to return `structs` and `unions` from functions to *mode*. Possible values of *mode* are "pcc", "reg", and "default" (the default). "default" or "pcc" means that `structs` are returned on the stack, while "reg" means that a `struct` or a `union` whose size is 1, 2, 4, or 8 bytes will be returned in a register.

`show struct-convention`

Show the current setting of the convention to return `structs` from functions.

21.4.2.1 x87 registers

GDB provides access to the x87 state through the following registers:

- `$st0` to `st7`: ST(0) to ST(7) floating-point registers
- `$fctrl`: control word register (FCW)
- `$fstat`: status word register (FSW)
- `$ftag`: tag word (FTW)
- `$fiseg`: last instruction pointer segment
- `$fioff`: last instruction pointer
- `$foseg`: last data pointer segment
- `$fooff`: last data pointer
- `$fop`: last opcode

21.4.2.2 Intel Control-Flow Enforcement Technology.

The *Intel Control-Flow Enforcement Technology* (Intel CET) provides two capabilities to defend against “Return-oriented Programming” and “call/jmp-oriented programming” style control-flow attacks:

- **Shadow Stack:** A shadow stack is a second stack for a program. It holds the return addresses pushed by the call instruction. The RET instruction pops the return addresses

from both call and shadow stack. If the return addresses from the two stacks do not match, the processor signals a control protection exception.

- Indirect Branch Tracking (IBT): When IBT is enabled, the CPU implements a state machine that tracks indirect `JMP` and `CALL` instructions. The state machine can be either `IDLE` or `WAIT_FOR_ENDBRANCH`. When a `JMP` or `CALL` is executed the state machine changes to the `WAIT_FOR_ENDBRANCH` state. In `WAIT_FOR_ENDBRANCH` state the next instruction in the program stream must be an `ENDBR` instruction, otherwise the processor signals a control protection exception. After executing a `ENDBR` instruction the state machine returns to the `IDLE` state.

Impact on GDB commands:

- Call/Print: Inferior calls in GDB reset the current PC to the beginning of the function that is called. No call instruction is executed, but the `RET` instruction actually is. To avoid a control protection exception due to the missing return address on the shadow stack, GDB pushes the new return address to the shadow stack and updates the shadow stack pointer.
- Step: With displaced stepping, GDB may run an out of line copy of a call instruction. In this case, the wrong return address is pushed to the shadow stack. GDB corrects this value to avoid a control protection exception. For more details on displaced stepping, see [displaced-stepping], page 779.

21.4.3 Alpha

See the following section.

21.4.4 MIPS

Alpha- and MIPS-based computers use an unusual stack frame, which sometimes requires GDB to search backward in the object code to find the beginning of a function.

To improve response time (especially for embedded applications, where GDB may be restricted to a slow serial line for this search) you may want to limit the size of this search, using one of these commands:

set heuristic-fence-post limit

Restrict GDB to examining at most *limit* bytes in its search for the beginning of a function. A value of 0 (the default) means there is no limit. However, except for 0, the larger the limit the more bytes **heuristic-fence-post** must search and therefore the longer it takes to run. You should only need to use this command when debugging a stripped executable.

show heuristic-fence-post

Display the current limit.

These commands are available *only* when GDB is configured for debugging programs on Alpha or MIPS processors.

Several MIPS-specific commands are available when debugging MIPS programs:

set mips abi arg

Tell GDB which MIPS ABI is used by the inferior. Possible values of *arg* are:

‘auto’ The default ABI associated with the current binary (this is the default).

`'o32'`
`'o64'`
`'n32'`
`'n64'`
`'eabi32'`
`'eabi64'`

show mips abi

Show the MIPS ABI used by GDB to debug the inferior.

set mips compression arg

Tell GDB which MIPS compressed ISA (Instruction Set Architecture) encoding is used by the inferior. GDB uses this for code disassembly and other internal interpretation purposes. This setting is only referred to when no executable has been associated with the debugging session or the executable does not provide information about the encoding it uses. Otherwise this setting is automatically updated from information provided by the executable.

Possible values of *arg* are `'mips16'` and `'micromips'`. The default compressed ISA encoding is `'mips16'`, as executables containing MIPS16 code frequently are not identified as such.

This setting is “sticky”; that is, it retains its value across debugging sessions until reset either explicitly with this command or implicitly from an executable.

The compiler and/or assembler typically add symbol table annotations to identify functions compiled for the MIPS16 or microMIPS ISAs. If these function-scope annotations are present, GDB uses them in preference to the global compressed ISA encoding setting.

show mips compression

Show the MIPS compressed ISA encoding used by GDB to debug the inferior.

set mipsfpu

show mipsfpu

See Section 21.3.6 [MIPS Embedded], page 353.

set mips mask-address arg

This command determines whether the most-significant 32 bits of 64-bit MIPS addresses are masked off. The argument *arg* can be `'on'`, `'off'`, or `'auto'`. The latter is the default setting, which lets GDB determine the correct value.

show mips mask-address

Show whether the upper 32 bits of MIPS addresses are masked off or not.

set remote-mips64-transfers-32bit-regs

This command controls compatibility with 64-bit MIPS targets that transfer data in 32-bit quantities. If you have an old MIPS 64 target that transfers 32 bits for some registers, like SR and FSR, and 64 bits for other registers, set this option to `'on'`.

show remote-mips64-transfers-32bit-regs

Show the current setting of compatibility with older MIPS 64 targets.

set debug mips

This command turns on and off debugging messages for the MIPS-specific target code in GDB.

show debug mips

Show the current setting of MIPS debugging messages.

21.4.5 HPPA

When GDB is debugging the HP PA architecture, it provides the following special commands:

set debug hppa

This command determines whether HPPA architecture-specific debugging messages are to be displayed.

show debug hppa

Show whether HPPA debugging messages are displayed.

maint print unwind *address*

This command displays the contents of the unwind table entry at the given *address*.

21.4.6 PowerPC

When GDB is debugging the PowerPC architecture, it provides a set of pseudo-registers to enable inspection of 128-bit wide Decimal Floating Point numbers stored in the floating point registers. These values must be stored in two consecutive registers, always starting at an even register like `f0` or `f2`.

The pseudo-registers go from `$d10` through `$d115`, and are formed by joining the even/odd register pairs `f0` and `f1` for `$d10`, `f2` and `f3` for `$d11` and so on.

For POWER7 processors, GDB provides a set of pseudo-registers, the 64-bit wide Extended Floating Point Registers (`'f32'` through `'f63'`).

21.4.7 Sparc64

21.4.7.1 ADI Support

The M7 processor supports an Application Data Integrity (ADI) feature that detects invalid data accesses. When software allocates memory and enables ADI on the allocated memory, it chooses a 4-bit version number, sets the version in the upper 4 bits of the 64-bit pointer to that data, and stores the 4-bit version in every cacheline of that data. Hardware saves the latter in spare bits in the cache and memory hierarchy. On each load and store, the processor compares the upper 4 VA (virtual address) bits to the cacheline's version. If there is a mismatch, the processor generates a version mismatch trap which can be either precise or disrupting. The trap is an error condition which the kernel delivers to the process as a SIGSEGV signal.

Note that only 64-bit applications can use ADI and need to be built with ADI-enabled.

Values of the ADI version tags, which are in granularity of a cacheline (64 bytes), can be viewed or modified.

adi (examine | x) [/ n] addr

The **adi examine** command displays the value of one ADI version tag per cacheline.

n is a decimal integer specifying the number in bytes; the default is 1. It specifies how much ADI version information, at the ratio of 1:ADI block size, to display.

addr is the address in user address space where you want GDB to begin displaying the ADI version tags.

Below is an example of displaying ADI versions of variable "shmaddr".

```
(gdb) adi x/100 shmaddr
0xffff800010002c000:    0 0
```

adi (assign | a) [/ n] addr = tag

The **adi assign** command is used to assign new ADI version tag to an address.

n is a decimal integer specifying the number in bytes; the default is 1. It specifies how much ADI version information, at the ratio of 1:ADI block size, to modify.

addr is the address in user address space where you want GDB to begin modifying the ADI version tags.

tag is the new ADI version tag.

For example, do the following to modify then verify ADI versions of variable "shmaddr":

```
(gdb) adi a/100 shmaddr = 7
(gdb) adi x/100 shmaddr
0xffff800010002c000:    7 7
```

21.4.8 S12Z

When GDB is debugging the S12Z architecture, it provides the following special command:

maint info bdccsr

This command displays the current value of the microprocessor's BDCCSR register.

21.4.9 AMD GPU

GDB supports debugging programs offloaded to AMD GPU devices using the AMD ROCm (<https://docs.amd.com/>) platform. GDB presents host threads alongside GPU wavefronts, allowing debugging both the host and device parts of the program simultaneously.

21.4.9.1 AMD GPU Architectures

The list of AMD GPU architectures supported by GDB depends on the version of the AMD Debugger API library used. See its documentation (https://docs.amd.com/bundle/ROCDebugger_User_and_API) for more details.

21.4.9.2 AMD GPU Device Driver and AMD ROCm Runtime

GDB requires a compatible AMD GPU device driver to be installed. A warning message is displayed if either the device driver version or the version of the debug support it implements

is unsupported. GDB will continue to function except no AMD GPU debugging will be possible.

GDB requires each agent to have compatible firmware installed by the device driver. A warning message is displayed if unsupported firmware is detected. GDB will continue to function except no AMD GPU debugging will be possible on the agent.

GDB requires a compatible AMD ROCm runtime to be loaded in order to detect AMD GPU code objects and wavefronts. A warning message is displayed if an unsupported AMD ROCm runtime is detected, or there is an error or restriction that prevents debugging. GDB will continue to function except no AMD GPU debugging will be possible.

21.4.9.3 AMD GPU Wavefronts

An AMD GPU wavefront is represented in GDB as a thread.

Note that some AMD GPU architectures may have restrictions on providing information about AMD GPU wavefronts created when GDB is not attached (see [AMD GPU Attaching Restrictions], page 369).

When scheduler-locking is in effect (see [set scheduler-locking], page 96), new wavefronts created by the resumed thread (either CPU thread or GPU wavefront) are held in the halt state.

21.4.9.4 AMD GPU Code Objects

The ‘`info sharedlibrary`’ command will show the AMD GPU code objects as file or memory URIs, together with the host’s shared libraries. For example:

```
(gdb) info sharedlibrary
From      To      Syms Read  Shared Object Library
0x1111  0x2222  Yes (*)    /lib64/ld-linux-x86-64.so.2
...
0x3333  0x4444  Yes (*)    /opt/rocm-4.5.0/.../libamd_comgr.so
0x5555  0x6666  Yes (*)    /lib/x86_64-linux-gnu/libtinfo.so.5
0x7777  0x8888  Yes        file:///tmp/a.out#offset=6477&size=10832
0x9999  0xaaaa  Yes (*)    memory://95557/mem#offset=0x1234&size=100
(*): Shared library is missing debugging information.
(gdb)
```

For a ‘file’ URI, the path portion is the file on disk containing the code object. The *offset* parameter is a 0-based offset in this file, to the start of the code object. If omitted, it defaults to 0. The *size* parameter is the size of the code object in bytes. If omitted, it defaults to the size of the file.

For a ‘memory’ URI, the path portion is the process id of the process owning the memory containing the code object. The *offset* parameter is the memory address where the code object is found, and the *size* parameter is its size in bytes.

AMD GPU code objects are loaded into each AMD GPU device separately. The ‘`info sharedlibrary`’ command may therefore show the same code object loaded multiple times. As a consequence, setting a breakpoint in AMD GPU code will result in multiple breakpoint locations if there are multiple AMD GPU devices.

21.4.9.5 AMD GPU Entity Target Identifiers and Convenience Variables

The AMD GPU entities have the following target identifier formats:

Thread Target ID

The AMD GPU thread target identifier (*systag*) string has the following format:

```
AMDGPU Wave agent-id:queue-id:dispatch-id:wave-id (work-group-x,work-group-
y,work-group-z)/work-group-thread-index
```

21.4.9.6 AMD GPU Signals

For AMD GPU wavefronts, GDB maps target conditions to stop signals in the following way:

SIGILL	Execution of an illegal instruction.
SIGTRAP	Execution of a <code>S_TRAP</code> instruction other than: • <code>S_TRAP 1</code> which is used by GDB to insert breakpoints. • <code>S_TRAP 2</code> which raises <code>SIGABRT</code>.
SIGABRT	Execution of a <code>S_TRAP 2</code> instruction.
SIGFPE	Execution of a floating point or integer instruction detects a condition that is enabled to raise a signal. The conditions include: • Floating point operation is invalid. • Floating point operation had subnormal input that was rounded to zero. • Floating point operation performed a division by zero. • Floating point operation produced an overflow result. The result was rounded to infinity. • Floating point operation produced an underflow result. A subnormal result was rounded to zero. • Floating point operation produced an inexact result. • Integer operation performed a division by zero. By default, these conditions are not enabled to raise signals. The <code>'set \$mode'</code> command can be used to change the AMD GPU wavefront's register that has bits controlling which conditions are enabled to raise signals. The <code>'print \$trapsts'</code> command can be used to inspect which conditions have been detected even if they are not enabled to raise a signal.
SIGBUS	Execution of an instruction that accessed global memory using an address that is outside the virtual address range.
SIGSEGV	Execution of an instruction that accessed a global memory page that is either not mapped or accessed with incompatible permissions.

If a single instruction raises more than one signal, they will be reported one at a time each time the wavefront is continued.

21.4.9.7 AMD GPU Memory Violation Reporting

A wavefront can report memory violation events. However, the program location at which they are reported may be after the machine instruction that caused them. This can result in the reported source statement being incorrect. The following commands can be used to control this behavior:

set amdgpu precise-memory mode

Controls how AMD GPU devices detect memory violations, where *mode* can be:

- off** The program location may not be immediately after the instruction that caused the memory violation. This is the default.
- on** Requests that the program location will be immediately after the instruction that caused a memory violation. Enabling this mode may make the AMD GPU device execution significantly slower as it has to wait for each memory operation to complete before executing the next instruction.

The **amdgpu precise-memory** parameter is per-inferior. When an inferior forks or execs, or the user uses the **clone-inferior** command, and an inferior is created as a result, the newly created inferior inherits the parameter value of the original inferior.

show amdgpu precise-memory

Displays the currently requested AMD GPU precise memory setting.

21.4.9.8 AMD GPU Logging

The **set debug amd-dbgapi** command can be used to enable diagnostic messages in the **amd-dbgapi** target. The **show debug amd-dbgapi** command displays the current setting. See [set debug amd-dbgapi], page 388.

The **set debug amd-dbgapi-lib log-level level** command can be used to enable diagnostic messages from the **amd-dbgapi** library (which GDB uses under the hood). The **show debug amd-dbgapi-lib log-level** command displays the current **amd-dbgapi** library log level. See [set debug amd-dbgapi-lib], page 388.

21.4.9.9 AMD GPU Restrictions

1. When in non-stop mode, wavefronts may not hit breakpoints inserted while not stopped, nor see memory updates made while not stopped, until the wavefront is next stopped. Memory updated by non-stopped wavefronts may not be visible until the wavefront is next stopped.
2. The HIP runtime performs deferred code object loading by default. AMD GPU code objects are not loaded until the first kernel is launched. Before then, all breakpoints have to be set as pending breakpoints.

If source line positions are used that only correspond to source lines in unloaded code objects, then GDB may not set pending breakpoints, and instead set breakpoints on the next following source line that maps to host code. This can result in unexpected breakpoint hits being reported. When the code object containing the source lines is loaded, the incorrect breakpoints will be removed and replaced by the correct ones. This problem can be avoided by only setting breakpoints in unloaded code objects using symbol or function names.

Setting the **HIP_ENABLE_DEFERRED_LOADING** environment variable to 0 can be used to disable deferred code object loading by the HIP runtime. This ensures all code objects will be loaded when the inferior reaches the beginning of the **main** function.

3. If no CPU thread is running, then ‘Ctrl-C’ is not able to stop AMD GPU threads. This can happen for example if you enable `scheduler-locking` after the whole program stopped, and then resume an AMD GPU thread. The only way to unblock the situation is to kill the GDB process.

- 4.

By default, for some architectures, the AMD GPU device driver causes all AMD GPU wavefronts created when GDB is not attached to be unable to report the dispatch associated with the wavefront, or the wavefront’s work-group position. The ‘`info threads`’ command will display this missing information with a ‘?’.

This does not affect wavefronts created while GDB is attached which are always capable of reporting this information.

If the `HSA_ENABLE_DEBUG` environment variable is set to ‘1’ when the AMD ROCm runtime is initialized, then this information will be available for all architectures even for wavefronts created when GDB was not attached.

21.4.10 RISC-V

When GDB is debugging a RISC-V architecture, it provides the following special commands:

`set riscv numeric-register-names [on|off]`

This command controls whether GDB displays RISC-V register names using their numeric or abi names. When ‘on’, GDB displays registers by their numeric names (e.g. ‘x1’). When ‘off’, GDB displays registers by their abi names (e.g. ‘ra’). The default is ‘off’.

`show riscv numeric-register-names`

Show whether GDB will refer to registers by their numeric names.

22 Controlling GDB

You can alter the way GDB interacts with you by using the `set` command. For commands controlling how GDB displays data, see Section 10.9 [Print Settings], page 159. Other settings are described here.

22.1 Prompt

GDB indicates its readiness to read a command by printing a string called the *prompt*. This string is normally `(gdb)`. You can change the prompt string with the `set prompt` command. For instance, when debugging GDB with GDB, it is useful to change the prompt in one of the GDB sessions so that you can always tell which one you are talking to.

Note: `set prompt` does not add a space for you after the prompt you set. This allows you to set a prompt which ends in a space or a prompt that does not.

`set prompt newprompt`

Directs GDB to use *newprompt* as its prompt string henceforth.

For example, this will set a blue-colored “(gdb)” prompt:

```
set prompt \001\033[0;34m\002(gdb)\001\033[0m\002
```

It uses `\001` and `\002` to begin and end a sequence of non-printing characters, to make sure they're not counted in the string length.

`show prompt`

Prints a line of the form: `Gdb's prompt is: your-prompt`

Versions of GDB that ship with Python scripting enabled have prompt extensions. The commands for interacting with these extensions are:

`set extended-prompt prompt`

Set an extended prompt that allows for substitutions. See Section 23.3.4.3 [gdb.prompt], page 535, for a list of escape sequences that can be used for substitution. Any escape sequences specified as part of the prompt string are replaced with the corresponding strings each time the prompt is displayed.

For example:

```
set extended-prompt Current working directory: \w (gdb)
```

Note that when an extended-prompt is set, it takes control of the *prompt_hook* hook. See [prompt_hook], page 414, for further information.

`show extended-prompt`

Prints the extended prompt. Any escape sequences specified as part of the prompt string with `set extended-prompt`, are replaced with the corresponding strings each time the prompt is displayed.

22.2 Command Editing

GDB reads its input commands via the *Readline* interface. This GNU library provides consistent behavior for programs which provide a command line interface to the user. Advantages are GNU Emacs-style or vi-style inline editing of commands, `csh`-like history substitution, and a storage and recall of command history across debugging sessions.

You may control the behavior of command line editing in GDB with the command **set**.

```
set editing
set editing on
    Enable command line editing (enabled by default).

set editing off
    Disable command line editing.

show editing
    Show whether command line editing is enabled.
```

See Chapter 33 [Command Line Editing], page 733, for more details about the Readline interface. Users unfamiliar with GNU Emacs or vi are encouraged to read that chapter.

GDB sets the Readline application name to ‘gdb’. This is useful for conditions in `.inputrc`.

GDB defines a bindable Readline command, **operate-and-get-next**. This is bound to `C-o` by default. This command accepts the current line for execution and fetches the next line relative to the current line from the history for editing. Any argument is ignored.

It is possible to discover which version of Readline GDB is using with the **show configuration** command (see [show configuration], page 31).

22.3 Command History

GDB can keep track of the commands you type during your debugging sessions, so that you can be certain of precisely what happened. Use these commands to manage the GDB command history facility.

GDB uses the GNU History library, a part of the Readline package, to provide the history facility. See Chapter 34 [Using History Interactively], page 759, for the detailed description of the History library.

To issue a command to GDB without affecting certain aspects of the state which is seen by users, prefix it with ‘**server**’ (see Section 28.2 [Server Prefix], page 716). This means that this command will not affect the command history, nor will it affect GDB’s notion of which command to repeat if **RET** is pressed on a line by itself.

The server prefix does not affect the recording of values into the value history; to print a value without recording it into the value history, use the **output** command instead of the **print** command.

Here is the description of GDB commands related to command history.

```
set history filename [fname]
    Set the name of the GDB command history file to fname. This is the file where
    GDB reads an initial command history list, and where it writes the command
    history from this session when it exits. You can access this list through history
    expansion or through the history command editing characters listed below.
    This file defaults to the value of the environment variable GDBHISTFILE, or to
    ./gdb_history (./_gdb_history on MS-DOS) if this variable is not set.

    The GDBHISTFILE environment variable is read after processing any GDB ini-
    tialization files (see Section 2.1.3 [Startup], page 17) and after processing any
    commands passed using command line options (for example, -ex).
```

If the *fname* argument is not given, or if the `GDBHISTFILE` is the empty string then GDB will neither try to load an existing history file, nor will it try to save the history on exit.

set history save

set history save on

Record command history in a file, whose name may be specified with the **set history filename** command. By default, this option is disabled. The command history will be recorded when GDB exits. If **set history filename** is set to the empty string then history saving is disabled, even when **set history save** is on.

set history save off

Don't record the command history into the file specified by **set history filename** when GDB exits.

set history size size

set history size unlimited

Set the number of commands which GDB keeps in its history list. This defaults to the value of the environment variable `GDBHISTSIZE`, or to 256 if this variable is not set. Non-numeric values of `GDBHISTSIZE` are ignored. If *size* is **unlimited** or if `GDBHISTSIZE` is either a negative number or the empty string, then the number of commands GDB keeps in the history list is unlimited.

The `GDBHISTSIZE` environment variable is read after processing any GDB initialization files (see Section 2.1.3 [Startup], page 17) and after processing any commands passed using command line options (for example, **-ex**).

set history remove-duplicates count

set history remove-duplicates unlimited

Control the removal of duplicate history entries in the command history list. If *count* is non-zero, GDB will look back at the last *count* history entries and remove the first entry that is a duplicate of the current entry being added to the command history list. If *count* is **unlimited** then this lookbehind is unbounded. If *count* is 0, then removal of duplicate history entries is disabled.

Only history entries added during the current session are considered for removal. This option is set to 0 by default.

save history filename

Occasionally you may want to save your history to a file. This command will do just that, saving the commands in your current session to the named file. Note that you should normally edit the saved history before using **source** to reload it, as the **save history** command itself will be at the end of the file.

Alternatively, you can use **server save history** to avoid this issue, as the **server** prefix prevents the command from being recorded (see Section 28.2 [Server Prefix], page 716).

History expansion assigns special meaning to the character **!**. See Section 34.1.1 [Event Designators], page 759, for more details.

Since **!** is also the logical not operator in C, history expansion is off by default. If you decide to enable history expansion with the **set history expansion on** command, you may

sometimes need to follow `!` (when it is used as logical not, in an expression) with a space or a tab to prevent it from being expanded. The readline history facilities do not attempt substitution on the strings `!=` and `!()`, even when history expansion is enabled.

The commands to control history expansion are:

`set history expansion on`

`set history expansion`

Enable history expansion. History expansion is off by default.

`set history expansion off`

Disable history expansion.

`show history`

`show history filename`

`show history save`

`show history size`

`show history expansion`

These commands display the state of the GDB history parameters. `show history` by itself displays all four states.

`show commands`

Display the last ten commands in the command history.

`show commands n`

Print ten commands centered on command number *n*.

`show commands +`

Print ten commands just after the commands last printed.

22.4 Screen Size

Certain commands to GDB may produce large amounts of information output to the screen. To help you read all of it, GDB pauses and asks you for input at the end of each page of output. Type `RET` when you want to see one more page of output, `q` to discard the remaining output, or `c` to continue without paging for the rest of the current command. Also, the screen width setting determines when to wrap lines of output. Depending on what is being printed, GDB tries to break the line at a readable place, rather than simply letting it overflow onto the following line.

Normally GDB knows the size of the screen from the terminal driver software. For example, on Unix GDB uses the termcap data base together with the value of the `TERM` environment variable and the `stty rows` and `stty cols` settings. If this is not correct, you can override it with the `set height` and `set width` commands:

`set height lpp`

`set height unlimited`

`show height`

`set width cpl`

`set width unlimited`

`show width`

These `set` commands specify a screen height of *lpp* lines and a screen width of *cpl* characters. The associated `show` commands display the current settings.

If you specify a height of either **unlimited** or zero lines, GDB does not pause during output no matter how long the output is. This is useful if output is to a file or to an editor buffer.

Likewise, you can specify **'set width unlimited'** or **'set width 0'** to prevent GDB from wrapping its output.

set pagination on

set pagination off

Turn the output pagination on or off; the default is on. Turning pagination off is the alternative to **set height unlimited**. Note that running GDB with the **--batch** option (see Section 2.1.2 [Mode Options], page 13) also automatically disables pagination.

show pagination

Show the current pagination mode.

22.5 Output Styling

GDB can style its output on a capable terminal. This is enabled by default on most systems, but disabled by default when in batch mode (see Section 2.1.2 [Mode Options], page 13). Various style settings are available; and styles can also be disabled entirely.

set style enabled 'on|off'

Enable or disable all styling. The default is host-dependent, with most hosts defaulting to **'on'**.

If the **NO_COLOR** environment variable is set to a non-empty value, then GDB will change this to **'off'** at startup.

show style enabled

Show the current state of styling.

set style emoji 'auto|on|off'

Enable or disable the use of emoji. On most hosts, the default is **'auto'**, meaning that emoji will only be used if the host character set is **'UTF-8'**; however, on Windows the default is **'off'** when using the console, unless the console's output codepage is 65001, the Windows UTF-8 codepage. Note that disabling styling as a whole will also prevent emoji display.

Currently, emoji are printed whenever GDB reports an error or a warning.

show style emoji

Show the current state of emoji output.

set style sources 'on|off'

Enable or disable source code styling. This affects whether source code, such as the output of the **list** command, is styled. The default is **'on'**. Note that source styling only works if styling in general is enabled, and if a source highlighting library is available to GDB.

There are two ways that highlighting can be done. First, if GDB was linked with the GNU Source Highlight library, then it is used. Otherwise, if GDB was configured with Python scripting support, and if the Python Pygments package is available, then it will be used.

show style sources

Show the current state of source code styling.

set style warning-prefix

show style warning-prefix

set style error-prefix

show style error-prefix

These commands control the prefix that is printed before warnings and errors, respectively. This functionality is intended for use with emoji display, and so the prefixes are only displayed if emoji styling is enabled. The defaults are the warning sign emoji for warnings, and the cross mark emoji for errors.

set style tui-current-position 'on|off'

Enable or disable styling of the source and assembly code highlighted by the TUI's current position indicator. The default is 'off'. See Chapter 25 [GDB Text User Interface], page 599.

show style tui-current-position

Show whether the source and assembly code highlighted by the TUI's current position indicator is styled.

set style disassembler enabled 'on|off'

Enable or disable disassembler styling. This affects whether disassembler output, such as the output of the **disassemble** command, is styled. Disassembler styling only works if styling in general is enabled (with **set style enabled on**), and if a source highlighting library is available to GDB.

The two source highlighting libraries that GDB could use to style disassembler output are; GDB's builtin disassembler, or the Python Pygments package.

GDB's first choice will be to use the builtin disassembler for styling, this usually provides better results, being able to style different types of instruction operands differently. However, the builtin disassembler is not able to style all architectures.

For architectures that the builtin disassembler is unable to style, GDB will fall back to use the Python Pygments package where possible. In order to use the Python Pygments package, GDB must be built with Python support, and the Pygments package must be installed.

If neither of these options are available then GDB will produce unstyled disassembler output, even when this setting is 'on'.

To discover if the current architecture supports styling using the builtin disassembler library see [**maint show libopcodes-styling enabled**], page 792.

show style disassembler enabled

Show the current state of disassembler styling.

Subcommands of **set style** control specific forms of styling. These subcommands all follow the same pattern: each style-able object can be styled with a foreground color, a background color, and an intensity.

For example, the style of file names can be controlled using the **set style filename** group of commands:

set style filename background color

Set the background to *color*. *color* can be **‘none’** (meaning the terminal’s default color), a name of one of the eight standard colors of ISO/IEC 6429, index from 0 to 255 into terminal’s color palette or a hexadecimal RGB triplet in **‘#RRGGBB’** format for 24-bit TrueColor.

Valid color names are **‘black’**, **‘red’**, **‘green’**, **‘yellow’**, **‘blue’**, **‘magenta’**, **‘cyan’**, and **‘white’**.

Integers 0 to 7 are the synonyms for the standard colors. Integers 8-15 are used for the so-called bright colors from the aixterm extended 16-color palette. Integers 16-255 are the indexes into xterm extended 256-color palette (usually 6x6x6 cube plus gray ramp). In general, 256-color palette is terminal dependent and sometimes can be changed with OSC 4 sequences, e.g. **“\033]4;1;rgb:00/FF/00\033\\”**. A hexadecimal 24-bit TrueColor is specified in the format **‘#RRGGBB’** where RR, GG and BB are the 2-digit hexadecimal integers specifying the intensity of the red, green and blue color components, respectively.

It is the responsibility of the user to verify that the terminal supports the specified colors.

set style filename foreground color

Set the foreground to *color*. *color* can be given in the same ways as for the background.

set style filename intensity value

Set the intensity to *value*. Valid intensities are **‘normal’** (the default), **‘bold’**, and **‘dim’**.

The **show style** command and its subcommands are styling a style name in their output using its own style. So, use **show style** to see the complete list of styles, their characteristics and the visual aspect of each style.

The style-able objects are:

command Control the styling of any GDB commands that are displayed by GDB. By default, this style’s intensity is bold.

filename Control the styling of file names and URLs. By default, this style’s foreground color is green.

function Control the styling of function names. These are managed with the **set style function** family of commands. By default, this style’s foreground color is yellow.

This style is also used for symbol names in styled disassembler output if GDB is using its builtin disassembler library for styling (see [**set style disassembler enabled**], page 376).

line-number

Control the styling of line numbers. By default, this style’s intensity is dim.

variable Control the styling of variable names. These are managed with the **set style variable** family of commands. By default, this style’s foreground color is cyan.

- address** Control the styling of addresses. These are managed with the **set style address** family of commands. By default, this style's foreground color is blue. This style is also used for addresses in styled disassembler output if GDB is using its builtin disassembler library for styling (see [**set style disassembler enabled**], page 376).
- version** Control the styling of GDB's version number text. By default, this style's foreground color is magenta and it has bold intensity. The version number is displayed in two places, the output of **show version**, and when GDB starts up. In order to control how GDB styles the version number at startup, add the **set style version** family of commands to the early initialization command file (see Section 2.1.4 [Initialization Files], page 18).
- title** Control the styling of titles. These are managed with the **set style title** family of commands. By default, this style's intensity is bold. The title style is used when displaying the header line of a list, for example the first line of the output of **info breakpoints** (see Section 5.1.1 [Set Breaks], page 60).
- highlight** Control the styling of highlightings. These are managed with the **set style highlight** family of commands. By default, this style's foreground color is red. Commands are using the highlight style to draw the user attention to some specific parts of their output. For example, the command **apropos -v REGEXP** uses the highlight style to mark the documentation parts matching *regexp*.
- metadata** Control the styling of data annotations added by GDB to data it displays. By default, this style's intensity is dim. Metadata annotations include the '**repeats n times**' annotation for suppressed display of repeated array elements (see Section 10.9 [Print Settings], page 159), '**<unavailable>**' and '**<error descr>**' annotations for errors and '**<optimized-out>**' annotations for optimized-out values in displaying stack frame information in backtraces (see Section 8.2 [Backtrace], page 116), etc.
- tui-border** Control the styling of the TUI border. Note that, unlike other styling options, only the color of the border can be controlled via **set style**. This was done for compatibility reasons, as TUI controls to set the border's intensity predated the addition of general styling to GDB. See Section 25.6 [TUI Configuration], page 605.
- tui-active-border** Control the styling of the active TUI border; that is, the TUI window that has the focus.
- disassembler comment** Control the styling of comments in the disassembler output. These are managed with the **set style disassembler comment** family of commands. This style is only used when GDB is styling using its builtin disassembler library (see [**set style disassembler enabled**], page 376). By default, this style's intensity is dim, and its foreground color is white.

disassembler immediate

Control the styling of numeric operands in the disassembler output. These are managed with the **set style disassembler immediate** family of commands. This style is not used for instruction operands that represent addresses, in that case the ‘**disassembler address**’ style is used. This style is only used when GDB is styling using its builtin disassembler library. By default, this style’s foreground color is blue.

disassembler address

Control the styling of address operands in the disassembler output. This is an alias for the ‘**address**’ style.

disassembler symbol

Control the styling of symbol names in the disassembler output. This is an alias for the ‘**function**’ style.

disassembler mnemonic

Control the styling of instruction mnemonics in the disassembler output. These are managed with the **set style disassembler mnemonic** family of commands. This style is also used for assembler directives, e.g. **.byte**, **.word**, etc. This style is only used when GDB is styling using its builtin disassembler library. By default, this style’s foreground color is green.

disassembler register

Control the styling of register operands in the disassembler output. These are managed with the **set style disassembler register** family of commands. This style is only used when GDB is styling using its builtin disassembler library. By default, this style’s foreground color is red.

22.6 Numbers

You can always enter numbers in octal, decimal, or hexadecimal in GDB by the usual conventions: octal numbers begin with ‘0’, decimal numbers end with ‘.’, and hexadecimal numbers begin with ‘0x’. Numbers that neither begin with ‘0’ or ‘0x’, nor end with a ‘.’ are, by default, entered in base 10; likewise, the default display for numbers—when no particular format is specified—is base 10. You can change the default base for both input and output with the commands described below.

set input-radix base

Set the default base for numeric input. Supported choices for *base* are decimal 8, 10, or 16. The base must itself be specified either unambiguously or using the current input radix; for example, any of

```
set input-radix 012
set input-radix 10.
set input-radix 0xa
```

sets the input base to decimal. On the other hand, ‘**set input-radix 10**’ leaves the input radix unchanged, no matter what it was, since ‘10’, being without any leading or trailing signs of its base, is interpreted in the current radix. Thus, if the current radix is 16, ‘10’ is interpreted in hex, i.e. as 16 decimal, which doesn’t change the radix.

set output-radix *base*

Set the default base for numeric display. Supported choices for *base* are decimal 8, 10, or 16. The base must itself be specified either unambiguously or using the current input radix.

show input-radix

Display the current default base for numeric input.

show output-radix

Display the current default base for numeric display.

set radix [*base*]

show radix

These commands set and show the default base for both input and output of numbers. **set radix** sets the radix of input and output to the same base; without an argument, it resets the radix back to its default value of 10.

22.7 Configuring the Current ABI

GDB can determine the *ABI* (Application Binary Interface) of your application automatically. However, sometimes you need to override its conclusions. Use these commands to manage GDB’s view of the current ABI.

One GDB configuration can debug binaries for multiple operating system targets, either via remote debugging or native emulation. GDB will autodetect the *OS ABI* (Operating System ABI) in use, but you can override its conclusion using the **set osabi** command. One example where this is useful is in debugging of binaries which use an alternate C library (e.g. UCLIBC for GNU/Linux) which does not have the same identifying marks that the standard C library for your platform provides.

When GDB is debugging the AArch64 architecture, it provides a “Newlib” OS ABI. This is useful for handling **setjmp** and **longjmp** when debugging binaries that use the NEWLIB C library. The “Newlib” OS ABI can be selected by **set osabi Newlib**.

show osabi

Show the OS ABI currently in use.

set osabi With no argument, show the list of registered available OS ABI’s.

set osabi *abi*

Set the current OS ABI to *abi*.

Generally, the way that an argument of type **float** is passed to a function depends on whether the function is prototyped. For a prototyped (i.e. ANSI/ISO style) function, **float** arguments are passed unchanged, according to the architecture’s convention for **float**. For unprototyped (i.e. K&R style) functions, **float** arguments are first promoted to type **double** and then passed.

Unfortunately, some forms of debug information do not reliably indicate whether a function is prototyped. If GDB calls a function that is not marked as prototyped, it consults **set coerce-float-to-double**.

set coerce-float-to-double

set coerce-float-to-double on

Arguments of type `float` will be promoted to `double` when passed to an unprototyped function. This is the default setting.

set coerce-float-to-double off

Arguments of type `float` will be passed directly to unprototyped functions.

show coerce-float-to-double

Show the current setting of promoting `float` to `double`.

GDB needs to know the ABI used for your program's C++ objects. The correct C++ ABI depends on which C++ compiler was used to build your application. GDB only fully supports programs with a single C++ ABI; if your program contains code using multiple C++ ABI's or if GDB can not identify your program's ABI correctly, you can tell GDB which ABI to use. Currently supported ABI's include "gnu-v2", for g++ versions before 3.0, "gnu-v3", for g++ versions 3.0 and later, and "hpaCC" for the HP ANSI C++ compiler. Other C++ compilers may use the "gnu-v2" or "gnu-v3" ABI's as well. The default setting is "auto".

show cp-abi

Show the C++ ABI currently in use.

set cp-abi

With no argument, show the list of supported C++ ABI's.

set cp-abi abi

set cp-abi auto

Set the current C++ ABI to *abi*, or return to automatic detection.

22.8 Automatically loading associated files

GDB sometimes reads files with commands and settings automatically, without being explicitly told so by the user. We call this feature *auto-loading*. While auto-loading is useful for automatically adapting GDB to the needs of your project, it can sometimes produce unexpected results or introduce security risks (e.g., if the file comes from untrusted sources).

There are various kinds of files GDB can automatically load. In addition to these files, GDB supports auto-loading code written in various extension languages. See Section 23.5 [Auto-loading extensions], page 592.

Note that loading of these associated files (including the local `.gdbinit` file) requires accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

For these reasons, GDB includes commands and options to let you control when to auto-load files and which files should be auto-loaded.

set auto-load off

Globally disable loading of all auto-loaded files. You may want to use this command with the `-iex` option (see [Option -init-eval-command], page 17) such as:

```
$ gdb -iex "set auto-load off" untrusted-executable corefile
```

Be aware that system init file (see Section C.6 [System-wide configuration], page 775) and init files from your home directory (see [Home Directory Init

File], page 19) still get read (as they come from generally trusted directories). To prevent GDB from auto-loading even those init files, use the `-nx` option (see Section 2.1.2 [Mode Options], page 13), in addition to `set auto-load no`.

`show auto-load`

Show whether auto-loading of each specific ‘auto-load’ file(s) is enabled or disabled.

```
(gdb) show auto-load
gdb-scripts: Auto-loading of canned sequences of commands scripts is on.
libthread-db: Auto-loading of inferior specific libthread_db is on.
local-gdbinit: Auto-loading of .gdbinit script from current directory
                is on.
python-scripts: Auto-loading of Python scripts is on.
safe-path: List of directories from which it is safe to auto-load files
            is $debugdir:$datadir/auto-load.
scripts-directory: List of directories from which to load auto-loaded scripts
                   is $debugdir:$datadir/auto-load.
```

`info auto-load`

Print whether each specific ‘auto-load’ file(s) have been auto-loaded or not.

```
(gdb) info auto-load
gdb-scripts:
Loaded Script
Yes      /home/user/gdb/gdb-gdb.gdb
libthread-db: No auto-loaded libthread-db.
local-gdbinit: Local .gdbinit file "/home/user/gdb/.gdbinit" has been
                loaded.
python-scripts:
Loaded Script
Yes      /home/user/gdb/gdb-gdb.py
```

These are GDB control commands for the auto-loading:

See [set auto-load off], page 381.	Disable auto-loading globally.
See [show auto-load], page 382.	Show setting of all kinds of files.
See [info auto-load], page 382.	Show state of all kinds of files.
See [set auto-load gdb-scripts], page 405.	Control for GDB command scripts.
See [show auto-load gdb-scripts], page 405.	Show setting of GDB command scripts.
See [info auto-load gdb-scripts], page 405.	Show state of GDB command scripts.
See [set auto-load python-scripts], page 532.	Control for GDB Python scripts.
See [show auto-load python-scripts], page 532.	Show setting of GDB Python scripts.
See [info auto-load python-scripts], page 532.	Show state of GDB Python scripts.
See [set auto-load guile-scripts], page 590.	Control for GDB Guile scripts.
See [show auto-load guile-scripts], page 590.	Show setting of GDB Guile scripts.
See [info auto-load guile-scripts], page 590.	Show state of GDB Guile scripts.
See [set auto-load scripts-directory], page 593.	Control for GDB auto-loaded scripts location.
See [show auto-load scripts-directory], page 593.	Show GDB auto-loaded scripts location.
See [add-auto-load-scripts-directory], page 593.	Add directory for auto-loaded scripts location list.
See [set auto-load local-gdbinit], page 383.	Control for init file in the current directory.

See [show auto-load local-gdbinit], page 383.	Show setting of init file in the current directory.
See [info auto-load local-gdbinit], page 383.	Show state of init file in the current directory.
See [set auto-load libthread-db], page 383.	Control for thread debugging library.
See [show auto-load libthread-db], page 383.	Show setting of thread debugging library.
See [info auto-load libthread-db], page 384.	Show state of thread debugging library.
See [set auto-load safe-path], page 384.	Control directories trusted for automatic loading.
See [show auto-load safe-path], page 384.	Show directories trusted for automatic loading.
See [add-auto-load-safe-path], page 384.	Add directory trusted for automatic loading.

22.8.1 Automatically loading init file in the current directory

By default, GDB reads and executes the canned sequences of commands from init file (if any) in the current working directory, see [Init File in the Current Directory during Startup], page 20.

Note that loading of this local `.gdbinit` file also requires accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

set auto-load local-gdbinit [on|off]

Enable or disable the auto-loading of canned sequences of commands (see Section 23.1 [Sequences], page 397) found in init file in the current directory.

show auto-load local-gdbinit

Show whether auto-loading of canned sequences of commands from init file in the current directory is enabled or disabled.

info auto-load local-gdbinit

Print whether canned sequences of commands from init file in the current directory have been auto-loaded.

22.8.2 Automatically loading thread debugging library

This feature is currently present only on GNU/Linux native hosts.

GDB reads in some cases thread debugging library from places specific to the inferior (see [set libthread-db-search-path], page 52).

The special ‘`libthread-db-search-path`’ entry ‘`$sdir`’ is processed without checking this ‘`set auto-load libthread-db`’ switch as system libraries have to be trusted in general. In all other cases of ‘`libthread-db-search-path`’ entries GDB checks first if ‘`set auto-load libthread-db`’ is enabled before trying to open such thread debugging library.

Note that loading of this debugging library also requires accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

set auto-load libthread-db [on|off]

Enable or disable the auto-loading of inferior specific thread debugging library.

show auto-load libthread-db

Show whether auto-loading of inferior specific thread debugging library is enabled or disabled.

info auto-load libthread-db

Print the list of all loaded inferior specific thread debugging libraries and for each such library print list of inferior *pids* using it.

22.8.3 Security restriction for auto-loading

As the files of inferior can come from untrusted source (such as submitted by an application user) GDB does not always load any files automatically. GDB provides the ‘**set auto-load safe-path**’ setting to list directories trusted for loading files not explicitly requested by user. Each directory can also be a shell wildcard pattern.

If the path is not set properly you will see a warning and the file will not get loaded:

```
$ ./gdb -q ./gdb
Reading symbols from /home/user/gdb/gdb...
warning: File "/home/user/gdb/gdb-gdb.gdb" auto-loading has been
declined by your 'auto-load safe-path' set
to "$debugdir:$datadir/auto-load".
warning: File "/home/user/gdb/gdb-gdb.py" auto-loading has been
declined by your 'auto-load safe-path' set
to "$debugdir:$datadir/auto-load".
```

To instruct GDB to go ahead and use the init files anyway, invoke GDB like this:

```
$ gdb -q -iex "set auto-load safe-path /home/user/gdb" ./gdb
```

The list of trusted directories is controlled by the following commands:

set auto-load safe-path [directories]

Set the list of directories (and their subdirectories) trusted for automatic loading and execution of scripts. You can also enter a specific trusted file. Each directory can also be a shell wildcard pattern; wildcards do not match directory separator - see `FNМ_PATHNAME` for system function `fnmatch` (see Section “Wildcard Matching” in *GNU C Library Reference Manual*). If you omit *directories*, ‘**auto-load safe-path**’ will be reset to its default value as specified during GDB compilation.

The list of directories uses path separator (‘:’ on GNU and Unix systems, ‘;’ on MS-Windows and MS-DOS) to separate directories, similarly to the `PATH` environment variable.

show auto-load safe-path

Show the list of directories trusted for automatic loading and execution of scripts.

add-auto-load-safe-path

Add an entry (or list of entries) to the list of directories trusted for automatic loading and execution of scripts. Multiple entries may be delimited by the host platform path separator in use.

This variable defaults to what `--with-auto-load-dir` has been configured to (see [with-auto-load-dir], page 593). `$debugdir` and `$datadir` substitution applies the same as for

[`set auto-load scripts-directory`], page 593. The default `set auto-load safe-path` value can be also overridden by GDB configuration option `--with-auto-load-safe-path`.

Setting this variable to `/` disables this security protection, corresponding GDB configuration option is `--without-auto-load-safe-path`. This variable is supposed to be set to the system directories writable by the system superuser only. Users can add their source directories in init files in their home directories (see [Home Directory Init File], page 19). See also deprecated init file in the current directory (see [Init File in the Current Directory during Startup], page 20).

To force GDB to load the files it declined to load in the previous example, you could use one of the following ways:

```
~/gdbinit: 'add-auto-load-safe-path ~/src/gdb'
```

Specify this trusted directory (or a file) as additional component of the list. You have to specify also any existing directories displayed by by `'show auto-load safe-path'` (such as `'/usr:/bin'` in this example).

```
gdb -iex "set auto-load safe-path /usr:/bin:~/src/gdb" ...
```

Specify this directory as in the previous case but just for a single GDB session.

```
gdb -iex "set auto-load safe-path /" ...
```

Disable auto-loading safety for a single GDB session. This assumes all the files you debug during this GDB session will come from trusted sources.

```
./configure --without-auto-load-safe-path
```

During compilation of GDB you may disable any auto-loading safety. This assumes all the files you will ever debug with this GDB come from trusted sources.

On the other hand you can also explicitly forbid automatic files loading which also suppresses any such warning messages:

```
gdb -iex "set auto-load no" ...
```

You can use GDB command-line option for a single GDB session.

```
~/gdbinit: 'set auto-load no'
```

Disable auto-loading globally for the user (see [Home Directory Init File], page 19). While it is improbable, you could also use system init file instead (see Section C.6 [System-wide configuration], page 775).

This setting applies to the file names as entered by user. If no entry matches GDB tries as a last resort to also resolve all the file names into their canonical form (typically resolving symbolic links) and compare the entries again. GDB already canonicalizes most of the filenames on its own before starting the comparison so a canonical form of directories is recommended to be entered.

22.8.4 Displaying files tried for auto-load

For better visibility of all the file locations where you can place scripts to be auto-loaded with inferior — or to protect yourself against accidental execution of untrusted scripts — GDB provides a feature for printing all the files attempted to be loaded. Both existing and non-existing files may be printed.

For example the list of directories from which it is safe to auto-load files (see Section 22.8.3 [Auto-loading safe path], page 384) applies also to canonicalized filenames which may not be too obvious while setting it up.

```
(gdb) set debug auto-load on
(gdb) file ~/src/t/true
auto-load: Loading canned sequences of commands script "/tmp/true-gdb.gdb"
          for objfile "/tmp/true".
auto-load: Updating directories of "/usr:/opt".
auto-load: Using directory "/usr".
auto-load: Using directory "/opt".
warning: File "/tmp/true-gdb.gdb" auto-loading has been declined
          by your 'auto-load safe-path' set to "/usr:/opt".
```

set debug auto-load [on|off]

Set whether to print the filenames attempted to be auto-loaded.

show debug auto-load

Show whether printing of the filenames attempted to be auto-loaded is turned on or off.

22.9 Optional Warnings and Messages

By default, GDB is silent about its inner workings. If you are running on a slow machine, you may want to use the **set verbose** command. This makes GDB tell you when it does a lengthy internal operation, so you will not think it has crashed.

Currently, the messages controlled by **set verbose** are those which announce that the symbol table for a source file is being read; see **symbol-file** in Section 18.1 [Commands to Specify Files], page 293.

set verbose on

Enables GDB output of certain informational messages.

set verbose off

Disables GDB output of certain informational messages.

show verbose

Displays whether **set verbose** is on or off.

By default, if GDB encounters bugs in the symbol table of an object file, it is silent; but if you are debugging a compiler, you may find this information useful (see Section 18.7 [Errors Reading Symbol Files], page 310).

set complaints limit

Permits GDB to output *limit* complaints about each type of unusual symbols before becoming silent about the problem. Set *limit* to zero to suppress all complaints; set it to a large number to prevent complaints from being suppressed.

show complaints

Displays how many symbol complaints GDB is permitted to produce.

By default, GDB is cautious, and asks what sometimes seems to be a lot of stupid questions to confirm certain commands. For example, if you try to run a program which is already running:

```
(gdb) run
```

```
The program being debugged has been started already.  
Start it from the beginning? (y or n)
```

If you are willing to unflinchingly face the consequences of your own commands, you can disable this “feature”:

```
set confirm off
```

Disables confirmation requests. Note that running GDB with the `--batch` option (see Section 2.1.2 [Mode Options], page 13) also automatically disables confirmation requests.

```
set confirm on
```

Enables confirmation requests (the default).

```
show confirm
```

Displays state of confirmation requests.

If you need to debug user-defined commands or sourced files you may find it useful to enable *command tracing*. In this mode each command will be printed as it is executed, prefixed with one or more ‘+’ symbols, the quantity denoting the call depth of each command.

```
set trace-commands on
```

Enable command tracing.

```
set trace-commands off
```

Disable command tracing.

```
show trace-commands
```

Display the current state of command tracing.

22.10 Optional Messages about Internal Happenings

GDB has commands that enable optional debugging messages from various GDB subsystems; normally these commands are of interest to GDB maintainers, or when reporting a bug. This section documents those commands.

```
set exec-done-display
```

Turns on or off the notification of asynchronous commands’ completion. When on, GDB will print a message when an asynchronous command finishes its execution. The default is off.

```
show exec-done-display
```

Displays the current setting of asynchronous command completion notification.

```
set debug aarch64
```

Turns on or off display of debugging messages related to ARM AArch64. The default is off.

```
show debug aarch64
```

Displays the current state of displaying debugging messages related to ARM AArch64.

```
set debug arch
```

Turns on or off display of gdbarch debugging info. The default is off

show debug arch

Displays the current state of displaying gdbarch debugging info.

set debug aix-thread

Display debugging messages about inner workings of the AIX thread module.

show debug aix-thread

Show the current state of AIX thread debugging info display.

set debug amd-dbgapi-lib

show debug amd-dbgapi-lib

The **set debug amd-dbgapi-lib log-level *level*** command can be used to enable diagnostic messages from the ‘amd-dbgapi’ library, where *level* can be:

off	no logging is enabled
error	fatal errors are reported
warning	fatal errors and warnings are reported
info	fatal errors, warnings, and info messages are reported
verbose	all messages are reported

The **show debug amd-dbgapi-lib log-level** command displays the current amd-dbgapi library log level.

set debug amd-dbgapi

show debug amd-dbgapi

The ‘**set debug amd-dbgapi**’ command can be used to enable diagnostic messages in the ‘amd-dbgapi’ target. The ‘**show debug amd-dbgapi**’ command displays the current setting. See [set debug amd-dbgapi], page 388.

set debug check-physname

Check the results of the “physname” computation. When reading DWARF debugging information for C++, GDB attempts to compute each entity’s name. GDB can do this computation in two different ways, depending on exactly what information is present. When enabled, this setting causes GDB to compute the names both ways and display any discrepancies.

show debug check-physname

Show the current state of “physname” checking.

set debug coff-pe-read

Control display of debugging messages related to reading of COFF/PE exported symbols. The default is off.

show debug coff-pe-read

Displays the current state of displaying debugging messages related to reading of COFF/PE exported symbols.

set debug ctf [on | off]

Turns on or off display of debugging messages related to reading CTF debug info. The default is off.

show debug ctf

Displays the current state of CTF debug info reading messages.

set debug dwarf-die
Dump DWARF DIEs after they are read in. The value is the number of nesting levels to print. A value of zero turns off the display.

show debug dwarf-die
Show the current state of DWARF DIE debugging.

set debug dwarf-line
Turns on or off display of debugging messages related to reading DWARF line tables. The default is 0 (off). A value of 1 provides basic information. A value greater than 1 provides more verbose information.

show debug dwarf-line
Show the current state of DWARF line table debugging.

set debug dwarf-read
Turns on or off display of debugging messages related to reading DWARF debug info. The default is 0 (off). A value of 1 provides basic information. A value greater than 1 provides more verbose information.

show debug dwarf-read
Show the current state of DWARF reader debugging.

set debug displaced
Turns on or off display of GDB debugging info for the displaced stepping support. The default is off.

show debug displaced
Displays the current state of displaying GDB debugging info related to displaced stepping.

set debug event
Turns on or off display of GDB event debugging info. The default is off.

show debug event
Displays the current state of displaying GDB event debugging info.

set debug event-loop
Controls output of debugging info about the event loop. The possible values are ‘off’, ‘all’ (shows all debugging info) and ‘all-except-ui’ (shows all debugging info except those about UI-related events).

show debug event-loop
Shows the current state of displaying debugging info about the event loop.

set debug expression
Turns on or off display of debugging info about GDB expression parsing. The default is off.

show debug expression
Displays the current state of displaying debugging info about GDB expression parsing.

set debug fbsd-lwp
Turns on or off debugging messages from the FreeBSD LWP debug support.

show debug fbsd-lwp
Show the current state of FreeBSD LWP debugging messages.

set debug fbsd-nat
Turns on or off debugging messages from the FreeBSD native target.

show debug fbsd-nat
Show the current state of FreeBSD native target debugging messages.

set debug fortran-array-slicing
Turns on or off display of GDB Fortran array slicing debugging info. The default is off.

show debug fortran-array-slicing
Displays the current state of displaying GDB Fortran array slicing debugging info.

set debug frame
Turns on or off display of GDB frame debugging info. The default is off.

show debug frame
Displays the current state of displaying GDB frame debugging info.

set debug gnu-ifunc
Turn on or off debugging messages related to GNU ifunc resolution.

show debug gnu-ifunc
Show the current state of GNU ifunc resolution debugging messages.

set debug gnu-nat
Turn on or off debugging messages from the GNU/Hurd debug support.

show debug gnu-nat
Show the current state of GNU/Hurd debugging messages.

set debug infrun
Turns on or off display of GDB debugging info for running the inferior. The default is off. `infrun.c` contains GDB's runtime state machine used for implementing operations such as single-stepping the inferior.

show debug infrun
Displays the current state of GDB inferior debugging.

set debug infcall
Turns on or off display of debugging info related to inferior function calls made by GDB.

show debug infcall
Displays the current state of GDB inferior function call debugging.

set debug jit
Turn on or off debugging messages from JIT debug support.

show debug jit
Displays the current state of GDB JIT debugging.

set debug linux-nat [on|off]
Turn on or off debugging messages from the Linux native target debug support.

show debug linux-nat
Show the current state of Linux native target debugging messages.

set debug linux-namespaces
Turn on or off debugging messages from the Linux namespaces debug support.

show debug linux-namespaces
Show the current state of Linux namespaces debugging messages.

set debug mach-o
Control display of debugging messages related to Mach-O symbols processing.
The default is off.

show debug mach-o
Displays the current state of displaying debugging messages related to reading of COFF/PE exported symbols.

set debug notification
Turn on or off debugging messages about remote async notification. The default is off.

show debug notification
Displays the current state of remote async notification debugging messages.

set debug observer
Turns on or off display of GDB observer debugging. This includes info such as the notification of observable events.

show debug observer
Displays the current state of observer debugging.

set debug overload
Turns on or off display of GDB C++ overload debugging info. This includes info such as ranking of functions, etc. The default is off.

show debug overload
Displays the current state of displaying GDB C++ overload debugging info.

set debug parser
Turns on or off the display of expression parser debugging output. Internally, this sets the `yydebug` variable in the expression parser. See Section “Tracing Your Parser” in *Bison*, for details. The default is off.

show debug parser
Show the current state of expression parser debugging.

set debug remote
Turns on or off display of reports on all packets sent back and forth across the serial line to the remote machine. The info is printed on the GDB standard output stream. The default is off.

show debug remote
Displays the state of display of remote packets.

set debug remote-packet-max-chars

Sets the maximum number of characters to display for each remote packet when **set debug remote** is on. This is useful to prevent GDB from displaying lengthy remote packets and polluting the console.

The default value is 512, which means GDB will truncate each remote packet after 512 bytes.

Setting this option to **unlimited** will disable truncation and will output the full length of the remote packets.

show debug remote-packet-max-chars

Displays the number of bytes to output for remote packet debugging.

set debug separate-debug-file

Turns on or off display of debug output about separate debug file search.

show debug separate-debug-file

Displays the state of separate debug file search debug output.

set debug serial

Turns on or off display of GDB serial debugging info. The default is off.

show debug serial

Displays the current state of displaying GDB serial debugging info.

set debug solib

Turns on or off display of debugging messages related to shared libraries. The default is off.

show debug solib

Show the current state of solib debugging messages.

set debug symbol-lookup

Turns on or off display of debugging messages related to symbol lookup. The default is 0 (off). A value of 1 provides basic information. A value greater than 1 provides more verbose information.

show debug symbol-lookup

Show the current state of symbol lookup debugging messages.

set debug symfile

Turns on or off display of debugging messages related to symbol file functions. The default is off. See Section 18.1 [Files], page 293.

show debug symfile

Show the current state of symbol file debugging messages.

set debug symtab-create

Turns on or off display of debugging messages related to symbol table creation. The default is 0 (off). A value of 1 provides basic information. A value greater than 1 provides more verbose information.

show debug symtab-create

Show the current state of symbol table creation debugging.

set debug target

Turns on or off display of GDB target debugging info. This info includes what is going on at the target level of GDB, as it happens. The default is 0. Set it to 1 to track events, and to 2 to also track the value of large memory transfers.

show debug target

Displays the current state of displaying GDB target debugging info.

set debug timestamp

Turns on or off display of timestamps with GDB debugging info. When enabled, seconds and microseconds are displayed before each debugging message.

show debug timestamp

Displays the current state of displaying timestamps with GDB debugging info.

set debug varobj

Turns on or off display of GDB variable object debugging info. The default is off.

show debug varobj

Displays the current state of displaying GDB variable object debugging info.

set debug xml

Turn on or off debugging messages for built-in XML parsers.

show debug xml

Displays the current state of XML debugging messages.

set debug breakpoints

Turns on or off display of GDB debugging info for breakpoint insertion and removal. The default is off.

show debug breakpoints

Displays the current state of displaying GDB debugging info for breakpoint insertion and removal.

22.11 Other Miscellaneous Settings

set interactive-mode

If **on**, forces GDB to assume that GDB was started in a terminal. In practice, this means that GDB should wait for the user to answer queries generated by commands entered at the command prompt. If **off**, forces GDB to operate in the opposite mode, and it uses the default answers to all queries. If **auto** (the default), GDB tries to determine whether its standard input is a terminal, and works in interactive-mode if it is, non-interactively otherwise.

In the vast majority of cases, the debugger should be able to guess correctly which mode should be used. But this setting can be useful in certain specific cases, such as running a MinGW GDB inside a cygwin window.

show interactive-mode

Displays whether the debugger is operating in interactive mode or not.

set suppress-cli-notifications

If **on**, command-line-interface (CLI) notifications that are printed by GDB are suppressed. If **off**, the notifications are printed as usual. The default value is **off**. CLI notifications occur when you change the selected context or when the program being debugged stops, as detailed below.

User-selected context changes:

When you change the selected context (i.e. the current inferior, thread and/or the frame), GDB prints information about the new context. For example, the default behavior is below:

```
(gdb) inferior 1
[Switching to inferior 1 [process 634] (/tmp/test)]
[Switching to thread 1 (process 634)]
#0  main () at test.c:3
3      return 0;
(gdb)
```

When the notifications are suppressed, the new context is not printed:

```
(gdb) set suppress-cli-notifications on
(gdb) inferior 1
(gdb)
```

The program being debugged stops:

When the program you are debugging stops (e.g. because of hitting a breakpoint, completing source-stepping, an interrupt, etc.), GDB prints information about the stop event. For example, below is a breakpoint hit:

```
(gdb) break test.c:3
Breakpoint 2 at 0x5555555555155: file test.c, line 3.
(gdb) continue
Continuing.

Breakpoint 2, main () at test.c:3
3      return 0;
(gdb)
```

When the notifications are suppressed, the output becomes:

```
(gdb) break test.c:3
Breakpoint 2 at 0x5555555555155: file test.c, line 3.
(gdb) set suppress-cli-notifications on
(gdb) continue
Continuing.
(gdb)
```

Suppressing CLI notifications may be useful in scripts to obtain a reduced output from a list of commands.

show local-environment [varname]

Print the value of the local environment variable *varname*. If you do not supply *varname*, print the names and values of all local environment variables. A local environment variable is used when GDB executes a local command, such as by the **shell** command.

set local-environment varname [=value]

Set environment variable *varname* to *value*. The value changes for GDB itself. The *value* may be any string; the values of environment variables are just strings, and any interpretation is supplied by your program itself. The *value* parameter is optional; if it is eliminated, the variable is set to a null value.

unset local-environment varname

Remove variable *varname* from the local environment. This is different from ‘**set local-environment varname =**’; **unset local-environment** removes the variable from the environment, rather than assigning it an empty value.

show suppress-cli-notifications

Displays whether printing CLI notifications is suppressed or not.

23 Extending GDB

GDB provides several mechanisms for extension. GDB also provides the ability to automatically load extensions when it reads a file for debugging. This allows the user to automatically customize GDB for the program being debugged.

To facilitate the use of extension languages, GDB is capable of evaluating the contents of a file. When doing so, GDB can recognize which extension language is being used by looking at the filename extension. Files with an unrecognized filename extension are always treated as a GDB Command Files. See Section 23.1.3 [Command files], page 401.

You can control how GDB evaluates these files with the following setting:

set script-extension off

All scripts are always evaluated as GDB Command Files.

set script-extension soft

The debugger determines the scripting language based on filename extension. If this scripting language is supported, GDB evaluates the script using that language. Otherwise, it evaluates the file as a GDB Command File.

set script-extension strict

The debugger determines the scripting language based on filename extension, and evaluates the script using that language. If the language is not supported, then the evaluation fails.

show script-extension

Display the current value of the `script-extension` option.

23.1 Canned Sequences of Commands

Aside from breakpoint commands (see Section 5.1.7 [Breakpoint Command Lists], page 79), GDB provides two ways to store sequences of commands for execution as a unit: user-defined commands and command files.

23.1.1 User-defined Commands

A *user-defined command* is a sequence of GDB commands to which you assign a new name as a command. This is done with the `define` command.

User commands may accept an unlimited number of arguments separated by whitespace. Arguments are accessed within the user command via `$arg0...$argN`. The arguments are text substitutions, so they may reference variables, use complex expressions, or even perform inferior functions calls. Note, however, that this textual substitution means that working with certain arguments is difficult. For example, there is no way for the user to pass an argument containing a space; and while stringifying an argument can be done using an expression like `"$arg1"`, this will fail if the argument contains a quote. For more complicated and robust commands, we recommend writing them in Python; see Section 23.3.2.21 [CLI Commands In Python], page 471.

A trivial example:

```
define adder
  print $arg0 + $arg1 + $arg2
end
```

To execute the command use:

```
adder 1 2 3
```

This defines the command **adder**, which prints the sum of its three arguments.

In addition, **\$argc** may be used to find out how many arguments have been passed.

```
define adder
  if $argc == 2
    print $arg0 + $arg1
  end
  if $argc == 3
    print $arg0 + $arg1 + $arg2
  end
end
```

Combining with the **eval** command (see [eval], page 404) makes it easier to process a variable number of arguments:

```
define adder
  set $i = 0
  set $sum = 0
  while $i < $argc
    eval "set $sum = $sum + $arg%d", $i
    set $i = $i + 1
  end
  print $sum
end
```

define *commandname*

Define a command named *commandname*. If there is already a command by that name, you are asked to confirm that you want to redefine it. The argument *commandname* may be a bare command name consisting of letters, numbers, dashes, dots, and underscores. It may also start with any predefined or user-defined prefix command. For example, ‘**define target my-target**’ creates a user-defined ‘**target my-target**’ command.

The definition of the command is made up of other GDB command lines, which are given following the **define** command. The end of these commands is marked by a line containing **end**.

document *commandname*

Document the user-defined command *commandname*, so that it can be accessed by **help**. The command *commandname* must already be defined. This command reads lines of documentation just as **define** reads the lines of the command definition, ending with **end**. After the **document** command is finished, **help** on command *commandname* displays the documentation you have written.

You may use the **document** command again to change the documentation of a command. Redefining the command with **define** does not change the documentation.

It is also possible to document user-defined aliases. The alias documentation will then be used by the **help** and **apropos** commands instead of the documentation of the aliased command. Documenting a user-defined alias is particularly useful when defining an alias as a set of nested **with** commands (see Section 23.2.1 [Command aliases default args], page 406).

define-prefix *commandname*

Define or mark the command *commandname* as a user-defined prefix command. Once marked, *commandname* can be used as prefix command by the **define** command. Note that **define-prefix** can be used with a not yet defined *commandname*. In such a case, *commandname* is defined as an empty user-defined command. In case you redefine a command that was marked as a user-defined prefix command, the subcommands of the redefined command are kept (and GDB indicates so to the user).

Example:

```
(gdb) define-prefix abc
(gdb) define-prefix abc def
(gdb) define abc def
Type commands for definition of "abc def".
End with a line saying just "end".
>echo command initial def\n
>end
(gdb) define abc def ghi
Type commands for definition of "abc def ghi".
End with a line saying just "end".
>echo command ghi\n
>end
(gdb) define abc def
Keeping subcommands of prefix command "def".
Redefine command "def"? (y or n) y
Type commands for definition of "abc def".
End with a line saying just "end".
>echo command def\n
>end
(gdb) abc def ghi
command ghi
(gdb) abc def
command def
(gdb)
```

dont-repeat

Used inside a user-defined command, this tells GDB that this command should not be repeated when the user hits **RET** (see Section 3.1 [Command Syntax], page 23).

help user-defined

List all user-defined commands and all python commands defined in class **COMMAND_USER**. The first line of the documentation or docstring is included (if any).

`show user`

`show user commandname`

Display the GDB commands used to define *commandname* (but not its documentation). If no *commandname* is given, display the definitions for all user-defined commands. This does not work for user-defined python commands.

`show max-user-call-depth`

`set max-user-call-depth`

The value of `max-user-call-depth` controls how many recursion levels are allowed in user-defined commands before GDB suspects an infinite recursion and aborts the command. This does not apply to user-defined python commands.

`save user filename`

Save all user-defined commands to the file *filename*. This command writes out the user-defined commands as a script that can be re-read into GDB using the `source` command.

In addition to the above commands, user-defined commands frequently use control flow commands, described in Section 23.1.3 [Command Files], page 401.

When user-defined commands are executed, the commands of the definition are not printed. An error in any command stops execution of the user-defined command.

If used interactively, commands that would ask for confirmation proceed without asking when used inside a user-defined command. Many GDB commands that normally print messages to say what they are doing omit the messages when used in a user-defined command.

23.1.2 User-defined Command Hooks

You may define *hooks*, which are a special kind of user-defined command. Whenever you run the command `'foo'`, if the user-defined command `'hook-foo'` exists, it is executed (with no arguments) before that command.

A hook may also be defined which is run after the command you executed. Whenever you run the command `'foo'`, if the user-defined command `'hookpost-foo'` exists, it is executed (with no arguments) after that command. Post-execution hooks may exist simultaneously with pre-execution hooks, for the same command.

It is valid for a hook to call the command which it hooks. If this occurs, the hook is not re-executed, thereby avoiding infinite recursion.

In addition, a pseudo-command, `'stop'` exists. Defining (`'hook-stop'`) makes the associated commands execute every time execution stops in your program: before breakpoint commands are run, displays are printed, or the stack frame is printed.

For example, to ignore `SIGALRM` signals while single-stepping, but treat them normally during normal execution, you could define:

```
define hook-stop
handle SIGALRM nopass
end
```

```
define hook-run
handle SIGALRM pass
end
```

```
define hook-continue
```



```

handle SIGALRM pass
end

```

As a further example, to hook at the beginning and end of the `echo` command, and to add extra text to the beginning and end of the message, you could define:

```

define hook-echo
echo <<<---
end

define hookpost-echo
echo --->>>\n
end

(gdb) echo Hello World
<<<---Hello World--->>>
(gdb)

```

You can define a hook for any single-word command in GDB, but not for command aliases; you should define a hook for the basic command name, e.g. `backtrace` rather than `bt`. You can hook a multi-word command by adding `hook-` or `hookpost-` to the last word of the command, e.g. ‘`define target hook-remote`’ to add a hook to ‘`target remote`’.

If an error occurs during the execution of your hook, execution of GDB commands stops and GDB issues a prompt (before the command that you actually typed had a chance to run).

If you try to define a hook which does not match any known command, you get a warning from the `define` command.

23.1.3 Command Files

A command file for GDB is a text file made of lines that are GDB commands. Comments (lines starting with `#`) may also be included. An empty line in a command file does nothing; it does not mean to repeat the last command, as it would from the terminal.

You can request the execution of a command file with the `source` command. Note that the `source` command is also used to evaluate scripts that are not Command Files. The exact behavior can be configured using the `script-extension` setting. See Chapter 23 [Extending GDB], page 397.

```
source [-s] [-v] filename
```

Execute the command file *filename*.

The lines in a command file are generally executed sequentially, unless the order of execution is changed by one of the *flow-control commands* described below. The commands are not printed as they are executed. An error in any command terminates execution of the command file and control is returned to the console.

GDB first searches for *filename* in the current directory. If the file is not found there, and *filename* does not specify a directory, then GDB also looks for the file on the source search path (specified with the ‘`directory`’ command); except that `$cdir` is not searched because the compilation directory is not relevant to scripts.

If `-s` is specified, then GDB searches for *filename* on the search path even if *filename* specifies a directory. The search is done by appending *filename* to each element of the search path. So, for example, if *filename* is `mylib/myscript` and the search path contains `/home/user`

then GDB will look for the script `/home/user/mylib/myscript`. The search is also done if *filename* is an absolute path. For example, if *filename* is `/tmp/myscript` and the search path contains `/home/user` then GDB will look for the script `/home/user/tmp/myscript`. For DOS-like systems, if *filename* contains a drive specification, it is stripped before concatenation. For example, if *filename* is `d:myscript` and the search path contains `c:/tmp` then GDB will look for the script `c:/tmp/myscript`.

If `-v`, for verbose mode, is given then GDB displays each command as it is executed. The option must be given before *filename*, and is interpreted as part of the filename anywhere else.

Commands that would ask for confirmation if used interactively proceed without asking when used in a command file. Many GDB commands that normally print messages to say what they are doing omit the messages when called from command files.

GDB also accepts command input from standard input. In this mode, normal output goes to standard output and error output goes to standard error. Errors in a command file supplied on standard input do not terminate execution of the command file—execution continues with the next command.

```
gdb < cmds > log 2>&1
```

(The syntax above will vary depending on the shell used.) This example will execute commands from the file `cmds`. All output and errors would be directed to `log`.

Since commands stored on command files tend to be more general than commands typed interactively, they frequently need to deal with complicated situations, such as different or unexpected values of variables and symbols, changes in how the program being debugged is built, etc. GDB provides a set of flow-control commands to deal with these complexities. Using these commands, you can write complex scripts that loop over data structures, execute commands conditionally, etc.

if

else This command allows to include in your script conditionally executed commands. The **if** command takes a single argument, which is an expression to evaluate. It is followed by a series of commands that are executed only if the expression is true (its value is nonzero). There can then optionally be an **else** line, followed by a series of commands that are only executed if the expression was false. The end of the list is marked by a line containing **end**.

while This command allows to write loops. Its syntax is similar to **if**: the command takes a single argument, which is an expression to evaluate, and must be followed by the commands to execute, one per line, terminated by an **end**. These commands are called the *body* of the loop. The commands in the body of **while** are executed repeatedly as long as the expression evaluates to true.

loop_break

This command exits the **while** loop in whose body it is included. Execution of the script continues after that **while** **end** line.

loop_continue

This command skips the execution of the rest of the body of commands in the **while** loop in whose body it is included. Execution branches to the beginning of the **while** loop, where it evaluates the controlling expression.

end Terminate the block of commands that are the body of **if**, **else**, or **while** flow-control commands.

23.1.4 Commands for Controlled Output

During the execution of a command file or a user-defined command, normal GDB output is suppressed; the only output that appears is what is explicitly printed by the commands in the definition. This section describes three commands useful for generating exactly the output you want.

echo text Print *text*. Nonprinting characters can be included in *text* using C escape sequences, such as ‘\n’ to print a newline. **No newline is printed unless you specify one.** In addition to the standard C escape sequences, a backslash followed by a space stands for a space. This is useful for displaying a string with spaces at the beginning or the end, since leading and trailing spaces are otherwise trimmed from all arguments. To print ‘ and foo = ’, use the command ‘echo \ and foo = \ ’.

A backslash at the end of *text* can be used, as in C, to continue the command onto subsequent lines. For example,

```
echo This is some text\n\
which is continued\n\
onto several lines.\n
```

produces the same output as

```
echo This is some text\n
echo which is continued\n
echo onto several lines.\n
```

output expression

Print the value of *expression* and nothing but that value: no newlines, no ‘\$nn = ’. The value is not entered in the value history either. See Section 10.1 [Expressions], page 147, for more information on expressions.

output/fmt expression

Print the value of *expression* in format *fmt*. You can use the same formats as for **print**. See Section 10.5 [Output Formats], page 152, for more information.

printf template, expressions...

Print the values of one or more *expressions* under the control of the string *template*. To print several values, make *expressions* be a comma-separated list of individual expressions, which may be either numbers or pointers. Their values are printed as specified by *template*, exactly as a C program would do by executing the code below:

```
printf (template, expressions...);
```

As in C **printf**, ordinary characters in *template* are printed verbatim, while *conversion specification* introduced by the ‘%’ character cause subsequent *expressions* to be evaluated, their values converted and formatted according to type and style information encoded in the conversion specifications, and then printed.

For example, you can print two values in hex like this:

```
printf "foo, bar-foo = 0x%x, 0x%x\n", foo, bar-foo
```

`printf` supports all the standard C conversion specifications, including the flags and modifiers between the ‘%’ character and the conversion letter, with the following exceptions:

- The argument-ordering modifiers, such as ‘2\$’, are not supported.
- The modifier ‘*’ is not supported for specifying precision or width.
- The ‘.’ flag (for separation of digits into groups according to `LC_NUMERIC`) is not supported.
- The type modifiers ‘hh’, ‘j’, ‘t’, and ‘z’ are not supported.
- The conversion letter ‘n’ (as in ‘%n’) is not supported.
- The conversion letters ‘a’ and ‘A’ are not supported.

Note that the ‘ll’ type modifier is supported only if the underlying C implementation used to build GDB supports the `long long int` type, and the ‘L’ type modifier is supported only if `long double` type is available.

As in C, `printf` supports simple backslash-escape sequences, such as `\n`, ‘\t’, ‘\\’, ‘\”’, ‘\a’, and ‘\f’, that consist of backslash followed by a single character. Octal and hexadecimal escape sequences are not supported.

Additionally, `printf` supports conversion specifications for DFP (*Decimal Floating Point*) types using the following length modifiers together with a floating point specifier. letters:

- ‘H’ for printing `Decimal32` types.
- ‘D’ for printing `Decimal64` types.
- ‘DD’ for printing `Decimal128` types.

If the underlying C implementation used to build GDB has support for the three length modifiers for DFP types, other modifiers such as width and precision will also be available for GDB to use.

In case there is no such C support, no additional modifiers will be available and the value will be printed in the standard way.

Here’s an example of printing DFP types using the above conversion letters:

```
printf "D32: %Hf - D64: %Df - D128: %DDf\n", 1.2345df, 1.2E10dd, 1.2E1dl
```

Additionally, `printf` supports a special ‘%V’ output format. This format prints the string representation of an expression just as GDB would produce with the standard `print` command (see Chapter 10 [Examining Data], page 143):

```
(gdb) print array
$1 = {0, 1, 2, 3, 4, 5}
(gdb) printf "Array is: %V\n", array
Array is: {0, 1, 2, 3, 4, 5}
```

It is possible to include print options with the ‘%V’ format by placing them in ‘[...]’ immediately after the ‘%V’, like this:

```
(gdb) printf "Array is: %V[-array-indexes on]\n", array
Array is: {[0] = 0, [1] = 1, [2] = 2, [3] = 3, [4] = 4, [5] = 5}
```

If you need to print a literal ‘[’ directly after a ‘%V’, then just include an empty print options list:

```
(gdb) printf "Array is: %V[] [Hello]\n", array
Array is: {0, 1, 2, 3, 4, 5}[Hello]
```

`eval template, expressions...`

Convert the values of one or more *expressions* under the control of the string *template* to a command line, and call it.

23.1.5 Controlling auto-loading native GDB scripts

When a new object file is read (for example, due to the `file` command, or because the inferior has loaded a shared library), GDB will look for the command file `objfile-gdb.gdb`. See Section 23.5 [Auto-loading extensions], page 592.

Auto-loading can be enabled or disabled, and the list of auto-loaded scripts can be printed.

`set auto-load gdb-scripts [on|off]`

Enable or disable the auto-loading of canned sequences of commands scripts.

`show auto-load gdb-scripts`

Show whether auto-loading of canned sequences of commands scripts is enabled or disabled.

`info auto-load gdb-scripts [regex]`

Print the list of all canned sequences of commands scripts that GDB auto-loaded.

If *regex* is supplied only canned sequences of commands scripts with matching names are printed.

23.2 Command Aliases

Aliases allow you to define alternate spellings for existing commands. For example, if a new GDB command defined in Python (see Section 23.3 [Python], page 407) has a long name, it is handy to have an abbreviated version of it that involves less typing.

GDB itself uses aliases. For example ‘s’ is an alias of the ‘step’ command even though it is otherwise an ambiguous abbreviation of other commands like ‘set’ and ‘show’.

Aliases are also used to provide shortened or more common versions of multi-word commands. For example, GDB provides the ‘tty’ alias of the ‘set inferior-tty’ command.

You can define a new alias with the ‘alias’ command.

`alias [-a] [--] alias = command [default-args]`

alias specifies the name of the new alias. Each word of *alias* must consist of letters, numbers, dashes and underscores.

command specifies the name of an existing command that is being aliased.

command can also be the name of an existing alias. In this case, *command* cannot be an alias that has default arguments.

The ‘-a’ option specifies that the new alias is an abbreviation of the command. Abbreviations are not used in command completion.

The ‘--’ option specifies the end of options, and is useful when *alias* begins with a dash.

You can specify *default-args* for your alias. These *default-args* will be automatically added before the alias arguments typed explicitly on the command line.

For example, the below defines an alias `btfullall` that shows all local variables and all frame arguments:

```
(gdb) alias btfullall = backtrace -full -frame-arguments all
```

For more information about *default-args*, see Section 23.2.1 [Default Arguments], page 406.

Here is a simple example showing how to make an abbreviation of a command so that there is less to type. Suppose you were tired of typing ‘`disas`’, the current shortest unambiguous abbreviation of the ‘`disassemble`’ command and you wanted an even shorter version named ‘`di`’. The following will accomplish this.

```
(gdb) alias -a di = disas
```

Note that aliases are different from user-defined commands. With a user-defined command, you also need to write documentation for it with the ‘`document`’ command. An alias automatically picks up the documentation of the existing command.

Here is an example where we make ‘`elms`’ an abbreviation of ‘`elements`’ in the ‘`set print elements`’ command. This is to show that you can make an abbreviation of any part of a command.

```
(gdb) alias -a set print elms = set print elements
(gdb) alias -a show print elms = show print elements
(gdb) set p elms 200
(gdb) show p elms
Limit on string chars or array elements to print is 200.
```

Note that if you are defining an alias of a ‘`set`’ command, and you want to have an alias for the corresponding ‘`show`’ command, then you need to define the latter separately.

Unambiguously abbreviated commands are allowed in *command* and *alias*, just as they are normally.

```
(gdb) alias -a set pr elms = set p ele
```

Finally, here is an example showing the creation of a one word alias for a more complex command. This creates alias ‘`spe`’ of the command ‘`set print elements`’.

```
(gdb) alias spe = set print elements
(gdb) spe 20
```

23.2.1 Default Arguments

You can tell GDB to always prepend some default arguments to the list of arguments provided explicitly by the user when using a user-defined alias.

If you repeatedly use the same arguments or options for a command, you can define an alias for this command and tell GDB to automatically prepend these arguments or options to the list of arguments you type explicitly when using the alias¹.

For example, if you often use the command `thread apply all` specifying to work on the threads in ascending order and to continue in case it encounters an error, you can tell GDB to automatically prepend the `-ascending` and `-c` options by using:

```
(gdb) alias thread apply asc-all = thread apply all -ascending -c
```

Once you have defined this alias with its default args, any time you type the `thread apply asc-all` followed by some arguments, GDB will execute `thread apply all -ascending -c some arguments`.

¹ GDB could easily accept default arguments for pre-defined commands and aliases, but it was deemed this would be confusing, and so is not allowed.

To have even less to type, you can also define a one word alias:

```
(gdb) alias t_a_c = thread apply all -ascending -c
```

As usual, unambiguous abbreviations can be used for *alias* and *default-args*.

The different aliases of a command do not share their default args. For example, you define a new alias `bt_ALL` showing all possible information and another alias `bt_SMALL` showing very limited information using:

```
(gdb) alias bt_ALL = backtrace -entry-values both -frame-arg all \
-past-main -past-entry -full
(gdb) alias bt_SMALL = backtrace -entry-values no -frame-arg none \
-past-main off -past-entry off
```

(For more on using the `alias` command, see Section 23.2 [Aliases], page 405.)

Default args are not limited to the arguments and options of *command*, but can specify nested commands if *command* accepts such a nested command as argument. For example, the below defines `faalocalsoftype` that lists the frames having locals of a certain type, together with the matching local vars:

```
(gdb) alias faalocalsoftype = frame apply all info locals -q -t
(gdb) faalocalsoftype int
#1 0x55554f5e in sleeper_or_burner (v=0xdf50) at sleepers.c:86
i = 0
ret = 21845
```

This is also very useful to define an alias for a set of nested `with` commands to have a particular combination of temporary settings. For example, the below defines the alias `pp10` that pretty prints an expression argument, with a maximum of 10 elements if the expression is a string or an array:

```
(gdb) alias pp10 = with print pretty -- with print elements 10 -- print
```

This defines the alias `pp10` as being a sequence of 3 commands. The first part `with print pretty --` temporarily activates the setting `set print pretty`, then launches the command that follows the separator `--`. The command following the first part is also a `with` command that temporarily changes the setting `set print elements` to 10, then launches the command that follows the second separator `--`. The third part `print` is the command the `pp10` alias will launch, using the temporary values of the settings and the arguments explicitly given by the user. For more information about the `with` command usage, see Section 3.2 [Command Settings], page 23.

By default, asking the help for an alias shows the documentation of the aliased command. When the alias is a set of nested commands, `help` of an alias shows the documentation of the first command. This help is not particularly useful for an alias such as `pp10`. For such an alias, it is useful to give a specific documentation using the `document` command (see Section 23.1.1 [Define], page 397).

23.3 Extending GDB using Python

You can extend GDB using the Python programming language (<http://www.python.org/>). This feature is available only if GDB was configured using `--with-python`.

Python scripts used by GDB should be installed in `data-directory/python`, where *data-directory* is the data directory as determined at GDB startup (see Section 18.8 [Data Files], page 312). This directory, known as the *python directory*, is automatically added to the

Python Search Path in order to allow the Python interpreter to locate all scripts installed at this location.

Additionally, GDB commands and convenience functions which are written in Python and are located in the `data-directory/python/gdb/command` or `data-directory/python/gdb/function` directories are automatically imported when GDB starts.

23.3.1 Python Commands

GDB provides two commands for accessing the Python interpreter, and one related setting:

```
python-interactive [command]
pi [command]
```

Without an argument, the `python-interactive` command can be used to start an interactive Python prompt. To return to GDB, type the EOF character (e.g., *Ctrl-D* on an empty prompt).

Alternatively, a single-line Python command can be given as an argument and evaluated. If the command is an expression, the result will be printed; otherwise, nothing will be printed. For example:

```
(gdb) python-interactive 2 + 3
5
```

```
python [command]
py [command]
```

The `python` command can be used to evaluate Python code.

If given an argument, the `python` command will evaluate the argument as a Python command. For example:

```
(gdb) python print(23)
23
```

If you do not provide an argument to `python`, it will act as a multi-line command, like `define`. In this case, the Python script is made up of subsequent command lines, given after the `python` command. This command list is terminated using a line containing `end`. For example:

```
(gdb) python
>print(23)
>end
23
```

```
set python print-stack
```

By default, GDB will print only the message component of a Python exception when an error occurs in a Python script. This can be controlled using `set python print-stack`: if `full`, then full Python stack printing is enabled; if `none`, then Python stack and message printing is disabled; if `message`, the default, only the message component of the error is printed.

```
set python ignore-environment [on|off]
```

By default this option is ‘off’, and, when GDB initializes its internal Python interpreter, the Python interpreter will check the environment for variables that will effect how it behaves, for example `PYTHONHOME`, and `PYTHONPATH`².

² See the ENVIRONMENT VARIABLES section of `man 1 python` for a comprehensive list.

If this option is set to ‘on’ before Python is initialized then Python will ignore all such environment variables. As Python is initialized early during GDB’s startup process, then this option must be placed into the early initialization file (see Section 2.1.4 [Initialization Files], page 18) to have the desired effect.

This option is equivalent to passing `-E` to the real `python` executable.

set python dont-write-bytecode [auto|on|off]

When this option is ‘off’, then, once GDB has initialized the Python interpreter, the interpreter will byte-compile any Python modules that it imports and write the byte code to disk in `.pyc` files.

If this option is set to ‘on’ before Python is initialized then Python will no longer write the byte code to disk. As Python is initialized early during GDB’s startup process, then this option must be placed into the early initialization file (see Section 2.1.4 [Initialization Files], page 18) to have the desired effect.

By default this option is set to ‘auto’. In this mode, provided the `python ignore-environment` setting is ‘off’, the environment variable `PYTHONDONTWRITEBYTECODE` is examined to see if it should write out byte-code or not. `PYTHONDONTWRITEBYTECODE` is considered to be off/disabled either when set to the empty string or when the environment variable doesn’t exist. All other settings, including those which don’t seem to make sense, indicate that it’s on/enabled.

This option is equivalent to passing `-B` to the real `python` executable.

It is also possible to execute a Python script from the GDB interpreter:

source script-name

The script name must end with ‘.py’ and GDB must be configured to recognize the script language based on filename extension using the `script-extension` setting. See Chapter 23 [Extending GDB], page 397.

The following commands are intended to help debug GDB itself:

set debug py-breakpoint on|off

show debug py-breakpoint

When ‘on’, GDB prints debug messages related to the Python breakpoint API. This is ‘off’ by default.

set debug py-unwind on|off

show debug py-unwind

When ‘on’, GDB prints debug messages related to the Python unwinder API. This is ‘off’ by default.

23.3.2 Python API

You can get quick online help for GDB’s Python API by issuing the command `python help (gdb)`.

Functions and methods which have two or more optional arguments allow them to be specified using keyword syntax. This allows passing some optional arguments while skipping others. Example: `gdb.some_function ('foo', bar = 1, baz = 2)`.

23.3.2.1 Basic Python

At startup, GDB overrides Python's `sys.stdout` and `sys.stderr` to print using GDB's output-paging streams. A Python program which outputs to one of these streams may have its output interrupted by the user (see Section 22.4 [Screen Size], page 374). In this situation, a Python `KeyboardInterrupt` exception is thrown.

Some care must be taken when writing Python code to run in GDB. Two things worth noting in particular:

- GDB installs handlers for `SIGCHLD` and `SIGINT`. Python code must not override these, or even change the options using `sigaction`. If your program changes the handling of these signals, GDB will most likely stop working correctly. Note that it is unfortunately common for GUI toolkits to install a `SIGCHLD` handler. When creating a new Python thread, you can use `gdb.blocked_signals` or `gdb.Thread` to handle this correctly; see Section 23.3.2.2 [Threading in GDB], page 415.
- GDB takes care to mark its internal file descriptors as close-on-exec. However, this cannot be done in a thread-safe way on all platforms. Your Python programs should be aware of this and should both create new file descriptors with the close-on-exec flag set and arrange to close unneeded file descriptors before starting a child process.

GDB introduces a new Python module, named `gdb`. All methods and classes added by GDB are placed in this module. GDB automatically `imports` the `gdb` module for use in all scripts evaluated by the `python` command.

Some types of the `gdb` module come with a textual representation (accessible through the `repr` or `str` functions). These are offered for debugging purposes only, expect them to change over time.

`gdb.PYTHONDIR` [Variable]

A string containing the python directory (see Section 23.3 [Python], page 407).

`gdb.execute (command [, from_tty [, to_string [, styling ]]])` [Function]

Evaluate *command*, a string, as a GDB CLI command. If a GDB exception happens while *command* runs, it is translated as described in Section 23.3.2.3 [Exception Handling], page 416.

The *from_tty* flag specifies whether GDB ought to consider this command as having originated from the user invoking it interactively. It must be a boolean value. If omitted, it defaults to `False`.

By default, any output produced by *command* is sent to GDB's standard output (and to the log output if logging is turned on). If the *to_string* parameter is `True`, then output will be collected by `gdb.execute` and returned as a string. The default is `False`, in which case the return value is `None`. If *to_string* is `True`, the GDB virtual terminal will be temporarily set to unlimited width and height, and its pagination will be disabled; see Section 22.4 [Screen Size], page 374.

When *styling* is `True`, the output, whether sent to standard output, or to a string, will have styling applied, if GDB's standard output supports styling, and `show style enabled` is `on`. When *styling* is `False` then no styling is applied. The default for *styling* is `True` when *to_string* is `False`, and `False` when *to_string* is `True`.

`gdb.breakpoints ()` [Function]

Return a sequence holding all of GDB's breakpoints. See Section 23.3.2.33 [Breakpoints In Python], page 503, for more information. In GDB version 7.11 and earlier, this function returned `None` if there were no breakpoints. This peculiarity was subsequently fixed, and now `gdb.breakpoints` returns an empty sequence in this case.

`gdb.rbreak (regex [, minsyms [, throttle [, symtabs ]]])` [Function]

Return a Python list holding a collection of newly set `gdb.Breakpoint` objects matching function names defined by the *regex* pattern. If the *minsyms* keyword is `True`, all system functions (those not explicitly defined in the inferior) will also be included in the match. The *throttle* keyword takes an integer that defines the maximum number of pattern matches for functions matched by the *regex* pattern. If the number of matches exceeds the integer value of *throttle*, a `RuntimeError` will be raised and no breakpoints will be created. If *throttle* is not defined then there is no imposed limit on the maximum number of matches and breakpoints to be created. The *symtabs* keyword takes a Python iterable that yields a collection of `gdb.Symtab` objects and will restrict the search to those functions only contained within the `gdb.Symtab` objects.

`gdb.parameter (parameter)` [Function]

Return the value of a GDB *parameter* given by its name, a string; the parameter name string may contain spaces if the parameter has a multi-part name. For example, 'print object' is a valid parameter name.

If the named parameter does not exist, this function throws a `gdb.error` (see Section 23.3.2.3 [Exception Handling], page 416). Otherwise, the parameter's value is converted to a Python value of the appropriate type, and returned.

`gdb.set_parameter (name, value)` [Function]

Sets the gdb parameter *name* to *value*. As with `gdb.parameter`, the parameter name string may contain spaces if the parameter has a multi-part name.

`gdb.with_parameter (name, value)` [Function]

Create a Python context manager (for use with the Python `with` statement) that temporarily sets the gdb parameter *name* to *value*. On exit from the context, the previous value will be restored.

This uses `gdb.parameter` in its implementation, so it can throw the same exceptions as that function.

For example, it's sometimes useful to evaluate some Python code with a particular gdb language:

```
with gdb.with_parameter('language', 'pascal'):
    ... language-specific operations
```

`gdb.history (number)` [Function]

Return a value from GDB's value history (see Section 10.11 [Value History], page 172). The *number* argument indicates which history element to return. If *number* is negative, then GDB will take its absolute value and count backward from the last element (i.e., the most recent element) to find the value to return. If *number* is zero, then GDB will return the most recent element. If the element specified by *number* doesn't exist in the value history, a `gdb.error` exception will be raised.

If no exception is raised, the return value is always an instance of `gdb.Value` (see Section 23.3.2.4 [Values From Inferior], page 417).

`gdb.add_history (value)` [Function]

Takes *value*, an instance of `gdb.Value` (see Section 23.3.2.4 [Values From Inferior], page 417), and appends the value this object represents to GDB's value history (see Section 10.11 [Value History], page 172), and return an integer, its history number. If *value* is not a `gdb.Value`, it is converted using the `gdb.Value` constructor. If *value* can't be converted to a `gdb.Value` then a `TypeError` is raised.

When a command implemented in Python prints a single `gdb.Value` as its result, then placing the value into the history will allow the user convenient access to those values via CLI history facilities.

`gdb.history_count ()` [Function]

Return an integer indicating the number of values in GDB's value history (see Section 10.11 [Value History], page 172).

`gdb.convenience_variable (name)` [Function]

Return the value of the convenience variable (see Section 10.12 [Convenience Vars], page 173) named *name*. *name* must be a string. The name should not include the '\$' that is used to mark a convenience variable in an expression. If the convenience variable does not exist, then `None` is returned.

`gdb.set_convenience_variable (name, value)` [Function]

Set the value of the convenience variable (see Section 10.12 [Convenience Vars], page 173) named *name*. *name* must be a string. The name should not include the '\$' that is used to mark a convenience variable in an expression. If *value* is `None`, then the convenience variable is removed. Otherwise, if *value* is not a `gdb.Value` (see Section 23.3.2.4 [Values From Inferior], page 417), it is converted using the `gdb.Value` constructor.

`gdb.parse_and_eval (expression [, global_context])` [Function]

Parse *expression*, which must be a string, as an expression in the current language, evaluate it, and return the result as a `gdb.Value`.

global_context, if provided, is a boolean indicating whether the parsing should be done in the global context. The default is `'False'`, meaning that the current frame or current static context should be used.

This function can be useful when implementing a new command (see Section 23.3.2.21 [CLI Commands In Python], page 471, see Section 23.3.2.22 [GDB/MI Commands In Python], page 474), as it provides a way to parse the command's argument as an expression. It is also useful simply to compute values.

`gdb.find_pc_line (pc)` [Function]

Return the `gdb.Symtab_and_line` object corresponding to the *pc* value. See Section 23.3.2.31 [Symbol Tables In Python], page 500. If an invalid value of *pc* is passed as an argument, then the `symtab` and `line` attributes of the returned `gdb.Symtab_and_line` object will be `None` and 0 respectively. This is identical to `gdb.current_progspace().find_pc_line(pc)` and is included for historical compatibility.

`gdb.write` (*string* [, *stream*] [, *style*]) [Function]

Print a string to GDB's paginated output stream. The optional *stream* determines the stream to print to. The default stream is GDB's standard output stream. Possible stream values are:

`gdb.STDOUT`
GDB's standard output stream.

`gdb.STDERR`
GDB's standard error stream.

`gdb.STDLOG`
GDB's log stream (see Section 2.4 [Logging Output], page 22).

The *style* should be a **`gdb.Style`** object (see Section 23.3.2.37 [Styles In Python], page 510), or **`None`** (the default). If *style* is **`None`** then the current style for *stream* will be applied to *text*. If *style* is a **`gdb.Style`** object, then this style is applied to *text*, after which the default output style is restored.

Writing to **`sys.stdout`** or **`sys.stderr`** will automatically call this function and will automatically direct the output to the relevant stream.

`gdb.flush` ([*stream*]) [Function]

Flush the buffer of a GDB paginated stream so that the contents are displayed immediately. GDB will flush the contents of a stream automatically when it encounters a newline in the buffer. The optional *stream* determines the stream to flush. The default stream is GDB's standard output stream. Possible stream values are:

`gdb.STDOUT`
GDB's standard output stream.

`gdb.STDERR`
GDB's standard error stream.

`gdb.STDLOG`
GDB's log stream (see Section 2.4 [Logging Output], page 22).

Flushing **`sys.stdout`** or **`sys.stderr`** will automatically call this function for the relevant stream.

`gdb.warning` (*text*) [Function]

Print a warning message to GDB's standard output stream. The warning message is the warning prefix (see [warning-prefix], page 376), the string **`'warning: '`**, and then *text*, which must be a non-empty string.

Due to the warning prefix, *text* should not begin with a capital letter (except for proper nouns), and *text* should end with a period.

`gdb.target_charset` () [Function]

Return the name of the current target character set (see Section 10.21 [Character Sets], page 190). This differs from **`gdb.parameter('target-charset')`** in that **`'auto'`** is never returned.

- `gdb.target_wide_charset ()`** [Function]
 Return the name of the current target wide character set (see Section 10.21 [Character Sets], page 190). This differs from `gdb.parameter('target-wide-charset')` in that 'auto' is never returned.
- `gdb.host_charset ()`** [Function]
 Return a string, the name of the current host character set (see Section 10.21 [Character Sets], page 190). This differs from `gdb.parameter('host-charset')` in that 'auto' is never returned.
- `gdb.solib_name (address)`** [Function]
 Return the name of the shared library holding the given *address* as a string, or `None`. This is identical to `gdb.current_progspace().solib_name(address)` and is included for historical compatibility.
- `gdb.decode_line ([expression])`** [Function]
 Return locations of the line specified by *expression*, or of the current line if no argument was given. This function returns a Python tuple containing two elements. The first element contains a string holding any unparsed section of *expression* (or `None` if the expression has been fully parsed). The second element contains either `None` or another tuple that contains all the locations that match the expression represented as `gdb.Symtab_and_line` objects (see Section 23.3.2.31 [Symbol Tables In Python], page 500). If *expression* is provided, it is decoded the way that GDB's inbuilt `break` or `edit` commands do (see Section 9.2 [Location Specifications], page 128).
- `gdb.prompt_hook (current_prompt)`** [Function]
 If *prompt_hook* is callable, GDB will call the method assigned to this operation before a prompt is displayed by GDB.
 The parameter *current_prompt* contains the current GDB prompt. This method must return a Python string, or `None`. If a string is returned, the GDB prompt will be set to that string. If `None` is returned, GDB will continue to use the current prompt.
 Some prompts cannot be substituted in GDB. Secondary prompts such as those used by `readline` for command input, and annotation related prompts are prohibited from being changed.
- `gdb.architecture_names ()`** [Function]
 Return a list containing all of the architecture names that the current build of GDB supports. Each architecture name is a string. The names returned in this list are the same names as are returned from `gdb.Architecture.name` (see [Architecture.name], page 512).
- `gdb.connections`** [Function]
 Return a list of `gdb.TargetConnection` objects, one for each currently active connection (see Section 23.3.2.40 [Connections In Python], page 514). The connection objects are in no particular order in the returned list.
- `gdb.format_address (address [, progspace, architecture])`** [Function]
 Return a string in the format '*addr* <*symbol+offset*>', where *addr* is *address* formatted in hexadecimal, *symbol* is the symbol whose address is the nearest to *address* and below it in memory, and *offset* is the offset from *symbol* to *address* in decimal.

If no suitable *symbol* was found, then the `<symbol+offset>` part is not included in the returned string, instead the returned string will just contain the *address* formatted as hexadecimal. How far GDB looks back for a suitable symbol can be controlled with `set print max-symbolic-offset` (see Section 10.9 [Print Settings], page 159).

Additionally, the returned string can include file name and line number information when `set print symbol-filename on` (see Section 10.9 [Print Settings], page 159), in this case the format of the returned string is `'addr <symbol+offset> at filename:line-number'`.

The *progspace* is the `gdb.Progspace` in which *symbol* is looked up, and *architecture* is used when formatting *addr*, e.g. in order to determine the size of an address in bytes. If neither *progspace* or *architecture* are passed, then by default GDB will use the program space and architecture of the currently selected inferior, thus, the following two calls are equivalent:

```
gdb.format_address(address)
gdb.format_address(address,
                    gdb.selected_inferior().progspace,
                    gdb.selected_inferior().architecture())
```

It is not valid to only pass one of *progspace* or *architecture*, either they must both be provided, or neither must be provided (and the defaults will be used).

This method uses the same mechanism for formatting address, symbol, and offset information as core GDB does in commands such as *disassemble*.

Here are some examples of the possible string formats:

```
0x00001042
0x00001042 <symbol+16>
0x00001042 <symbol+16 at file.c:123>
```

`gdb.current_language ()` [Function]

Return the name of the current language as a string. Unlike `gdb.parameter('language')`, this function will never return `'auto'`. If a `gdb.Frame` object is available (see Section 23.3.2.28 [Frames In Python], page 490), the `language` method might be preferable in some cases, as that is not affected by the user's language setting.

23.3.2.2 Threading in GDB

GDB is not thread-safe. If your Python program uses multiple threads, you must be careful to only call GDB-specific functions in the GDB thread. GDB provides some functions to help with this.

`gdb.blocked_signals ()` [Function]

As mentioned earlier (see Section 23.3.2.1 [Basic Python], page 410), certain signals must be delivered to the GDB main thread. The `blocked_signals` function returns a context manager that will block these signals on entry. This can be used when starting a new thread to ensure that the signals are blocked there, like:

```
with gdb.blocked_signals():
    start_new_thread()
```

`gdb.Thread` [class]

This is a subclass of Python's `threading.Thread` class. It overrides the `start` method to call `blocked_signals`, making this an easy-to-use drop-in replacement for creating threads that will work well in GDB.

`gdb.interrupt ()` [Function]

This causes GDB to react as if the user had typed a control-C character at the terminal. That is, if the inferior is running, it is interrupted; if a GDB command is executing, it is stopped; and if a Python command is running, `KeyboardInterrupt` will be raised.

Unlike most Python APIs in GDB, `interrupt` is thread-safe.

`gdb.post_event (event)` [Function]

Put *event*, a callable object taking no arguments, into GDB's internal event queue. This callable will be invoked at some later point, during GDB's event processing. Events posted using `post_event` will be run in the order in which they were posted; however, there is no way to know when they will be processed relative to other events inside GDB.

Unlike most Python APIs in GDB, `post_event` is thread-safe. For example:

```
(gdb) python
>import threading
>
>class Writer():
> def __init__(self, message):
>     self.message = message;
> def __call__(self):
>     gdb.write(self.message)
>
>class MyThread1 (threading.Thread):
> def run (self):
>     gdb.post_event(Writer("Hello "))
>
>class MyThread2 (threading.Thread):
> def run (self):
>     gdb.post_event(Writer("World\n"))
>
>MyThread1().start()
>MyThread2().start()
>end
(gdb) Hello World
```

23.3.2.3 Exception Handling

When executing the `python` command, Python exceptions uncaught within the Python code are translated to calls to GDB error-reporting mechanism. If the command that called `python` does not handle the error, GDB will terminate it and print an error message. Exactly what will be printed depends on `set python print-stack` (see Section 23.3.1 [Python Commands], page 408). Example:

```
(gdb) python print(foo)
Traceback (most recent call last):
  File "<string>", line 1, in <module>
NameError: name 'foo' is not defined
```

GDB errors that happen in GDB commands invoked by Python code are converted to Python exceptions. The type of the Python exception depends on the error.

`gdb.error`

This is the base class for most exceptions generated by GDB. It is derived from `RuntimeError`, for compatibility with earlier versions of GDB.

If an error occurring in GDB does not fit into some more specific category, then the generated exception will have this type.

`gdb.MemoryError`

This is a subclass of `gdb.error` which is thrown when an operation tried to access invalid memory in the inferior.

`KeyboardInterrupt`

User interrupt (via `C-c` or by typing `q` at a pagination prompt) is translated to a Python `KeyboardInterrupt` exception.

In all cases, your exception handler will see the GDB error message as its value and the Python call stack backtrace at the Python statement closest to where the GDB error occurred as the traceback.

When implementing GDB commands in Python via `gdb.Command`, or functions via `gdb.Function`, it is useful to be able to throw an exception that doesn't cause a traceback to be printed. For example, the user may have invoked the command incorrectly. GDB provides a special exception class that can be used for this purpose.

`gdb.GdbError`

When thrown from a command or function, this exception will cause the command or function to fail, but the Python stack will not be displayed. GDB does not throw this exception itself, but rather recognizes it when thrown from user Python code. Example:

```
(gdb) python
>class HelloWorld (gdb.Command):
>  """Greet the whole world."""
>  def __init__ (self):
>    super (HelloWorld, self).__init__ ("hello-world", gdb.COMMAND_USER)
>  def invoke (self, args, from_tty):
>    argv = gdb.string_to_argv (args)
>    if len (argv) != 0:
>      raise gdb.GdbError ("hello-world takes no arguments")
>    print ("Hello, World!")
>HelloWorld ()
>end
(gdb) hello-world 42
hello-world takes no arguments
```

23.3.2.4 Values From Inferior

GDB provides values it obtains from the inferior program in an object of type `gdb.Value`. GDB uses this object for its internal bookkeeping of the inferior's values, and for fetching values when necessary.

Inferior values that are simple scalars can be used directly in Python expressions that are valid for the value's data type. Here's an example for an integer or floating-point value `some_val`:

```
bar = some_val + 2
```

As result of this, `bar` will also be a `gdb.Value` object whose values are of the same type as those of `some_val`. Valid Python operations can also be performed on `gdb.Value` objects representing a `struct` or `class` object. For such cases, the overloaded operator (if present), is used to perform the operation. For example, if `val1` and `val2` are `gdb.Value` objects

representing instances of a `class` which overloads the `+` operator, then one can use the `+` operator in their Python script as follows:

```
val3 = val1 + val2
```

The result of the operation `val3` is also a `gdb.Value` object corresponding to the value returned by the overloaded `+` operator. In general, overloaded operators are invoked for the following operations: `+` (binary addition), `-` (binary subtraction), `*` (multiplication), `/`, `%`, `<<`, `>>`, `|`, `&`, `^`.

Inferior values that are structures or instances of some class can be accessed using the Python *dictionary syntax*. For example, if `some_val` is a `gdb.Value` instance holding a structure, you can access its `foo` element with:

```
bar = some_val['foo']
```

Again, `bar` will also be a `gdb.Value` object. Structure elements can also be accessed by using `gdb.Field` objects as subscripts (see Section 23.3.2.5 [Types In Python], page 425, for more information on `gdb.Field` objects). For example, if `foo_field` is a `gdb.Field` object corresponding to element `foo` of the above structure, then `bar` can also be accessed as follows:

```
bar = some_val[foo_field]
```

If a `gdb.Value` has array or pointer type, an integer index can be used to access elements.

```
result = some_array[23]
```

A `gdb.Value` that represents a function can be executed via inferior function call. Any arguments provided to the call must match the function's prototype, and must be provided in the order specified by that prototype.

For example, `some_val` is a `gdb.Value` instance representing a function that takes two integers as arguments. To execute this function, call it like so:

```
result = some_val (10,20)
```

Any values returned from a function call will be stored as a `gdb.Value`.

The following attributes are provided:

Value.address [Variable]

If this object is addressable, this read-only attribute holds a `gdb.Value` object representing the address. Otherwise, this attribute holds `None`.

Value.is_optimized_out [Variable]

This read-only boolean attribute is true if the compiler optimized out this value, or any part of this value, and thus it is not available for fetching from the inferior.

Value.is_unavailable [Variable]

This read-only boolean attribute is true if this value, or any part of this value, is not available to GDB. Where an optimized out value has been removed from the program by the compiler, an unavailable value does exist in the program, but GDB is unable to fetch it.

Some reasons why this might occur include, but are not limited to: a core file format that GDB doesn't fully understand; during live debugging if the debug API has no mechanism to access the required state, e.g. the kernel gives an error when trying to read a particular register set; or reading a value from GDB's history, when only a partial value was stored, e.g. due to the `max-value-size` setting (see [max-value-size], page 195).

Value.type [Variable]

The type of this `gdb.Value`. The value of this attribute is a `gdb.Type` object (see Section 23.3.2.5 [Types In Python], page 425).

Value.dynamic_type [Variable]

The dynamic type of this `gdb.Value`. This uses the object's virtual table and the C++ run-time type information (RTTI) to determine the dynamic type of the value. If this value is of class type, it will return the class in which the value is embedded, if any. If this value is of pointer or reference to a class type, it will compute the dynamic type of the referenced object, and return a pointer or reference to that type, respectively. In all other cases, it will return the value's static type.

Note that this feature will only work when debugging a C++ program that includes RTTI for the object in question. Otherwise, it will just return the static type of the value as in `ptype foo` (see Chapter 16 [Symbols], page 265).

Value.is_lazy [Variable]

The value of this read-only boolean attribute is `True` if this `gdb.Value` has not yet been fetched from the inferior. GDB does not fetch values until necessary, for efficiency. For example:

```
myval = gdb.parse_and_eval ('somevar')
```

The value of `somevar` is not fetched at this time. It will be fetched when the value is needed, or when the `fetch_lazy` method is invoked.

Value.bytes [Variable]

The value of this attribute is a `bytes` object containing the bytes that make up this `Value`'s complete value in little endian order. If the complete contents of this value are not available then accessing this attribute will raise an exception.

This attribute can also be assigned to. The new value should be a buffer object (e.g. a `bytes` object), the length of the new buffer must exactly match the length of this `Value`'s type. The bytes values in the new buffer should be in little endian order.

As with `Value.assign` (see [Value.assign], page 420), if this value cannot be assigned to, then an exception will be thrown.

The following methods are provided:

Value.__init__ (val) [Function]

Many Python values can be converted directly to a `gdb.Value` via this object initializer. Specifically:

Python boolean

A Python boolean is converted to the boolean type from the current language.

Python integer

A Python integer is converted to the C `long` type for the current architecture.

Python long

A Python long is converted to the C `long long` type for the current architecture.

Python float

A Python float is converted to the C `double` type for the current architecture.

Python string

A Python string is converted to a target string in the current target language using the current target encoding. If a character cannot be represented in the current target encoding, then an exception is thrown.

`gdb.Value`

If `val` is a `gdb.Value`, then a copy of the value is made.

`gdb.LazyString`

If `val` is a `gdb.LazyString` (see Section 23.3.2.35 [Lazy Strings In Python], page 508), then the lazy string's `value` method is called, and its result is used.

`Value.__init__ (val, type)` [Function]

This second form of the `gdb.Value` constructor returns a `gdb.Value` of type `type` where the value contents are taken from the Python buffer object specified by `val`. The number of bytes in the Python buffer object must be greater than or equal to the size of `type`.

If `type` is `None` then this version of `__init__` behaves as though `type` was not passed at all.

`Value.assign (rhs)` [Function]

Assign `rhs` to this value, and return `None`. If this value cannot be assigned to, or if the assignment is invalid for some reason (for example a type-checking failure), an exception will be thrown.

`Value.cast (type)` [Function]

Return a new instance of `gdb.Value` that is the result of casting this instance to the type described by `type`, which must be a `gdb.Type` object. If the cast cannot be performed for some reason, this method throws an exception.

`Value.dereference ()` [Function]

For pointer data types, this method returns a new `gdb.Value` object whose contents is the object pointed to by the pointer. For example, if `foo` is a C pointer to an `int`, declared in your C program as

```
int *foo;
```

then you can use the corresponding `gdb.Value` to access what `foo` points to like this:

```
bar = foo.dereference ()
```

The result `bar` will be a `gdb.Value` object holding the value pointed to by `foo`.

A similar function `Value.reference_value` exists which also returns `gdb.Value` objects corresponding to the values pointed to by pointer values (and additionally, values referenced by reference values). However, the behavior of `Value.dereference` differs from `Value.reference_value` by the fact that the behavior of `Value.dereference` is identical to applying the C unary operator `*` on a given value. For example, consider a reference to a pointer `ptrref`, declared in your C++ program as

```
typedef int *intptr;
```

```
...
int val = 10;
intptr ptr = &val;
intptr &ptrref = ptr;
```

Though `ptrref` is a reference value, one can apply the method `Value.dereference` to the `gdb.Value` object corresponding to it and obtain a `gdb.Value` which is identical to that corresponding to `val`. However, if you apply the method `Value.referenced_value`, the result would be a `gdb.Value` object identical to that corresponding to `ptr`.

```
py_ptrref = gdb.parse_and_eval ("ptrref")
py_val = py_ptrref.dereference ()
py_ptr = py_ptrref.referenced_value ()
```

The `gdb.Value` object `py_val` is identical to that corresponding to `val`, and `py_ptr` is identical to that corresponding to `ptr`. In general, `Value.dereference` can be applied whenever the C unary operator `*` can be applied to the corresponding C value. For those cases where applying both `Value.dereference` and `Value.referenced_value` is allowed, the results obtained need not be identical (as we have seen in the above example). The results are however identical when applied on `gdb.Value` objects corresponding to pointers (`gdb.Value` objects with type code `TYPE_CODE_PTR`) in a C/C++ program.

`Value.referenced_value ()` [Function]

For pointer or reference data types, this method returns a new `gdb.Value` object corresponding to the value referenced by the pointer/reference value. For pointer data types, `Value.dereference` and `Value.referenced_value` produce identical results. The difference between these methods is that `Value.dereference` cannot get the values referenced by reference values. For example, consider a reference to an `int`, declared in your C++ program as

```
int val = 10;
int &ref = val;
```

then applying `Value.dereference` to the `gdb.Value` object corresponding to `ref` will result in an error, while applying `Value.referenced_value` will result in a `gdb.Value` object identical to that corresponding to `val`.

```
py_ref = gdb.parse_and_eval ("ref")
er_ref = py_ref.dereference ()      # Results in error
py_val = py_ref.referenced_value () # Returns the referenced value
```

The `gdb.Value` object `py_val` is identical to that corresponding to `val`.

`Value.reference_value ()` [Function]

Return a `gdb.Value` object which is a reference to the value encapsulated by this instance.

`Value.const_value ()` [Function]

Return a `gdb.Value` object which is a `const` version of the value encapsulated by this instance.

`Value.dynamic_cast (type)` [Function]

Like `Value.cast`, but works as if the C++ `dynamic_cast` operator were used. Consult a C++ reference for details.

Value.reinterpret_cast (*type*) [Function]

Like **Value.cast**, but works as if the C++ **reinterpret_cast** operator were used. Consult a C++ reference for details.

Value.format_string (...) [Function]

Convert a **gdb.Value** to a string, similarly to what the **print** command does. Invoked with no arguments, this is equivalent to calling the **str** function on the **gdb.Value**. The representation of the same value may change across different versions of GDB, so you shouldn't, for instance, parse the strings returned by this method.

All the arguments are keyword only. If an argument is not specified, the current global default setting is used.

raw **True** if pretty-printers (see Section 10.10 [Pretty Printing], page 170) should not be used to format the value. **False** if enabled pretty-printers matching the type represented by the **gdb.Value** should be used to format it.

pretty_arrays **True** if arrays should be pretty printed to be more convenient to read, **False** if they shouldn't (see **set print array** in Section 10.9 [Print Settings], page 159).

pretty_structs **True** if structs should be pretty printed to be more convenient to read, **False** if they shouldn't (see **set print pretty** in Section 10.9 [Print Settings], page 159).

array_indexes **True** if array indexes should be included in the string representation of arrays, **False** if they shouldn't (see **set print array-indexes** in Section 10.9 [Print Settings], page 159).

symbols **True** if the string representation of a pointer should include the corresponding symbol name (if one exists), **False** if it shouldn't (see **set print symbol** in Section 10.9 [Print Settings], page 159).

unions **True** if unions which are contained in other structures or unions should be expanded, **False** if they shouldn't (see **set print union** in Section 10.9 [Print Settings], page 159).

address **True** if the string representation of a pointer should include the address, **False** if it shouldn't (see **set print address** in Section 10.9 [Print Settings], page 159).

nibbles **True** if binary values should be displayed in groups of four bits, known as nibbles. **False** if it shouldn't (see Section 10.9 [Print Settings], page 159).

deref_refs **True** if C++ references should be resolved to the value they refer to, **False** (the default) if they shouldn't. Note that, unlike for the **print** command, references are not automatically expanded when using the **format_string** method or the **str** function. There is no global **print** setting to change the default behavior.

actual_objects

True if the representation of a pointer to an object should identify the *actual* (derived) type of the object rather than the *declared* type, using the virtual function table. **False** if the *declared* type should be used. (See **set print object** in Section 10.9 [Print Settings], page 159).

static_members

True if static members should be included in the string representation of a C++ object, **False** if they shouldn't (see **set print static-members** in Section 10.9 [Print Settings], page 159).

max_characters

Number of string characters to print, 0 to follow **max_elements**, or **UINT_MAX** to print an unlimited number of characters (see **set print characters** in Section 10.9 [Print Settings], page 159).

max_elements

Number of array elements to print, or 0 to print an unlimited number of elements (see **set print elements** in Section 10.9 [Print Settings], page 159).

max_depth

The maximum depth to print for nested structs and unions, or -1 to print an unlimited number of elements (see **set print max-depth** in Section 10.9 [Print Settings], page 159).

repeat_threshold

Set the threshold for suppressing display of repeated array elements, or 0 to represent all elements, even if repeated. (See **set print repeats** in Section 10.9 [Print Settings], page 159).

format

A string containing a single character representing the format to use for the returned string. For instance, 'x' is equivalent to using the GDB command **print** with the /x option and formats the value as a hexadecimal number.

styling

True if GDB should apply styling to the returned string. When styling is applied, the returned string might contain ANSI terminal escape sequences. Escape sequences will only be included if styling is turned on, see Section 22.5 [Output Styling], page 375. Additionally, GDB only styles some value contents, so not every output string will contain escape sequences.

When **False**, which is the default, no output styling is applied.

summary

True when just a summary should be printed. In this mode, scalar values are printed in their entirety, but aggregates such as structures or unions are omitted. This mode is used by **set print frame-arguments scalars** (see Section 10.9 [Print Settings], page 159).

Value.to_array () [Function]

If this value is array-like (see [Type.is_array_like], page 426), then this method converts it to an array, which is returned. If this value is already an array, it is simply returned. Otherwise, an exception is throw.

Value.string ([encoding[, errors[, length]]) [Function]

If this `gdb.Value` represents a string, then this method converts the contents to a Python string. Otherwise, this method will throw an exception.

Values are interpreted as strings according to the rules of the current language. If the optional *length* argument is given, the string will be converted to that length, and will include any embedded zeroes that the string may contain. Otherwise, for languages where the string is zero-terminated, the entire string will be converted.

For example, in C-like languages, a value is a string if it is a pointer to or an array of characters or ints of type `wchar_t`, `char16_t`, or `char32_t`.

If the optional *encoding* argument is given, it must be a string naming the encoding of the string in the `gdb.Value`, such as "ascii", "iso-8859-6" or "utf-8". It accepts the same encodings as the corresponding argument to Python's `string.decode` method, and the Python codec machinery will be used to convert the string. If *encoding* is not given, or if *encoding* is the empty string, then either the `target-charset` (see Section 10.21 [Character Sets], page 190) will be used, or a language-specific encoding will be used, if the current language is able to supply one.

The optional *errors* argument is the same as the corresponding argument to Python's `string.decode` method.

If the optional *length* argument is given, the string will be fetched and converted to the given length.

Value.lazy_string ([encoding [, length]]) [Function]

This method attempts to convert this `gdb.Value` to a `gdb.LazyString` (see Section 23.3.2.35 [Lazy Strings In Python], page 508). Values of array or pointer type can be converted; for other types, this method will throw an exception.

If the optional *encoding* argument is given, it must be a string naming the encoding of the `gdb.LazyString`. Some examples are: 'ascii', 'iso-8859-6' or 'utf-8'. If the *encoding* argument is an encoding that GDB does recognize, GDB will raise an error.

When a lazy string is printed, the GDB encoding machinery is used to convert the string during printing. If the optional *encoding* argument is not provided, or is an empty string, GDB will automatically select the encoding most suitable for the string type. For further information on encoding in GDB please see Section 10.21 [Character Sets], page 190.

If the optional *length* argument is given, the string will be fetched and encoded to the length of characters specified. If the *length* argument is not provided, the string will be fetched and encoded until a null of appropriate width is found.

Value.fetch_lazy () [Function]

If the `gdb.Value` object is currently a lazy value (`gdb.Value.is_lazy` is `True`), then the value is fetched from the inferior. Any errors that occur in the process will produce a Python exception.

If the `gdb.Value` object is not a lazy value, this method has no effect.
 This method does not return a value.

23.3.2.5 Types In Python

GDB represents types from the inferior using the class `gdb.Type`.

The following type-related functions are available in the `gdb` module:

`gdb.lookup_type (name [, block])` [Function]

This function looks up a type by its *name*, which must be a string.

If *block* is given, then *name* is looked up in that scope. Otherwise, it is searched for globally.

Ordinarily, this function will return an instance of `gdb.Type`. If the named type cannot be found, it will throw an exception.

Integer types can be found without looking them up by name. See Section 23.3.2.38 [Architectures In Python], page 512, for the `integer_type` method.

If the type is a structure or class type, or an enum type, the fields of that type can be accessed using the Python *dictionary syntax*. For example, if `some_type` is a `gdb.Type` instance holding a structure type, you can access its `foo` field with:

```
bar = some_type['foo']
```

`bar` will be a `gdb.Field` object; see below under the description of the `Type.fields` method for a description of the `gdb.Field` class.

An instance of `Type` has the following attributes:

`Type.alignof` [Variable]

The alignment of this type, in bytes. Type alignment comes from the debugging information; if it was not specified, then GDB will use the relevant ABI to try to determine the alignment. In some cases, even this is not possible, and zero will be returned.

`Type.code` [Variable]

The type code for this type. The type code will be one of the `TYPE_CODE_` constants defined below.

`Type.dynamic` [Variable]

A boolean indicating whether this type is dynamic. In some situations, such as Rust `enum` types or Ada variant records, the concrete type of a value may vary depending on its contents. That is, the declared type of a variable, or the type returned by `gdb.lookup_type` may be dynamic; while the type of the variable's value will be a concrete instance of that dynamic type.

For example, consider this code:

```
int n;
int array[n];
```

Here, at least conceptually (whether your compiler actually does this is a separate issue), examining `gdb.lookup_symbol("array", ...).type` could yield a `gdb.Type` which reports a size of `None`. This is the dynamic type.

However, examining `gdb.parse_and_eval("array").type` would yield a concrete type, whose length would be known.

Type.name [Variable]
The name of this type. If this type has no name, then `None` is returned.

Type.sizeof [Variable]
The size of this type, in target `char` units. Usually, a target's `char` type will be an 8-bit byte. However, on some unusual platforms, this type may have a different size. A dynamic type may not have a fixed size; in this case, this attribute's value will be `None`.

Type.tag [Variable]
The tag name for this type. The tag name is the name after `struct`, `union`, or `enum` in C and C++; not all languages have this concept. If this type has no tag name, then `None` is returned.

Type.objfile [Variable]
The `gdb.Objfile` that this type was defined in, or `None` if there is no associated objfile.

Type.is_scalar [Variable]
This property is `True` if the type is a scalar type, otherwise, this property is `False`. Examples of non-scalar types include structures, unions, and classes.

Type.is_signed [Variable]
For scalar types (those for which `Type.is_scalar` is `True`), this property is `True` if the type is signed, otherwise this property is `False`.
Attempting to read this property for a non-scalar type (a type for which `Type.is_scalar` is `False`), will raise a `ValueError`.

Type.is_array_like [Variable]
A boolean indicating whether this type is array-like.
Some languages have array-like objects that are represented internally as structures. For example, this is true for a Rust slice type, or for an Ada unconstrained array. GDB may know about these types. This determination is done based on the language from which the type originated.

Type.is_string_like [Variable]
A boolean indicating whether this type is string-like. Like `Type.is_array_like`, this is determined based on the originating language of the type.

The following methods are provided:

Type.fields () [Function]
Return the fields of this type. The behavior depends on the type code:

- For structure and union types, this method returns the fields.
- Enum types have one field per enum constant.
- Function and method types have one field per parameter. The base types of C++ classes are also represented as fields.
- Array types have one field representing the array's range.

- If the type does not fit into one of these categories, a `TypeError` is raised.

Each field is a `gdb.Field` object, with some pre-defined attributes:

<code>bitpos</code>	This attribute is not available for <code>enum</code> or <code>static</code> (as in C++) fields. The value is the position, counting in bits, from the start of the containing type. Note that, in a dynamic type, the position of a field may not be constant. In this case, the value will be <code>None</code> . Also, a dynamic type may have fields that do not appear in a corresponding concrete type.
<code>enumval</code>	This attribute is only available for <code>enum</code> fields, and its value is the enumeration member's integer representation.
<code>name</code>	The name of the field, or <code>None</code> for anonymous fields.
<code>artificial</code>	This is <code>True</code> if the field is artificial, usually meaning that it was provided by the compiler and not the user. This attribute is always provided, and is <code>False</code> if the field is not artificial.
<code>is_base_class</code>	This is <code>True</code> if the field represents a base class of a C++ structure. This attribute is always provided, and is <code>False</code> if the field is not a base class of the type that is the argument of <code>fields</code> , or if that type was not a C++ class.
<code>bitsize</code>	If the field is packed, or is a bitfield, then this will have a non-zero value, which is the size of the field in bits. Otherwise, this will be zero; in this case the field's size is given by its type.
<code>type</code>	The type of the field. This is usually an instance of <code>Type</code> , but it can be <code>None</code> in some situations.
<code>parent_type</code>	The type which contains this field. This is an instance of <code>gdb.Type</code> .

`Type.array (n1 [, n2])` [Function]

Return a new `gdb.Type` object which represents an array of this type. If one argument is given, it is the inclusive upper bound of the array; in this case the lower bound is zero. If two arguments are given, the first argument is the lower bound of the array, and the second argument is the upper bound of the array. An array's length must not be negative, but the bounds can be.

`Type.vector (n1 [, n2])` [Function]

Return a new `gdb.Type` object which represents a vector of this type. If one argument is given, it is the inclusive upper bound of the vector; in this case the lower bound is zero. If two arguments are given, the first argument is the lower bound of the vector, and the second argument is the upper bound of the vector. A vector's length must not be negative, but the bounds can be.

The difference between an `array` and a `vector` is that arrays behave like in C: when used in expressions they decay to a pointer to the first element whereas vectors are treated as first class values.

- Type.const ()** [Function]
Return a new `gdb.Type` object which represents a `const`-qualified variant of this type.
- Type.volatile ()** [Function]
Return a new `gdb.Type` object which represents a `volatile`-qualified variant of this type.
- Type.unqualified ()** [Function]
Return a new `gdb.Type` object which represents an unqualified variant of this type. That is, the result is neither `const` nor `volatile`.
- Type.range ()** [Function]
Return a Python `Tuple` object that contains two elements: the low bound of the argument type and the high bound of that type. If the type does not have a range, GDB will raise a `gdb.error` exception (see Section 23.3.2.3 [Exception Handling], page 416).
- Type.reference ()** [Function]
Return a new `gdb.Type` object which represents a reference to this type.
- Type.pointer ()** [Function]
Return a new `gdb.Type` object which represents a pointer to this type.
- Type.strip_typedefs ()** [Function]
Return a new `gdb.Type` that represents the real type, after removing all layers of typedefs.
- Type.target ()** [Function]
Return a new `gdb.Type` object which represents the target type of this type.
For a pointer type, the target type is the type of the pointed-to object. For an array type (meaning C-like arrays), the target type is the type of the elements of the array. For a function or method type, the target type is the type of the return value. For a complex type, the target type is the type of the elements. For a typedef, the target type is the aliased type.
If the type does not have a target, this method will throw an exception.
- Type.template_argument (n [, block])** [Function]
If this `gdb.Type` is an instantiation of a template, this will return a new `gdb.Value` or `gdb.Type` which represents the value of the *n*th template argument (indexed starting at 0).
If this `gdb.Type` is not a template type, or if the type has fewer than *n* template arguments, this will throw an exception. Ordinarily, only C++ code will have template types.
If *block* is given, then *name* is looked up in that scope. Otherwise, it is searched for globally.
- Type.optimized_out ()** [Function]
Return `gdb.Value` instance of this type whose value is optimized out. This allows a frame decorator to indicate that the value of an argument or a local variable is not known.

Each type has a code, which indicates what category this type falls into. The available type categories are represented by constants defined in the `gdb` module:

`gdb.TYPE_CODE_PTR`

The type is a pointer.

`gdb.TYPE_CODE_ARRAY`

The type is an array.

`gdb.TYPE_CODE_STRUCT`

The type is a structure.

`gdb.TYPE_CODE_UNION`

The type is a union.

`gdb.TYPE_CODE_ENUM`

The type is an enum.

`gdb.TYPE_CODE_FLAGS`

A bit flags type, used for things such as status registers.

`gdb.TYPE_CODE_FUNC`

The type is a function.

`gdb.TYPE_CODE_INT`

The type is an integer type.

`gdb.TYPE_CODE_FLT`

A floating point type.

`gdb.TYPE_CODE_VOID`

The special type `void`.

`gdb.TYPE_CODE_SET`

A Pascal set type.

`gdb.TYPE_CODE_RANGE`

A range type, that is, an integer type with bounds.

`gdb.TYPE_CODE_STRING`

A string type. Note that this is only used for certain languages with language-defined string types; C strings are not represented this way.

`gdb.TYPE_CODE_BITSTRING`

A string of bits. It is deprecated.

`gdb.TYPE_CODE_ERROR`

An unknown or erroneous type.

`gdb.TYPE_CODE_METHOD`

A method type, as found in C++.

`gdb.TYPE_CODE_METHODPTR`

A pointer-to-member-function.

`gdb.TYPE_CODE_MEMBERPTR`

A pointer-to-member.

`gdb.TYPE_CODE_REF`
A reference type.

`gdb.TYPE_CODE_RVALUE_REF`
A C++11 rvalue reference type.

`gdb.TYPE_CODE_CHAR`
A character type.

`gdb.TYPE_CODE_BOOL`
A boolean type.

`gdb.TYPE_CODE_COMPLEX`
A complex float type.

`gdb.TYPE_CODE_TYPEDEF`
A typedef to some other type.

`gdb.TYPE_CODE_NAMESPACE`
A C++ namespace.

`gdb.TYPE_CODE_DECFLOAT`
A decimal floating point type.

`gdb.TYPE_CODE_INTERNAL_FUNCTION`
A function internal to GDB. This is the type used to represent convenience functions.

`gdb.TYPE_CODE_XMETHOD`
A method internal to GDB. This is the type used to represent xmethods (see Section 23.3.2.16 [Writing an Xmethod], page 452).

`gdb.TYPE_CODE_FIXED_POINT`
A fixed-point number.

`gdb.TYPE_CODE_NAMESPACE`
A Fortran namelist.

Further support for types is provided in the `gdb.types` Python module (see Section 23.3.4.2 [gdb.types], page 533).

23.3.2.6 Pretty Printing API

A pretty-printer is just an object that holds a value and implements a specific interface, defined here. An example output is provided (see Section 10.10 [Pretty Printing], page 170).

Because GDB did not document extensibility for pretty-printers, by default GDB will assume that only the basic pretty-printer methods may be available. The basic methods are marked as such, below.

To allow extensibility, GDB provides the `gdb.ValuePrinter` base class. This class does not provide any attributes or behavior, but instead serves as a tag that can be recognized by GDB. For such printers, GDB reserves all attributes starting with a lower-case letter. That is, in the future, GDB may add a new method or attribute to the pretty-printer protocol, and `gdb.ValuePrinter`-based printers are expected to handle this gracefully. A simple way to do this would be to use a leading underscore (or two, following the Python name-mangling scheme) to any attributes local to the implementation.

pretty_printer.children (self) [Function]

GDB will call this method on a pretty-printer to compute the children of the pretty-printer's value.

This method must return an object conforming to the Python iterator protocol. Each item returned by the iterator must be a tuple holding two elements. The first element is the “name” of the child; the second element is the child's value. The value can be any Python object which is convertible to a GDB value.

This is a basic method, and is optional. If it does not exist, GDB will act as though the value has no children.

For efficiency, the **children** method should lazily compute its results. This will let GDB read as few elements as necessary, for example when various print settings (see Section 10.9 [Print Settings], page 159) or **-var-list-children** (see Section 27.17 [GDB/MI Variable Objects], page 660) limit the number of elements to be displayed.

Children may be hidden from display based on the value of ‘**set print max-depth**’ (see Section 10.9 [Print Settings], page 159).

pretty_printer.display_hint (self) [Function]

The CLI may call this method and use its result to change the formatting of a value. The result will also be supplied to an MI consumer as a ‘**displayhint**’ attribute of the variable being printed.

This is a basic method, and is optional. If it does exist, this method must return a string or the special value **None**.

Some display hints are predefined by GDB:

- ‘**array**’ Indicate that the object being printed is “array-like”. The CLI uses this to respect parameters such as **set print elements** and **set print array**.
- ‘**map**’ Indicate that the object being printed is “map-like”, and that the children of this value can be assumed to alternate between keys and values.
- ‘**string**’ Indicate that the object being printed is “string-like”. If the printer's **to_string** method returns a Python string of some kind, then GDB will call its internal language-specific string-printing function to format the string. For the CLI this means adding quotation marks, possibly escaping some characters, respecting **set print elements**, and the like.

The special value **None** causes GDB to apply the default display rules.

pretty_printer.to_string (self) [Function]

GDB will call this method to display the string representation of the value passed to the object's constructor.

This is a basic method, and is optional.

When printing from the CLI, if the **to_string** method exists, then GDB will prepend its result to the values returned by **children**. Exactly how this formatting is done is dependent on the display hint, and may change as more hints are added. Also, depending on the print settings (see Section 10.9 [Print Settings], page 159), the CLI may print just the result of **to_string** in a stack trace, omitting the result of **children**.

If this method returns a string, it is printed verbatim.

Otherwise, if this method returns an instance of `gdb.Value`, then GDB prints this value. This may result in a call to another pretty-printer.

If instead the method returns a Python value which is convertible to a `gdb.Value`, then GDB performs the conversion and prints the resulting value. Again, this may result in a call to another pretty-printer. Python scalars (integers, floats, and booleans) and strings are convertible to `gdb.Value`; other types are not.

Finally, if this method returns `None` then no further operations are performed in this method and nothing is printed.

If the result is not one of these types, an exception is raised.

`pretty_printer.num_children ()` [Function]

This is not a basic method, so GDB will only ever call it for objects derived from `gdb.ValuePrinter`.

If available, this method should return the number of children. `None` may be returned if the number can't readily be computed.

`pretty_printer.child (n)` [Function]

This is not a basic method, so GDB will only ever call it for objects derived from `gdb.ValuePrinter`.

If available, this method should return the child item (that is, a tuple holding the name and value of this child) indicated by `n`. Indices start at zero.

GDB provides a function which can be used to look up the default pretty-printer for a `gdb.Value`:

`gdb.default_visualizer (value)` [Function]

This function takes a `gdb.Value` object as an argument. If a pretty-printer for this value exists, then it is returned. If no such printer exists, then this returns `None`.

Normally, a pretty-printer can respect the user's print settings (including temporarily applied settings, such as `'/x'`) simply by calling `Value.format_string` (see Section 23.3.2.4 [Values From Inferior], page 417). However, these settings can also be queried directly:

`gdb.print_options ()` [Function]

Return a dictionary whose keys are the valid keywords that can be given to `Value.format_string`, and whose values are the user's settings. During a `print` or other operation, the values will reflect any flags that are temporarily in effect.

```
(gdb) python print(gdb.print_options ()['max_elements'])
200
```

23.3.2.7 Selecting Pretty-Printers

GDB provides several ways to register a pretty-printer: globally, per program space, and per objfile. When choosing how to register your pretty-printer, a good rule is to register it with the smallest scope possible: that is prefer a specific objfile first, then a program space, and only register a printer globally as a last resort.

`gdb.pretty_printers` [Variable]

The Python list `gdb.pretty_printers` contains an array of functions or callable objects that have been registered via addition as a pretty-printer. Printers in this list are called **global printers**, they're available when debugging all inferiors.

Each `gdb.Progspace` contains a `pretty_printers` attribute. Each `gdb.Objfile` also contains a `pretty_printers` attribute.

Each function on these lists is passed a single `gdb.Value` argument and should return a pretty-printer object conforming to the interface definition above (see Section 23.3.2.6 [Pretty Printing API], page 430). If a function cannot create a pretty-printer for the value, it should return `None`.

GDB first checks the `pretty_printers` attribute of each `gdb.Objfile` in the current program space and iteratively calls each enabled lookup routine in the list for that `gdb.Objfile` until it receives a pretty-printer object. If no pretty-printer is found in the objfile lists, GDB then searches the pretty-printer list of the current program space, calling each enabled function until an object is returned. After these lists have been exhausted, it tries the global `gdb.pretty_printers` list, again calling each enabled function until an object is returned.

The order in which the objfiles are searched is not specified. For a given list, functions are always invoked from the head of the list, and iterated over sequentially until the end of the list, or a printer object is returned.

For various reasons a pretty-printer may not work. For example, the underlying data structure may have changed and the pretty-printer is out of date.

The consequences of a broken pretty-printer are severe enough that GDB provides support for enabling and disabling individual printers. For example, if `print frame-arguments` is on, a backtrace can become highly illegible if any argument is printed with a broken printer.

Pretty-printers are enabled and disabled by attaching an `enabled` attribute to the registered function or callable object. If this attribute is present and its value is `False`, the printer is disabled, otherwise the printer is enabled.

23.3.2.8 Writing a Pretty-Printer

A pretty-printer consists of two parts: a lookup function to detect if the type is supported, and the printer itself.

Here is an example showing how a `std::string` printer might be written. See Section 23.3.2.6 [Pretty Printing API], page 430, for details on the API this class must provide. Note that this example uses the `gdb.ValuePrinter` base class, and is careful to use a leading underscore for its local state.

```
class StdStringPrinter(gdb.ValuePrinter):
    "Print a std::string"

    def __init__(self, val):
        self.__val = val

    def to_string(self):
        return self.__val['_M_dataplus']['_M_p']

    def display_hint(self):
        return 'string'
```

And here is an example showing how a lookup function for the printer example above might be written.

```
def str_lookup_function(val):
    lookup_tag = val.type.tag
    if lookup_tag is None:
        return None
    regex = re.compile("^std::basic_string<char,.*>$")
    if regex.match(lookup_tag):
        return StdStringPrinter(val)
    return None
```

The example lookup function extracts the value's type, and attempts to match it to a type that it can pretty-print. If it is a type the printer can pretty-print, it will return a printer object. If not, it returns `None`.

We recommend that you put your core pretty-printers into a Python package. If your pretty-printers are for use with a library, we further recommend embedding a version number into the package name. This practice will enable GDB to load multiple versions of your pretty-printers at the same time, because they will have different names.

You should write auto-loaded code (see Section 23.3.3 [Python Auto-loading], page 532) such that it can be evaluated multiple times without changing its meaning. An ideal auto-load file will consist solely of `imports` of your printer modules, followed by a call to a register pretty-printers with the current objfile.

Taken as a whole, this approach will scale nicely to multiple inferiors, each potentially using a different library version. Embedding a version number in the Python package name will ensure that GDB is able to load both sets of printers simultaneously. Then, because the search for pretty-printers is done by objfile, and because your auto-loaded code took care to register your library's printers with a specific objfile, GDB will find the correct printers for the specific version of the library used by each inferior.

To continue the `std::string` example (see Section 23.3.2.6 [Pretty Printing API], page 430), this code might appear in `gdb.libstdcxx.v6`:

```
def register_printers(objfile):
    objfile.pretty_printers.append(str_lookup_function)
```

And then the corresponding contents of the auto-load file would be:

```
import gdb.libstdcxx.v6
gdb.libstdcxx.v6.register_printers(gdb.current_objfile())
```

The previous example illustrates a basic pretty-printer. There are a few things that can be improved on. The printer doesn't have a name, making it hard to identify in a list of installed printers. The lookup function has a name, but lookup functions can have arbitrary, even identical, names.

Second, the printer only handles one type, whereas a library typically has several types. One could install a lookup function for each desired type in the library, but one could also have a single lookup function recognize several types. The latter is the conventional way this is handled. If a pretty-printer can handle multiple data types, then its *subprinters* are the printers for the individual data types.

The `gdb.printing` module provides a formal way of solving these problems (see Section 23.3.4.1 [gdb.printing], page 533). Here is another example that handles multiple types.

These are the types we are going to pretty-print:

```
struct foo { int a, b; };
struct bar { struct foo x, y; };
```

Here are the printers:

```
class fooPrinter(gdb.ValuePrinter):
    """Print a foo object."""

    def __init__(self, val):
        self.__val = val

    def to_string(self):
        return ("a=<" + str(self.__val["a"]) +
                "> b=<" + str(self.__val["b"]) + ">")

class barPrinter(gdb.ValuePrinter):
    """Print a bar object."""

    def __init__(self, val):
        self.__val = val

    def to_string(self):
        return ("x=<" + str(self.__val["x"]) +
                "> y=<" + str(self.__val["y"]) + ">")
```

This example doesn't need a lookup function, that is handled by the `gdb.printing` module. Instead a function is provided to build up the object that handles the lookup.

```
import gdb.printing

def build_pretty_printer():
    pp = gdb.printing.RegexpCollectionPrettyPrinter(
        "my_library")
    pp.add_printer('foo', '^foo$', fooPrinter)
    pp.add_printer('bar', '^bar$', barPrinter)
    return pp
```

And here is the autoload support:

```
import gdb.printing
import my_library
gdb.printing.register_pretty_printer(
    gdb.current_objfile(),
    my_library.build_pretty_printer())
```

Finally, when this printer is loaded into GDB, here is the corresponding output of `'info pretty-printer'`:

```
(gdb) info pretty-printer
my_library.so:
  my_library
    foo
    bar
```

23.3.2.9 Type Printing API

GDB provides a way for Python code to customize type display. This is mainly useful for substituting canonical typedef names for types.

A *type printer* is just a Python object conforming to a certain protocol. A simple base class implementing the protocol is provided; see Section 23.3.4.2 [gdb.types], page 533. A type printer must supply at least:

enabled [Instance Variable of `type_printer`]
 A boolean which is True if the printer is enabled, and False otherwise. This is manipulated by the `enable type-printer` and `disable type-printer` commands.

name [Instance Variable of `type_printer`]
 The name of the type printer. This must be a string. This is used by the `enable type-printer` and `disable type-printer` commands.

instantiate (*self*) [Method on `type_printer`]
 This is called by GDB at the start of type-printing. It is only called if the type printer is enabled. This method must return a new object that supplies a `recognize` method, as described below.

When displaying a type, say via the `ptype` command, GDB will compute a list of type recognizers. This is done by iterating first over the per-objfile type printers (see Section 23.3.2.27 [Objfiles In Python], page 487), followed by the per-progspace type printers (see Section 23.3.2.26 [Progspace In Python], page 484), and finally the global type printers.

GDB will call the `instantiate` method of each enabled type printer. If this method returns `None`, then the result is ignored; otherwise, it is appended to the list of recognizers.

Then, when GDB is going to display a type name, it iterates over the list of recognizers. For each one, it calls the recognition function, stopping if the function returns a non-`None` value. The recognition function is defined as:

recognize (*self*, *type*) [Method on `type_recognizer`]
 If *type* is not recognized, return `None`. Otherwise, return a string which is to be printed as the name of *type*. The *type* argument will be an instance of `gdb.Type` (see Section 23.3.2.5 [Types In Python], page 425).

GDB uses this two-pass approach so that type printers can efficiently cache information without holding on to it too long. For example, it can be convenient to look up type information in a type printer and hold it for a recognizer's lifetime; if a single pass were done then type printers would have to make use of the event system in order to avoid holding information that could become stale as the inferior changed.

23.3.2.10 Filtering Frames

Frame filters are Python objects that manipulate the visibility of a frame or frames when a backtrace (see Section 8.2 [Backtrace], page 116) is printed by GDB.

Only commands that print a backtrace, or, in the case of GDB/MI commands (see Chapter 27 [GDB/MI], page 609), those that return a collection of frames are affected. The commands that work with frame filters are:

backtrace (see [The backtrace command], page 116), **-stack-list-frames** (see [The -stack-list-frames command], page 657), **-stack-list-variables** (see [The -stack-list-variables command], page 659), **-stack-list-arguments** (see [The -stack-list-arguments command], page 655) and **-stack-list-locals** (see [The -stack-list-locals command], page 659).

A frame filter works by taking an iterator as an argument, applying actions to the contents of that iterator, and returning another iterator (or, possibly, the same iterator it

was provided in the case where the filter does not perform any operations). Typically, frame filters utilize tools such as the Python's `itertools` module to work with and create new iterators from the source iterator. Regardless of how a filter chooses to apply actions, it must not alter the underlying GDB frame or frames, or attempt to alter the call-stack within GDB. This preserves data integrity within GDB. Frame filters are executed on a priority basis and care should be taken that some frame filters may have been executed before, and that some frame filters will be executed after.

An important consideration when designing frame filters, and well worth reflecting upon, is that frame filters should avoid unwinding the call stack if possible. Some stacks can run very deep, into the tens of thousands in some cases. To search every frame when a frame filter executes may be too expensive at that step. The frame filter cannot know how many frames it has to iterate over, and it may have to iterate through them all. This ends up duplicating effort as GDB performs this iteration when it prints the frames. If the filter can defer unwinding frames until frame decorators are executed, after the last filter has executed, it should. See Section 23.3.2.11 [Frame Decorator API], page 438, for more information on decorators. Also, there are examples for both frame decorators and filters in later chapters. See Section 23.3.2.12 [Writing a Frame Filter], page 441, for more information.

The Python dictionary `gdb.frame_filters` contains key/object pairings that comprise a frame filter. Frame filters in this dictionary are called **global** frame filters, and they are available when debugging all inferiors. These frame filters must register with the dictionary directly. In addition to the **global** dictionary, there are other dictionaries that are loaded with different inferiors via auto-loading (see Section 23.3.3 [Python Auto-loading], page 532). The two other areas where frame filter dictionaries can be found are: `gdb.Progspace` which contains a `frame_filters` dictionary attribute, and each `gdb.Objfile` object which also contains a `frame_filters` dictionary attribute.

When a command is executed from GDB that is compatible with frame filters, GDB combines the **global**, `gdb.Progspace` and all `gdb.Objfile` dictionaries currently loaded. All of the `gdb.Objfile` dictionaries are combined, as several frames, and thus several object files, might be in use. GDB then prunes any frame filter whose **enabled** attribute is **False**. This pruned list is then sorted according to the **priority** attribute in each filter.

Once the dictionaries are combined, pruned and sorted, GDB creates an iterator which wraps each frame in the call stack in a `FrameDecorator` object, and calls each filter in order. The output from the previous filter will always be the input to the next filter, and so on.

Frame filters have a mandatory interface which each frame filter must implement, defined here:

`FrameFilter.filter` (*iterator*) [Function]

GDB will call this method on a frame filter when it has reached the order in the priority list for that filter.

For example, if there are four frame filters:

Name	Priority
<code>Filter1</code>	5
<code>Filter2</code>	10
<code>Filter3</code>	100
<code>Filter4</code>	1

The order that the frame filters will be called is:

```
Filter3 -> Filter2 -> Filter1 -> Filter4
```

Note that the output from `Filter3` is passed to the input of `Filter2`, and so on.

This `filter` method is passed a Python iterator. This iterator contains a sequence of frame decorators that wrap each `gdb.Frame`, or a frame decorator that wraps another frame decorator. The first filter that is executed in the sequence of frame filters will receive an iterator entirely comprised of default `FrameDecorator` objects. However, after each frame filter is executed, the previous frame filter may have wrapped some or all of the frame decorators with their own frame decorator. As frame decorators must also conform to a mandatory interface, these decorators can be assumed to act in a uniform manner (see Section 23.3.2.11 [Frame Decorator API], page 438).

This method must return an object conforming to the Python iterator protocol. Each item in the iterator must be an object conforming to the frame decorator interface. If a frame filter does not wish to perform any operations on this iterator, it should return that iterator untouched.

This method is not optional. If it does not exist, GDB will raise and print an error.

FrameFilter.name [Variable]

The `name` attribute must be Python string which contains the name of the filter displayed by GDB (see Section 8.6 [Frame Filter Management], page 124). This attribute may contain any combination of letters or numbers. Care should be taken to ensure that it is unique. This attribute is mandatory.

FrameFilter.enabled [Variable]

The `enabled` attribute must be Python boolean. This attribute indicates to GDB whether the frame filter is enabled, and should be considered when frame filters are executed. If `enabled` is `True`, then the frame filter will be executed when any of the backtrace commands detailed earlier in this chapter are executed. If `enabled` is `False`, then the frame filter will not be executed. This attribute is mandatory.

FrameFilter.priority [Variable]

The `priority` attribute must be Python integer. This attribute controls the order of execution in relation to other frame filters. There are no imposed limits on the range of `priority` other than it must be a valid integer. The higher the `priority` attribute, the sooner the frame filter will be executed in relation to other frame filters. Although `priority` can be negative, it is recommended practice to assume zero is the lowest priority that a frame filter can be assigned. Frame filters that have the same priority are executed in unsorted order in that priority slot. This attribute is mandatory. 100 is a good default priority.

23.3.2.11 Decorating Frames

Frame decorators are sister objects to frame filters (see Section 23.3.2.10 [Frame Filter API], page 436). Frame decorators are applied by a frame filter and can only be used in conjunction with frame filters.

The purpose of a frame decorator is to customize the printed content of each `gdb.Frame` in commands where frame filters are executed. This concept is called decorating a frame. Frame decorators decorate a `gdb.Frame` with Python code contained within each API call.

This separates the actual data contained in a `gdb.Frame` from the decorated data produced by a frame decorator. This abstraction is necessary to maintain integrity of the data contained in each `gdb.Frame`.

Frame decorators have a mandatory interface, defined below.

GDB already contains a frame decorator called `FrameDecorator`. This contains substantial amounts of boilerplate code to decorate the content of a `gdb.Frame`. It is recommended that other frame decorators inherit and extend this object, and only to override the methods needed.

`FrameDecorator` is defined in the Python module `gdb.FrameDecorator`, so your code can import it like:

```
from gdb.FrameDecorator import FrameDecorator
```

FrameDecorator.elided (*self*) [Function]

The `elided` method groups frames together in a hierarchical system. An example would be an interpreter, where multiple low-level frames make up a single call in the interpreted language. In this example, the frame filter would elide the low-level frames and present a single high-level frame, representing the call in the interpreted language, to the user.

The `elided` function must return an iterable and this iterable must contain the frames that are being elided wrapped in a suitable frame decorator. If no frames are being elided this function may return an empty iterable, or `None`. Elided frames are indented from normal frames in a CLI backtrace, or in the case of GDB/MI, are placed in the `children` field of the eliding frame.

It is the frame filter's task to also filter out the elided frames from the source iterator. This will avoid printing the frame twice.

FrameDecorator.function (*self*) [Function]

This method returns the name of the function in the frame that is to be printed.

This method must return a Python string describing the function, or `None`.

If this function returns `None`, GDB will not print any data for this field.

FrameDecorator.address (*self*) [Function]

This method returns the address of the frame that is to be printed.

This method must return a Python numeric integer type of sufficient size to describe the address of the frame, or `None`.

If this function returns a `None`, GDB will not print any data for this field.

FrameDecorator.filename (*self*) [Function]

This method returns the filename and path associated with this frame.

This method must return a Python string containing the filename and the path to the object file backing the frame, or `None`.

If this function returns a `None`, GDB will not print any data for this field.

FrameDecorator.line (*self*): [Function]

This method returns the line number associated with the current position within the function addressed by this frame.

This method must return a Python integer type, or `None`.

If this function returns a `None`, GDB will not print any data for this field.

FrameDecorator.frame_args (*self*) [Function]

This method must return an iterable, or `None`. Returning an empty iterable, or `None` means frame arguments will not be printed for this frame. This iterable must contain objects that implement two methods, described here.

This object must implement a `symbol` method which takes a single `self` parameter and must return a `gdb.Symbol` (see Section 23.3.2.30 [Symbols In Python], page 495), or a Python string. The object must also implement a `value` method which takes a single `self` parameter and must return a `gdb.Value` (see Section 23.3.2.4 [Values From Inferior], page 417), a Python value, or `None`. If the `value` method returns `None`, and the `argument` method returns a `gdb.Symbol`, GDB will look-up and print the value of the `gdb.Symbol` automatically.

A brief example:

```
class SymValueWrapper():

    def __init__(self, symbol, value):
        self.sym = symbol
        self.val = value

    def value(self):
        return self.val

    def symbol(self):
        return self.sym

class SomeFrameDecorator()
...
...
    def frame_args(self):
        args = []
        try:
            block = self.inferior_frame.block()
        except:
            return None

        # Iterate over all symbols in a block. Only add
        # symbols that are arguments.
        for sym in block:
            if not sym.is_argument:
                continue
            args.append(SymValueWrapper(sym, None))

        # Add example synthetic argument.
        args.append(SymValueWrapper('foo', 42))

        return args
```

FrameDecorator.frame_locals (*self*) [Function]

This method must return an iterable or `None`. Returning an empty iterable, or `None` means frame local arguments will not be printed for this frame.

The object interface, the description of the various strategies for reading frame locals, and the example are largely similar to those described in the `frame_args` function, (see [The frame filter `frame_args` function], page 440). Below is a modified example:

```
class SomeFrameDecorator()
...
...
    def frame_locals(self):
        vars = []
        try:
            block = self.inferior_frame.block()
        except:
            return None

        # Iterate over all symbols in a block. Add all
        # symbols, except arguments.
        for sym in block:
            if sym.is_argument:
                continue
            vars.append(SymValueWrapper(sym, None))

        # Add an example of a synthetic local variable.
        vars.append(SymValueWrapper('bar', 99))

        return vars
```

`FrameDecorator.inferior_frame (self):` [Function]

This method must return the underlying `gdb.Frame` that this frame decorator is decorating. GDB requires the underlying frame for internal frame information to determine how to print certain values when printing a frame.

23.3.2.12 Writing a Frame Filter

There are three basic elements that a frame filter must implement: it must correctly implement the documented interface (see Section 23.3.2.10 [Frame Filter API], page 436), it must register itself with GDB, and finally, it must decide if it is to work on the data provided by GDB. In all cases, whether it works on the iterator or not, each frame filter must return an iterator. A bare-bones frame filter follows the pattern in the following example.

```
import gdb

class FrameFilter():

    def __init__(self):
        # Frame filter attribute creation.
        #
        # 'name' is the name of the filter that GDB will display.
        #
        # 'priority' is the priority of the filter relative to other
        # filters.
        #
        # 'enabled' is a boolean that indicates whether this filter is
        # enabled and should be executed.

        self.name = "Foo"
        self.priority = 100
        self.enabled = True
```

```

    # Register this frame filter with the global frame_filters
    # dictionary.
    gdb.frame_filters[self.name] = self

    def filter(self, frame_iter):
        # Just return the iterator.
        return frame_iter

```

The frame filter in the example above implements the three requirements for all frame filters. It implements the API, self registers, and makes a decision on the iterator (in this case, it just returns the iterator untouched).

The first step is attribute creation and assignment, and as shown in the comments the filter assigns the following attributes: **name**, **priority** and whether the filter should be enabled with the **enabled** attribute.

The second step is registering the frame filter with the dictionary or dictionaries that the frame filter has interest in. As shown in the comments, this filter just registers itself with the global dictionary **gdb.frame_filters**. As noted earlier, **gdb.frame_filters** is a dictionary that is initialized in the **gdb** module when GDB starts. What dictionary a filter registers with is an important consideration. Generally, if a filter is specific to a set of code, it should be registered either in the **objfile** or **progspace** dictionaries as they are specific to the program currently loaded in GDB. The global dictionary is always present in GDB and is never unloaded. Any filters registered with the global dictionary will exist until GDB exits. To avoid filters that may conflict, it is generally better to register frame filters against the dictionaries that more closely align with the usage of the filter currently in question. See Section 23.3.3 [Python Auto-loading], page 532, for further information on auto-loading Python scripts.

GDB takes a hands-off approach to frame filter registration, therefore it is the frame filter's responsibility to ensure registration has occurred, and that any exceptions are handled appropriately. In particular, you may wish to handle exceptions relating to Python dictionary key uniqueness. It is mandatory that the dictionary key is the same as frame filter's **name** attribute. When a user manages frame filters (see Section 8.6 [Frame Filter Management], page 124), the names GDB will display are those contained in the **name** attribute.

The final step of this example is the implementation of the **filter** method. As shown in the example comments, we define the **filter** method and note that the method must take an iterator, and also must return an iterator. In this bare-bones example, the frame filter is not very useful as it just returns the iterator untouched. However this is a valid operation for frame filters that have the **enabled** attribute set, but decide not to operate on any frames.

In the next example, the frame filter operates on all frames and utilizes a frame decorator to perform some work on the frames. See Section 23.3.2.11 [Frame Decorator API], page 438, for further information on the frame decorator interface.

This example works on inlined frames. It highlights frames which are inlined by tagging them with an "[inlined]" tag. By applying a frame decorator to all frames with the Python **itertools** **imap** method, the example defers actions to the frame decorator. Frame decorators are only processed when GDB prints the backtrace.

This introduces a new decision making topic: whether to perform decision making operations at the filtering step, or at the printing step. In this example's approach, it does

not perform any filtering decisions at the filtering step beyond mapping a frame decorator to each frame. This allows the actual decision making to be performed when each frame is printed. This is an important consideration, and well worth reflecting upon when designing a frame filter. An issue that frame filters should avoid is unwinding the stack if possible. Some stacks can run very deep, into the tens of thousands in some cases. To search every frame to determine if it is inlined ahead of time may be too expensive at the filtering step. The frame filter cannot know how many frames it has to iterate over, and it would have to iterate through them all. This ends up duplicating effort as GDB performs this iteration when it prints the frames.

In this example decision making can be deferred to the printing step. As each frame is printed, the frame decorator can examine each frame in turn when GDB iterates. From a performance viewpoint, this is the most appropriate decision to make as it avoids duplicating the effort that the printing step would undertake anyway. Also, if there are many frame filters unwinding the stack during filtering, it can substantially delay the printing of the backtrace which will result in large memory usage, and a poor user experience.

```
class InlineFilter():

    def __init__(self):
        self.name = "InlinedFrameFilter"
        self.priority = 100
        self.enabled = True
        gdb.frame_filters[self.name] = self

    def filter(self, frame_iter):
        frame_iter = itertools.imap(InlinedFrameDecorator,
                                    frame_iter)

        return frame_iter
```

This frame filter is somewhat similar to the earlier example, except that the `filter` method applies a frame decorator object called `InlinedFrameDecorator` to each element in the iterator. The `imap` Python method is light-weight. It does not proactively iterate over the iterator, but rather creates a new iterator which wraps the existing one.

Below is the frame decorator for this example.

```
class InlinedFrameDecorator(FrameDecorator):

    def __init__(self, fobj):
        super(InlinedFrameDecorator, self).__init__(fobj)

    def function(self):
        frame = self.inferior_frame()
        name = str(frame.name())

        if frame.type() == gdb.INLINE_FRAME:
            name = name + " [inlined]"

        return name
```

This frame decorator only defines and overrides the `function` method. It lets the supplied `FrameDecorator`, which is shipped with GDB, perform the other work associated with printing this frame.

The combination of these two objects create this output from a backtrace:

```
#0  0x004004e0 in bar () at inline.c:11
```

```
#1 0x00400566 in max [inlined] (b=6, a=12) at inline.c:21
#2 0x00400566 in main () at inline.c:31
```

So in the case of this example, a frame decorator is applied to all frames, regardless of whether they may be inlined or not. As GDB iterates over the iterator produced by the frame filters, GDB executes each frame decorator which then makes a decision on what to print in the `function` callback. Using a strategy like this is a way to defer decisions on the frame content to printing time.

Eliding Frames

It might be that the above example is not desirable for representing inlined frames, and a hierarchical approach may be preferred. If we want to hierarchically represent frames, the `elided` frame decorator interface might be preferable.

This example approaches the issue with the `elided` method. This example is quite long, but very simplistic. It is out-of-scope for this section to write a complete example that comprehensively covers all approaches of finding and printing inlined frames. However, this example illustrates the approach an author might use.

This example comprises of three sections.

```
class InlineFrameFilter():

    def __init__(self):
        self.name = "InlinedFrameFilter"
        self.priority = 100
        self.enabled = True
        gdb.frame_filters[self.name] = self

    def filter(self, frame_iter):
        return ElidingInlineIterator(frame_iter)
```

This frame filter is very similar to the other examples. The only difference is this frame filter is wrapping the iterator provided to it (`frame_iter`) with a custom iterator called `ElidingInlineIterator`. This again defers actions to when GDB prints the backtrace, as the iterator is not traversed until printing.

The iterator for this example is as follows. It is in this section of the example where decisions are made on the content of the backtrace.

```
class ElidingInlineIterator:
    def __init__(self, ii):
        self.input_iterator = ii

    def __iter__(self):
        return self

    def next(self):
        frame = next(self.input_iterator)

        if frame.inferior_frame().type() != gdb.INLINE_FRAME:
            return frame

        try:
            eliding_frame = next(self.input_iterator)
        except StopIteration:
            return frame
        return ElidingFrameDecorator(eliding_frame, [frame])
```

This iterator implements the Python iterator protocol. When the `next` function is called (when GDB prints each frame), the iterator checks if this frame decorator, `frame`, is wrapping an inlined frame. If it is not, it returns the existing frame decorator untouched. If it is wrapping an inlined frame, it assumes that the inlined frame was contained within the next oldest frame, `eliding_frame`, which it fetches. It then creates and returns a frame decorator, `ElidingFrameDecorator`, which contains both the elided frame, and the eliding frame.

```
class ElidingInlineDecorator(FrameDecorator):

    def __init__(self, frame, elided_frames):
        super(ElidingInlineDecorator, self).__init__(frame)
        self.frame = frame
        self.elided_frames = elided_frames

    def elided(self):
        return iter(self.elided_frames)
```

This frame decorator overrides one function and returns the inlined frame in the `elided` method. As before it lets `FrameDecorator` do the rest of the work involved in printing this frame. This produces the following output.

```
#0 0x004004e0 in bar () at inline.c:11
#2 0x00400529 in main () at inline.c:25
#1 0x00400529 in max (b=6, a=12) at inline.c:15
```

In that output, `max` which has been inlined into `main` is printed hierarchically. Another approach would be to combine the `function` method, and the `elided` method to both print a marker in the inlined frame, and also show the hierarchical relationship.

23.3.2.13 Unwinding Frames in Python

In GDB terminology “unwinding” is the process of finding the previous frame (that is, caller’s) from the current one. An unwinder has three methods. The first one checks if it can handle given frame (“sniff” it). For the frames it can sniff an unwinder provides two additional methods: it can return frame’s ID, and it can fetch registers from the previous frame. A running GDB maintains a list of the unwinders and calls each unwinder’s sniffer in turn until it finds the one that recognizes the current frame. There is an API to register an unwinder.

The unwinders that come with GDB handle standard frames. However, mixed language applications (for example, an application running Java Virtual Machine) sometimes use frame layouts that cannot be handled by the GDB unwinders. You can write Python code that can handle such custom frames.

You implement a frame unwinder in Python as a class with which has two attributes, `name` and `enabled`, with obvious meanings, and a single method `__call__`, which examines a given frame and returns an object (an instance of `gdb.UnwindInfo` class) describing it. If an unwinder does not recognize a frame, it should return `None`. The code in GDB that enables writing unwinders in Python uses this object to return frame’s ID and previous frame registers when GDB core asks for them.

An unwinder should do as little work as possible. Some otherwise innocuous operations can cause problems (even crashes, as this code is not well-hardened yet). For example, making an inferior call from an unwinder is unadvisable, as an inferior call will reset GDB’s stack unwinding process, potentially causing re-entrant unwinding.

Unwinder Input

An object passed to an unwinder (a `gdb.PendingFrame` instance) provides a method to read frame's registers:

`PendingFrame.read_register (register)` [Function]

This method returns the contents of *register* in the frame as a `gdb.Value` object. For a description of the acceptable values of *register* see `[Frame.read_register]`, page 492. If *register* does not name a register for the current architecture, this method will throw an exception.

Note that this method will always return a `gdb.Value` for a valid register name. This does not mean that the value will be valid. For example, you may request a register that an earlier unwinder could not unwind—the value will be unavailable. Instead, the `gdb.Value` returned from this method will be lazy; that is, its underlying bits will not be fetched until it is first used. So, attempting to use such a value will cause an exception at the point of use.

The type of the returned `gdb.Value` depends on the register and the architecture. It is common for registers to have a scalar type, like `long long`; but many other types are possible, such as pointer, pointer-to-function, floating point or vector types.

It also provides a factory method to create a `gdb.UnwindInfo` instance to be returned to GDB:

`PendingFrame.create_unwind_info (frame_id)` [Function]

Returns a new `gdb.UnwindInfo` instance identified by given *frame_id*. The *frame_id* is used internally by GDB to identify the frames within the current thread's stack. The attributes of *frame_id* determine what type of frame is created within GDB:

sp, pc The frame is identified by the given stack address and PC. The stack address must be chosen so that it is constant throughout the lifetime of the frame, so a typical choice is the value of the stack pointer at the start of the function—in the DWARF standard, this would be the “Call Frame Address”.

This is the most common case by far. The other cases are documented for completeness but are only useful in specialized situations.

sp, pc, special

The frame is identified by the stack address, the PC, and a “special” address. The special address is used on architectures that can have frames that do not change the stack, but which are still distinct, for example the IA-64, which has a second stack for registers. Both *sp* and *special* must be constant throughout the lifetime of the frame.

sp The frame is identified by the stack address only. Any other stack frame with a matching *sp* will be considered to match this frame. Inside gdb, this is called a “wild frame”. You will never need this.

Each attribute value should either be an instance of `gdb.Value` or an integer.

A helper class is provided in the `gdb.unwinder` module that can be used to represent a frame-id (see `[gdb.unwinder.FrameId]`, page 448).

PendingFrame.architecture () [Function]

Return the `gdb.Architecture` (see Section 23.3.2.38 [Architectures In Python], page 512) for this `gdb.PendingFrame`. This represents the architecture of the particular frame being unwound.

PendingFrame.level () [Function]

Return an integer, the stack frame level for this frame. See Section 8.1 [Stack Frames], page 115.

PendingFrame.name () [Function]

Returns the function name of this pending frame, or `None` if it can't be obtained.

PendingFrame.is_valid () [Function]

Returns true if the `gdb.PendingFrame` object is valid, false if not. A pending frame object becomes invalid when the call to the unwinder, for which the pending frame was created, returns.

All `gdb.PendingFrame` methods, except this one, will raise an exception if the pending frame object is invalid at the time the method is called.

PendingFrame.pc () [Function]

Returns the pending frame's resume address.

PendingFrame.block () [Function]

Return the pending frame's code block (see Section 23.3.2.29 [Blocks In Python], page 493). If the frame does not have a block – for example, if there is no debugging information for the code in question – then this will raise a `RuntimeError` exception.

PendingFrame.function () [Function]

Return the symbol for the function corresponding to this pending frame. See Section 23.3.2.30 [Symbols In Python], page 495.

PendingFrame.find_sal () [Function]

Return the pending frame's symtab and line object (see Section 23.3.2.31 [Symbol Tables In Python], page 500).

PendingFrame.language () [Function]

Return the language of this frame, as a string, or `None`.

Unwinder Output: UnwindInfo

Use `PendingFrame.create_unwind_info` method described above to create a `gdb.UnwindInfo` instance. Use the following method to specify caller registers that have been saved in this frame:

gdb.UnwindInfo.add_saved_register (register, value) [Function]

register identifies the register, for a description of the acceptable values see [Frame.read_register], page 492. *value* is a register value (a `gdb.Value` object).

The `gdb.unwinder` Module

GDB comes with a `gdb.unwinder` module which contains the following classes:

`gdb.unwinder.Unwinder` [class]

The `Unwinder` class is a base class from which user created unwinders can derive, though it is not required that unwinders derive from this class, so long as any user created unwinder has the required `name` and `enabled` attributes.

`gdb.unwinder.Unwinder.__init__ (name)` [Function]

The *name* is a string used to reference this unwinder within some GDB commands (see [Managing Registered Unwinders], page 449).

`gdb.unwinder.name` [Variable]

A read-only attribute which is a string, the name of this unwinder.

`gdb.unwinder.enabled` [Variable]

A modifiable attribute containing a boolean; when `True`, the unwinder is enabled, and will be used by GDB. When `False`, the unwinder has been disabled, and will not be used.

`gdb.unwinder.FrameId` [class]

This is a class suitable for being used as the frame-id when calling `gdb.PendingFrame.create_unwind_info`. It is not required to use this class, any class with the required attribute (see [gdb.PendingFrame.create_unwind_info], page 446) will be accepted, but in most cases this class will be sufficient.

`gdb.unwinder.FrameId` has the following method:

`gdb.unwinder.FrameId.__init__ (sp, pc, special = None)` [Function]

The *sp* and *pc* arguments are required and should be either a `gdb.Value` object, or an integer.

The *special* argument is optional; if specified, it should be a `gdb.Value` object, or an integer.

`gdb.unwinder.FrameId` has the following read-only attributes:

`gdb.unwinder.sp` [Variable]

The *sp* value passed to the constructor.

`gdb.unwinder.pc` [Variable]

The *pc* value passed to the constructor.

`gdb.unwinder.special` [Variable]

The *special* value passed to the constructor, or `None` if no such value was passed.

Registering an Unwinder

Object files and program spaces can have unwinders registered with them. In addition, you can register unwinders globally.

The `gdb.unwinders` module provides the function to register an unwinder:

`gdb.unwinder.register_unwinder (locus, unwinder, replace=False)` [Function]

locus specifies to which unwinder list to prepend the *unwinder*. It can be either an object file (see Section 23.3.2.27 [Objfiles In Python], page 487), a program space (see Section 23.3.2.26 [Progspace In Python], page 484), or `None`, in which case the unwinder is registered globally. The newly added *unwinder* will be called before any other unwinder from the same locus. Two unwinders in the same locus cannot have the same name. An attempt to add an unwinder with an already existing name raises an exception unless *replace* is `True`, in which case the old unwinder is deleted and the new unwinder is registered in its place.

GDB first calls the unwinders from all the object files in no particular order, then the unwinders from the current program space, then the globally registered unwinders, and finally the unwinders builtin to GDB.

Unwinder Skeleton Code

Here is an example of how to structure a user created unwinder:

```
from gdb.unwinder import Unwinder, FrameId

class MyUnwinder(Unwinder):
    def __init__(self):
        super().__init__("MyUnwinder_Name")

    def __call__(self, pending_frame):
        if not <we recognize frame>:
            return None

        # Create a FrameID. Usually the frame is identified by a
        # stack pointer and the function address.
        sp = ... compute a stack address ...
        pc = ... compute function address ...
        unwind_info = pending_frame.create_unwind_info(FrameId(sp, pc))

        # Find the values of the registers in the caller's frame and
        # save them in the result:
        unwind_info.add_saved_register(<register-number>, <register-value>)
        ....

        # Return the result:
        return unwind_info

gdb.unwinder.register_unwinder(<locus>, MyUnwinder(), <replace>)
```

Managing Registered Unwinders

GDB defines 3 commands to manage registered unwinders. These are:

`info unwinder [ locus [ name-regex ] ]`

Lists all registered unwinders. Arguments *locus* and *name-regex* are both optional and can be used to filter which unwinders are listed.

The *locus* argument should be either *global*, *progspace*, or the name of an object file. Only unwinders registered for the specified locus will be listed.

The *name-regex* is a regular expression used to match against unwinder names. When trying to match against unwinder names that include a string enclose *name-regex* in quotes.

```
disable unwinder [ locus [ name-regex ] ]
```

The *locus* and *name-regex* are interpreted as in *info unwinder* above, but instead of listing the matching unwinders, all of the matching unwinders are disabled. The **enabled** field of each matching unwinder is set to **False**.

```
enable unwinder [ locus [ name-regex ] ]
```

The *locus* and *name-regex* are interpreted as in *info unwinder* above, but instead of listing the matching unwinders, all of the matching unwinders are enabled. The **enabled** field of each matching unwinder is set to **True**.

23.3.2.14 Xmethods In Python

Xmethods are additional methods or replacements for existing methods of a C++ class. This feature is useful for those cases where a method defined in C++ source code could be inlined or optimized out by the compiler, making it unavailable to GDB. For such cases, one can define an xmethod to serve as a replacement for the method defined in the C++ source code. GDB will then invoke the xmethod, instead of the C++ method, to evaluate expressions. One can also use xmethods when debugging with core files. Moreover, when debugging live programs, invoking an xmethod need not involve running the inferior (which can potentially perturb its state). Hence, even if the C++ method is available, it is better to use its replacement xmethod if one is defined.

The xmethods feature in Python is available via the concepts of an *xmethod matcher* and an *xmethod worker*. To implement an xmethod, one has to implement a matcher and a corresponding worker for it (more than one worker can be implemented, each catering to a different overloaded instance of the method). Internally, GDB invokes the **match** method of a matcher to match the class type and method name. On a match, the **match** method returns a list of matching *worker* objects. Each worker object typically corresponds to an overloaded instance of the xmethod. They implement a **get_arg_types** method which returns a sequence of types corresponding to the arguments the xmethod requires. GDB uses this sequence of types to perform overload resolution and picks a winning xmethod worker. A winner is also selected from among the methods GDB finds in the C++ source code. Next, the winning xmethod worker and the winning C++ method are compared to select an overall winner. In case of a tie between a xmethod worker and a C++ method, the xmethod worker is selected as the winner. That is, if a winning xmethod worker is found to be equivalent to the winning C++ method, then the xmethod worker is treated as a replacement for the C++ method. GDB uses the overall winner to invoke the method. If the winning xmethod worker is the overall winner, then the corresponding xmethod is invoked via the **__call__** method of the worker object.

If one wants to implement an xmethod as a replacement for an existing C++ method, then they have to implement an equivalent xmethod which has exactly the same name and takes arguments of exactly the same type as the C++ method. If the user wants to invoke the C++ method even though a replacement xmethod is available for that method, then they can disable the xmethod.

See Section 23.3.2.15 [Xmethod API], page 451, for API to implement xmethods in Python. See Section 23.3.2.16 [Writing an Xmethod], page 452, for implementing xmethods in Python.

23.3.2.15 Xmethod API

The GDB Python API provides classes, interfaces and functions to implement, register and manipulate xmethods. See Section 23.3.2.14 [Xmethods In Python], page 450.

An xmethod matcher should be an instance of a class derived from **XMethodMatcher** defined in the module `gdb.xmethod`, or an object with similar interface and attributes. An instance of **XMethodMatcher** has the following attributes:

name [Variable]
The name of the matcher.

enabled [Variable]
A boolean value indicating whether the matcher is enabled or disabled.

methods [Variable]
A list of named methods managed by the matcher. Each object in the list is an instance of the class **XMethod** defined in the module `gdb.xmethod`, or any object with the following attributes:

name Name of the xmethod which should be unique for each xmethod managed by the matcher.

enabled A boolean value indicating whether the xmethod is enabled or disabled.

The class **XMethod** is a convenience class with same attributes as above along with the following constructor:

XMethod.__init__ (*self*, *name*) [Function]
Constructs an enabled xmethod with name *name*.

The **XMethodMatcher** class has the following methods:

XMethodMatcher.__init__ (*self*, *name*) [Function]
Constructs an enabled xmethod matcher with name *name*. The **methods** attribute is initialized to **None**.

XMethodMatcher.match (*self*, *class_type*, *method_name*) [Function]
Derived classes should override this method. It should return a xmethod worker object (or a sequence of xmethod worker objects) matching the *class_type* and *method_name*. *class_type* is a `gdb.Type` object, and *method_name* is a string value. If the matcher manages named methods as listed in its **methods** attribute, then only those worker objects whose corresponding entries in the **methods** list are enabled should be returned.

An xmethod worker should be an instance of a class derived from **XMethodWorker** defined in the module `gdb.xmethod`, or support the following interface:

XMethodWorker.get_arg_types (*self*) [Function]
This method returns a sequence of `gdb.Type` objects corresponding to the arguments that the xmethod takes. It can return an empty sequence or **None** if the xmethod does not take any arguments. If the xmethod takes a single argument, then a single `gdb.Type` object corresponding to it can be returned.

XMethodWorker.get_result_type (*self*, *args) [Function]

This method returns a `gdb.Type` object representing the type of the result of invoking this xmethod. The *args* argument is the same tuple of arguments that would be passed to the `__call__` method of this worker.

XMethodWorker.__call__ (*self*, *args) [Function]

This is the method which does the *work* of the xmethod. The *args* arguments is the tuple of arguments to the xmethod. Each element in this tuple is a `gdb.Value` object. The first element is always the **this** pointer value.

For GDB to lookup xmethods, the xmethod matchers should be registered using the following function defined in the module `gdb.xmethod`:

register_xmethod_matcher (*locus*, *matcher*, *replace=False*) [Function]

The *matcher* is registered with *locus*, replacing an existing matcher with the same name as *matcher* if *replace* is `True`. *locus* can be a `gdb.Objfile` object (see Section 23.3.2.27 [Objfiles In Python], page 487), or a `gdb.Progspace` object (see Section 23.3.2.26 [Progspace In Python], page 484), or `None`. If it is `None`, then *matcher* is registered globally.

23.3.2.16 Writing an Xmethod

Implementing xmethods in Python will require implementing xmethod matchers and xmethod workers (see Section 23.3.2.14 [Xmethods In Python], page 450). Consider the following C++ class:

```
class MyClass
{
public:
    MyClass (int a) : a_(a) { }

    int geta (void) { return a_; }
    int operator+ (int b);

private:
    int a_;
};

int
MyClass::operator+ (int b)
{
    return a_ + b;
}
```

Let us define two xmethods for the class `MyClass`, one replacing the method `geta`, and another adding an overloaded flavor of `operator+` which takes a `MyClass` argument (the C++ code above already has an overloaded `operator+` which takes an `int` argument). The xmethod matcher can be defined as follows:

```
class MyClass_geta(gdb.xmethod.XMethod):
    def __init__(self):
        gdb.xmethod.XMethod.__init__(self, 'geta')

    def get_worker(self, method_name):
        if method_name == 'geta':
            return MyClassWorker_geta()
```

```

class MyClass_sum(gdb.xmethod.XMethod):
    def __init__(self):
        gdb.xmethod.XMethod.__init__(self, 'sum')

    def get_worker(self, method_name):
        if method_name == 'operator+':
            return MyClassWorker_plus()

class MyClassMatcher(gdb.xmethod.XMethodMatcher):
    def __init__(self):
        gdb.xmethod.XMethodMatcher.__init__(self, 'MyClassMatcher')
        # List of methods 'managed' by this matcher
        self.methods = [MyClass_geta(), MyClass_sum()]

    def match(self, class_type, method_name):
        if class_type.tag != 'MyClass':
            return None
        workers = []
        for method in self.methods:
            if method.enabled:
                worker = method.get_worker(method_name)
                if worker:
                    workers.append(worker)

        return workers

```

Notice that the `match` method of `MyClassMatcher` returns a worker object of type `MyClassWorker_geta` for the `geta` method, and a worker object of type `MyClassWorker_plus` for the `operator+` method. This is done indirectly via helper classes derived from `gdb.xmethod.XMethod`. One does not need to use the `methods` attribute in a matcher as it is optional. However, if a matcher manages more than one xmethod, it is a good practice to list the xmethods in the `methods` attribute of the matcher. This will then facilitate enabling and disabling individual xmethods via the `enable/disable` commands. Notice also that a worker object is returned only if the corresponding entry in the `methods` attribute of the matcher is enabled.

The implementation of the worker classes returned by the matcher setup above is as follows:

```

class MyClassWorker_geta(gdb.xmethod.XMethodWorker):
    def get_arg_types(self):
        return None

    def get_result_type(self, obj):
        return gdb.lookup_type('int')

    def __call__(self, obj):
        return obj['a_']

class MyClassWorker_plus(gdb.xmethod.XMethodWorker):
    def get_arg_types(self):
        return gdb.lookup_type('MyClass')

    def get_result_type(self, obj):

```

```

        return gdb.lookup_type('int')

    def __call__(self, obj, other):
        return obj['a_'] + other['a_']

```

For GDB to actually lookup a xmethod, it has to be registered with it. The matcher defined above is registered with GDB globally as follows:

```
gdb.xmethod.register_xmethod_matcher(None, MyClassMatcher())
```

If an object `obj` of type `MyClass` is initialized in C++ code as follows:

```
MyClass obj(5);
```

then, after loading the Python script defining the xmethod matchers and workers into GDB, invoking the method `geta` or using the operator `+` on `obj` will invoke the xmethods defined above:

```
(gdb) p obj.geta()
$1 = 5
```

```
(gdb) p obj + obj
$2 = 10
```

Consider another example with a C++ template class:

```

template <class T>
class MyTemplate
{
public:
    MyTemplate () : dsize_(10), data_ (new T [10]) { }
    ~MyTemplate () { delete [] data_; }

    int footprint (void)
    {
        return sizeof (T) * dsize_ + sizeof (MyTemplate<T>);
    }

private:
    int dsize_;
    T *data_;
};

```

Let us implement an xmethod for the above class which serves as a replacement for the `footprint` method. The full code listing of the xmethod workers and xmethod matchers is as follows:

```

class MyTemplateWorker_footprint(gdb.xmethod.XMethodWorker):
    def __init__(self, class_type):
        self.class_type = class_type

    def get_arg_types(self):
        return None

    def get_result_type(self):
        return gdb.lookup_type('int')

    def __call__(self, obj):
        return (self.class_type.sizeof +
                obj['dsize_'] *
                self.class_type.template_argument(0).sizeof)

```

```

class MyTemplateMatcher_footprint(gdb.xmethod.XMethodMatcher):
    def __init__(self):
        gdb.xmethod.XMethodMatcher.__init__(self, 'MyTemplateMatcher')

    def match(self, class_type, method_name):
        if (re.match('MyTemplate<[ \t\n]*[_a-zA-Z][_a-zA-Z0-9]*>',
                    class_type.tag) and
            method_name == 'footprint'):
            return MyTemplateWorker_footprint(class_type)

```

Notice that, in this example, we have not used the `methods` attribute of the matcher as the matcher manages only one xmethod. The user can enable/disable this xmethod by enabling/disabling the matcher itself.

23.3.2.17 Inferiors In Python

Programs which are being run under GDB are called inferiors (see Section 4.9 [Inferiors Connections and Programs], page 43). Python scripts can access information about and manipulate inferiors controlled by GDB via objects of the `gdb.Inferior` class.

The following inferior-related functions are available in the `gdb` module:

`gdb.inferiors ()` [Function]
Return a tuple containing all inferior objects.

`gdb.selected_inferior ()` [Function]
Return an object representing the current inferior.

A `gdb.Inferior` object has the following attributes:

`Inferior.num` [Variable]
ID of inferior, as assigned by GDB. You can use this to make Python breakpoints inferior-specific, for example (see [The Breakpoint.inferior attribute], page 505).

`Inferior.connection` [Variable]
The `gdb.TargetConnection` for this inferior (see Section 23.3.2.40 [Connections In Python], page 514), or `None` if this inferior has no connection.

`Inferior.connection_num` [Variable]
ID of inferior's connection as assigned by GDB, or `None` if the inferior is not connected to a target. See Section 4.9 [Inferiors Connections and Programs], page 43. This is equivalent to `gdb.Inferior.connection.num` in the case where `gdb.Inferior.connection` is not `None`.

`Inferior.pid` [Variable]
Process ID of the inferior, as assigned by the underlying operating system.

`Inferior.was_attached` [Variable]
Boolean signaling whether the inferior was created using 'attach', or started by GDB itself.

`Inferior.main_name` [Variable]
A string holding the name of this inferior's "main" function, if it can be determined. If the name of main is not known, this is `None`.

Inferior.progspace [Variable]

The inferior's program space. See Section 23.3.2.26 [Progspace In Python], page 484.

Inferior.arguments [Variable]

The inferior's command line arguments, if known. This corresponds to the **set args** and **show args** commands. See Section 4.3 [Arguments], page 38.

When accessed, the value is a string holding all the arguments. The contents are quoted as they would be when passed to the shell. If there are no arguments, the value is **None**.

Either a string or a sequence of strings can be assigned to this attribute. When a string is assigned, it is assumed to have any necessary quoting for the shell; when a sequence is assigned, the quoting is applied by GDB.

Inferior.corefile [Variable]

If a core file has been loaded into this inferior (see [core-file command], page 294), then this contains a **gdb.Corefile** object that represents the loaded core file (see Section 23.3.2.45 [Core Files In Python], page 531).

If no core file has been loaded into this inferior, then this attribute contains **None**.

A **gdb.Inferior** object has the following methods:

Inferior.is_valid () [Function]

Returns **True** if the **gdb.Inferior** object is valid, **False** if not. A **gdb.Inferior** object will become invalid if the inferior no longer exists within GDB. All other **gdb.Inferior** methods will throw an exception if it is invalid at the time the method is called.

Inferior.threads () [Function]

This method returns a tuple holding all the threads which are valid when it is called. If there are no valid threads, the method will return an empty tuple.

Inferior.architecture () [Function]

Return the **gdb.Architecture** (see Section 23.3.2.38 [Architectures In Python], page 512) for this inferior. This represents the architecture of the inferior as a whole. Some platforms can have multiple architectures in a single address space, so this may not match the architecture of a particular frame (see Section 23.3.2.28 [Frames In Python], page 490).

Inferior.read_memory (address, length) [Function]

Read *length* addressable memory units from the inferior, starting at *address*. Returns a **memoryview** object, which behaves much like an array or a string. It can be modified and given to the **Inferior.write_memory** function.

Inferior.write_memory (address, buffer [, length]) [Function]

Write the contents of *buffer* to the inferior, starting at *address*. The *buffer* parameter must be a Python object which supports the buffer protocol, i.e., a string, an array or the object returned from **Inferior.read_memory**. If given, *length* determines the number of addressable memory units from *buffer* to be written.

Inferior.search_memory (*address*, *length*, *pattern*) [Function]

Search a region of the inferior memory starting at *address* with the given *length* using the search pattern supplied in *pattern*. The *pattern* parameter must be a Python object which supports the buffer protocol, i.e., a string, an array or the object returned from `gdb.read_memory`. Returns a Python Long containing the address where the pattern was found, or None if the pattern could not be found.

Inferior.thread_from_handle (*handle*) [Function]

Return the thread object corresponding to *handle*, a thread library specific data structure such as `pthread_t` for pthreads library implementations.

The function `Inferior.thread_from_thread_handle` provides the same functionality, but use of `Inferior.thread_from_thread_handle` is deprecated.

The environment that will be passed to the inferior can be changed from Python by using the following methods. These methods only take effect when the inferior is started – they will not affect an inferior that is already executing.

Inferior.clear_env () [Function]

Clear the current environment variables that will be passed to this inferior.

Inferior.set_env (*name*, *value*) [Function]

Set the environment variable *name* to have the indicated value. Both parameters must be strings.

Inferior.unset_env (*name*) [Function]

Unset the environment variable *name*. *name* must be a string.

One may add arbitrary attributes to `gdb.Inferior` objects in the usual Python way. This is useful if, for example, one needs to do some extra record keeping associated with the inferior.

When selecting a name for a new attribute, avoid starting the new attribute name with a lower case letter; future attributes added by GDB will start with a lower case letter. Additionally, avoid starting attribute names with two underscore characters, as these could clash with Python builtin attribute names.

In this contrived example we record the time when an inferior last stopped:

```
(gdb) python
import datetime

def thread_stopped(event):
    if event.inferior_thread is not None:
        thread = event.inferior_thread
    else:
        thread = gdb.selected_thread()
    inferior = thread.inferior
    inferior._last_stop_time = datetime.datetime.today()

gdb.events.stop.connect(thread_stopped)
```

```
(gdb) file /tmp/hello
Reading symbols from /tmp/hello...
(gdb) start
Temporary breakpoint 1 at 0x401198: file /tmp/hello.c, line 18.
Starting program: /tmp/hello

Temporary breakpoint 1, main () at /tmp/hello.c:18
18  printf ("Hello World\n");
(gdb) python print(gdb.selected_inferior()._last_stop_time)
2024-01-04 14:48:41.347036
```

23.3.2.18 Events In Python

GDB provides a general event facility so that Python code can be notified of various state changes, particularly changes that occur in the inferior.

An *event* is just an object that describes some state change. The type of the object and its attributes will vary depending on the details of the change. All the existing events are described below.

In order to be notified of an event, you must register an event handler with an *event registry*. An event registry is an object in the `gdb.events` module which dispatches particular events. A registry provides methods to register and unregister event handlers:

EventRegistry.connect (*object*) [Function]

Add the given callable *object* to the registry. This object will be called when an event corresponding to this registry occurs.

EventRegistry.disconnect (*object*) [Function]

Remove the given *object* from the registry. Once removed, the object will no longer receive notifications of events.

Here is an example:

```
def exit_handler (event):
    print ("event type: exit")
    if hasattr (event, 'exit_code'):
        print ("exit code: %d" % (event.exit_code))
    else:
        print ("exit code not available")

gdb.events.exited.connect (exit_handler)
```

In the above example we connect our handler `exit_handler` to the registry `events.exited`. Once connected, `exit_handler` gets called when the inferior exits. The argument *event* in this example is of type `gdb.ExitedEvent`. As you can see in the example the `ExitedEvent` object has an attribute which indicates the exit code of the inferior.

Some events can be thread specific when GDB is running in non-stop mode. When represented in Python, these events all extend `gdb.ThreadEvent`. This event is a base class and is never emitted directly; instead, events which are emitted by this or other modules might extend this event. Examples of these events are `gdb.BreakpointEvent` and `gdb.ContinueEvent`. `gdb.ThreadEvent` holds the following attributes:

ThreadEvent.inferior_thread [Variable]

In non-stop mode this attribute will be set to the specific thread which was involved in the emitted event. Otherwise, it will be set to `None`.

The following is a listing of the event registries that are available and details of the events they emit:

events.cont

Emits `gdb.ContinueEvent`, which extends `gdb.ThreadEvent`. This event indicates that the inferior has been continued after a stop. For inherited attribute refer to `gdb.ThreadEvent` above.

events.exited

Emits `events.ExitedEvent`, which indicates that the inferior has exited. `events.ExitedEvent` has two attributes:

ExitedEvent.exit_code [Variable]

An integer representing the exit code, if available, which the inferior has returned. (The exit code could be unavailable if, for example, GDB detaches from the inferior.) If the exit code is unavailable, the attribute does not exist.

ExitedEvent.inferior [Variable]

A reference to the inferior which triggered the `exited` event.

events.stop

Emits `gdb.StopEvent`, which extends `gdb.ThreadEvent`.

Indicates that the inferior has stopped. All events emitted by this registry extend `gdb.StopEvent`. As a child of `gdb.ThreadEvent`, `gdb.StopEvent` will indicate the stopped thread when GDB is running in non-stop mode. Refer to `gdb.ThreadEvent` above for more details.

`gdb.StopEvent` has the following additional attributes:

StopEvent.details [Variable]

A dictionary holding any details relevant to the stop. The exact keys and values depend on the type of stop, but are identical to the corresponding MI output (see Section 27.7.3 [GDB/MI Async Records], page 618).

A dictionary was used for this (rather than adding attributes directly to the event object) so that the MI keys could be used unchanged.

When a `StopEvent` results from a `finish` command, it will also hold the return value from the function, if that is available. This will be an entry named `'return-value'` in the `details` dictionary. The value of this entry will be a `gdb.Value` object.

Emits `gdb.SignalEvent`, which extends `gdb.StopEvent`.

This event indicates that the inferior or one of its threads has received a signal. `gdb.SignalEvent` has the following attributes:

SignalEvent.stop_signal [Variable]

A string representing the signal received by the inferior. A list of possible signal values can be obtained by running the command `info signals` in the GDB command prompt.

Also emits `gdb.BreakpointEvent`, which extends `gdb.StopEvent`.

`gdb.BreakpointEvent` event indicates that one or more breakpoints have been hit, and has the following attributes:

BreakpointEvent.breakpoints [Variable]

A sequence containing references to all the breakpoints (type `gdb.Breakpoint`) that were hit. See Section 23.3.2.33 [Breakpoints In Python], page 503, for details of the `gdb.Breakpoint` object.

BreakpointEvent.breakpoint [Variable]

A reference to the first breakpoint that was hit. This attribute is maintained for backward compatibility and is now deprecated in favor of the `gdb.BreakpointEvent.breakpoints` attribute.

events.new_objfile

Emits `gdb.NewObjFileEvent` which indicates that a new object file has been loaded by GDB. `gdb.NewObjFileEvent` has one attribute:

NewObjFileEvent.new_objfile [Variable]

A reference to the object file (`gdb.Objfile`) which has been loaded. See Section 23.3.2.27 [Objfiles In Python], page 487, for details of the `gdb.Objfile` object.

events.free_objfile

Emits `gdb.FreeObjFileEvent` which indicates that an object file is about to be removed from GDB. One reason this can happen is when the inferior calls `dlclose`. `gdb.FreeObjFileEvent` has one attribute:

FreeObjFileEvent.objfile [Variable]

A reference to the object file (`gdb.Objfile`) which will be unloaded. See Section 23.3.2.27 [Objfiles In Python], page 487, for details of the `gdb.Objfile` object.

events.clear_objfiles

Emits `gdb.ClearObjFilesEvent` which indicates that the list of object files for a program space has been reset. `gdb.ClearObjFilesEvent` has one attribute:

ClearObjFilesEvent.progspace [Variable]

A reference to the program space (`gdb.Progspace`) whose objfile list has been cleared. See Section 23.3.2.26 [Progspace In Python], page 484.

events.inferior_call

Emits events just before and after a function in the inferior is called by GDB. Before an inferior call, this emits an event of type

`gdb.InferiorCallPreEvent`, and after an inferior call, this emits an event of type `gdb.InferiorCallPostEvent`.

`gdb.InferiorCallPreEvent`

Indicates that a function in the inferior is about to be called.

`InferiorCallPreEvent.ptid` [Variable]

The thread in which the call will be run.

`InferiorCallPreEvent.address` [Variable]

The location of the function to be called.

`gdb.InferiorCallPostEvent`

Indicates that a function in the inferior has just been called.

`InferiorCallPostEvent.ptid` [Variable]

The thread in which the call was run.

`InferiorCallPostEvent.address` [Variable]

The location of the function that was called.

`events.memory_changed`

Emits `gdb.MemoryChangedEvent` which indicates that the memory of the inferior has been modified by the GDB user, for instance via a command like `set *addr = value`. The event has the following attributes:

`MemoryChangedEvent.address` [Variable]

The start address of the changed region.

`MemoryChangedEvent.length` [Variable]

Length in bytes of the changed region.

`events.register_changed`

Emits `gdb.RegisterChangedEvent` which indicates that a register in the inferior has been modified by the GDB user.

`RegisterChangedEvent.frame` [Variable]

A `gdb.Frame` object representing the frame in which the register was modified.

`RegisterChangedEvent.regnum` [Variable]

Denotes which register was modified.

`events.breakpoint_created`

This is emitted when a new breakpoint has been created. The argument that is passed is the new `gdb.Breakpoint` object.

`events.breakpoint_modified`

This is emitted when a breakpoint has been modified in some way. The argument that is passed is the new `gdb.Breakpoint` object.

events.breakpoint_deleted

This is emitted when a breakpoint has been deleted. The argument that is passed is the `gdb.Breakpoint` object. When this event is emitted, the `gdb.Breakpoint` object will already be in its invalid state; that is, the `is_valid` method will return `False`.

events.before_prompt

This event carries no payload. It is emitted each time GDB presents a prompt to the user.

events.new_inferior

This is emitted when a new inferior is created. Note that the inferior is not necessarily running; in fact, it may not even have an associated executable.

The event is of type `gdb.NewInferiorEvent`. This has a single attribute:

NewInferiorEvent.inferior [Variable]
The new inferior, a `gdb.Inferior` object.

events.inferior_deleted

This is emitted when an inferior has been deleted. Note that this is not the same as process exit; it is notified when the inferior itself is removed, say via `remove-inferiors`.

The event is of type `gdb.InferiorDeletedEvent`. This has a single attribute:

InferiorDeletedEvent.inferior [Variable]
The inferior that is being removed, a `gdb.Inferior` object.

events.new_thread

This is emitted when GDB notices a new thread. The event is of type `gdb.NewThreadEvent`, which extends `gdb.ThreadEvent`. This has a single attribute:

NewThreadEvent.inferior_thread [Variable]
The new thread.

events.thread_exited

This is emitted when GDB notices a thread has exited. The event is of type `gdb.ThreadExitedEvent` which extends `gdb.ThreadEvent`. This has a single attribute:

ThreadExitedEvent.inferior_thread [Variable]
The exiting thread.

events.selected_context

This is emitted when the user directly, or indirectly, causes the selected inferior context to change. The context consists of the currently selected inferior, thread, and frame. Examples commands that trigger this event are *inferior*, *thread*, and *frame*.

The event is of type `gdb.SelectedContextEvent` and has the following attributes:

SelectedContextEvent.inferior [Variable]
 The currently selected inferior. This is of type `gdb.Inferior` (see Section 23.3.2.17 [Inferiors In Python], page 455).

SelectedContextEvent.thread [Variable]
 The currently selected thread. If not `None` then this is of type `gdb.InferiorThread` (see Section 23.3.2.19 [Threads In Python], page 465). If switching to an inferior that is not yet started, then the `thread` attribute will be `None`.

SelectedContextEvent.frame [Variable]
 The currently selected frame. If not `None` then this is of type `gdb.Frame` (see Section 23.3.2.28 [Frames In Python], page 490). If switching to an inferior that is not yet started, then the `frame` attribute will be `None`.

In some cases GDB might emit the `selected_context` event even when the context has not changed. The state within the event will always reflect the state of the current inferior. These unnecessary events could be removed in future releases of GDB.

`events.gdb_exiting`

This is emitted when GDB exits. This event is not emitted if GDB exits as a result of an internal error, or after an unexpected signal. The event is of type `gdb.GdbExitingEvent`, which has a single attribute:

GdbExitingEvent.exit_code [Variable]
 An integer, the value of the exit code GDB will return.

`events.connection_removed`

This is emitted when GDB removes a connection (see Section 23.3.2.40 [Connections In Python], page 514). The event is of type `gdb.ConnectionEvent`. This has a single read-only attribute:

ConnectionEvent.connection [Variable]
 The `gdb.TargetConnection` that is being removed.

`events.executable_changed`

Emits `gdb.ExecutableChangedEvent` which indicates that the `gdb.Progspace.executable_filename` has changed.

This event is emitted when either the value of `gdb.Progspace.executable_filename` has changed to name a different file, or the executable file named by `gdb.Progspace.executable_filename` has changed on disk, and GDB has therefore reloaded it.

ExecutableChangedEvent.progspace [Variable]
 The `gdb.Progspace` in which the current executable has changed. The file name of the updated executable will be visible in `gdb.Progspace.executable_filename` (see Section 23.3.2.26 [Progspace In Python], page 484).

`ExecutableChangedEvent.reload` [Variable]

This attribute will be `True` if the value of `gdb.Progspace.executable_filename` didn't change, but the file it names changed on disk instead, and GDB reloaded it.

When this attribute is `False`, the value in `gdb.Progspace.executable_filename` was changed to name a different file.

Remember that GDB tracks the executable file and the symbol file separately, these are visible as `gdb.Progspace.executable_filename` and `gdb.Progspace.filename` respectively. When using the `file` command, GDB updates both of these fields, but the executable file is updated first, so when this event is emitted, the executable filename will have changed, but the symbol filename might still hold its previous value.

`events.corefile_changed`

Emits `gdb.CorefileChangedEvent` which indicates that the core file associated with a `gdb.Inferior` has changed, either a new core file has been loaded, or the existing core file has been unloaded (see Section 23.3.2.45 [Core Files In Python], page 531).

`CorefileChangedEvent.inferior` [Variable]

The `gdb.Inferior` in which the core file has changed (see Section 23.3.2.17 [Inferiors In Python], page 455).

`events.new_progspace`

This is emitted when GDB adds a new program space (see Section 23.3.2.26 [Program Spaces In Python], page 484). The event is of type `gdb.NewProgspaceEvent`, and has a single read-only attribute:

`NewProgspaceEvent.progspace` [Variable]

The `gdb.Progspace` that was added to GDB.

No `NewProgspaceEvent` is emitted for the very first program space, which is assigned to the first inferior. This first program space is created within GDB before any Python scripts are sourced.

`events.free_progspace`

This is emitted when GDB removes a program space (see Section 23.3.2.26 [Program Spaces In Python], page 484), for example as a result of the `remove-inferiors` command (see [remove-inferiors], page 46). The event is of type `gdb.FreeProgspaceEvent`, and has a single read-only attribute:

`FreeProgspaceEvent.progspace` [Variable]

The `gdb.Progspace` that is about to be removed from GDB.

`events.tui_enabled`

This is emitted when the TUI is enabled or disabled. The event is of type `gdb.TuiEnabledEvent`, which has a single read-only attribute:

`TuiEnabledEvent.enabled` [Variable]

If the TUI has just been enabled, this is `True`; otherwise it is `False`.

23.3.2.19 Threads In Python

Python scripts can access information about, and manipulate inferior threads controlled by GDB, via objects of the `gdb.InferiorThread` class.

The following thread-related functions are available in the `gdb` module:

`gdb.selected_thread()` [Function]
 This function returns the thread object for the selected thread. If there is no selected thread, this will return `None`.

To get the list of threads for an inferior, use the `Inferior.threads()` method. See Section 23.3.2.17 [Inferiors In Python], page 455.

A `gdb.InferiorThread` object has the following attributes:

`InferiorThread.name` [Variable]
 The name of the thread. If the user specified a name using `thread name`, then this returns that name. Otherwise, if an OS-supplied name is available, then it is returned. Otherwise, this returns `None`.

This attribute can be assigned to. The new value must be a string object, which sets the new name, or `None`, which removes any user-specified thread name.

`InferiorThread.num` [Variable]
 The per-inferior number of the thread, as assigned by GDB.

`InferiorThread.global_num` [Variable]
 The global ID of the thread, as assigned by GDB. You can use this to make Python breakpoints thread-specific, for example (see [The Breakpoint.thread attribute], page 505).

`InferiorThread.ptid` [Variable]
 ID of the thread, as assigned by the operating system. This attribute is a tuple containing three integers. The first is the Process ID (PID); the second is the Lightweight Process ID (LWPID), and the third is the Thread ID (TID). Either the LWPID or TID may be 0, which indicates that the operating system does not use that identifier.

`InferiorThread.ptid_string` [Variable]
 This read-only attribute contains a string representing `InferiorThread.ptid`. This is the string that GDB uses in the ‘Target Id’ column in the *info threads* output (see [‘info threads’], page 50).

`InferiorThread.inferior` [Variable]
 The inferior this thread belongs to. This attribute is represented as a `gdb.Inferior` object. This attribute is not writable.

`InferiorThread.details` [Variable]
 A string containing target specific thread state information. The format of this string varies by target. If there is no additional state information for this thread, then this attribute contains `None`.

For example, on a GNU/Linux system, a thread that is in the process of exiting will return the string ‘Exiting’. For remote targets the `details` string will be obtained with the ‘`qThreadExtraInfo`’ remote packet, if the target supports it (see [‘`qThreadExtraInfo`’], page 832).

GDB displays the `details` string as part of the ‘Target Id’ column, in the `info threads` output (see [‘`info threads`’], page 50).

A `gdb.InferiorThread` object has the following methods:

<code>InferiorThread.is_valid ()</code>	[Function]
Returns <code>True</code> if the <code>gdb.InferiorThread</code> object is valid, <code>False</code> if not. A <code>gdb.InferiorThread</code> object will become invalid if the thread exits, or the inferior that the thread belongs is deleted. All other <code>gdb.InferiorThread</code> methods will throw an exception if it is invalid at the time the method is called.	
<code>InferiorThread.switch ()</code>	[Function]
This changes GDB’s currently selected thread to the one represented by this object.	
<code>InferiorThread.is_stopped ()</code>	[Function]
Return a Boolean indicating whether the thread is stopped.	
<code>InferiorThread.is_running ()</code>	[Function]
Return a Boolean indicating whether the thread is running.	
<code>InferiorThread.is_exited ()</code>	[Function]
Return a Boolean indicating whether the thread is exited.	
<code>InferiorThread.handle ()</code>	[Function]
Return the thread object’s handle, represented as a Python <code>bytes</code> object. A <code>gdb.Value</code> representation of the handle may be constructed via <code>gdb.Value(bufobj, type)</code> where <code>bufobj</code> is the Python <code>bytes</code> representation of the handle and <code>type</code> is a <code>gdb.Type</code> for the handle type.	

One may add arbitrary attributes to `gdb.InferiorThread` objects in the usual Python way. This is useful if, for example, one needs to do some extra record keeping associated with the thread.

See [choosing attribute names], page 457, for guidance on selecting a suitable name for new attributes.

In this contrived example we record the time when a thread last stopped:

```
(gdb) python
import datetime

def thread_stopped(event):
    if event.inferior_thread is not None:
        thread = event.inferior_thread
    else:
        thread = gdb.selected_thread()
    thread._last_stop_time = datetime.datetime.today()

gdb.events.stop.connect(thread_stopped)
```

```
(gdb) file /tmp/hello
Reading symbols from /tmp/hello...
(gdb) start
Temporary breakpoint 1 at 0x401198: file /tmp/hello.c, line 18.
Starting program: /tmp/hello

Temporary breakpoint 1, main () at /tmp/hello.c:18
18  printf ("Hello World\n");
(gdb) python print(gdb.selected_thread()._last_stop_time)
2024-01-04 14:48:41.347036
```

23.3.2.20 Recordings In Python

The following recordings-related functions (see Chapter 7 [Process Record and Replay], page 105) are available in the `gdb` module:

`gdb.start_recording ([method], [format])` [Function]
 Start a recording using the given *method* and *format*. If no *format* is given, the default format for the recording method is used. If no *method* is given, the default method will be used. Returns a `gdb.Record` object on success. Throw an exception on failure.

The following strings can be passed as *method*:

- "full"
- "btrace": Possible values for *format*: "pt", "bts" or leave out for default format.

`gdb.current_recording ()` [Function]
 Access a currently running recording. Return a `gdb.Record` object on success. Return `None` if no recording is currently active.

`gdb.stop_recording ()` [Function]
 Stop the current recording. Throw an exception if no recording is currently active. All record objects become invalid after this call.

A `gdb.Record` object has the following attributes:

`Record.method` [Variable]
 A string with the current recording method, e.g. `full` or `btrace`.

`Record.format` [Variable]
 A string with the current recording format, e.g. `bt`, `pts` or `None`.

`Record.begin` [Variable]
 A method specific instruction object representing the first instruction in this recording.

`Record.end` [Variable]
 A method specific instruction object representing the current instruction, that is not actually part of the recording.

`Record.replay_position` [Variable]
 The instruction representing the current replay position. If there is no replay active, this will be `None`.

Record.instruction_history [Variable]
A list with all recorded instructions.

Record.function_call_history [Variable]
A list with all recorded function call segments.

A `gdb.Record` object has the following methods:

Record.goto (*instruction*) [Function]
Move the replay position to the given *instruction*.

Record.clear () [Function]
Clear the trace data of the current recording. This forces re-decoding of the trace for successive commands.

The common `gdb.Instruction` class that recording method specific instruction objects inherit from, has the following attributes:

Instruction.pc [Variable]
An integer representing this instruction's address.

Instruction.data [Variable]
A `memoryview` object holding the raw instruction data.

Instruction.decoded [Variable]
A human readable string with the disassembled instruction.

Instruction.size [Variable]
The size of the instruction in bytes.

Additionally `gdb.RecordInstruction` has the following attributes:

RecordInstruction.number [Variable]
An integer identifying this instruction. `number` corresponds to the numbers seen in `record instruction-history` (see Chapter 7 [Process Record and Replay], page 105).

RecordInstruction.sal [Variable]
A `gdb.Symtab_and_line` object representing the associated symtab and line of this instruction. May be `None` if no debug information is available.

RecordInstruction.is_speculative [Variable]
A boolean indicating whether the instruction was executed speculatively.

If an error occurred during recording or decoding a recording, this error is represented by a `gdb.RecordGap` object in the instruction list. It has the following attributes:

RecordGap.number [Variable]
An integer identifying this gap. `number` corresponds to the numbers seen in `record instruction-history` (see Chapter 7 [Process Record and Replay], page 105).

RecordGap.error_code [Variable]
 A numerical representation of the reason for the gap. The value is specific to the current recording method.

RecordGap.error_string [Variable]
 A human readable string with the reason for the gap.

Some GDB features write auxiliary information into the execution history. This information is represented by a `gdb.RecordAuxiliary` object in the instruction list. It has the following attributes:

RecordAuxiliary.number [Variable]
 An integer identifying this auxiliary. *number* corresponds to the numbers seen in `record instruction-history` (see Chapter 7 [Process Record and Replay], page 105).

RecordAuxiliary.data [Variable]
 A string representation of the auxiliary data.

A `gdb.RecordFunctionSegment` object has the following attributes:

RecordFunctionSegment.number [Variable]
 An integer identifying this function segment. *number* corresponds to the numbers seen in `record function-call-history` (see Chapter 7 [Process Record and Replay], page 105).

RecordFunctionSegment.symbol [Variable]
 A `gdb.Symbol` object representing the associated symbol. May be `None` if no debug information is available.

RecordFunctionSegment.level [Variable]
 An integer representing the function call's stack level. May be `None` if the function call is a gap.

RecordFunctionSegment.instructions [Variable]
 A list of `gdb.RecordInstruction` or `gdb.RecordGap` objects associated with this function call.

RecordFunctionSegment.up [Variable]
 A `gdb.RecordFunctionSegment` object representing the caller's function segment. If the call has not been recorded, this will be the function segment to which control returns. If neither the call nor the return have been recorded, this will be `None`.

RecordFunctionSegment.prev [Variable]
 A `gdb.RecordFunctionSegment` object representing the previous segment of this function call. May be `None`.

RecordFunctionSegment.next [Variable]
 A `gdb.RecordFunctionSegment` object representing the next segment of this function call. May be `None`.

The following example demonstrates the usage of these objects and functions to create a function that will rewind a record to the last time a function in a different file was executed. This would typically be used to track the execution of user provided callback functions in a library which typically are not visible in a back trace.

```
def bringback ():
    rec = gdb.current_recording ()
    if not rec:
        return

    insn = rec.instruction_history
    if len (insn) == 0:
        return

    try:
        position = insn.index (rec.replay_position)
    except:
        position = -1
    try:
        filename = insn[position].sal.symtab.fullname ()
    except:
        filename = None

    for i in reversed (insn[:position]):
        try:
            current = i.sal.symtab.fullname ()
        except:
            current = None

        if filename == current:
            continue

        rec.goto (i)
    return
```

Another possible application is to write a function that counts the number of code executions in a given line range. This line range can contain parts of functions or span across several functions and is not limited to be contiguous.

```
def countrange (filename, linerange):
    count = 0

    def filter_only (file_name):
        for call in gdb.current_recording ().function_call_history:
            try:
                if file_name in call.symbol.symtab.fullname ():
                    yield call
            except:
                pass

    for c in filter_only (filename):
        for i in c.instructions:
            try:
                if i.sal.line in linerange:
                    count += 1
                    break;
            except:
                pass
```

```
return count
```

23.3.2.21 CLI Commands In Python

You can implement new GDB CLI commands in Python. A CLI command is implemented using an instance of the `gdb.Command` class, most commonly using a subclass.

Command.__init__ (*name*, *command_class* [, *completer_class* [, [Function]
prefix]])

The object initializer for `Command` registers the new command with GDB. This initializer is normally invoked from the subclass' own `__init__` method.

name is the name of the command. If *name* consists of multiple words, then the initial words are looked for as prefix commands. In this case, if one of the prefix commands does not exist, an exception is raised.

There is no support for multi-line commands.

command_class should be one of the 'COMMAND_' constants defined below. This argument tells GDB how to categorize the new command in the help system.

completer_class is an optional argument. If given, it should be one of the 'COMPLETE_' constants defined below. This argument tells GDB how to perform completion for this command. If not given, GDB will attempt to complete using the object's `complete` method (see below); if no such method is found, an error will occur when completion is attempted.

prefix is an optional argument. If `True`, then the new command is a prefix command; sub-commands of this command may be registered.

The help text for the new command is taken from the Python documentation string for the command's class, if there is one. If no documentation string is provided, the default value 'This command is not documented.' is used.

Command.dont_repeat () [Function]

By default, a GDB command is repeated when the user enters a blank line at the command prompt. A command can suppress this behavior by invoking the `dont_repeat` method at some point in its `invoke` method (normally this is done early in case of exception). This is similar to the user command `dont-repeat`, see Section 23.1.1 [Define], page 397.

Command.invoke (*argument*, *from_tty*) [Function]

This method is called by GDB when this command is invoked.

argument is a string. It is the argument to the command, after leading and trailing whitespace has been stripped.

from_tty is a boolean argument. When true, this means that the command was entered by the user at the terminal; when false it means that the command came from elsewhere.

If this method throws an exception, it is turned into a GDB `error` call. Otherwise, the return value is ignored.

For non-prefix commands (see [Command.__init__], page 471), the `invoke` method is required. For prefix commands the `invoke` method is optional. Only prefix commands that need to handle unknown sub-commands should implement the `invoke` method.

For prefix commands that don't implement `invoke`, if the prefix command is used without a sub-command name then GDB will display the help text for every sub-command, unless the prefix command is a `show` sub-command, in which case GDB will list the values of all sub-commands.

To break *argument* up into an argv-like string use `gdb.string_to_argv`. This function behaves identically to GDB's internal argument lexer `buildargv`. It is recommended to use this for consistency. Arguments are separated by spaces and may be quoted. Example:

```
print(gdb.string_to_argv ("1 2\ \\\"3 '4 \"5' \"6 '7\""))
['1', '2 "3', '4 "5', "6 '7"]
```

Command.complete (*text*, *word*) [Function]

This method is called by GDB when the user attempts completion on this command. All forms of completion are handled by this method, that is, the TAB and M-? key bindings (see Section 3.3 [Completion], page 24), and the `complete` command (see Section 3.6 [Help], page 29).

The arguments *text* and *word* are both strings; *text* holds the complete command line up to the cursor's location, while *word* holds the last word of the command line; this is computed using a word-breaking heuristic.

The `complete` method can return several values:

- If the return value is a sequence, the contents of the sequence are used as the completions. It is up to `complete` to ensure that the contents actually do complete the word. A zero-length sequence is allowed, it means that there were no completions available. Only string elements of the sequence are used; other elements in the sequence are ignored.
- If the return value is one of the 'COMPLETE_' constants defined below, then the corresponding GDB-internal completion function is invoked, and its result is used.
- All other results are treated as though there were no available completions.

When a new command is registered, it must be declared as a member of some general class of commands. This is used to classify top-level commands in the on-line help system; note that prefix commands are not listed under their own category but rather that of their top-level command. The available classifications are represented by constants defined in the `gdb` module:

`gdb.COMMAND_NONE`

The command does not belong to any particular class. A command in this category will not be displayed in any of the help categories.

`gdb.COMMAND_RUNNING`

The command is related to running the inferior. For example, `start`, `step`, and `continue` are in this category. Type *help running* at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_DATA`

The command is related to data or variables. For example, `call`, `find`, and `print` are in this category. Type *help data* at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_STACK`

The command has to do with manipulation of the stack. For example, **`backtrace`**, **`frame`**, and **`return`** are in this category. Type **`help stack`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_FILES`

This class is used for file-related commands. For example, **`file`**, **`list`** and **`section`** are in this category. Type **`help files`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_SUPPORT`

This should be used for “support facilities”, generally meaning things that are useful to the user when interacting with GDB, but not related to the state of the inferior. For example, **`help`**, **`make`**, and **`shell`** are in this category. Type **`help support`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_STATUS`

The command is an ‘**`info`**’-related command, that is, related to the state of GDB itself. For example, **`info`**, **`macro`**, and **`show`** are in this category. Type **`help status`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_BREAKPOINTS`

The command has to do with breakpoints. For example, **`break`**, **`clear`**, and **`delete`** are in this category. Type **`help breakpoints`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_TRACEPOINTS`

The command has to do with tracepoints. For example, **`trace`**, **`actions`**, and **`tfind`** are in this category. Type **`help tracepoints`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_TUI`

The command has to do with the text user interface (see Chapter 25 [TUI], page 599). Type **`help tui`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_USER`

The command is a general purpose command for the user, and typically does not fit in one of the other categories. Type **`help user-defined`** at the GDB prompt to see a list of commands in this category, as well as the list of gdb macros (see Section 23.1 [Sequences], page 397).

`gdb.COMMAND_OBSCURE`

The command is only used in unusual circumstances, or is not of general interest to users. For example, **`checkpoint`**, **`fork`**, and **`stop`** are in this category. Type **`help obscure`** at the GDB prompt to see a list of commands in this category.

`gdb.COMMAND_MAINTENANCE`

The command is only useful to GDB maintainers. The **`maintenance`** and **`flushregs`** commands are in this category. Type **`help internals`** at the GDB prompt to see a list of commands in this category.

A new command can use a predefined completion function, either by specifying it via an argument at initialization, or by returning it from the `complete` method. These predefined completion constants are all defined in the `gdb` module:

`gdb.COMPLETE_NONE`

This constant means that no completion should be done.

`gdb.COMPLETE_FILENAME`

This constant means that filename completion should be performed.

`gdb.COMPLETE_LOCATION`

This constant means that location completion should be done. See Section 9.2 [Location Specifications], page 128.

`gdb.COMPLETE_COMMAND`

This constant means that completion should examine GDB command names.

`gdb.COMPLETE_SYMBOL`

This constant means that completion should be done using symbol names as the source.

`gdb.COMPLETE_EXPRESSION`

This constant means that completion should be done on expressions. Often this means completing on symbol names, but some language parsers also have support for completing on field names.

The following code snippet shows how a trivial CLI command can be implemented in Python:

```
class HelloWorld (gdb.Command):
    """Greet the whole world."""

    def __init__ (self):
        super (HelloWorld, self).__init__ ("hello-world", gdb.COMMAND_USER)

    def invoke (self, arg, from_tty):
        print ("Hello, World!")

HelloWorld ()
```

The last line instantiates the class, and is necessary to trigger the registration of the command with GDB. Depending on how the Python code is read into GDB, you may need to import the `gdb` module explicitly.

23.3.2.22 GDB/MI Commands In Python

It is possible to add GDB/MI (see Chapter 27 [GDB/MI], page 609) commands implemented in Python. A GDB/MI command is implemented using an instance of the `gdb.MICommand` class, most commonly using a subclass.

`MICommand.__init__ (name)` [Function]

The object initializer for `MICommand` registers the new command with GDB. This initializer is normally invoked from the subclass' own `__init__` method.

name is the name of the command. It must be a valid name of a GDB/MI command, and in particular must start with a hyphen (-). Reusing the name of a built-in

GDB/MI is not allowed, and a `RuntimeError` will be raised. Using the name of an GDB/MI command previously defined in Python is allowed, the previous command will be replaced with the new command.

MICommand.invoke (*arguments*) [Function]

This method is called by GDB when the new MI command is invoked.

arguments is a list of strings. Note, that `--thread` and `--frame` arguments are handled by GDB itself therefore they do not show up in *arguments*.

If this method raises an exception, then it is turned into a GDB/MI `^error` response. Only `gdb.GdbError` exceptions (or its sub-classes) should be used for reporting errors to users, any other exception type is treated as a failure of the `invoke` method, and the exception will be printed to the error stream according to the `set python print-stack` setting (see `[set python print-stack]`, page 408).

If this method returns `None`, then the GDB/MI command will return a `^done` response with no additional values.

Otherwise, the return value must be a dictionary, which is converted to a GDB/MI *result-record* (see Section 27.4.2 [GDB/MI Output Syntax], page 613). The keys of this dictionary must be strings, and are used as *variable* names in the *result-record*, these strings must comply with the naming rules detailed below. The values of this dictionary are recursively handled as follows:

- If the value is Python sequence or iterator, it is converted to GDB/MI *list* with elements converted recursively.
- If the value is Python dictionary, it is converted to GDB/MI *tuple*. Keys in that dictionary must be strings, which comply with the *variable* naming rules detailed below. Values are converted recursively.
- Otherwise, value is first converted to a Python string using `str ()` and then converted to GDB/MI *const*.

The strings used for *variable* names in the GDB/MI output must follow the following rules; the string must be at least one character long, the first character must be in the set `[a-zA-Z]`, while every subsequent character must be in the set `[-_a-zA-Z0-9]`.

An instance of `MICommand` has the following attributes:

MICommand.name [Variable]

A string, the name of this GDB/MI command, as was passed to the `__init__` method. This attribute is read-only.

MICommand.installed [Variable]

A boolean value indicating if this command is installed ready for a user to call from the command line. Commands are automatically installed when they are instantiated, after which this attribute will be `True`.

If later, a new command is created with the same name, then the original command will become uninstalled, and this attribute will be `False`.

This attribute is read-write, setting this attribute to `False` will uninstall the command, removing it from the set of available commands. Setting this attribute to `True` will install the command for use. If there is already a Python command with

this name installed, the currently installed command will be uninstalled, and this command installed in its stead.

The following code snippet shows how some trivial MI commands can be implemented in Python:

```
class MIEcho(gdb.MICommand):
    """Echo arguments passed to the command."""

    def __init__(self, name, mode):
        self._mode = mode
        super(MIEcho, self).__init__(name)

    def invoke(self, argv):
        if self._mode == 'dict':
            return { 'dict': { 'argv' : argv } }
        elif self._mode == 'list':
            return { 'list': argv }
        else:
            return { 'string': ", ".join(argv) }

MIEcho("-echo-dict", "dict")
MIEcho("-echo-list", "list")
MIEcho("-echo-string", "string")
```

The last three lines instantiate the class three times, creating three new GDB/MI commands `-echo-dict`, `-echo-list`, and `-echo-string`. Each time a subclass of `gdb.MICommand` is instantiated, the new command is automatically registered with GDB.

Depending on how the Python code is read into GDB, you may need to import the `gdb` module explicitly.

The following example shows a GDB session in which the above commands have been added:

```
(gdb)
-echo-dict abc def ghi
^done,dict={argv=["abc","def","ghi"]}
(gdb)
-echo-list abc def ghi
^done,list=["abc","def","ghi"]
(gdb)
-echo-string abc def ghi
^done,string="abc, def, ghi"
(gdb)
```

Conversely, it is possible to execute GDB/MI commands from Python, with the results being a Python object and not a specially-formatted string. This is done with the `gdb.execute_mi` function.

`gdb.execute_mi (command [, arg]...)` [Function]

Invoke a GDB/MI command. *command* is the name of the command, a string. The arguments, *arg*, are passed to the command. Each argument must also be a string.

This function returns a Python dictionary whose contents reflect the corresponding GDB/MI command's output. Refer to the documentation for these commands for details. Lists are represented as Python lists, and tuples are represented as Python dictionaries.

If the command fails, it will raise a Python exception.

Here is how this works using the commands from the example above:

```
(gdb) python print(gdb.execute_mi("-echo-dict", "abc", "def", "ghi"))
{'dict': {'argv': ['abc', 'def', 'ghi']}}
(gdb) python print(gdb.execute_mi("-echo-list", "abc", "def", "ghi"))
{'list': ['abc', 'def', 'ghi']}
(gdb) python print(gdb.execute_mi("-echo-string", "abc", "def", "ghi"))
{'string': 'abc, def, ghi'}
```

23.3.2.23 GDB/MI Notifications In Python

It is possible to emit GDB/MI notifications from Python. Use the `gdb.notify_mi` function to do that.

`gdb.notify_mi (name [, data])` [Function]

Emit a GDB/MI asynchronous notification. *name* is the name of the notification, consisting of alphanumeric characters and a hyphen (-). *data* is any additional data to be emitted with the notification, passed as a Python dictionary. This argument is optional. The dictionary is converted to a GDB/MI *result* records (see Section 27.4.2 [GDB/MI Output Syntax], page 613) the same way as result of Python MI command (see Section 23.3.2.22 [GDB/MI Commands In Python], page 474).

If *data* is `None` then no additional values are emitted.

While using existing notification names (see Section 27.7.3 [GDB/MI Async Records], page 618) with `gdb.notify_mi` is allowed, users are encouraged to prefix user-defined notification with a hyphen (-) to avoid possible conflict. GDB will never introduce notification starting with hyphen.

Here is how to emit `--connection-removed` whenever a connection to remote GDB server is closed (see Section 23.3.2.40 [Connections In Python], page 514):

```
def notify_connection_removed(event):
    data = {"id": event.connection.num, "type": event.connection.type}
    gdb.notify_mi("--connection-removed", data)
```

```
gdb.events.connection_removed.connect(notify_connection_removed)
```

Then, each time a connection is closed, there will be a notification on MI channel:

```
--connection-removed,id="1",type="remote"
```

23.3.2.24 Parameters In Python

You can implement new GDB parameters using Python. A new parameter is implemented as an instance of the `gdb.Parameter` class.

Parameters are exposed to the user via the `set` and `show` commands. See Section 3.6 [Help], page 29.

There are many parameters that already exist and can be set in GDB. Two examples are: `set follow fork` and `set charset`. Setting these parameters influences certain behavior in GDB. Similarly, you can define parameters that can be used to influence behavior in custom Python scripts and commands.

Parameter.__init__ (*name*, *command_class*, *parameter_class* [, *enum_sequence*] [Function])

The object initializer for **Parameter** registers the new parameter with GDB. This initializer is normally invoked from the subclass' own `__init__` method.

name is the name of the new parameter. If *name* consists of multiple words, then the initial words are looked for as prefix parameters. An example of this can be illustrated with the `set print` set of parameters. If *name* is `print foo`, then `print` will be searched as the prefix parameter. In this case the parameter can subsequently be accessed in GDB as `set print foo`.

If *name* consists of multiple words, and no prefix parameter group can be found, an exception is raised.

command_class should be one of the 'COMMAND_' constants (see Section 23.3.2.21 [CLI Commands In Python], page 471). This argument tells GDB how to categorize the new parameter in the help system.

parameter_class should be one of the 'PARAM_' constants defined below. This argument tells GDB the type of the new parameter; this information is used for input validation and completion.

If *parameter_class* is `PARAM_ENUM`, then *enum_sequence* must be a sequence of strings. These strings represent the possible values for the parameter.

If *parameter_class* is not `PARAM_ENUM`, then the presence of a fourth argument will cause an exception to be thrown.

The help text for the new parameter includes the Python documentation string from the parameter's class, if there is one. If there is no documentation string, a default value is used. The documentation string is included in the output of the parameters `help set` and `help show` commands, and should be written taking this into account. If the documentation string for the parameter's class is the empty string then GDB will only use `Parameter.set_doc` or `Parameter.show_doc` (see below) in the `help` output.

Parameter.set_doc [Variable]

If this attribute exists, and is a string, then its value is used as the first part of the help text for this parameter's `set` command. The second part of the help text is taken from the documentation string for the parameter's class, if there is one.

The value of `set_doc` should give a brief summary specific to the set action, this text is only displayed when the user runs the `help set` command for this parameter. The class documentation should be used to give a fuller description of what the parameter does, this text is displayed for both the `help set` and `help show` commands.

The `set_doc` value is examined when `Parameter.__init__` is invoked; subsequent changes have no effect.

Parameter.show_doc [Variable]

If this attribute exists, and is a string, then its value is used as the first part of the help text for this parameter's `show` command. The second part of the help text is taken from the documentation string for the parameter's class, if there is one.

The value of `show_doc` should give a brief summary specific to the show action, this text is only displayed when the user runs the `help show` command for this parameter.

The class documentation should be used to give a fuller description of what the parameter does, this text is displayed for both the `help set` and `help show` commands.

The `show_doc` value is examined when `Parameter.__init__` is invoked; subsequent changes have no effect.

`Parameter.value` [Variable]

The `value` attribute holds the underlying value of the parameter. It can be read and assigned to just as any other attribute. GDB does validation when assignments are made.

There are two methods that may be implemented in any `Parameter` class. These are:

`Parameter.get_set_string (self)` [Function]

If this method exists, GDB will call it when a *parameter*'s value has been changed via the `set` API (for example, `set foo off`). The `value` attribute has already been populated with the new value and may be used in output. This method must return a string. If the returned string is not empty, GDB will present it to the user.

If this method raises the `gdb.GdbError` exception (see Section 23.3.2.3 [Exception Handling], page 416), then GDB will print the exception's string and the `set` command will fail. Note, however, that the `value` attribute will not be reset in this case. So, if your parameter must validate values, it should store the old value internally and reset the exposed value, like so:

```
class ExampleParam (gdb.Parameter):
    def __init__ (self, name):
        super (ExampleParam, self).__init__ (name,
                                              gdb.COMMAND_DATA,
                                              gdb.PARAM_BOOLEAN)
        self.value = True
        self.saved_value = True
    def validate(self):
        return False
    def get_set_string (self):
        if not self.validate():
            self.value = self.saved_value
            raise gdb.GdbError('Failed to validate')
        self.saved_value = self.value
        return ""
```

`Parameter.get_show_string (self, svalue)` [Function]

GDB will call this method when a *parameter*'s `show` API has been invoked (for example, `show foo`). The argument `svalue` receives the string representation of the current value. This method must return a string.

When a new parameter is defined, its type must be specified. The available types are represented by constants defined in the `gdb` module:

`gdb.PARAM_BOOLEAN`

The value is a plain boolean. The Python boolean values, `True` and `False` are the only valid values.

`gdb.PARAM_AUTO_BOOLEAN`

The value has three possible states: `true`, `false`, and `'auto'`. In Python, `true` and `false` are represented using boolean constants, and `'auto'` is represented using `None`.

`gdb.PARAM_UIINTEGER`

The value is an unsigned integer. The value of `None` should be interpreted to mean “unlimited” (literal `'unlimited'` can also be used to set that value), and the value of 0 is reserved and should not be used.

`gdb.PARAM_INTEGER`

The value is a signed integer. The value of `None` should be interpreted to mean “unlimited” (literal `'unlimited'` can also be used to set that value), and the value of 0 is reserved and should not be used.

`gdb.PARAM_STRING`

The value is a string. When the user modifies the string, any escape sequences, such as `'\t'`, `'\f'`, and octal escapes, are translated into corresponding characters and encoded into the current host charset.

`gdb.PARAM_STRING_NOESCAPE`

The value is a string. When the user modifies the string, escapes are passed through untranslated.

`gdb.PARAM_OPTIONAL_FILENAME`

The value is either a filename (a string), or `None`.

`gdb.PARAM_FILENAME`

The value is a filename. This is just like `PARAM_STRING_NOESCAPE`, but uses file names for completion.

`gdb.PARAM_ZINTEGER`

The value is a signed integer. This is like `PARAM_INTEGER`, except that 0 is allowed and the value of `None` is not supported.

`gdb.PARAM_ZUIINTEGER`

The value is an unsigned integer. This is like `PARAM_UIINTEGER`, except that 0 is allowed and the value of `None` is not supported.

`gdb.PARAM_ZUIINTEGER_UNLIMITED`

The value is a signed integer. This is like `PARAM_INTEGER` including that the value of `None` should be interpreted to mean “unlimited” (literal `'unlimited'` can also be used to set that value), except that 0 is allowed, and the value cannot be negative, except the special value -1 is returned for the setting of “unlimited”.

`gdb.PARAM_ENUM`

The value is a string, which must be one of a collection string constants provided when the parameter is created.

`gdb.PARAM_COLOR`

The value is `gdb.Color` instance.

When creating multiple new parameters using `gdb.Parameter`, it is often desirable to create a prefix command that can be used to group related parameters together, for example, if you wished to add the parameters `plugin-name feature-1` and `plugin-name feature-2`, then the `plugin-name` would need to be a prefix command (see Section 23.3.2.21 [CLI Commands In Python], page 471).

However, when creating parameters, you will almost always need to create two prefix commands, one as a `set` sub-command, and one as a `show` sub-command. GDB provides the `gdb.ParameterPrefix` helper class to make creation of these two prefixes easier.

ParameterPrefix.__init__ (*name*, *command_class*, *doc* = None) [Function]

The object initializer for `ParameterPrefix` registers two new `gdb.Command` prefixes, one as a `set` sub-command, and the other as a `show` sub-command.

name, a string, is the name of the new prefix, without either `set` or `show`, similar to the *name* passed to `gdb.Parameter` (see Section 23.3.2.24 [Parameters In Python], page 477). For example, to create the prefixes `set plugin-name` and `show plugin-name`, you would pass the string `plugin-name`.

command_class should be one of the ‘COMMAND_’ constants (see Section 23.3.2.21 [CLI Commands In Python], page 471). This argument tells GDB how to categorize the new parameter prefixes in the help system.

There are a number of ways in which the help text for the two new prefix commands can be provided. If the *doc* parameter is not `None`, then this will be used as the documentation string for both prefix commands.

If *doc* is `None`, but `gdb.ParameterPrefix` has been sub-classed, then the prefix command documentation will be taken from sub-classes documentation string (i.e., the `__doc__` attribute).

If *doc* is `None`, and there is no `__doc__` string, then the default value ‘This command is not documented.’ is used.

When writing the help text, keep in mind that the same text is used for both the `set` and `show` prefix commands.

ParameterPrefix.invoke_set (*argument*, *from_tty*) [Function]

If a sub-class defines this method, then GDB will call this when the prefix command is used with an unknown sub-command. The *argument* and *from_tty* parameters are the same as for `gdb.Command.invoke` (see [Command.invoke], page 471).

If this method throws an exception, it is turned into a GDB `error` call. Otherwise, the return value is ignored.

It is not required that a `ParameterPrefix` sub-class override this method. Usually, a parameter prefix only exists as a means to group related parameters together. GDB handles this use case automatically with no need to implement `invoke_set`.

ParameterPrefix.invoke_show (*argument*, *from_tty*) [Function]

This is like the `invoke_set` method, but for the `show` prefix command. As with `invoke_set`, implementation of this method is optional, and usually not required.

`ParameterPrefix.dont_repeat ()` [Function]

Like `Command.dont_repeat` (see [Command.dont_repeat], page 471), this can be called from `ParameterPrefix.invoke_set` or `ParameterPrefix.invoke_show` to prevent the prefix commands from being repeated.

Here is a small example that uses `gdb.ParameterPrefix` along with `gdb.Parameter` to create two new parameters *plugin-name feature-1* and *plugin-name feature-2*. As neither `invoke_set` or `invoke_show` is needed, this example does not sub-class `gdb.ParameterPrefix`:

```
class ExampleParam(gdb.Parameter):
    def __init__(self, name):
        super().__init__(name, gdb.COMMAND_DATA, gdb.PARAM_BOOLEAN)
        self.value = True

gdb.ParameterPrefix("plugin-name", gdb.COMMAND_NONE,
    """An example parameter prefix.

    This groups together some parameters.""")
ExampleParam("plugin-name feature-1")
ExampleParam("plugin-name feature-2")
```

The helper class `gdb.StyleParameterSet` exists to make it easier to create new styles. GDB places style settings under ‘`show style ...`’ and ‘`set style ...`’, an example of a style is ‘`filename`’. Each style is really a prefix command (see Section 23.3.2.21 [CLI Commands In Python], page 471), with sub-commands ‘`foreground`’, ‘`background`’, and optionally, ‘`intensity`’.

It is simple enough to create a new style using two `gdb.Command` objects for the prefix commands (one for ‘`set`’, and one for ‘`show`’), and three `gdb.Parameter` objects, one each for the ‘`foreground`’, ‘`background`’, and ‘`intensity`’. You would also want to take care to craft the help text so that the new style behaves the same as the existing styles.

Or, you can use the `gdb.StyleParameterSet` class, which takes care of all this, as the following example shows:

```
(gdb) python s = gdb.StyleParameterSet("my-style")
(gdb) show style my-style
style my-style background: The "my-style" style background color is: none
style my-style foreground: The "my-style" style foreground color is: none
style my-style intensity: The "my-style" style display intensity is: normal
(gdb)
```

You might also want to group a number of styles within a new prefix, similar to how GDB groups disassembler related styles within the ‘`style disassembler`’ prefix. This can be done using `gdb.ParameterPrefix` (see [gdb.ParameterPrefix], page 481), as in this example:

```
(gdb) python gdb.ParameterPrefix("style group", gdb.COMMAND_NONE)
(gdb) python s_a = gdb.StyleParameterSet("group aa")
(gdb) python s_b = gdb.StyleParameterSet("group bb")
(gdb) show style group
style group aa background: The "group aa" style background color is: none
style group aa foreground: The "group aa" style foreground color is: none
style group aa intensity: The "group aa" style display intensity is: normal
style group bb background: The "group bb" style background color is: none
style group bb foreground: The "group bb" style foreground color is: none
style group bb intensity: The "group bb" style display intensity is: normal
(gdb)
```

The `gdb.StyleParameterSet` class has the following methods and attributes: